On-Device Training Library

Complete Beginner's Reference

Every File · Every Line · Every Concept Explained

A comprehensive guide to the ODT C library for on-device training on the STM32 Nucleo-F746ZG microcontroller. Written for beginners — every line of source code is explained in plain language, with definitions for all C syntax and concepts.

Mohamed Atef | Master's Thesis | 2026

Table of Contents

Chapter 1

Introduction

What is this project and why does it exist?

1.1 The Problem: Machine Learning on a Tiny Chip

Modern machine learning models are trained on powerful servers with gigabytes of RAM and dedicated GPUs that cost thousands of dollars. But many real-world applications — wearable devices, industrial sensors, wildlife monitors, hearing aids — need to run AI locally, on a tiny chip with no internet connection and less RAM than a floppy disk.

The STM32 Nucleo-F746ZG is one such chip. It has a 32-bit ARM Cortex-M7 processor running at 216 MHz, 320 kilobytes of RAM, and 1 megabyte of flash storage. For comparison, a single image from your phone camera takes about 10 megabytes. Running even a tiny neural network on this chip is a serious engineering challenge.

This project — the On-Device Training (ODT) library — solves that problem. It is a C library (not Python, not TensorFlow) that can train and run neural networks entirely on the STM32, without any connection to a PC. The application is speaker identification from audio, as part of a Master's thesis combining two research ideas: FQT (Fixed-point Quantization Training) and RigL (sparse training).

1.2 What is FQT?

Floating-point numbers (like 0.12345678) need 32 bits each. That is too expensive for a chip with 320 KB of RAM. Fixed-point quantization replaces each 32-bit float with a smaller integer (say, an 8-bit or 16-bit number) plus a scale factor. For example, the float 0.5 might be stored as the integer 64 with a scale of 1/128. This is called SYM_INT32 quantization in the ODT library.

WORKED EXAMPLE

Example: storing the value 0.75 as SYM_INT32 with scale=0.01: mantissa = round(0.75 / 0.01) = round(75.0) = 75 Stored as: integer 75 (4 bytes) To recover: 75 × 0.01 = 0.75 With 8-bit storage (qMaxBits=8), values -127 to +127 are representable. With 16-bit storage (qMaxBits=16), values -32767 to +32767 are representable. FQT trains the network to be accurate even with this limited precision.

1.3 What is RigL?

Most neural networks are 'dense': every neuron is connected to every other neuron. But research (Evci et al., 2020 — RigL paper) shows that you can remove most connections (make the network 'sparse') and still get good accuracy. This is important on the STM32 because fewer connections means fewer multiplications, which means faster inference and less memory.

RigL works by periodically removing the smallest-magnitude weights and adding new connections where the gradients are largest. The ODT library has partial support for RigL — Chapter 10 explains exactly what is implemented and what is still missing.

1.4 The 8-Layer Architecture

The ODT library is organised in 8 layers, where each layer builds on the ones below. This document has one chapter per layer. Understanding the layers in order is the key to understanding the whole system.

Layer	Files	What it provides
0 — Foundation	Common.h	Debug macros — printing error/warning/debug messages
1 — Primitives	DTypes.h/.c	Read/write individual numbers as raw bytes
2 — Data Model	Quantization.h/.c	How numbers are stored: float, int, quantized
3 — Core Data	DataStorage.h, Tensor.h, TensorConversion.h	The tensor — the central data container
5 — Arithmetic	Arithmetic.h/.c + Add/Sub/Mul/Div/Matmul/etc.	All math operations on tensors
6 — Layers	Layer.h + Linear/Conv1d/Relu/Softmax/etc.	Neural network layers (forward + backward)
7 — Training	LossFunction.h + Optimizer.h + TrainingLoop	Loss functions, SGD optimizer, training loop
Support	RNG, Serialize, DataLoader, UserAPI	Random numbers, saving/loading, data pipeline

1.5 How to Read This Document

Each chapter follows the same structure: (1) what the file does and why it exists; (2) the complete source code with every line explained; (3) relations — which files depend on this file and which files this file uses; (4) what is missing for RigL. C syntax concepts are explained in blue boxes whenever they first appear. Read the chapters in order — later chapters assume knowledge from earlier ones.

■ BEGINNER TIP

If you are new to C: start by reading Chapter 2 very carefully. It introduces the most important C concepts (preprocessor, macros, #ifndef). Later chapters build on these. Every time you see a 'C CONCEPT' box, stop and read it before continuing.

Chapter 2

Layer 0 — Common.h

The Debug Macro System

Section 2.1 — What This File Does

■ WHAT THIS FILE DOES

Common.h provides a debug printing system for the entire ODT library. It defines four macros: PRINT_ERROR, PRINT_WARNING, PRINT_INFO, and PRINT_DEBUG. Each macro calls printf() to output a message, but PRINT_WARNING/INFO/DEBUG are silenced in production builds (zero code size, zero runtime cost). PRINT_ERROR is always active because errors are fatal and must always be reported. Every .c file in the library includes Common.h as its first include.

Think of Common.h as the 'alarm system' of the library. During development, you can turn on verbose output to see exactly what is happening. When you compile for the final product (flashing to the STM32), all the verbose output is automatically removed by the compiler, so the firmware is as small and fast as possible.

Section 2.2 — Complete Source Code

```
// Common.h — src/common/include/Common.h
#ifndef COMMON_H // line 1
#define COMMON_H // line 2
#include <stdio.h> // line 4
#include <stdlib.h> // line 5
#include <stdbool.h> // line 6
#ifndef SOURCE_FILE // line 8
#define SOURCE_FILE "UNKNOWN" // line 9
#endif // line 10
#if defined DEBUG_MODE_DEBUG // line 12
#define DLEVEL 3 // line 13
#elif defined DEBUG_MODE_INFO // line 14
#define DLEVEL 2 // line 15
#elif defined DEBUG_MODE_WARNING // line 16
#define DLEVEL 1 // line 17
#else // line 18
#define DLEVEL 0 // line 19
#endif // line 20
#define PRINT_ERROR(str, ...) do { // line 22
printf("[%s] ERROR: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \
} while(false) // line 24
#if DLEVEL >= 1 // line 26
#define PRINT_WARNING(str, ...) do { \
printf("[%s] WARNING: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \
} while(false) // line 29
#else
#define PRINT_WARNING(str, ...) do { } while(false)
#endif
#if DLEVEL >= 2
#define PRINT_INFO(str, ...) do { \
printf("[%s] INFO: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \
} while(false)
#else
#define PRINT_INFO(str, ...) do { } while(false)
#endif
#if DLEVEL >= 3
#define PRINT_DEBUG(str, ...) do { \
printf("[%s] DEBUG: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \
} while(false)
#else
#define PRINT_DEBUG(str, ...) do { } while(false)
#endif
#endif // COMMON_H
```

Section 2.3 — Line-by-Line Explanation

■ C CONCEPT: #ifndef / #define / #endif — Header Guard

A header guard stops a .h file from being included more than once in the same compilation. Without it, if two .c files both include Common.h, the compiler would see all the definitions twice and give 'redefinition' errors. #ifndef COMMON_H → 'if the name COMMON_H is NOT yet defined...' #define COMMON_H → '...then define it now (as an empty symbol)' #endif → '...end of the conditional block' The first time Common.h is included, COMMON_H is not defined, so the block executes. The second time it is included, COMMON_H IS already defined, so the entire block is skipped. The name COMMON_H is arbitrary — it is the convention to use the filename in uppercase with dots replaced by underscores.

1	`#ifndef COMMON_H`	Start of the header guard. 'ifndef' = 'if not defined'. If the preprocessor symbol COMMON_H has not been defined yet, process everything until the matching #endif. This prevents the file from being processed twice.

2	`#define COMMON_H`	Define the guard symbol. After this line, COMMON_H is defined (its value is empty). If this file is included again later, line 1 will be false, and the whole file is skipped.
4	`#include <stdio.h>`	Include the standard I/O header. This gives us access to the printf() function. Angle brackets < > mean this is a system header (part of the C standard library). Double quotes would mean a project-local file.
5	`#include <stdlib.h>`	Include the standard library header. This gives us exit(), malloc(), free(), and more. We need exit() so that PRINT_ERROR can halt the program after a fatal error.
6	`#include <stdbool.h>`	Include the boolean header. This defines the bool type (true/false). Without this, C has no boolean — it uses 0/1 integers instead. stdbool.h is a C99 addition; before C99, programmers had to define bool themselves.
8	`#ifndef SOURCE_FILE`	Each .c file defines SOURCE_FILE as its filename before including Common.h: #define SOURCE_FILE "RELU". If a file forgets to do this, this guard provides the fallback 'UNKNOWN'.
9	`#define SOURCE_FILE "UNKNOWN"`	Fallback value for SOURCE_FILE. The debug messages will show [UNKNOWN] as the source, which is a signal to the developer to add a proper definition.
12	`#if defined DEBUG_MODE_DEBUG`	Compile-time conditional: check if the compiler was given -DDEBUG_MODE_DEBUG. '#if defined X' is equivalent to '#ifdef X'. This starts a chain of three levels of debug verbosity.
13	`#define DLEVEL 3`	If DEBUG_MODE_DEBUG is defined, set DLEVEL to 3 (most verbose). DLEVEL controls which print macros are active: level 3 enables ALL of them.
14	`#elif defined DEBUG_MODE_INFO`	'elif' = 'else if'. Only checked if line 12 was false. If DEBUG_MODE_INFO is defined (but not DEBUG_MODE_DEBUG), set DLEVEL=2.
16	`#elif defined DEBUG_MODE_WARNING`	If only DEBUG_MODE_WARNING is defined, set DLEVEL=1 (warnings only).
18	`#else`	No debug flag was defined — this is the default production build.
19	`#define DLEVEL 0`	DLEVEL=0 means: only PRINT_ERROR is active. All other macros expand to empty code. The compiler sees 'do { } while(false)' and removes it entirely. This is how the library achieves zero overhead in production.
22	`#define PRINT_ERROR(str, ...) do { \`	Define the PRINT_ERROR macro. It takes a format string 'str' plus variadic arguments '...' (zero or more extra parameters, like printf). The \ at the end continues the macro definition to the next line. PRINT_ERROR has NO DLEVEL guard — it is ALWAYS active.
23	`printf("[%s] ERROR: " str "\n", SOURCE_FILE, ##__VA_ARGS__); \`	'[%s] ERROR: ' and 'str' are two string literals next to each other. The C preprocessor concatenates adjacent string literals automatically. So PRINT_ERROR("value is %d", x) becomes printf("[RELU] ERROR: value is %d\n", x). ##__VA_ARGS__ expands to the variadic arguments; ## removes the leading comma if there are no extra arguments (prevents a syntax error for PRINT_ERROR("msg")).

24	`} while(false)`	The do { } while(false) wrapper is a safe macro pattern. It makes the macro behave like a single statement in all contexts: if (err) PRINT_ERROR("bad!"); ← works correctly with the wrapper Without the wrapper, the macro expands to two statements, and only the first is guarded by the if. The do-while forces both into one compound statement.
26	`#if DLEVEL >= 1`	PRINT_WARNING is only compiled into the binary if DLEVEL is 1, 2, or 3. The preprocessor evaluates DLEVEL >= 1 at compile time — this is not a runtime check.

■ C CONCEPT: The Preprocessor — What It Does

The C preprocessor runs BEFORE the compiler. It handles all lines starting with #. It performs three main jobs: 1. File inclusion (#include) — copies the entire contents of a header file in place 2. Macro expansion (#define) — replaces macro names with their defined text 3. Conditional compilation (#if, #ifdef, #ifndef) — includes or excludes blocks of code The preprocessor sees the source code as raw text. It does not understand C syntax — it only does text substitution. The result of preprocessing is given to the C compiler, which then does the actual translation to machine code.

■ C CONCEPT: Variadic Macros — ... and __VA_ARGS__

A variadic macro accepts a variable number of arguments, like printf(). #define MY_PRINT(fmt, ...) printf(fmt, __VA_ARGS__) The ... in the parameter list means 'zero or more extra arguments'. Inside the macro body, __VA_ARGS__ expands to those extra arguments. The ## before __VA_ARGS__ is a special GNU extension: it removes the comma before __VA_ARGS__ when there are no extra arguments. Without it: MY_PRINT("hello") → printf("hello",) ← syntax error! With ##: MY_PRINT("hello") → printf("hello") ← correct

Section 2.4 — Relations With Other Files

■ RELATIONS WITH OTHER FILES

Common.h is the only file with NO dependencies — it is Layer 0. Every other .c file in the library includes Common.h as its first include. This makes PRINT_ERROR, PRINT_WARNING etc. available everywhere.

File	Direction	Why
Common.h	→ (no dependencies)	Layer 0: depends on nothing
Every .c file in the library	← includes Common.h	All .c files need PRINT_ERROR for error reporting
stdio.h (C standard library)	← includes (system)	Provides printf() which PRINT_ERROR uses
stdlib.h (C standard library)	← includes (system)	Provides exit() for fatal error termination
stdbool.h (C standard library)	← includes (system)	Provides bool type for do-while(false) safety pattern

2.5 The SOURCE_FILE Convention

Every .c file in the library starts with the line:

```
#define SOURCE_FILE "FILENAME"
```

This line must appear BEFORE the #include "Common.h" line, because Common.h uses SOURCE_FILE in its macro definitions. When PRINT_ERROR is called from Relu.c, the output shows [RELU] so the developer immediately knows which file triggered the error. On a UART terminal connected to the STM32, this is often the

only debugging information available.

═■ WORKED EXAMPLE

How a .c file starts (example from Relu.c): #define SOURCE_FILE "RELU" ← must come FIRST #include <stdlib.h> #include <string.h> #include "Common.h" ← SOURCE_FILE must be defined before this #include "Relu.h" When you see: [RELU] ERROR: Unknown QType! You immediately know: the error came from Relu.c, the switch statement fell through to default because an unexpected quantization type was used.

Section 2.5 — What Is Missing for RigL in This File

■ RigL GAP IN THIS FILE

Common.h has no RigL gap. It is a utility file that provides debug printing. RigL does not require any changes here. However, when implementing RigL operations (pruning, regrowing), the developer will use PRINT_DEBUG to log sparsity changes during training — so Common.h will naturally be used by the new RigL code.

Chapter 3

Layer 1 — DTypes.h / DTypes.c

Byte-Level Data Types and Raw Memory Access

Section 3.1 — What This File Does

■ WHAT THIS FILE DOES

DTypes.h and DTypes.c provide the most fundamental operations in the library: reading and writing individual numbers (int32_t and float) as raw bytes. All tensor data is stored as a raw array of uint8_t (unsigned bytes). To read or write a value, you need to assemble/disassemble its bytes. DTypes.c provides safe functions to do this using memcpy. Every arithmetic operation in the library eventually calls these functions.

Why store data as bytes? Because uint8_t is the universal currency of memory in C. A tensor might hold float32 values today and int32 values tomorrow. By storing everything as raw bytes, the tensor_t struct does not need to know what type of data it holds — only the functions that read/write it need to know. This is the foundation of the library's type flexibility.

■ C CONCEPT: What is uint8_t?

uint8_t is an 8-bit unsigned integer — a single byte, holding values 0 to 255. The 'u' stands for unsigned (no negative values). '8' is the bit width. 't' is a C convention for type aliases (typedef). Data type widths: uint8_t = 1 byte = 8 bits (values 0 to 255) int32_t = 4 bytes = 32 bits (values -2,147,483,648 to +2,147,483,647) float = 4 bytes = 32 bits (IEEE 754 floating point) size_t = 4 or 8 bytes (platform-dependent, always unsigned) These types are defined in <stdint.h> (for int32_t, uint8_t) and <stddef.h> (for size_t). Using them instead of 'int' or 'long' guarantees exact sizes on all platforms.

Section 3.2 — DTypes.h: The Header File

```
// DTypes.h — src/tensor/include/DTypes.h
#ifndef ENV5_RUNTIME_DTYPES_H
#define ENV5_RUNTIME_DTYPES_H
#include <stdint.h>
#include <stddef.h>
// Read functions
int32_t readBytesAsInt32(const uint8_t *bytes);
float readBytesAsFloat(const uint8_t *bytes);
// Write functions
void writeInt32ToByteArray(int32_t value, uint8_t *bytes);
void writeFloatToByteArray(float value, uint8_t *bytes);
// Bit-level operations (for BOOL/sparse types)
void writeBitsToByteArray(uint8_t *byteArray, size_t startBit,
size_t endBit, uint8_t data);
uint8_t readBitsFromByteArray(const uint8_t *byteArray, size_t startBit,
size_t endBit);
#endif // ENV5_RUNTIME_DTYPES_H
```

The header declares 6 functions. The first four are the workhorses used everywhere. The last two (writeBitsToByteArray, readBitsFromByteArray) handle sub-byte operations for BOOL tensors (where each element takes only 1 bit) and sparse tensors. Every file that needs to read or write tensor values includes DTypes.h.

Section 3.3 — DTypes.c: The Implementation

```
// DTypes.c — src/tensor/DTypes.c
#define SOURCE_FILE "DTYPES"
#include <string.h> // for memcpy
#include <stdint.h>
#include "Common.h"
#include "DTypes.h"
int32_t readBytesAsInt32(const uint8_t *bytes) {
int32_t value;
memcpy(&value, bytes, sizeof(int32_t));
return value;
}
float readBytesAsFloat(const uint8_t *bytes) {
float value;
memcpy(&value, bytes, sizeof(float));
return value;
}
void writeInt32ToByteArray(int32_t value, uint8_t *bytes) {
memcpy(bytes, &value, sizeof(int32_t));
}
void writeFloatToByteArray(float value, uint8_t *bytes) {
memcpy(bytes, &value, sizeof(float));
}
void writeBitsToByteArray(uint8_t *byteArray, size_t startBit,
size_t endBit, uint8_t data) {
size_t byteIndex = startBit / 8;
size_t bitStart = startBit % 8;
size_t bitEnd = endBit % 8;
uint8_t mask = ((1u << (bitEnd - bitStart)) - 1) << bitStart;
byteArray[byteIndex] = (byteArray[byteIndex] & ~mask) | ((data << bitStart) & mask);
}
uint8_t readBitsFromByteArray(const uint8_t *byteArray, size_t startBit,
size_t endBit) {
size_t byteIndex = startBit / 8;
size_t bitStart = startBit % 8;
size_t bitEnd = endBit % 8;
uint8_t mask = ((1u << (bitEnd - bitStart)) - 1) << bitStart;
return (byteArray[byteIndex] & mask) >> bitStart;
}
```

Section 3.4 — Line-by-Line Explanation

3.4.1 readBytesAsInt32

■ C CONCEPT: memcpy — The Safe Way to Reinterpret Memory

memcpy(destination, source, size) copies 'size' bytes from source to destination. Why not just cast? You might think: why not write *(int32_t*)bytes? This would tell the CPU: 'treat the memory at address bytes as an int32_t'. But this is UNDEFINED BEHAVIOUR in C when the pointer is misaligned or the pointer types violate strict aliasing rules (the compiler can assume pointers to different types never point to the same memory). memcpy is the CORRECT way: it copies the raw bytes without any type assumptions. The compiler knows memcpy semantics and can optimize it to a single 32-bit load instruction on ARM Cortex-M7 anyway. Example: uint8_t bytes[4] = {0x00, 0x00, 0x80, 0x3F}; // IEEE 754 for 1.0f float value; memcpy(&value, bytes, sizeof(float)); // value is now 1.0f

1-3	`int32_t readBytesAsInt32(const uint8_t *bytes) {`	Function signature. 'const uint8_t *bytes' means: a pointer to bytes that we promise not to modify (const). The 'const' is important for correctness and lets the compiler pass read-only memory safely.

2	`int32_t value;`	Declare a local int32_t variable. It is uninitialized — its value is garbage until memcpy fills it on the next line. We do NOT initialize it to 0 because memcpy immediately overwrites all 4 bytes anyway.
3	`memcpy(&value, bytes, sizeof(int32_t));`	Copy sizeof(int32_t) = 4 bytes from 'bytes' into 'value'. '&value' takes the address of the local variable — gives memcpy a pointer to write to. sizeof(int32_t) ensures we copy exactly 4 bytes regardless of the platform. After this line, 'value' holds the integer stored in bytes[0..3].
4	`return value;`	Return the assembled integer. The caller receives it as a regular int32_t.

═■ WORKED EXAMPLE

Example: reading the integer 305 from a byte array. 305 in binary = 0x00000131 = bytes [0x31, 0x01, 0x00, 0x00] (little-endian ARM) uint8_t data[4] = {0x31, 0x01, 0x00, 0x00}; int32_t result = readBytesAsInt32(data); // result = 305 Little-endian means: the LEAST significant byte comes first in memory. ARM Cortex-M7 (and most modern CPUs) are little-endian. 0x31 is the least significant byte of 305; 0x00 is the most significant byte.

3.4.2 readBytesAsFloat

■ C CONCEPT: IEEE 754 — How Floats are Stored

A 32-bit float (float in C) is stored in IEEE 754 format: Bit 31: sign (0 = positive, 1 = negative) Bits 30-23: exponent (8 bits, biased by 127) Bits 22-0: mantissa (23 bits, fraction part) The value = (-1)^sign × 2^(exponent-127) × 1.mantissa Example: 1.0 in IEEE 754: Sign=0, Exponent=127 (stored as 01111111), Mantissa=0 Bytes: 0x00, 0x00, 0x80, 0x3F (little-endian) This is why readBytesAsFloat uses memcpy — we need to take the 4 raw bytes and have the CPU interpret them as a float WITHOUT changing their bit pattern.

1-3	`float readBytesAsFloat(const uint8_t *bytes) {`	Identical pattern to readBytesAsInt32, but returns float instead of int32_t.
2	`float value;`	Local float variable — 4 bytes on the stack, same as int32_t.
3	`memcpy(&value, bytes, sizeof(float));`	Copy 4 bytes from the array into the float variable. The CPU does NOT convert any values — it copies the exact bit pattern. If bytes holds the IEEE 754 representation of 0.5, value will be 0.5f.

3.4.3 writeInt32ToByteArray and writeFloatToByteArray

1	`void writeInt32ToByteArray(int32_t value, uint8_t *bytes) {`	Reverse operation: write an int32_t into a byte array. 'void' means no return value — the result is written through the pointer 'bytes'. The caller is responsible for ensuring 'bytes' points to at least 4 valid bytes.
2	`memcpy(bytes, &value, sizeof(int32_t));`	Copy 4 bytes FROM 'value' (taking its address with &) INTO 'bytes'. After this, bytes[0..3] hold the little-endian representation of 'value'. For value=305: bytes becomes [0x31, 0x01, 0x00, 0x00].

3.4.4 writeBitsToByteArray — Bit-Level Operations

■ C CONCEPT: Bit Manipulation — Masks, Shifts, OR, AND

A bitmask is an integer with specific bits set to 1. Combining a mask with bitwise operations lets you set, clear, or read individual bits. Operators: & (AND) — 1 & 1 = 1; anything & 0 = 0 → clears bits | (OR) — 0 | 1 = 1; anything | 1 = 1 → sets bits ~ (NOT) — inverts all bits → complement << (left shift) — multiply by 2 per shift → moves bits left >> (right shift) — divide by 2 per shift → moves bits right Example: isolate bits 2-4 of byte 0b10110110: mask = ((1<<(4-2)) - 1) << 2 = (4-1) << 2 = 3 << 2 = 0b00001100 result = (0b10110110 & 0b00001100) >> 2 = 0b00000110 >> 2 = 0b00000001 = 1

2	`size_t byteIndex = startBit / 8;`	Find which byte contains the bit we want. Bits are numbered 0..N. Bit 13 is in byte 13/8 = byte 1 (integer division).
3	`size_t bitStart = startBit % 8;`	Offset within the byte. Bit 13 is at position 13%8 = 5 within byte 1.
5	`uint8_t mask = ((1u << (bitEnd - bitStart)) - 1) << bitStart;`	Build a bitmask covering exactly the bits we want to write. '1u' is unsigned 1 to prevent signed overflow on shift. (1u << width) - 1 creates width consecutive 1-bits starting at bit 0. Shifting left by bitStart moves them to the correct position. Example: startBit=2, endBit=5: mask = ((1<<3)-1) << 2 = 0b00011100
6	`byteArray[byteIndex] = (byteArray[byteIndex] & ~mask) | ((data << bitStart) & mask);`	Three steps in one line: 1. 'byteArray[byteIndex] & ~mask' — clear the target bits (AND with inverted mask) 2. '(data << bitStart) & mask' — place the new data in the correct bit positions 3. '... | ...' — OR them together to get the final byte with the new bits set This is the standard 'clear-then-set' bit manipulation pattern.

Section 3.5 — Relations With Other Files

File	Direction	Why
DTypes.h/.c	→ depends on	<string.h> for memcpy, Common.h for SOURCE_FILE
Arithmetic.c	← uses DTypes	All element read/write: readBytesAsInt32/Float, writeXxx
Matmul.c	← uses DTypes	Reading matrix elements, writing results
TensorConversion.c	← uses DTypes	Reading/writing during type conversion
Sgd.c	← uses DTypes	Reading/writing weight and gradient values
Serialize.c / Deserialize.c	← uses DTypes	Writing tensor bytes to/from byte stream
Log.c / Sum.c / MinMax.c	← uses DTypes	Reading element values for reduction ops
Bernoulli.c / Distributions.c	← uses DTypes	Writing sampled values to tensors

■ RigL GAP IN THIS FILE

DTypes.h/.c has no direct RigL gap. However, RigL uses a sparsity mask (a tensor of booleans — one bit per weight). writeBitsToByteArray and readBitsFromByteArray are exactly the functions needed to work with this mask. They already exist! The gap is not in DTypes but in the higher-level RigL logic that uses them.

Chapter 4

Layer 2 — Quantization.h / Quantization.c

The Quantization Type System

Section 4.1 — What This File Does

■ WHAT THIS FILE DOES

Quantization.h defines HOW numbers are stored in every tensor. It provides three things: (1) the qtype_t enum — six possible storage formats; (2) config structs — parameters for each format (scale, zero-point, bit depth); (3) init functions — create and configure quantization objects. Every tensor carries a quantization_t that says 'my values are stored as X'. This is Layer 2 because all higher layers need to know the storage format before they can read or write tensor values.

Understanding quantization is essential for understanding the whole library. A tensor does not just store raw numbers — it stores numbers in a compressed format. The quantization_t attached to each tensor tells you how to decode those numbers back to their real values.

■ C CONCEPT: What is Quantization? (Plain Language)

Quantization is the process of representing a continuous value (like 0.12345) as a small integer (like 12) plus a scale factor (0.01). Real value = integer × scale 0.12 = 12 × 0.01 Why bother? Because integers use less memory and are faster to compute: float (32 bits) × float (32 bits) → 32-bit result (needs FPU, 1 cycle) int8 (8 bits) × int8 (8 bits) → 16-bit result (integer mult, ~0.5 cycles) The price you pay is precision loss. 0.12345 becomes 0.12 (rounded to 2 decimals). FQT (Fixed-point Quantization Training) trains the model to be accurate despite this precision loss.

Section 4.2 — Quantization.h: The Header File

```
// Quantization.h — src/tensor/include/Quantization.h (key parts)
#ifndef ENV5_RUNTIME_QUANTIZATION_H
#define ENV5_RUNTIME_QUANTIZATION_H
#include <stdint.h>
#include <stddef.h>
// ■■ qtype enum: the six storage formats ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
typedef enum qtype {
INT32, // raw signed 32-bit integer, no scale
FLOAT32, // IEEE 754 single-precision float
SYM_INT32, // symmetric quantized: real = mantissa × scale
SYM, // symmetric, configurable bits (< 32)
ASYM, // asymmetric: real = (mantissa - zeroPoint) × scale
BOOL // 1-bit boolean (packed: 8 per byte)
} qtype_t;
// ■■ qConfig structs: one per quantization type ■■■■■■■■■■■■■■■■■■■■■■■■■
typedef struct int32QConfig {
// empty — INT32 has no extra config
} int32QConfig_t;
typedef struct float32QConfig {
// empty — FLOAT32 has no extra config
} float32QConfig_t;
typedef struct symInt32QConfig {
float scale;
uint8_t qMaxBits; // e.g. 16 → mantissa in [-32767, 32767]
uint8_t roundingMode;
} symInt32QConfig_t;
typedef struct symQConfig {
float scale;
uint8_t qMaxBits;
uint8_t roundingMode;
} symQConfig_t;
typedef struct asymQConfig {
float scale;
int32_t zeroPoint;
uint8_t qMaxBits;
uint8_t roundingMode;
} asymQConfig_t;
typedef struct boolQConfig {
// empty — booleans have no extra config
} boolQConfig_t;
// ■■ quantization_t: the unified quantization descriptor ■■■■■■■■■■■■■■■■
typedef struct quantization {
qtype_t type; // which format?
void *qConfig; // pointer to the format-specific config struct
} quantization_t;
// ■■ rounding modes ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
typedef enum roundingMode {
HALF_AWAY, // round 0.5 away from zero (standard rounding)
SR_HALF_AWAY, // stochastic rounding: adds noise before rounding
} roundingMode_t;
// ■■ init functions ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
void initInt32Quantization(quantization_t *q);
void initFloat32Quantization(quantization_t *q);
void initSymInt32Quantization(quantization_t *q, symInt32QConfig_t *cfg);
void initBoolQuantization(quantization_t *q, boolQConfig_t *cfg);
void initInt32QConfig(int32QConfig_t *cfg);
void initFloat32QConfig(float32QConfig_t *cfg);
void initSymInt32QConfig(roundingMode_t rm, symInt32QConfig_t *cfg);
size_t calcNumberOfBytesForData(quantization_t *q, size_t numberOfElements);
size_t calcBitsPerElement(quantization_t *q);
#endif // ENV5_RUNTIME_QUANTIZATION_H
```

Section 4.3 — Line-by-Line: The qtype_t Enum

■ C CONCEPT: typedef enum — Defining an Enumeration Type

An enum (enumeration) is a type whose values are named integer constants. typedef enum myEnum { A, B, C } myEnum_t; A=0, B=1, C=2 automatically (or you can assign values manually: A=5). 'typedef' creates a type alias so you can write 'myEnum_t x' instead of 'enum myEnum x'. Enums are useful because they let you write readable code: if (q->type == FLOAT32) { ... } ← clear if (q->type == 1) { ... } ← confusing — what does 1 mean?

INT32	`INT32 — raw 32-bit integer`	The value IS the integer, no scale factor. Used for: raw index tensors, intermediate accumulation buffers. Memory: 4 bytes per element. No extra config (int32QConfig_t is empty).
FLOAT32	`FLOAT32 — IEEE 754 float`	The standard floating-point format. Used for: weights during FLOAT32 training, gradients, activations when using float paths. Memory: 4 bytes per element. No extra config (float32QConfig_t is empty). This is the simplest and most precise option, but uses the most memory.
SYM_INT32	`SYM_INT32 — symmetric fixed-point, 32-bit mantissa`	Real value = mantissa × scale. The mantissa is stored as int32_t (4 bytes). Scale is a float stored in symInt32QConfig_t. qMaxBits limits the range: if qMaxBits=16, mantissas stay in [-32767, +32767]. Used for: FQT training — both weights and activations. Centred at zero — no zero-point needed.
SYM	`SYM — symmetric, sub-32-bit mantissa`	Same as SYM_INT32 but qMaxBits can be 8, 4, 2, etc. Allows even more aggressive compression. Used for inference-only deployments.
ASYM	`ASYM — asymmetric fixed-point`	Real value = (mantissa - zeroPoint) × scale. The zeroPoint allows the range to be shifted: e.g. ReLU outputs are always >= 0, so ASYM can map integer 0 to real 0.0 and cover [0, 2×scale×qMax] efficiently. Used for activations after ReLU in pure-integer inference.
BOOL	`BOOL — 1-bit boolean`	Each element is 0 or 1, stored as 1 bit (8 elements per byte). Used for: sparsity masks (which weights are active), Bernoulli dropout masks. Memory: (N + 7) / 8 bytes for N elements (ceiling division).

4.3.1 The quantization_t Struct

■ C CONCEPT: void* — The Generic Pointer

void* is a pointer to 'nothing in particular' — it can point to any type. You cannot dereference a void* directly (the compiler doesn't know the size). You must cast it to the correct type first: void *ptr = &myCfg; // store any address symInt32QConfig_t *cfg = ptr; // retrieve as specific type In quantization_t, qConfig is void* because it might point to any of six config struct types depending on q->type. The code always checks q->type first, then casts qConfig to the appropriate type. This is a C polymorphism pattern.

Field	Type	Meaning
type	qtype_t	Which storage format: INT32, FLOAT32, SYM_INT32, SYM, ASYM, or BOOL
qConfig	void*	Pointer to the format-specific config struct. Must be cast to the right type based on 'type'. NULL for INT32 and FLOAT32 (they need no config).

═■ WORKED EXAMPLE

How to read a SYM_INT32 tensor's scale: quantization_t *q = tensor->quantization; // get quantization if (q->type == SYM_INT32) { // check the type FIRST symInt32QConfig_t *cfg = q->qConfig; // cast void* to correct type float scale = cfg->scale; // now safe to read scale } Why check the type first? Because if q->type were FLOAT32, qConfig would point to a float32QConfig_t (an empty struct), and reading scale from it would give garbage.

4.3.2 symInt32QConfig_t — Configuration for SYM_INT32

Field	Type	Meaning
scale	float	The scale factor. real_value = mantissa × scale. Updated by requantisation.
qMaxBits	uint8_t	Bit depth of the mantissa. e.g. 16 → mantissa in [-32767, 32767]. 8 → mantissa in [-127, 127].
roundingMode	uint8_t	How to round when converting: HALF_AWAY (standard) or SR_HALF_AWAY (stochastic, reduces bias during FQT).

4.3.3 calcNumberOfBytesForData — Memory Size Calculator

This function tells you how many bytes a tensor needs based on its quantization type and number of elements. It is used when allocating tensor data buffers.

```
size_t calcNumberOfBytesForData(quantization_t *q, size_t numberOfElements) {
switch(q->type) {
case INT32: return numberOfElements * sizeof(int32_t); // 4N bytes
case FLOAT32: return numberOfElements * sizeof(float); // 4N bytes
case SYM_INT32:return numberOfElements * sizeof(int32_t); // 4N bytes
case BOOL: return (numberOfElements + 7) / 8; // ceil(N/8) bytes
// SYM/ASYM: depends on qMaxBits...
default: PRINT_ERROR("Unknown qtype"); exit(1);
}
}
```

4N bytes for INT32/FLOAT32/SYM_INT32	`All three use 4 bytes per element (32-bit storage).`	Despite different semantics (raw int, float, scaled int), they all store one 32-bit value per element. The difference is only in how you interpret those bits.
(N+7)/8 for BOOL	`Ceiling division: pack 8 booleans per byte.`	(N+7)/8 is the ceiling of N/8. For N=10: (10+7)/8 = 17/8 = 2 bytes. Bit 0 of byte 0 is element 0; bit 7 of byte 0 is element 7; bit 0 of byte 1 is element 8.

═■ WORKED EXAMPLE

Memory comparison for a 1000-element tensor: FLOAT32: 1000 × 4 = 4000 bytes (3.9 KB) SYM_INT32: 1000 × 4 = 4000 bytes (3.9 KB) — same space as FLOAT32! SYM (8-bit): 1000 × 1 = 1000 bytes (1.0 KB) — 4× smaller BOOL: (1000+7)/8 = 125 bytes (0.1 KB) — 32× smaller than float SYM_INT32 saves memory over FLOAT32 ONLY if qMaxBits < 32, or if the precision loss allows fewer bits. Otherwise the memory is the same but you gain in compute speed on integer-only hardware.

Section 4.4 — The Init Functions

Every quantization_t must be initialised before use. The init functions set the type field and configure the qConfig pointer. Here is the implementation of the most important ones:

```
// From Quantization.c:
void initInt32Quantization(quantization_t *q) {
q->type = INT32;
q->qConfig = NULL; // INT32 needs no config
}
void initFloat32Quantization(quantization_t *q) {
q->type = FLOAT32;
q->qConfig = NULL; // FLOAT32 needs no config
}
void initSymInt32QConfig(roundingMode_t rm, symInt32QConfig_t *cfg) {
cfg->scale = 1.0f; // neutral scale: real value = mantissa × 1 = mantissa
cfg->qMaxBits = 16; // default: 16-bit mantissa range
cfg->roundingMode = rm;
}
void initSymInt32Quantization(quantization_t *q, symInt32QConfig_t *cfg) {
q->type = SYM_INT32;
q->qConfig = cfg; // store pointer to caller-provided config
}
```

q->type = INT32;	Set the type field to indicate raw integer storage.	This tells any code reading this tensor: 'read these bytes as int32_t, no conversion.'
q->qConfig = NULL;	NULL means 'no config struct'. INT32 and FLOAT32 need no extra parameters.	Any code that checks q->type first and only reads qConfig for SYM_INT32 will never accidentally dereference this NULL pointer.
cfg->scale = 1.0f;	Initialize scale to 1.0 (neutral). With scale=1.0, real = mantissa × 1 = mantissa.	The scale will be updated to a meaningful value after the first data is loaded or after the first requantisation pass. Starting at 1.0 is safe.
cfg->qMaxBits = 16;	Default: 16-bit mantissa range ([-32767, 32767]).	qMax = 2^(qMaxBits-1) - 1 = 2^15 - 1 = 32767. This is a good balance: small enough to save memory, large enough for training precision.
q->qConfig = cfg;	Store the pointer to the caller's config struct.	The quantization_t does NOT own the config — it just stores the address. The caller must ensure cfg stays valid as long as the tensor exists.

Section 4.5 — Relations With Other Files

File	Direction	Why
Quantization.h/.c	→ depends on	DTypes.h (indirectly), stdint.h, stddef.h
Tensor.h	← includes Quantization.h	tensor_t has a quantization_t* field
TensorConversion	← uses Quantization	Reads scale, qMaxBits, type to do type conversion
Arithmetic.c / Matmul.c	← uses Quantization	Reads scale to convert int accumulations to real
Sgd.c	← uses Quantization	Checks q->type to dispatch float vs SYM_INT32 update

LinearApi.c / Conv1dApi.c	← uses Quantization	Creates quantization_t objects for each tensor slot
LossFunction.h / MSE.c	← uses Quantization	Checks tensor type for forward/backward dispatch

■ RigL GAP IN THIS FILE

Quantization.h needs no changes for basic RigL. However, the BOOL type (sparsity mask) is critical for RigL: each weight's 'active' bit is stored as a BOOL tensor. The existing BOOL infrastructure in Quantization.h/DTypes.c is sufficient. What is MISSING for RigL is not in this file but in the arithmetic layer: the mask-aware forward pass (skip multiplications for inactive weights) and the gradient tracking for inactive weights are not yet implemented.

Chapter 5

Layer 3 — DataStorage.h and Tensor.h / Tensor.c

The Central Data Abstraction

Section 5.1 — DataStorage.h: The Dataset Container

■ WHAT THIS FILE DOES

DataStorage.h defines how raw datasets (training examples) are stored in memory. It provides two structs: dataEntry_t (one item in the dataset) and dataStorage_t (a collection of items). The DataLoader uses dataStorage_t to iterate over training data. DataStorage.c is currently empty — the structs are the interface; the DataLoader implements the logic.

```
// DataStorage.h — src/data_loader/include/DataStorage.h
#ifndef DATASTORAGE_H
#define DATASTORAGE_H
#include <stddef.h>
#include <stdint.h>
typedef struct dataEntry {
uint8_t *dataPTR; // pointer to raw bytes for this entry
size_t numberOfElements; // how many elements the entry contains
} dataEntry_t;
typedef struct dataStorage {
uint8_t *data; // flat byte buffer holding all entries
size_t size; // total bytes in the buffer
dataEntry_t *entries; // array of entry descriptors
size_t numberOfEntries;// how many entries are in this dataset
} dataStorage_t;
#endif // DATASTORAGE_H
```

5.1.1 dataEntry_t — One Training Example

Field	Type	Meaning
dataPTR	uint8_t*	Pointer to this entry's raw bytes inside the flat buffer. This is NOT a copy — it points into the dataStorage_t's data buffer.
numberOfElements	size_t	How many elements this entry contains. e.g. for a 1D signal of length 128, numberOfElements = 128.

5.1.2 dataStorage_t — The Full Dataset

Field	Type	Meaning
data	uint8_t*	The flat byte buffer. All entries are packed here consecutively. dataEntry_t::dataPTR points into this buffer.
size	size_t	Total byte count of the data buffer.
entries	dataEntry_t*	Array of numberOfEntries dataEntry_t descriptors. entries[i].dataPTR = data + i * bytesPerEntry.
numberOfEntries	size_t	The dataset size. The DataLoader iterates [0, numberOfEntries).

■ C CONCEPT: Pointer-into-buffer Pattern

Instead of allocating memory separately for each training example, all data lives in ONE flat buffer. Entry descriptors then hold pointers into that buffer. This is memory-efficient (no malloc per entry) and cache-friendly (all data is contiguous in memory). Example: 4 entries, each 10 bytes: data = [e0_bytes... e1_bytes... e2_bytes... e3_bytes...] // 40 bytes total entries[0].dataPTR = &data[0]; // points to start of entry 0 entries[1].dataPTR = &data[10]; // points to start of entry 1 entries[2].dataPTR = &data[20]; // ... The DataLoader then passes entries[i].dataPTR to the getSample callback which reads it and writes into the batch tensor.

Section 5.2 — Tensor.h: The Central Data Structure

■ WHAT THIS FILE DOES

Tensor.h defines the most important struct in the entire library: tensor_t. A tensor is an N-dimensional array of numbers. In ML, every piece of data — inputs, weights, biases, gradients, activations — is stored as a tensor. Tensor.h also defines shape_t (the dimensions), parameter_t (a tensor pair for weight + gradient), sparsity_t (future sparse support), and all utility functions that work on tensors.

■ C CONCEPT: What is a Tensor?

A tensor is a generalization of arrays: Scalar (0D): a single number — 42.0 Vector (1D): [1.0, 2.0, 3.0] — shape [3] Matrix (2D): [[1,2],[3,4],[5,6]] — shape [3,2] 3D tensor: a batch of matrices — shape [batch, rows, cols] In neural networks: Input batch: shape [batch_size, input_features] Linear weights: shape [output_features, input_features] Conv1d weights: shape [out_channels, in_channels, kernel_size] In memory, all tensors are stored as a FLAT 1D array of bytes. The shape_t tells you how to interpret the flat array as N-dimensional.

```
// Tensor.h — src/tensor/include/Tensor.h (key structs)
typedef struct shape {
size_t numberOfDimensions; // e.g. 2 for a matrix
size_t *dimensions; // e.g. {3, 4} for a 3x4 matrix
size_t *orderOfDimensions; // permutation for transposed views
} shape_t;
typedef struct tensor {
uint8_t *data; // flat byte array
shape_t *shape; // how to interpret the data
quantization_t *quantization; // how values are encoded
sparsity_t *sparsity; // sparse pattern (future use)
} tensor_t;
typedef struct parameter {
tensor_t *param; // the weight tensor
tensor_t *grad; // its gradient tensor (same shape)
} parameter_t;
```

Section 5.3 — shape_t in Detail

Field	Type	Meaning
numberOfDimensions	size_t	The number of axes. 1=vector, 2=matrix, 3=3D tensor, etc.
dimensions	size_t*	Array of size numberOfDimensions. dimensions[i] = length along axis i. For shape [3,4]: dimensions={3,4}, totalElements=3*4=12.
orderOfDimensions	size_t*	Permutation of [0..N-1]. Default: {0,1,2,...}. After transposeTensor(t,0,1), orderOfDimensions={1,0}. This enables O(1) transpose without moving data.

■ C CONCEPT: O(1) Transpose — No Data Movement

Transposing a matrix traditionally means moving all elements. For a 1000×1000 matrix, that's 1,000,000 copies. ODT uses a smarter approach: store a permutation of axis indices. When you transpose axes 0 and 1, instead of moving data, swap the axis order in orderOfDimensions. Accessing element [i][j] of the transposed matrix then maps to [j][i] of the original data. The indexing function applies the permutation. This is O(1): swapping two numbers regardless of tensor size. Essential for on-device training where moving data wastes memory and time.

5.3.1 setShape — Initialize a shape_t

```
// setShape fills in shape_t from caller-provided arrays
void setShape(shape_t *shape, size_t *dims, size_t numberOfDims,
size_t *orderOfDims) {
shape->numberOfDimensions = numberOfDims;
shape->dimensions = dims;
shape->orderOfDimensions = orderOfDims;
}
// Typical usage for a 3x4 matrix:
size_t dims[2] = {3, 4};
size_t order[2] = {0, 1}; // row-major (no transpose)
shape_t myShape;
setShape(&myShape, dims, 2, order);
```

■ NOTE

setShape does NOT allocate memory. It stores the pointers you provide. The caller is responsible for keeping dims[] and orderOfDims[] arrays alive. In practice they are declared as static or stack arrays in the user's setup code.

5.3.2 setOrderOfDimsForNewTensor — Default Order

```
// Fill orderOfDimensions with the identity permutation {0, 1, 2, ...}
void setOrderOfDimsForNewTensor(size_t numberOfDimensions,
size_t *orderOfDimensions) {
for (size_t i = 0; i < numberOfDimensions; i++) {
orderOfDimensions[i] = i;
}
}
```

for loop	`for (size_t i = 0; i < numberOfDimensions; i++)`	Iterate over all dimension indices, 0 to N-1.
body	`orderOfDimensions[i] = i;`	Set each axis to point to itself: no reordering. For a 3D tensor: orderOfDimensions = {0,1,2} means 'dimension 0 is still first, dimension 1 is still second, etc.'

Section 5.4 — tensor_t Field by Field

Field	Type	Meaning
data	uint8_t*	Raw bytes. For a FLOAT32 tensor of 12 elements: 48 bytes. For SYM_INT32: also 48 bytes (int32_t per element). For BOOL: ceil(N/8) bytes. This field is NEVER NULL for a valid tensor.
shape	shape_t*	Pointer to the shape descriptor. Gives dimensions and axis order. May be shared across multiple tensors (view semantics).
quantization	quantization_t*	Pointer to quantization. Tells you type (FLOAT32, SYM_INT32, etc.) and config (scale, qMaxBits, etc.). Must be cast correctly before use.
sparsity	sparsity_t*	Pointer to sparsity config. NULL for dense tensors (current state). Will hold sparsity mask location for RigL. Currently unused.

═■ WORKED EXAMPLE

Full tensor setup for a 3x4 FLOAT32 weight matrix: // 1. Allocate the data buffer (12 floats * 4 bytes = 48 bytes) static uint8_t weightData[48]; // 2. Set up quantization static quantization_t weightQ; initFloat32Quantization(&weightQ); // type=FLOAT32, qConfig=NULL // 3. Set up shape static size_t dims[2] = {3, 4}; static size_t order[2] = {0, 1}; static shape_t weightShape; setShape(&weightShape, dims, 2, order); // 4. Assemble the tensor static tensor_t weightTensor; setTensorValues(&weightTensor, weightData, &weightShape, &weightQ, NULL); // Now weightTensor.data points to weightData (48 bytes of floats) // weightTensor.shape->dimensions[0] = 3, [1] = 4 // weightTensor.quantization->type = FLOAT32

Section 5.5 — Key Functions in Tensor.c

5.5.1 calcNumberOfElementsByShape

```
size_t calcNumberOfElementsByShape(shape_t *shape) {
size_t numElem = 1;
size_t numberOfDimensions = shape->numberOfDimensions;
size_t *dimensions = shape->dimensions;
for (size_t i = 0; i < numberOfDimensions; i++) {
numElem *= dimensions[i];
}
return numElem;
}
```

size_t numElem = 1;	`Start with 1 (multiplicative identity).`	We multiply all dimension sizes together. Starting at 0 would give 0.
for loop	`for (size_t i = 0; i < numberOfDimensions; i++)`	Iterate over all dimensions.
numElem *= dimensions[i];	`Multiply running product by this dimension's size.`	For shape {3,4,5}: 1 × 3 = 3 → 3 × 4 = 12 → 12 × 5 = 60 elements total.
return numElem;	`Return total element count.`	A [3,4] matrix has 12 elements. A [2,3,4] cube has 24 elements.

5.5.2 calcBytesPerElement — Type-Aware Size

```
size_t calcBytesPerElement(quantization_t *quantization) {
switch (quantization->type) {
case INT32: return sizeof(int32_t); // 4 bytes
case FLOAT32: return sizeof(float); // 4 bytes
case SYM_INT32:return sizeof(int32_t); // 4 bytes
case SYM: {
symQConfig_t *cfg = quantization->qConfig;
return (size_t)ceilf((float)cfg->qBits / 8.0f); // e.g. 8 bits = 1 byte
}
case ASYM: {
asymQConfig_t *cfg = quantization->qConfig;
return (size_t)ceilf((float)cfg->qBits / 8.0f);
}
case BOOL: return 1; // 1 byte even though each element is 1 bit
default: PRINT_ERROR("Unknown QType!"); exit(1);
}
}
```

■ C CONCEPT: switch Statement in C

switch(expression) { case VALUE: ...; break; } tests an integer expression against multiple constant values. switch (x) { case 0: printf("zero"); break; case 1: printf("one"); break; default: printf("other"); break; } 'break' exits the switch. Without break, execution 'falls through' to the next case (a common C gotcha). The { ... } after a case with local variables (like cfg) creates a new scope to avoid redeclaration errors.

5.5.3 transposeTensor — O(1) Axis Swap

```
void transposeTensor(tensor_t *tensor, size_t dim0Index, size_t dim1Index) {
size_t *order = tensor->shape->orderOfDimensions;
size_t *dims = tensor->shape->dimensions;
// Swap the two entries in orderOfDimensions
size_t tmp = order[dim0Index];
order[dim0Index] = order[dim1Index];
order[dim1Index] = tmp;
// Also swap the physical dimension sizes
size_t tmpDim = dims[dim0Index];
dims[dim0Index] = dims[dim1Index];
dims[dim1Index] = tmpDim;
}
```

size_t tmp = order[dim0Index];	`Save the value we're about to overwrite (classic three-way swap).`	In C, a = b; b = a; would lose b's value. We need a temporary variable tmp to hold one value while we swap.
order[dim0Index] = order[dim1Index];	`Move dim1's axis number into dim0's slot.`	

order[dim1Index] = tmp;	`Move the saved dim0 value into dim1's slot.`	After these three lines, the two entries are swapped.
dims swap	`Also swap the sizes in the dimensions array.`	If you transpose a [3,4] matrix, the shape becomes [4,3]. Swapping dims[] reflects this without moving data.

5.5.4 copyTensor and tensorBoolGet/Set

```
// copyTensor: copy data bytes between two tensors of the same size
void copyTensor(tensor_t *dest, tensor_t *src) {
size_t bytes = calcBytesPerTensor(src);
memcpy(dest->data, src->data, bytes);
}
// tensorBoolGet: read a single 1-bit boolean from a BOOL tensor
bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex) {
size_t byteIdx = flatIndex / 8;
size_t bitIdx = flatIndex % 8;
return (tensor->data[byteIdx] >> bitIdx) & 1u;
}
// tensorBoolSet: write a single 1-bit boolean into a BOOL tensor
void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value) {
size_t byteIdx = flatIndex / 8;
size_t bitIdx = flatIndex % 8;
if (value)
tensor->data[byteIdx] |= (1u << bitIdx); // set bit
else
tensor->data[byteIdx] &= ~(1u << bitIdx); // clear bit
}
```

═■ WORKED EXAMPLE

Reading element 13 from a BOOL tensor: flatIndex = 13 byteIdx = 13 / 8 = 1 (element 13 is in byte 1) bitIdx = 13 % 8 = 5 (it is bit 5 of that byte) data[1] = 0b10100000 (example) data[1] >> 5 = 0b00000101 ... & 1 = 1 → element 13 is TRUE Writing FALSE to element 13: mask = ~(1u << 5) = ~0b00100000 = 0b11011111 data[1] &= 0b11011111 → bit 5 cleared, other bits unchanged

Section 5.6 — parameter_t: Weights and Gradients Together

A parameter_t bundles a weight tensor with its gradient tensor. In neural network training, every learnable weight has a corresponding gradient of the same shape. The optimizer (SGD) needs both: param to update and grad to read. Pairing them reduces the number of pointers you need to pass around.

```
typedef struct parameter {
tensor_t *param; // the weight tensor W
tensor_t *grad; // gradient dL/dW (same shape as param)
} parameter_t;
// Helper accessors (inline, never fail):
tensor_t *getParamFromParameter(parameter_t *p) { return p->param; }
tensor_t *getGradFromParameter(parameter_t *p) { return p->grad; }
```

═■ WORKED EXAMPLE

A linear layer has two parameter_t objects: one for weights, one for biases. linearConfig_t has: parameter_t weights → weights.param = weight tensor, weights.grad = dL/dW parameter_t bias → bias.param = bias tensor, bias.grad = dL/db During backward pass: linearCalcWeightGrads writes into weights.grad linearCalcBiasGrads writes into bias.grad During optimizer step: sgdStep reads weights.grad and updates weights.param sgdStep reads bias.grad and updates bias.param

Section 5.7 — sparsity_t: The RigL Hook

```
typedef enum { SPARSITY_TYPE_1, SPARSITY_TYPE_2 } sparsityType_t;
typedef struct sparsityConfig {
// currently empty
} sparsityConfig;
typedef struct sparsity {
sparsityType_t type;
sparsityConfig *config;
} sparsity_t;
// In tensor_t:
// sparsity_t *sparsity; // NULL for dense tensors
```

■ RigL GAP IN THIS FILE

The sparsity_t field in tensor_t is the planned integration point for RigL. Currently it is an empty stub — the config struct has no fields and is never read. To implement RigL you would: 1. Add a 'mask' field to sparsityConfig: a tensor_t* of type BOOL, same shape as the weight tensor. mask[i]=1 means weight i is active; 0 = pruned. 2. Add a 'sparsityRatio' float field: the target fraction of pruned weights. 3. In linearForward, check sparsity != NULL and skip elements where mask=0. 4. In the training loop, add a RigL update step that sorts weights by magnitude, prunes the smallest, and re-activates weights where the gradient is largest. The hook (sparsity_t *sparsity field) already exists in tensor_t. Only the mask logic, the arithmetic fast-path, and the RigL update step are missing.

Section 5.8 — TensorConversion.h / TensorConversion.c

■ WHAT THIS FILE DOES

TensorConversion provides the convertTensor function and the requantSymInt32Tensor functions. convertTensor converts a tensor from one quantization type to another (e.g. FLOAT32 → SYM_INT32 or SYM_INT32 → FLOAT32). This is essential for the FQT training loop: forward pass works in SYM_INT32, but intermediate buffers may need to be converted for loss computation or debug output.

```
// TensorConversion.h key declarations
// Function pointer type for a single type conversion
typedef void (*conversionFunction_t)(tensor_t *input, tensor_t *output);
// The dispatch table: conversionMatrix[fromType][toType]
extern conversionFunction_t conversionMatrix[6][6];
// Main entry point:
void convertTensor(tensor_t *inputTensor, tensor_t *outputTensor);
// Requantize SYM_INT32 → SYM_INT32 with fresh dynamic scale (two-pass):
void requantSymInt32Tensor(tensor_t *input, tensor_t *output);
// Requantize SYM_INT32 → SYM_INT32 into a caller-set target scale:
void requantSymInt32TensorToScale(tensor_t *input, tensor_t *output);
```

■ C CONCEPT: 2D Function Pointer Table — conversionMatrix[6][6]
Instead of a giant if/else chain for every pair of types, TensorConversion uses a 6×6 table indexed by (inputType, outputType). conversionMatrix[SYM_INT32][FLOAT32] = &symInt32ToFloat; conversionMatrix[FLOAT32][SYM_INT32] = &floatToSymInt32; ... convertTensor then does: conversionMatrix[input->quantization->type][output->quantization->type](input, output); This is O(1) dispatch with no branching. Adding a new type only requires filling in the relevant row/column of the table.

5.8.1 requantSymInt32Tensor — The Two-Pass Algorithm

requantSymInt32Tensor is the most important conversion function. It re-encodes a SYM_INT32 tensor into a new SYM_INT32 tensor with an automatically computed scale. The algorithm is the core of FQT.

■ C CONCEPT: Two-Pass Requantisation (FQT, Deutel 2025)
Pass A — Find the scale: absMax = max over all i of |mantissa_i × inScale| newScale = absMax / qMax (where qMax = 2^(qMaxBits-1) - 1) e.g.: absMax=0.95, qMaxBits=16 → qMax=32767 → newScale=0.95/32767 ≈ 2.9e-5 Pass B — Re-encode: for each i: out_i = clamp(round(realValue_i / newScale), -qMax, qMax) where realValue_i = mantissa_i × inScale Why two passes? Because you need to know the maximum value (Pass A) BEFORE you can compute the scale, and you need the scale BEFORE you can re-encode any value (Pass B). You cannot do this in one pass. IN-PLACE SAFE: both in and out can point to the same tensor, because Pass A only reads and Pass B reads-then-writes in the same order.

Section 5.9 — Relations With Other Files

File	Direction	Why
Tensor.h/.c	→ depends on	Quantization.h, DTypes.h, Common.h, MinMax.h
Arithmetic.c / Matmul.c	← uses Tensor	All operations take tensor_t* arguments
All Layer .c files	← uses Tensor	forward/backward operate on tensor_t*
Sgd.c / LossFunction	← uses Tensor	Read/write weights and gradients
TensorConversion	← uses Tensor	Converts tensor_t from one type to another
DataLoader.c	← uses Tensor + DataStorage	Loads data into batch tensor_t
Serialize.c	← uses Tensor	Saves tensor_t fields to byte stream

Chapter 6

Layer 4 — The Arithmetic Engine

Every operation explained line by line, plus RigL masked matmul

Section 6.1 — Why a Dedicated Arithmetic Layer?

■ WHAT THIS FILE DOES

Neural network computation is almost entirely arithmetic: multiply two numbers, add them, compare them, find the largest one. The Arithmetic layer separates WHAT to compute (the logic in Layer.c / Linear.c) from HOW to compute it (the math in Matmul.c / Add.c). This separation lets you swap implementations: for example, replacing a float matmul with a fixed-point SYM_INT32 matmul without changing the linear layer code at all.

■ C CONCEPT: Separation of Concerns

A design principle: each file has one job. Linear.c knows about layers but delegates all math to Matmul.c and Add.c. Matmul.c knows about the inner-product algorithm but reads values through DTypes.c. DTypes.c knows about bytes but nothing about tensors. This means you can change the matmul algorithm (e.g., add RigL masking) without touching Linear.c. You can change how bytes are read without touching either Matmul.c or Linear.c. Changes stay local.

Module	Key Functions	Purpose
Arithmetic.c	int32PointWiseArithmetic, floatPointWiseArithmetic, calcIndicesByRawIndex, calcTensorIndexByIndices	Generic element-wise engine + index math
Add.h/.c	addInt32Tensors, addFloat32TensorsInplace, addInt32TensorToSymInt32TensorInplace	All tensor addition variants
Sub.h/.c	subInt32Tensors, subFloat32Tensors	Tensor subtraction
Mul.h/.c	mulInt32Tensors, mulFloat32Tensors	Tensor multiplication
Div.h/.c	divInt32Tensors, divFloat32Tensors	Tensor division
Matmul.h/.c	matmulFloat32TensorsWithBias, matmulSymInt32TensorsWithBias	Matrix multiply O(M*K*N)
MinMax.h/.c	findAbsMaxFloat, findMaxInt32, findMinFloat	Scan tensor for extreme values
Sum.h/.c	sumFloat, sumInt32	Sum all elements
Rounding.h/.c	roundByMode, clamp	Quantize-aware rounding + range clamp
Log.h/.c	logFloat	Natural log for cross-entropy loss
Comparison.h/.c	gteFloatValue, gteSymInt32Zero	Threshold (used by ReLU)

Section 6.2 — Arithmetic.c: Index Calculation (Line by Line)

6.2.1 calcTensorIndexByIndices — Multi-dim Indices to Flat Offset

■ C CONCEPT: Row-Major Memory Layout

In C, multi-dimensional arrays are stored in 'row-major' order: the LAST dimension changes fastest. Matrix A[3][4] in memory: A[0][0] A[0][1] A[0][2] A[0][3] | A[1][0] A[1][1] ... flat: 0 1 2 3 | 4 5 Formula for [rows, cols]: flat_index = row * cols + col For [D1,D2,D3]: flat = i*(D2*D3) + j*D3 + k calcTensorIndexByIndices generalises this to any number of dimensions.

```
// Arithmetic.c — calcTensorIndexByIndices
// Input: shape {3,4}, indices {1,2}
// Output: flat memory offset = 6 (row 1, col 2 of 3×4 matrix)
size_t calcTensorIndexByIndices(size_t numberOfDimensions,
size_t *dimensions,
size_t *indices) {
// Start with the innermost (last) index
size_t index = indices[numberOfDimensions - 1];
size_t offset = dimensions[numberOfDimensions - 1];
// Walk backwards through outer dimensions
for (int i = (int)numberOfDimensions - 2; i >= 0; i--) {
index += indices[i] * offset;
offset *= dimensions[i];
}
return index;
}
```

1	`size_t index = indices[numberOfDimensions - 1];`	Start with the last (innermost) index. For shape {3,4}, indices {1,2}: index = indices[1] = 2. The last index contributes directly because each step in the last dimension is 1 element.
2	`size_t offset = dimensions[numberOfDimensions - 1];`	The 'stride' for the next outer dimension. For shape {3,4}: offset = 4. Moving one step in the outer (row) dimension jumps 4 elements in memory.
3	`for (int i = (int)numberOfDimensions - 2; i >= 0; i--)`	Walk backwards from second-to-last dimension to the first. CRITICAL: i must be int (signed), not size_t (unsigned). If i were size_t, then i-- at i=0 would wrap around to 18 quintillion, creating an infinite loop.
4	`index += indices[i] * offset;`	Add this dimension's contribution: its index multiplied by the stride. For i=0: index += 1 * 4 = 4. Total index = 2 + 4 = 6. Row 1, Col 2 of 3×4 = element 6. Correct!
5	`offset *= dimensions[i];`	Grow the stride for the next outer dimension. For a 3D tensor {2,3,4}: after inner dim: offset=4, after mid: offset=12, after outer: offset=24.

6.2.2 calcIndicesByRawIndex — Flat Offset to Multi-dim Indices

```
// Reverse: flat index → per-dimension indices
// Input: shape {3,4}, rawIndex=6
// Output: indices = {1, 2} (row 1, col 2)
void calcIndicesByRawIndex(size_t numberOfDims, size_t *dims,
size_t rawIndex, size_t *indices) {
// Phase 1: compute total elements
size_t offset = 1;
for (size_t i = 0; i < numberOfDims; i++) offset *= dims[i];
// Phase 2: peel off one dimension at a time (outermost first)
size_t restIndex = rawIndex;
for (size_t i = 0; i < numberOfDims; i++) {
offset /= dims[i];
indices[i] = restIndex / offset; // integer division
restIndex -= indices[i] * offset; // remainder
}
}
```

1	`size_t offset = 1; for (...) offset *= dims[i];`	Compute total elements. For {3,4}: offset = 1 × 3 × 4 = 12.
2	`offset /= dims[i];`	Each iteration: reduce offset to the stride for this dimension. i=0: offset = 12/3 = 4 (stride of the outer dimension is 4).
3	`indices[i] = restIndex / offset;`	How many full 'strides' fit in restIndex? That's the index for this dimension. i=0: indices[0] = 6 / 4 = 1. Correct: element 6 is in row 1.
4	`restIndex -= indices[i] * offset;`	Subtract the contribution of this dimension, leaving the remainder for inner dims. restIndex = 6 - 1*4 = 2. For the next iteration: 2 / 1 = 2. indices[1]=2. Correct!

6.2.3 int32PointWiseArithmetic — The Generic Operation Engine

This function applies ANY binary integer operation to matching elements of two tensors. The operation is passed as a function pointer, making this one function serve as add, subtract, multiply, divide, max, min, and more.

■ C CONCEPT: Function Pointers in C

A function pointer stores the address of a function. You can pass it to another function as an argument and call it. Definition of the type: typedef int32_t (*int32ArithFunc_t)(int32_t a, int32_t b); This means: a pointer to a function that takes two int32_t and returns int32_t. An actual function that matches: int32_t addInts(int32_t a, int32_t b) { return a + b; } Passing it: int32PointWiseArithmetic(A, B, addInts, out); Inside the function: arithmeticFunc(aVal, bVal) → calls addInts(aVal, bVal) This pattern avoids copy-pasting the loop for every operation type.

```
// typedef (from Arithmetic.h):
typedef int32_t (*int32ElementArithmeticFunc_t)(int32_t a, int32_t b);
void int32PointWiseArithmetic(
tensor_t *aTensor, tensor_t *bTensor,
int32ElementArithmeticFunc_t arithmeticFunc,
tensor_t *outputTensor) {
if (!doDimensionsMatch(aTensor, bTensor)) {
PRINT_ERROR("Dimensions don't match!"); exit(1);
}
size_t N = calcNumberOfElementsByTensor(aTensor);
size_t nDim = aTensor->shape->numberOfDimensions;
size_t aOrderedDims[nDim];
size_t bOrderedDims[nDim];
orderDims(aTensor, aOrderedDims);
orderDims(bTensor, bOrderedDims);
for (size_t i = 0; i < N; i++) {
size_t aIdx[nDim], bIdx[nDim];
calcIndicesByRawIndex(nDim, aOrderedDims, i, aIdx);
size_t aOff = calcElementIndexByIndices(nDim,
aTensor->shape->dimensions, aIdx,
aTensor->shape->orderOfDimensions);
calcIndicesByRawIndex(nDim, bOrderedDims, i, bIdx);
size_t bOff = calcElementIndexByIndices(nDim,
bTensor->shape->dimensions, bIdx,
bTensor->shape->orderOfDimensions);
int32_t aVal = readBytesAsInt32(aTensor->data + aOff*4);
int32_t bVal = readBytesAsInt32(bTensor->data + bOff*4);
int32_t res = arithmeticFunc(aVal, bVal);
size_t outOff = calcElementIndexByIndices(nDim,
outputTensor->shape->dimensions, aIdx,
outputTensor->shape->orderOfDimensions);
writeInt32ToByteArray(res, outputTensor->data + outOff*4);
}
}
```

1	`if (!doDimensionsMatch(aTensor, bTensor))`	Safety check: element-wise ops require identical shapes. doDimensionsMatch compares each dimension size. If they differ, print error and exit.
2	`size_t N = calcNumberOfElementsByTensor(aTensor);`	Total number of elements to process. Product of all dimensions.
3	`size_t aOrderedDims[nDim];`	VLA (Variable-Length Array): size nDim, allocated on the stack at runtime. This is C99 syntax. On STM32 the stack is ~4–8 KB, so nDim must be small (it always is: 2–4).
4	`orderDims(aTensor, aOrderedDims);`	Fill aOrderedDims with dimension sizes in the tensor's current view order. If the tensor is transposed, orderOfDimensions reorders which size goes where.
5	`for (size_t i = 0; i < N; i++)`	Iterate in logical order: element 0, 1, 2, ... N-1. The index math handles transposition.
6	`calcIndicesByRawIndex(nDim, aOrderedDims, i, aIdx);`	Convert flat iteration index i into multi-dim indices for tensor A in its logical order.
7	`size_t aOff = calcElementIndexByIndices(..., aTensor->shape->orderOfDimensions);`	Convert logical indices to PHYSICAL memory offset in A, respecting the transpose permutation.
8	`int32_t aVal = readBytesAsInt32(aTensor->data + aOff*4);`	Read A's element from the computed physical byte position. +aOff*4 because int32_t is 4 bytes.
9	`int32_t res = arithmeticFunc(aVal, bVal);`	Call the operation function via function pointer. This is where add, sub, mul, etc. happen.

10	`writeInt32ToByteArray(res, outputTensor->data + outOff*4);`	Write the result to the output tensor's physical memory location.

Section 6.3 — Add.h/.c: Every Addition Variant Explained

```
// The primitive: add two int32_t values
static int32_t addInt32s(int32_t a, int32_t b) { return a + b; }
static float addFloats(float a, float b) { return a + b; }
// 1. Add two INT32 tensors → output tensor
void addInt32Tensors(tensor_t *a, tensor_t *b, tensor_t *out) {
int32PointWiseArithmetic(a, b, addInt32s, out);
}
// 2. Add two FLOAT32 tensors, storing result back in 'a' (in-place)
void addFloat32TensorsInplace(tensor_t *a, tensor_t *b) {
floatPointWiseArithmeticInplace(a, b, addFloats);
}
// 3. Add a scalar INT32 value to every element of a tensor
void addInt32ElementWithInt32Tensor(int32_t element, tensor_t *t, tensor_t *out) {
size_t n = calcNumberOfElementsByTensor(t);
for (size_t i = 0; i < n; i++) {
int32_t val = readBytesAsInt32(t->data + i*4);
writeInt32ToByteArray(val + element, out->data + i*4);
}
}
// 4. Add raw INT32 gradient into SYM_INT32 weight mantissa (in-place)
void addInt32TensorToSymInt32TensorInplace(tensor_t *sym, tensor_t *raw) {
size_t n = calcNumberOfElementsByTensor(sym);
for (size_t i = 0; i < n; i++) {
int32_t s = readBytesAsInt32(sym->data + i*4);
int32_t r = readBytesAsInt32(raw->data + i*4);
writeInt32ToByteArray(s + r, sym->data + i*4);
}
}
```

1	`static int32_t addInt32s(int32_t a, int32_t b) { return a + b; }`	'static' means this function is private to Add.c — it cannot be called from other files. It simply returns a + b. This is the function passed as a function pointer to the engine.
2	`int32PointWiseArithmetic(a, b, addInt32s, out);`	Delegate to the generic engine, passing addInt32s as the operation. No loop written here — the engine provides the loop.
3	`floatPointWiseArithmeticInplace(a, b, addFloats);`	In-place variant: result stored back into 'a'. Used for gradient accumulation where you add multiple partial gradients into the same accumulator tensor.
4	`val + element → out`	Scalar broadcast: the same integer value 'element' is added to every tensor element. This is O(N): no index calculations needed, just a linear scan.
5	`addInt32TensorToSymInt32TensorInplace`	The most important addition function for FQT training. It adds a raw gradient (INT32 mantissa delta) into a weight's SYM_INT32 mantissa. The scale does NOT change. Only the stored integer (mantissa) changes. This is valid because the gradient was computed in the weight's own scale units.

Section 6.4 — Matmul.h/.c: Matrix Multiplication Line by Line

■ C CONCEPT: Matrix Multiplication — The Math

C = A × B, where A is [M, K] and B is [K, N]: C[i][j] = A[i][0]*B[0][j] + A[i][1]*B[1][j] + ... + A[i][K-1]*B[K-1][j] = sum_{k=0}^{K-1} A[i][k] * B[k][j] In neural networks (linear layer): A = input activations, shape [batch_size, in_features] M=batch, K=in B = weight matrix (stored transposed), shape [out_features, in_features] N=out, K=in C = output activations, shape [batch_size, out_features] M=batch, N=out B is stored as [N, K] = [out, in], so B[j][k] = weight for output j, input k. The formula becomes: C[i][j] = sum_k A[i][k] * B[j][k] (same as above, just explicit index).

6.4.1 matmulFloat32TensorsWithBias — Complete Line-by-Line

```
// Matmul.c — matmulFloat32TensorsWithBias
void matmulFloat32TensorsWithBias(tensor_t *A, tensor_t *B,
tensor_t *out, tensor_t *bias) {
// Extract dimensions
size_t M = getDimensionsByIndex(A, 0); // batch size
size_t K = getDimensionsByIndex(A, 1); // in_features
size_t N = getDimensionsByIndex(B, 0); // out_features
for (size_t i = 0; i < M; i++) { // for each sample in batch
for (size_t j = 0; j < N; j++) { // for each output neuron
float sum = 0.0f;
for (size_t k = 0; k < K; k++) { // dot product over in_features
float aVal = readBytesAsFloat(A->data + (i*K + k)*4);
float bVal = readBytesAsFloat(B->data + (j*K + k)*4);
sum += aVal * bVal;
}
float biasVal = readBytesAsFloat(bias->data + j*4);
float result = sum + biasVal;
writeFloatToByteArray(result, out->data + (i*N + j)*4);
}
}
}
```

1	`size_t M = getDimensionsByIndex(A, 0);`	getDimensionsByIndex(tensor, axis) returns tensor->shape->dimensions[axis] accounting for the orderOfDimensions permutation. M = batch size.
2	`size_t K = getDimensionsByIndex(A, 1);`	K = number of input features. This must equal B's second dimension (in_features). The shared dimension K is summed over — it disappears from the output.
3	`size_t N = getDimensionsByIndex(B, 0);`	N = number of output features. B is stored as [out, in] = [N, K]. The output has shape [M, N] = [batch, out_features].
4	`for (size_t i = 0; i < M; i++)`	Outer loop over batch samples. Each sample produces one row of the output.
5	`for (size_t j = 0; j < N; j++)`	Middle loop over output neurons. Each output neuron j has its own weight vector B[j,:].
6	`float sum = 0.0f;`	Accumulator for the dot product. Reset to zero for each output element. Initialized to 0.0f not 0 to ensure float semantics (not integer).
7	`for (size_t k = 0; k < K; k++)`	Inner loop: the dot product. Iterates over shared dimension K.
8	`float aVal = readBytesAsFloat(A->data + (i*K + k)*4);`	Read A[i][k]. Memory layout: row i starts at byte i*K*4, column k adds k*4. Combined: (i*K + k)*4. The *4 is because float takes 4 bytes.
9	`float bVal = readBytesAsFloat(B->data + (j*K + k)*4);`	Read B[j][k]. B is stored as [N, K]: row j starts at j*K, column k adds k. Note: B[j][k] = the weight connecting input k to output j. This is the 'transposed' convention: weights stored as [out, in] not [in, out].

10	`sum += aVal * bVal;`	Accumulate the product. After K iterations: sum = A[i,0]*B[j,0] + ... + A[i,K-1]*B[j,K-1]. This is the dot product of row i of A with row j of B.
11	`float biasVal = readBytesAsFloat(bias->data + j*4);`	Read bias for output neuron j. Each output neuron has ONE bias value, shared across all batch elements (same j, no i dependency).
12	`writeFloatToByteArray(result, out->data + (i*N + j)*4);`	Write result to out[i][j]. Output layout: row i at byte i*N*4, col j adds j*4.

6.4.2 matmulSymInt32TensorsWithBias — The FQT 64-bit Accumulation

```
// The SYM_INT32 matmul — key section:
symInt32QConfig_t *aCfg = (symInt32QConfig_t*)A->quantization->qConfig;
symInt32QConfig_t *bCfg = (symInt32QConfig_t*)B->quantization->qConfig;
symInt32QConfig_t *outCfg = (symInt32QConfig_t*)out->quantization->qConfig;
float scaleA = aCfg->scale;
float scaleB = bCfg->scale;
float scaleOut = outCfg->scale;
for (size_t i = 0; i < M; i++) {
for (size_t j = 0; j < N; j++) {
int64_t acc = 0; // 64-bit accumulator!
for (size_t k = 0; k < K; k++) {
int32_t aVal = readBytesAsInt32(A->data + (i*K+k)*4);
int32_t bVal = readBytesAsInt32(B->data + (j*K+k)*4);
acc += (int64_t)aVal * (int64_t)bVal; // safe: no overflow
}
// Convert mantissa product back to real value
float realSum = (float)acc * scaleA * scaleB;
// Add bias (bias has its own scale)
symInt32QConfig_t *biasCfg = (symInt32QConfig_t*)bias->quantization->qConfig;
int32_t biasM = readBytesAsInt32(bias->data + j*4);
realSum += (float)biasM * biasCfg->scale;
// Requantise to output scale
float qMax = (float)((1 << (outCfg->qMaxBits - 1)) - 1);
int32_t quantised = (int32_t)clamp(
roundByMode(realSum / scaleOut, outCfg->roundingMode),
-qMax, qMax);
writeInt32ToByteArray(quantised, out->data + (i*N+j)*4);
}
}
```

1	`symInt32QConfig_t *aCfg = (symInt32QConfig_t*)A->quantization->qConfig;`	Cast the void* qConfig pointer to the specific config type. A->quantization->qConfig is declared as void* to allow any config struct. The cast tells the compiler 'this void* is actually a symInt32QConfig_t*'. This is a SAFE cast because we know A has type SYM_INT32.
2	`float scaleA = aCfg->scale;`	Extract the scale factor. For SYM_INT32: real_value = mantissa * scale. scaleA converts A's mantissas back to real values.
3	`int64_t acc = 0;`	64-bit accumulator. CRITICAL: if we used int32_t, the accumulated sum of products would overflow for long vectors (K > 2). int64_t can hold up to ~9.2×10^18.
4	`(int64_t)aVal * (int64_t)bVal`	Cast BEFORE multiplication. Without the cast: int32_t * int32_t = int32_t (truncated). With the cast: the multiplication is performed in 64-bit arithmetic. For qMaxBits=16: max product = 32767^2 ≈ 10^9 which overflows int32 (max 2.1×10^9) after just 2 accumulations. 64-bit is essential.

5	`float realSum = (float)acc * scaleA * scaleB;`	Convert the integer dot product back to real-value space. acc = sum of (mantissa_A[i,k] × mantissa_B[j,k]). Multiplying by both scales: real_sum = sum of (real_A[i,k] × real_B[j,k]). This is the true matrix multiply result in floating point.
6	`realSum += (float)biasM * biasCfg->scale;`	Add bias in real-value space. Bias is also SYM_INT32, so convert its mantissa with its scale.
7	`float qMax = (float)((1 << (outCfg->qMaxBits - 1)) - 1);`	Compute the maximum representable mantissa for the output. 1 << (qMaxBits-1) = 2^(qMaxBits-1). Minus 1 for symmetric range: e.g., qMaxBits=16 → qMax=32767.
8	`roundByMode(realSum / scaleOut, outCfg->roundingMode)`	Requantise: divide real sum by output scale to get output mantissa, then round. The rounding mode (HALF_AWAY or SR_HALF_AWAY) is specified in the config.
9	`clamp(..., -qMax, qMax)`	Clip the result to the representable range. If the computation overflows the range, clamp prevents wrap-around (which would produce wrong signs).

■■ IMPORTANT

The (symInt32QConfig_t*)qConfig cast is only safe if the tensor truly has type SYM_INT32. Casting a FLOAT32 tensor's qConfig to symInt32QConfig_t and reading its fields would read garbage memory. The library relies on the programmer to call the correct function for the correct tensor type. There is no runtime type check protecting against wrong casts.

Section 6.5 — MinMax.h/.c: Full Line-by-Line

```
// MinMax.c — findAbsMaxFloat
float findAbsMaxFloat(uint8_t *bytes, size_t numberOfElements) {
float absMax = 0.0f;
for (size_t i = 0; i < numberOfElements; i++) {
float val = readBytesAsFloat(bytes + i * sizeof(float));
if (val < 0.0f) val = -val;
if (val > absMax) absMax = val;
}
return absMax;
}
// MinMax.c — findMaxInt32
int32_t findMaxInt32(uint8_t *bytes, size_t numberOfElements) {
int32_t maxVal = INT32_MIN;
for (size_t i = 0; i < numberOfElements; i++) {
int32_t val = readBytesAsInt32(bytes + i * sizeof(int32_t));
if (val > maxVal) maxVal = val;
}
return maxVal;
}
// MinMax.c — findAbsMaxSymInt32 (used in requant Pass A)
int32_t findAbsMaxSymInt32(tensor_t *tensor) {
size_t n = calcNumberOfElementsByTensor(tensor);
int32_t absMax = 0;
for (size_t i = 0; i < n; i++) {
int32_t val = readBytesAsInt32(tensor->data + i*4);
if (val < 0) val = -val;
if (val > absMax) absMax = val;
}
return absMax;
}
```

1	`float absMax = 0.0f;`	Initialize to zero. The absolute maximum is always >= 0, so starting at 0 is safe. We update whenever we find something larger.
2	`float val = readBytesAsFloat(bytes + i * sizeof(float));`	Pointer arithmetic: bytes + i*4 points to the start of the i-th float element. readBytesAsFloat reads 4 bytes starting there and reinterprets them as a float.
3	`if (val < 0.0f) val = -val;`	Manual absolute value. Same as fabsf(val) but avoids including math.h. Negating a negative float flips its sign bit, making it positive.
4	`if (val > absMax) absMax = val;`	Update the running maximum. After the loop, absMax holds the maximum magnitude.
5	`int32_t maxVal = INT32_MIN;`	INT32_MIN = -2,147,483,648 (smallest possible int32_t). Unlike findAbsMaxFloat, findMaxInt32 finds the SIGNED maximum (could be negative). Initializing to INT32_MIN guarantees any real value will be larger.

Section 6.6 — Rounding.h/.c: roundByMode and clamp

```
// Rounding.h
typedef enum { HALF_AWAY, SR_HALF_AWAY } roundingMode_t;
// Rounding.c
int32_t roundByMode(float input, roundingMode_t mode) {
if (mode == SR_HALF_AWAY) {
// Stochastic rounding: add noise in (-0.5, +0.5), then round
float noise = rngNextFloat(&globalRng) - 0.5f;
input += noise;
}
// Standard rounding: round half away from zero
return (int32_t)roundf(input);
}
float clamp(float input, float minVal, float maxVal) {
if (input < minVal) return minVal;
if (input > maxVal) return maxVal;
return input;
}
```

1	`typedef enum { HALF_AWAY, SR_HALF_AWAY } roundingMode_t;`	An enum (enumeration) defines named integer constants. HALF_AWAY = 0, SR_HALF_AWAY = 1 (automatically). Using named constants instead of 0/1 makes code self-documenting and type-safe.
2	`float noise = rngNextFloat(&globalRng) - 0.5f;`	Generate a uniform random float in [0,1) then shift to (-0.5, +0.5). rngNextFloat uses a pseudo-random number generator (Lehmer LCG in RNG.c). The &globalRng passes the RNG state by pointer so it gets updated.
3	`input += noise;`	Add the random noise to the value before rounding. A value of 0.3 becomes 0.3 + noise where noise ~ Uniform(-0.5, 0.5). Probability it rounds to 1: P(noise > 0.7) = 0.3. Expected result: 0.3×1 + 0.7×0 = 0.3. Unbiased!
4	`return (int32_t)roundf(input);`	C cast: convert float to int32_t. roundf() rounds to nearest integer (float). The (int32_t) cast then truncates the decimal part (which is now always 0.0 after rounding). Without roundf: (int32_t)3.7 would truncate to 3, not round to 4.
5	`if (input < minVal) return minVal;`	Clamp: if the value is below the minimum, return the minimum (not the input). Used after rounding to enforce the quantization range: clamp(-35000, -32767, 32767) = -32767.

■ C CONCEPT: Why Stochastic Rounding is Critical for FQT

Standard rounding always rounds 0.5 in the same direction. For small gradients (e.g., 0.001 in a scale=0.01 tensor), the mantissa change is 0.1, which ALWAYS rounds to 0 — the gradient is lost every step. Over 1000 gradient steps: standard rounding → weight never changes. Stochastic rounding: each step has 10% chance of applying the gradient. Expected weight change after 1000 steps = 100 × 0.1 = 10 gradient units. The weight DOES learn — just stochastically. This is why FQT can achieve competitive accuracy with INT16 weights. Without SR, small gradients vanish and training stalls.

Section 6.7 — RigL Masked Matrix Multiply (Full C Implementation)

■ RigL GAP IN THIS FILE
The standard matmulFloat32TensorsWithBias has no concept of sparsity. For RigL to save computation, we need a version that SKIPS elements where mask=0. This section shows the complete implementation of riglMaskedMatmulFloat32.

The key difference from the standard matmul: in the inner loop, before reading the weight B[j][k], we check the mask bit for that weight position. If the mask bit is 0 (this weight is pruned), we skip the multiply-and-add entirely. For 80% sparsity, this skips 80% of the inner loop iterations.

```
// ■■ riglMaskedMatmulFloat32 (to add to Matmul.c for RigL) ■■
//
// mask: a BOOL tensor of shape [out_features, in_features].
// mask=NULL means dense (no masking, same as standard matmul).
// mask[j*K + k] = 1 → weight B[j][k] is active (use it)
// mask[j*K + k] = 0 → weight B[j][k] is pruned (skip it)
void riglMaskedMatmulFloat32(tensor_t *A, tensor_t *B,
tensor_t *out, tensor_t *bias,
tensor_t *mask) {
size_t M = getDimensionsByIndex(A, 0); // batch
size_t K = getDimensionsByIndex(A, 1); // in_features
size_t N = getDimensionsByIndex(B, 0); // out_features
for (size_t i = 0; i < M; i++) {
for (size_t j = 0; j < N; j++) {
float sum = 0.0f;
for (size_t k = 0; k < K; k++) {
// ■■ RigL: skip pruned weights ■■
if (mask != NULL) {
size_t flatIdx = j * K + k;
if (!tensorBoolGet(mask, flatIdx)) continue;
}
// Normal multiply-accumulate
float aVal = readBytesAsFloat(A->data + (i*K+k)*4);
float bVal = readBytesAsFloat(B->data + (j*K+k)*4);
sum += aVal * bVal;
}
float biasVal = readBytesAsFloat(bias->data + j*4);
writeFloatToByteArray(sum + biasVal, out->data + (i*N+j)*4);
}
}
}
```

1	`void riglMaskedMatmulFloat32(..., tensor_t *mask)`	Same signature as standard matmul plus one extra argument: the mask tensor. Passing mask=NULL makes this function behave identically to the standard matmul. This allows calling the same function for both dense and sparse layers.
2	`if (mask != NULL)`	Guard: only check the mask if one was provided. Dense layers pass mask=NULL and pay no masking overhead.
3	`size_t flatIdx = j * K + k;`	Compute the flat index of weight B[j][k] in the weight matrix. The mask has the same shape as B: [N, K] = [out_features, in_features]. Weight B[j][k] corresponds to mask bit at position j*K + k.
4	`if (!tensorBoolGet(mask, flatIdx)) continue;`	tensorBoolGet reads one bit from the BOOL tensor. If the bit is 0 (weight is pruned): 'continue' jumps to the next k iteration, skipping the readBytesAsFloat and multiply-add entirely. For 80% sparsity: 80% of k iterations are skipped → ~5× speedup in the inner loop.

5	`sum += aVal * bVal;`	Only reached if mask bit is 1. Same as standard matmul. Active weights contribute to the output; pruned weights do not.

═■ WORKED EXAMPLE

Performance analysis for a linear layer [128, 64] at 80% sparsity: Standard matmul inner loop iterations: 128 × 64 × 64 = 524,288 RigL masked matmul (80% pruned, 20% active): Active weights: 0.20 × 128×64 = 1,638 Inner loop useful iterations: 128 × 1,638 ≈ 104,858 (plus 524,288 mask checks) Mask check cost: tensorBoolGet is one bit test — very cheap. MACS saved: (524,288 - 104,858) / 524,288 ≈ 80% On ARM Cortex-M7 at 216 MHz: Dense matmul: 524,288 MACs / ~43M MACs/sec ≈ 12ms Sparse matmul: 104,858 MACs ≈ 2.4ms (5× speedup)

Section 6.8 — Relations With Other Files

File	Direction	Why
Arithmetic.c	→ depends on	Tensor.h (tensor_t, shape_t), DTypes.h (readBytesAsInt32), Common.h
Add/Sub/Mul/Div.c	→ use Arithmetic.c	Call int32/floatPointWiseArithmetic with a function pointer lambda
Matmul.c	→ depends on	Tensor.h, DTypes.h, Rounding.h (clamp + roundByMode), TensorConversion.h
MinMax.c	→ depends on	DTypes.h only
Rounding.c	→ depends on	RNG.h (for stochastic rounding), math.h (roundf)
Linear.c	← uses Matmul	linearForwardFloat calls matmulFloat32TensorsWithBias
Conv1d.c	← uses Add + Matmul	conv forward loops call matmul-like accumulations
TensorConversion.c	← uses MinMax + Rounding	requantSymInt32: findAbsMax → roundByMode → clamp
Sgd.c	← uses Add	Weight update: addInt32TensorToSymInt32TensorInplace
RigL.c (future)	← uses Matmul	riglMaskedMatmulFloat32 replaces standard matmul

Chapter 7

Layer 5 — Neural Network Layers

Every forward and backward function explained line by line

Section 7.1 — Layer.h: The Dispatch System

■ WHAT THIS FILE DOES

Layer.h is the registry for all layer types. It defines layer_t — a base struct every layer shares — and a dispatch table of function pointers (forward, backward, update) keyed by layerType_t. When the training loop calls layerForward(layer, input, output), it looks up the correct forward function from the dispatch table based on layer->type.

```
// Layer.h — key types
typedef enum {
LINEAR, RELU, SOFTMAX, CONV1D, DROPOUT, FLATTEN
} layerType_t;
// Union: holds a pointer to ANY layer's config struct
typedef union layerConfig {
linearConfig_t *linear;
reluConfig_t *relu;
softmaxConfig_t *softmax;
conv1dConfig_t *conv1d;
dropoutConfig_t *dropout;
flattenConfig_t *flatten;
} layerConfig_t;
// The base layer struct — every layer is a layer_t
typedef struct layer {
layerType_t type; // which layer is this?
layerConfig_t config; // pointer to this layer's specific config
} layer_t;
// Dispatch function signatures:
typedef void (*layerForwardFunc_t)(layer_t*, tensor_t*, tensor_t*);
typedef void (*layerBackwardFunc_t)(layer_t*, tensor_t*, tensor_t*, tensor_t*);
// Public API:
void layerForward(layer_t *layer, tensor_t *input, tensor_t *output);
void layerBackward(layer_t *layer, tensor_t *lossGrad,
tensor_t *forwardInput, tensor_t *backwardOutput);
void layerUpdate(layer_t *layer, optimizer_t *opt, quantization_t *q);
```

■ C CONCEPT: union in C

A union is like a struct, but ALL fields share the same memory. The union is only as large as its largest field. struct example { int a; float b; } → sizeof = 8 bytes (a + b separate) union example { int a; float b; } → sizeof = 4 bytes (a and b OVERLAP) For layerConfig_t: all pointer types have the same size (4 bytes on 32-bit MCU). So the union is 4 bytes total. You can store any config pointer in it, but only reading the CORRECT field gives valid results. layer->config.linear is valid only if layer->type == LINEAR. Reading layer->config.relu when layer->type == LINEAR reads the same bytes but interprets them as a reluConfig_t pointer — undefined behaviour.

```
// Layer.c — dispatch implementation
// Table of forward functions, indexed by layerType_t
static layerForwardFunc_t forwardFuncs[] = {
[LINEAR] = linearForward,
[RELU] = reluForward,
[SOFTMAX] = softmaxForward,
[CONV1D] = conv1dForward,
[DROPOUT] = dropoutForward,
[FLATTEN] = flattenForward,
};
void layerForward(layer_t *layer, tensor_t *input, tensor_t *output) {
layerForwardFunc_t fn = forwardFuncs[layer->type];
if (fn == NULL) { PRINT_ERROR("Unknown layer type"); exit(1); }
fn(layer, input, output);
}
```

1	`static layerForwardFunc_t forwardFuncs[] = { [LINEAR]=linearForward, ... };`	A C99 designated initialiser for an array. [LINEAR]=linearForward sets element at index LINEAR (= 0) to linearForward. Equivalent to forwardFuncs[0]=linearForward. The static keyword means this array lives in the .data section (persistent global), not on the stack.
2	`layerForwardFunc_t fn = forwardFuncs[layer->type];`	O(1) lookup: use the layer's type as an array index to fetch the function pointer. No if/else chain needed. Adding a new layer type only requires adding one entry to the table.
3	`fn(layer, input, output);`	Call the function through the pointer. This dispatches to linearForward, reluForward, etc. The caller does not need to know which specific function runs.

Section 7.2 — Linear.h/.c: Fully-Connected Layer (Complete)

■ WHAT THIS FILE DOES

The Linear layer implements y = x × W^T + b. It is the fundamental building block of MLPs (multi-layer perceptrons). It has a forward pass (compute output from input), a weight gradient computation (for updating W), a bias gradient computation (for updating b), and an input gradient computation (for back-propagating the error).

Field	Type	Meaning
weights	parameter_t	The weight matrix W and its gradient dL/dW. Shape [out, in].
bias	parameter_t	The bias vector b and its gradient dL/db. Shape [out].
forwardInput	tensor_t*	POINTER to the input tensor from the last forward pass. Stored because the weight gradient backward pass needs it: dL/dW = dL/dY^T × X.
inputGradient	tensor_t*	Buffer for the gradient to pass back to the previous layer: dL/dX.

7.2.1 linearForwardFloat — Complete Line-by-Line

```
// Linear.c — linearForwardFloat
void linearForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
linearConfig_t *cfg = layer->config.linear;
// Store pointer to input for use in weight gradient backward pass
cfg->forwardInput = input;
// output = input × weights^T + bias
tensor_t *W = getParamFromParameter(&cfg->weights);
tensor_t *b = getParamFromParameter(&cfg->bias);
matmulFloat32TensorsWithBias(input, W, output, b);
}
```

1	`linearConfig_t *cfg = layer->config.linear;`	Access the linear-specific config through the union. layer->config is a union; .linear gives the linearConfig_t* field. This pointer was set when the layer was created.
2	`cfg->forwardInput = input;`	CRITICAL: store the input pointer. The backward pass for weights needs the forward input: dL/dW = dL/dY^T × X. Without this pointer, the backward pass would have no access to X. NOTE: this is a pointer, not a copy. The input tensor must remain valid until after the backward pass.
3	`tensor_t *W = getParamFromParameter(&cfg->weights);`	getParamFromParameter returns parameter->param (the weight tensor, not the gradient). The & takes the address of cfg->weights (a parameter_t) to pass it by pointer.
4	`matmulFloat32TensorsWithBias(input, W, output, b);`	The actual computation: output = input × W^T + b. W is stored as [out, in] so accessing B[j][k] = W[out_neuron][in_feature] gives the correct dot product without physically transposing W.

7.2.2 linearCalcWeightGradsFloat — Backward for Weights

■ C CONCEPT: Backpropagation for a Linear Layer

Forward: Y = X × W^T + b (X = input, W = weights, Y = output) Given dL/dY (gradient of loss w.r.t. output), compute: Weight gradient: dL/dW = dL/dY^T × X Shape check: dL/dY is [M, N], X is [M, K] dL/dY^T is [N, M], × X [M, K] → result [N, K] = [out, in] = same shape as W ✓ Bias gradient: dL/db = sum over batch of dL/dY dL/dY is [M, N], sum over M → result [N] = same shape as b ✓ Input gradient: dL/dX = dL/dY × W dL/dY is [M, N], W is [N, K] → result [M, K] = same shape as X ✓ This gradient flows to the PREVIOUS layer.

```
// Linear.c — linearCalcWeightGradsFloat
void linearCalcWeightGradsFloat(layer_t *layer,
tensor_t *lossGrad,
tensor_t *forwardInput) {
linearConfig_t *cfg = layer->config.linear;
tensor_t *wGrad = getGradFromParameter(&cfg->weights);
// dL/dW = dL/dY^T × X
// lossGrad is [M, N] = [batch, out]
// Transpose it to [N, M] to multiply with forwardInput [M, K]
transposeTensor(lossGrad, 0, 1); // now [N, M]
addFloat32TensorsInplace(wGrad, // accumulate gradient
matmulFloat32Result(lossGrad, forwardInput)); // [N,M]×[M,K]=[N,K]
transposeTensor(lossGrad, 0, 1); // restore [M, N]
}
```

1	`tensor_t *wGrad = getGradFromParameter(&cfg->weights);`	Get the gradient tensor (not the weight tensor). The gradient accumulates across mini-batch steps until sgdZeroGrad is called.
2	`transposeTensor(lossGrad, 0, 1);`	Transpose lossGrad from [M, N] to [N, M] in-place, O(1) (just swaps two size values). This allows us to compute dL/dY^T × X as a standard matmul.
3	`addFloat32TensorsInplace(wGrad, matmulFloat32Result(...))`	Accumulate: wGrad += dL/dY^T × X. Using += because gradients accumulate across multiple calls (e.g., gradient checkpointing). The gradient is zeroed by sgdZeroGrad before the next batch.
4	`transposeTensor(lossGrad, 0, 1);`	Restore lossGrad to [M, N]. Since transposeTensor is in-place and O(1), restoring takes the same time as transposing. This is important: lossGrad is shared with other layers; leaving it transposed would corrupt their computation.

7.2.3 linearCalcInputGradsFloat — Backward for Input

```
// Linear.c — linearCalcInputGradsFloat
void linearCalcInputGradsFloat(layer_t *layer,
tensor_t *lossGrad,
tensor_t *inputGradOut) {
linearConfig_t *cfg = layer->config.linear;
tensor_t *W = getParamFromParameter(&cfg->weights);
// dL/dX = dL/dY × W (W already in [out, in] = [N, K])
// lossGrad [M, N] × W [N, K] → inputGradOut [M, K]
matmulFloat32Tensors(lossGrad, W, inputGradOut);
}
```

1	`tensor_t *W = getParamFromParameter(&cfg->weights);`	The actual weight tensor (not transposed). Shape [N, K] = [out_features, in_features].
2	`matmulFloat32Tensors(lossGrad, W, inputGradOut);`	dL/dX = dL/dY × W. lossGrad [M,N] × W [N,K] = [M,K]. This is the gradient that flows to the PREVIOUS layer (e.g., a ReLU before this linear). It tells the previous layer: 'here is how changes to your output affect the loss.'

■ RigL GAP IN THIS FILE

For RigL, linearForwardFloat needs to be modified to check the weight tensor's sparsity: if (cfg->weights.param->sparsity != NULL) { riglMaskedMatmulFloat32(input, W, output, b, cfg->sparsityMask); } else { matmulFloat32TensorsWithBias(input, W, output, b); } The backward pass (linearCalcWeightGrads) does NOT need modification for RigL. Computing gradients for ALL weight positions (including pruned ones) is intentional: RigL uses those 'counterfactual gradients' in the grow step to decide which pruned weights should be reactivated.

Section 7.3 — Relu.h/.c: Rectified Linear Unit (Complete)

■ WHAT THIS FILE DOES

ReLU is f(x) = max(0, x). It is placed after linear layers to add non-linearity. Without non-linearities, any stack of linear layers can be collapsed into a single linear layer (because a linear function of a linear function is still linear). Non-linearity is what lets a network learn complex mappings like speaker ID.

■ C CONCEPT: Why ReLU Works So Well

ReLU has several properties that make it excellent for deep networks: 1. Non-saturating: for x > 0, gradient is exactly 1. Unlike sigmoid (gradient approaches 0 for large |x|), ReLU does not suffer from the vanishing gradient problem. 2. Sparse activation: roughly half of neurons output 0 (when x < 0). This creates natural sparsity in activations, making the network efficient. 3. Fast to compute: max(0, x) is a single comparison — no exp() or division needed. 4. Simple backward: gradient is 1 if x > 0, else 0. No complex chain rule required.

7.3.1 reluForwardFloat — Complete Line-by-Line

```
// Relu.c — reluForwardFloat
void reluForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
reluConfig_t *cfg = layer->config.relu;
// Store input pointer for backward pass
cfg->forwardInput = input;
// Apply ReLU: output[i] = max(0, input[i])
// gteFloatValue sets elements < 0 to 0, keeps elements >= 0 unchanged
gteFloatValue(input, 0.0f, 0.0f, output);
}
```

1	`cfg->forwardInput = input;`	ReLU's backward pass needs to know WHICH inputs were positive in the forward pass. For input x: if x > 0, gradient passes through (multiplied by 1). If x <= 0, gradient is blocked (multiplied by 0). We need the original input to determine which case applies.
2	`gteFloatValue(input, 0.0f, 0.0f, output);`	GTE = Greater Than or Equal. For each element: if element >= 0.0f, copy it to output unchanged. If element < 0.0f, write 0.0f (the belowVal argument) to output. This is exactly ReLU: f(x) = x if x >= 0, else 0.

7.3.2 reluBackwardFloat — Complete Line-by-Line

```
// Relu.c — reluBackwardFloat
// lossGrad: gradient from the layer ABOVE (dL/dY)
// forwardInput: the input that was fed to ReLU in the forward pass
// backwardOutput: gradient to pass to the layer BELOW (dL/dX)
void reluBackwardFloat(layer_t *layer, tensor_t *lossGrad,
tensor_t *forwardInput, tensor_t *backwardOutput) {
size_t n = calcNumberOfElementsByTensor(lossGrad);
for (size_t i = 0; i < n; i++) {
float x = readBytesAsFloat(forwardInput->data + i*4);
float dY = readBytesAsFloat(lossGrad->data + i*4);
float dX = (x > 0.0f) ? dY : 0.0f;
writeFloatToByteArray(dX, backwardOutput->data + i*4);
}
}
```

1	`size_t n = calcNumberOfElementsByTensor(lossGrad);`	Total number of elements. Same for input, output, and loss gradient (they all have equal shape).

2	`float x = readBytesAsFloat(forwardInput->data + i*4);`	Read the i-th element of the ORIGINAL input (stored during forward pass). We need to know if x was positive to determine if the gradient passes through.
3	`float dY = readBytesAsFloat(lossGrad->data + i*4);`	Read the gradient from the layer above: dL/dY[i]. This is the error signal coming back through the network.
4	`float dX = (x > 0.0f) ? dY : 0.0f;`	The ReLU backward rule: d/dx max(0,x) = 1 if x > 0, else 0 dL/dX[i] = dL/dY[i] × d/dx ReLU(x[i]) = dY if x>0, else 0 The ternary operator (condition ? val_if_true : val_if_false) selects between the two cases. Neurons that were active (x > 0) pass the gradient through unchanged. Neurons that were inactive (x <= 0) block the gradient (return 0).
5	`writeFloatToByteArray(dX, backwardOutput->data + i*4);`	Write the input gradient dL/dX[i] to the output buffer. This gradient is then passed to the previous layer's backward function.

Section 7.4 — Softmax.h/.c: Output Layer (Complete)

■ C CONCEPT: Softmax — Converting Logits to Probabilities

Softmax(x)[j] = exp(x[j]) / sum_{k=0}^{N-1} exp(x[k]) Properties: - All outputs in (0, 1) - All outputs sum to exactly 1.0 - Preserves relative ordering: largest logit → largest probability Numerical stability issue: exp(800) = overflow! Fix: Subtract the maximum logit before exp: Softmax(x)[j] = exp(x[j] - max(x)) / sum_k exp(x[k] - max(x)) This is mathematically identical but avoids overflow.

```
// Softmax.c — softmaxForwardFloat
void softmaxForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
size_t M = getDimensionsByIndex(input, 0); // batch size
size_t N = getDimensionsByIndex(input, 1); // num classes
for (size_t i = 0; i < M; i++) {
float *inRow = (float*)(input->data + i*N*4);
float *outRow = (float*)(output->data + i*N*4);
// Step 1: find max logit in this row (numerical stability)
float maxVal = inRow[0];
for (size_t j = 1; j < N; j++)
if (inRow[j] > maxVal) maxVal = inRow[j];
// Step 2: compute exp(x - maxVal) and sum
float sumExp = 0.0f;
for (size_t j = 0; j < N; j++) {
outRow[j] = expf(inRow[j] - maxVal);
sumExp += outRow[j];
}
// Step 3: normalise by sum
for (size_t j = 0; j < N; j++)
outRow[j] /= sumExp;
}
}
```

1	`float *inRow = (float*)(input->data + i*N*4);`	CAST: input->data is uint8_t*. Adding i*N*4 moves past i complete rows (each row is N floats = N*4 bytes). Casting to float* lets us access elements as inRow[j] = the j-th float in row i. This avoids calling readBytesAsFloat in the inner loop — direct pointer access is faster.
2	`float maxVal = inRow[0];`	Seed the maximum with the first element. Then scan the row to find the true maximum.
3	`outRow[j] = expf(inRow[j] - maxVal);`	expf is the float version of exp(). Subtracting maxVal makes the largest exponent 0 (exp(0)=1), and all others negative (exp(negative) < 1). No overflow is possible.
4	`sumExp += outRow[j];`	Accumulate the sum of all exponentials. This is the denominator of softmax.
5	`outRow[j] /= sumExp;`	Divide each element by the sum. Now all elements sum to 1. This is the normalisation step that makes the output a probability distribution.

Section 7.5 — Dropout.h/.c: Regularisation (Complete)

■ C CONCEPT: Dropout — Why it Prevents Overfitting

During training, dropout randomly sets a fraction p of activations to zero. Remaining activations are scaled by 1/(1-p) to preserve expected magnitude. Why it helps: The network cannot rely on any single neuron. Each training step uses a randomly different subset of the network. This forces every neuron to learn independently useful features rather than co-adapting to others. Inference (no dropout): All neurons active. The scaling factor 1/(1-p) applied during training compensates for the expected fraction that was dropped — output magnitude matches.

```
// Dropout.c — dropoutForwardFloat
void dropoutForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
dropoutConfig_t *cfg = layer->config.dropout;
// In inference mode: pass through unchanged
if (!cfg->isTraining) {
copyTensor(output, input);
return;
}
size_t n = calcNumberOfElementsByTensor(input);
float scale = 1.0f / (1.0f - cfg->dropoutRate);
for (size_t i = 0; i < n; i++) {
// Generate random float in [0, 1)
float r = rngNextFloat(&cfg->rng);
float val = readBytesAsFloat(input->data + i*4);
if (r < cfg->dropoutRate) {
// Dropped: set to zero
writeFloatToByteArray(0.0f, output->data + i*4);
// Mark as dropped in mask (for backward pass)
cfg->mask[i] = 0;
} else {
// Kept: scale up
writeFloatToByteArray(val * scale, output->data + i*4);
cfg->mask[i] = 1;
}
}
}
```

1	`if (!cfg->isTraining) { copyTensor(output, input); return; }`	During inference, dropout is disabled. The entire layer is a no-op: just copy input to output and return immediately. The ! operator negates the boolean: !true = false, !false = true.
2	`float scale = 1.0f / (1.0f - cfg->dropoutRate);`	Inverted dropout scaling. If dropoutRate=0.5, scale=2.0. Kept neurons are scaled UP by 2× during training. This ensures the expected output magnitude is the same as at inference time (where all neurons are active but not scaled).
3	`float r = rngNextFloat(&cfg->rng);`	Generate a random float uniformly in [0, 1). Each neuron independently gets a fresh random number.
4	`if (r < cfg->dropoutRate)`	A neuron is dropped if its random number falls below the dropout rate. With dropoutRate=0.5: each neuron has exactly 50% chance of being dropped. For 100 neurons: on average 50 will be dropped (but exact count varies randomly).
5	`cfg->mask[i] = 0; // or 1`	Store which neurons were dropped. The backward pass needs this: if a neuron was dropped (mask=0), its gradient is also 0 (the neuron didn't contribute). If it was kept (mask=1), the gradient is scaled by the same scale factor.

Section 7.6 — Conv1d.h/.c: 1D Convolution (Complete)

■ C CONCEPT: 1D Convolution — The Math

Input: X [batch, in_channels, length] Kernel: W [out_channels, in_channels, kernel_size] Output: Y [batch, out_channels, out_length] out_length = (length - kernel_size) / stride + 1 For each batch sample b, output channel oc, output position p: Y[b][oc][p] = sum_{ic=0}^{in_ch-1} sum_{k=0}^{ksize-1} X[b][ic][p*stride + k] × W[oc][ic][k] + bias[oc] Intuition: slide a small filter W[oc] over the input. At each position p, compute the dot product of the filter with the local window. Each filter detects a different local pattern.

```
// Conv1d.c — conv1dForwardFloat (simplified for clarity)
void conv1dForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
conv1dConfig_t *cfg = layer->config.conv1d;
cfg->forwardInput = input;
size_t B = getDimensionsByIndex(input, 0); // batch
size_t IC = getDimensionsByIndex(input, 1); // in_channels
size_t L = getDimensionsByIndex(input, 2); // length
size_t OC = cfg->outChannels; // out_channels
size_t KS = cfg->kernelSize; // kernel size
size_t STR = cfg->stride;
size_t OL = (L - KS) / STR + 1; // output length
tensor_t *W = getParamFromParameter(&cfg->weights); // [OC, IC, KS]
tensor_t *b = getParamFromParameter(&cfg->bias); // [OC]
for (size_t bat = 0; bat < B; bat++)
for (size_t oc = 0; oc < OC; oc++)
for (size_t p = 0; p < OL; p++) {
float sum = 0.0f;
for (size_t ic = 0; ic < IC; ic++)
for (size_t k = 0; k < KS; k++) {
// Input element at time position (p*STR + k), channel ic
size_t xIdx = bat*(IC*L) + ic*L + (p*STR + k);
float xVal = readBytesAsFloat(input->data + xIdx*4);
// Weight for output channel oc, input channel ic, position k
size_t wIdx = oc*(IC*KS) + ic*KS + k;
float wVal = readBytesAsFloat(W->data + wIdx*4);
sum += xVal * wVal;
}
float biasVal = readBytesAsFloat(b->data + oc*4);
size_t yIdx = bat*(OC*OL) + oc*OL + p;
writeFloatToByteArray(sum + biasVal, output->data + yIdx*4);
}
}
```

1	`size_t OL = (L - KS) / STR + 1;`	Output length formula: with a kernel of size KS and stride STR, we can fit the kernel at positions 0, STR, 2*STR, ..., L-KS. The number of positions is (L - KS)/STR + 1. Example: L=10, KS=3, STR=1 → OL=(10-3)/1+1=8.
2	`for (bat...) for (oc...) for (p...)`	Three outer loops: batch sample, output channel, and output position. Chained without braces (C allows this for single-statement bodies). The single statement is the { ... } block below.
3	`size_t xIdx = bat*(IC*L) + ic*L + (p*STR + k);`	Flat memory index for X[bat][ic][p*STR+k]. X is stored as [batch, channels, time] in row-major order. batch offset: bat*(IC*L). channel offset: ic*L. time offset: p*STR+k.
4	`size_t wIdx = oc*(IC*KS) + ic*KS + k;`	Flat memory index for W[oc][ic][k]. W is stored as [out_ch, in_ch, kernel_size]. out_ch offset: oc*(IC*KS). in_ch offset: ic*KS. kernel offset: k.
5	`size_t yIdx = bat*(OC*OL) + oc*OL + p;`	Flat memory index for Y[bat][oc][p]. Y stored as [batch, out_ch, out_len]. batch offset: bat*(OC*OL). out_ch offset: oc*OL. position: p.

Section 7.7 — Flatten.h/.c: Shape Transformation

```
// Flatten.c — flattenForwardFloat
// Converts [batch, channels, length] → [batch, channels*length]
// No data movement needed — just reshape the tensor
void flattenForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
size_t B = getDimensionsByIndex(input, 0); // batch
size_t N = calcNumberOfElementsByTensor(input) / B; // all other dims
// Copy data (flatten doesn't change values, only shape)
copyTensor(output, input);
// Reshape output to [B, N]
output->shape->dimensions[0] = B;
output->shape->dimensions[1] = N;
output->shape->numberOfDimensions = 2;
// Reset order of dimensions (not transposed)
output->shape->orderOfDimensions[0] = 0;
output->shape->orderOfDimensions[1] = 1;
}
```

1	`size_t N = calcNumberOfElementsByTensor(input) / B;`	Total elements divided by batch size = elements per sample. For [4, 16, 20]: total=4×16×20=1280, N=1280/4=320. Output shape: [4, 320].
2	`copyTensor(output, input);`	Copy all bytes from input to output. Flatten does NOT change the actual data — only the shape interpretation. The bytes are identical; only the shape metadata changes.
3	`output->shape->dimensions[0] = B; ...[1] = N;`	Overwrite the shape dimensions. Now output 'sees' the same bytes as a 2D matrix [B, N] instead of a 3D tensor [B, C, L]. The bytes are the same; the interpretation changed.
4	`output->shape->numberOfDimensions = 2;`	Update the dimension count. The shape now has only 2 dimensions instead of 3.

Section 7.8 — Relations With Other Files

File	Direction	Why
All Layer .c files	→ depend on	Layer.h, Tensor.h, Common.h
Linear.c	→ uses	Matmul.h (forward), Add.h (grad accumulation)
Relu.c	→ uses	Comparison.h (gteFloatValue), Tensor.h (calcNumberOfElements)
Softmax.c	→ uses	math.h (expf), Tensor.h, DTypes.h
Conv1d.c	→ uses	DTypes.h (readBytesAsFloat), Tensor.h
Dropout.c	→ uses	RNG.h (rngNextFloat), Tensor.h
Flatten.c	→ uses	Tensor.h (copyTensor, calcNumberOfElements)
Training loop	← uses all layers	Calls layerForward / layerBackward in sequence
Optimizer (Sgd.c)	← used by layerUpdate	Updates weights after each batch

Chapter 8

Layer 6 — Training Infrastructure

Loss, SGD, DataLoader, Training Loop — every line explained

Section 8.1 — LossFunction.h/.c: How the Network Measures Error

■ WHAT THIS FILE DOES

The loss function is the single number that tells the network how wrong it is. Training aims to minimise this number. LossFunction.h provides two variants: MSE (Mean Squared Error) for regression, and Cross-Entropy for classification. Speaker ID is a classification task, so Cross-Entropy is the standard choice.

■ C CONCEPT: Cross-Entropy Loss — The Math

For a single sample with true label class c and predicted probabilities p[0..N-1]: L = -log(p[c]) This measures how 'surprised' the model is by the correct answer. If p[c] = 1.0 (perfect confidence): L = -log(1) = 0. No surprise, no loss. If p[c] = 0.01 (very wrong): L = -log(0.01) ≈ 4.6. Very surprised, large loss. The gradient w.r.t. the softmax output (before log) simplifies beautifully: dL/d(logit[j]) = p[j] - 1_{j==c} (p[j] minus 1 if j is the correct class) For a batch of M samples: average these gradients over M.

```
// LossFunction.c — crossEntropyLossFloat
// predictions: softmax output [M, num_classes]
// labels: one-hot labels [M, num_classes] (or class indices)
float crossEntropyLossFloat(tensor_t *predictions, tensor_t *labels,
tensor_t *lossGrad) {
size_t M = getDimensionsByIndex(predictions, 0); // batch size
size_t N = getDimensionsByIndex(predictions, 1); // num classes
float totalLoss = 0.0f;
for (size_t i = 0; i < M; i++) {
float *pred = (float*)(predictions->data + i*N*4);
float *lab = (float*)(labels->data + i*N*4);
float *grad = (float*)(lossGrad->data + i*N*4);
for (size_t j = 0; j < N; j++) {
float eps = 1e-7f; // prevent log(0)
// Cross-entropy: -label * log(pred)
totalLoss -= lab[j] * logf(pred[j] + eps);
// Gradient: pred[j] - lab[j]
grad[j] = pred[j] - lab[j];
}
// Normalise gradient by batch size
for (size_t j = 0; j < N; j++) grad[j] /= (float)M;
}
return totalLoss / (float)M; // average loss over batch
}
```

1	`float *pred = (float*)(predictions->data + i*N*4);`	Cast and offset: point to row i of the predictions tensor. predictions->data is uint8_t*. Adding i*N*4 skips i complete rows (each N floats = N*4 bytes). Casting to float* enables direct array access: pred[j] = predictions[i][j].
2	`float eps = 1e-7f;`	A small constant (epsilon). logf(0) = -infinity, which would corrupt training. Adding eps ensures the argument to logf is always at least 1e-7, making logf(0 + eps) = logf(1e-7) ≈ -16.1 (large but finite).

3	`totalLoss -= lab[j] * logf(pred[j] + eps);`	Cross-entropy: L = -sum(label × log(prediction)). For one-hot labels: lab[j]=1 for the true class, 0 for all others. So only the log(pred[true_class]) term contributes. The others are zero.
4	`grad[j] = pred[j] - lab[j];`	The gradient of cross-entropy + softmax w.r.t. the logits. This elegant formula comes from the combined derivative of softmax and log. For true class j: grad[j] = p[j] - 1. For other classes: grad[j] = p[j].
5	`grad[j] /= (float)M;`	Average the gradient over the batch. Without this division, larger batches would produce larger gradients, making the effective learning rate batch-size-dependent. Dividing by M makes the gradient magnitude batch-size-independent.
6	`return totalLoss / (float)M;`	Return the average loss per sample. Useful for monitoring training progress.

Section 8.2 — Sgd.h/.c: Stochastic Gradient Descent (Complete)

■ C CONCEPT: SGD — The Basic Update Rule

Stochastic Gradient Descent is the simplest optimizer: w = w - lr * dL/dw where lr is the learning rate (e.g., 0.01), w is a weight, and dL/dw is the gradient. With weight decay (L2 regularisation): w = w - lr * (dL/dw + lambda * w) = w * (1 - lr*lambda) - lr * dL/dw Weight decay penalises large weights, preventing them from growing without bound. It acts as a force pulling all weights toward zero. With momentum (improves convergence speed): v = beta * v_prev + dL/dw (velocity accumulates gradient history) w = w - lr * v

Field	Type	Meaning
learningRate	float	Step size for weight updates. Typical values: 0.001 to 0.1.
weightDecay	float	L2 regularisation coefficient lambda. Typical: 0.0001 to 0.001.
momentum	float	Velocity retention factor beta. Typical: 0.9. Only in sgdStepM.

8.2.1 sgdZeroGrad — Reset Gradients Before Each Batch

```
// Sgd.c — sgdZeroGrad
void sgdZeroGrad(parameter_t *parameter) {
tensor_t *grad = getGradFromParameter(parameter);
size_t bytes = calcBytesPerTensor(grad);
// Zero all gradient bytes
memset(grad->data, 0, bytes);
// CRITICAL for SYM_INT32: restore scale to 1.0f after zeroing
if (grad->quantization->type == SYM_INT32) {
symInt32QConfig_t *cfg =
(symInt32QConfig_t*)grad->quantization->qConfig;
cfg->scale = 1.0f;
}
}
```

1	`size_t bytes = calcBytesPerTensor(grad);`	calcBytesPerTensor = calcBytesPerElement(grad->quantization) × calcNumberOfElements(grad). This gives the total number of bytes to zero.
2	`memset(grad->data, 0, bytes);`	Set all gradient bytes to 0. memset(ptr, value, count) fills 'count' bytes starting at ptr. The value 0 sets every byte to 0x00. For int32_t and float, all-zero bytes = value 0.
3	`if (grad->quantization->type == SYM_INT32) { cfg->scale = 1.0f; }`	CRITICAL BUG PREVENTION: memset zeros the entire memory, including the scale field inside the quantization config struct. scale=0.0f would make all subsequent 'real_value = mantissa × scale' computations yield 0.0, silently breaking training. After zeroing, scale must be explicitly restored to 1.0f (the identity: mantissa × 1.0 = mantissa).

■■ IMPORTANT

The scale restoration after sgdZeroGrad is a critical subtlety. If you call memset on a tensor_t's data buffer WITHOUT restoring scale, and the quantization config is embedded (not at a separate pointer), the scale becomes 0.0f. All future computations yield 0. This bug is hard to find because the network appears to train (gradients are computed, sgdStep runs) but weights never change. Always call sgdZeroGrad, never raw memset on a tensor.

8.2.2 sgdStepFloat — Complete Line-by-Line

```
// Sgd.c — sgdStepFloat
static void sgdStepFloat(sgd_t *sgd, parameter_t *parameter) {
tensor_t *w = getParamFromParameter(parameter);
tensor_t *g = getGradFromParameter(parameter);
size_t n = calcNumberOfElementsByTensor(w);
float lr = sgd->learningRate;
float wd = sgd->weightDecay;
for (size_t i = 0; i < n; i++) {
float wi = readBytesAsFloat(w->data + i*4);
float gi = readBytesAsFloat(g->data + i*4);
// Weight decay: pull weight toward zero
float grad_with_wd = gi + wd * wi;
// Gradient descent step
wi -= lr * grad_with_wd;
writeFloatToByteArray(wi, w->data + i*4);
}
}
```

1	`static void sgdStepFloat(...)`	'static' makes this function private to Sgd.c. The public API is sgdStep(), which dispatches to this function based on quantization type.
2	`float lr = sgd->learningRate; float wd = sgd->weightDecay;`	Cache the values in local variables. Avoids repeated pointer dereferences in the hot loop. On ARM Cortex-M7, loading from struct pointer every iteration is slightly slower than reading a local variable held in a register.
3	`float wi = readBytesAsFloat(w->data + i*4);`	Read weight i. Pointer arithmetic: w->data + i*4 is the byte address of element i. *4 because each float is 4 bytes.
4	`float grad_with_wd = gi + wd * wi;`	Add weight decay regularisation to the gradient. The effective gradient is: true_gradient + lambda × weight. This is equivalent to minimising (loss + lambda/2 × ||w||^2) — penalising large weights.
5	`wi -= lr * grad_with_wd;`	The actual SGD step: w = w - learning_rate × gradient. -= is shorthand for wi = wi - lr * grad_with_wd.
6	`writeFloatToByteArray(wi, w->data + i*4);`	Write the updated weight back to its byte position in the tensor.

8.2.3 sgdStepSymInt32 — FQT Weight Update (Complete Line-by-Line)

The SYM_INT32 SGD step must work with quantized mantissas. The key challenge: gradients come in as raw integer deltas (accumulated from backward passes), but the update rule requires floating-point arithmetic. The solution: convert to real values, update, then requantize.

```
// Sgd.c — sgdStepSymInt32
static void sgdStepSymInt32(sgd_t *sgd, parameter_t *parameter) {
tensor_t *w = getParamFromParameter(parameter);
tensor_t *g = getGradFromParameter(parameter);
size_t n = calcNumberOfElementsByTensor(w);
float lr = sgd->learningRate;
float wd = sgd->weightDecay;
symInt32QConfig_t *wCfg =
(symInt32QConfig_t*)w->quantization->qConfig;
symInt32QConfig_t *gCfg =
(symInt32QConfig_t*)g->quantization->qConfig;
float wScale = wCfg->scale;
float gScale = gCfg->scale;
float qMax = (float)((1 << (wCfg->qMaxBits-1)) - 1);
for (size_t i = 0; i < n; i++) {
int32_t wMantissa = readBytesAsInt32(w->data + i*4);
int32_t gMantissa = readBytesAsInt32(g->data + i*4);
// Convert mantissas to real floating-point values
float wReal = (float)wMantissa * wScale;
float gReal = (float)gMantissa * gScale;
// Weight decay + gradient descent in real space
float gTotal = gReal + wd * wReal;
float wNew = wReal - lr * gTotal;
// Requantize new weight back to SYM_INT32
float newMantissaF = wNew / wScale;
int32_t newMantissa = (int32_t)clamp(
roundByMode(newMantissaF, wCfg->roundingMode),
-qMax, qMax);
writeInt32ToByteArray(newMantissa, w->data + i*4);
}
}
```

1	`float wScale = wCfg->scale; float gScale = gCfg->scale;`	Cache both scales. wScale converts weight mantissas to real values. gScale converts gradient mantissas to real gradient values.
2	`float qMax = (float)((1 << (wCfg->qMaxBits-1)) - 1);`	Compute the clamp range. 1 << (qMaxBits-1) = 2^(qMaxBits-1). For qMaxBits=16: 1<<15 = 32768, minus 1 = 32767. Weights will be clamped to [-32767, +32767] after the update.
3	`float wReal = (float)wMantissa * wScale;`	Convert the stored integer mantissa to the real-valued weight. Example: mantissa=1500, scale=0.001 → wReal=1.5. All arithmetic happens in real (floating-point) space.
4	`float gReal = (float)gMantissa * gScale;`	Convert the gradient mantissa to the real-valued gradient. The gradient tensor's scale may differ from the weight's scale.
5	`float gTotal = gReal + wd * wReal;`	Add weight decay. The combined gradient includes both the computed gradient and the regularisation term (wd × weight).
6	`float wNew = wReal - lr * gTotal;`	The SGD update in real space. wNew is now a real-valued updated weight.
7	`float newMantissaF = wNew / wScale;`	Convert the updated real weight back to a mantissa: mantissa = real_value / scale. This is the inverse of the decode step.
8	`roundByMode(newMantissaF, wCfg->roundingMode)`	Round the float mantissa to the nearest integer, using the configured rounding mode (possibly stochastic rounding for small updates).
9	`clamp(..., -qMax, qMax)`	Ensure the new mantissa stays within the representable range. If the update pushes the weight out of range, clamp to the boundary.

8.2.4 sgdStepMFloat — SGD with Momentum

```
// Sgd.c — sgdStepMFloat (simplified)
// 'state' tensor holds the velocity v[i] for each weight
static void sgdStepMFloat(sgd_t *sgd, parameter_t *parameter, tensor_t *state) {
tensor_t *w = getParamFromParameter(parameter);
tensor_t *g = getGradFromParameter(parameter);
size_t n = calcNumberOfElementsByTensor(w);
float lr = sgd->learningRate;
float wd = sgd->weightDecay;
float beta = sgd->momentum; // e.g. 0.9
for (size_t i = 0; i < n; i++) {
float wi = readBytesAsFloat(w->data + i*4);
float gi = readBytesAsFloat(g->data + i*4);
float vi = readBytesAsFloat(state->data + i*4); // velocity
// Update velocity: v = beta*v + gradient
float vNew = beta * vi + (gi + wd * wi);
// Update weight: w = w - lr * v
float wNew = wi - lr * vNew;
writeFloatToByteArray(vNew, state->data + i*4); // save velocity
writeFloatToByteArray(wNew, w->data + i*4);
}
}
```

1	`float vi = readBytesAsFloat(state->data + i*4);`	Read the velocity for weight i from the separate state tensor. Momentum requires storing one extra float per weight — doubling memory. For large models on MCU, this may not be feasible. Basic SGD avoids this cost.
2	`float vNew = beta * vi + (gi + wd * wi);`	Update velocity: v = beta × v_old + gradient. beta=0.9 means 90% of the old velocity persists, 10% is 'new gradient'. This creates an exponential moving average of gradients: recent gradients matter more, but past gradients still influence.
3	`float wNew = wi - lr * vNew;`	Update weight using velocity instead of raw gradient. The velocity has accumulated direction over past steps, so it smooths out noisy gradients and accelerates convergence.
4	`writeFloatToByteArray(vNew, state->data + i*4);`	MUST save the new velocity for the next step. Without this, momentum has no memory — each step would be independent.

■ RigL GAP IN THIS FILE

RigL requires a mask-aware SGD step. The modification is simple: In sgdStepFloat, before updating weight i: if (cfg->mask != NULL && !tensorBoolGet(cfg->mask, i)) continue; Pruned weights (mask=0) must NOT be updated: - Their weight is pinned at 0 (they don't participate in forward) - Their gradient is still computed (needed for grow step) - But applying the gradient update would make them non-zero, defeating sparsity Active weights (mask=1) are updated normally. After the RigL grow step reactivates a pruned weight, it becomes mask=1 and immediately starts being updated by SGD.

Section 8.3 — DataLoader.h/.c: The Data Pipeline

■ WHAT THIS FILE DOES

DataLoader provides the mini-batch assembly system. It wraps a dataStorage_t dataset with shuffling, batching, and a callback. The training loop asks the DataLoader for the next batch; the DataLoader assembles N samples into a batch tensor and calls the getSample callback to convert raw bytes into model inputs.

Field	Type	Meaning
dataset	dataStorage_t*	Pointer to the raw dataset (all samples in flat buffer).
batchSize	size_t	How many samples per batch. Typical: 8, 16, 32.
shuffleIndices	size_t*	Index permutation array. Shuffled before each epoch.
currentIdx	size_t	Which sample in shuffleIndices to serve next.
getSample	getSampleFunc_t	Function pointer: converts raw bytes → input tensor.

```
// DataLoader.h — function pointer type for getSample callback
// Called for each sample in a batch. Reads from rawData, writes to inputTensor.
typedef void (*getSampleFunc_t)(uint8_t *rawData, size_t numElements,
tensor_t *inputTensor);
// DataLoader.c — dataLoaderGetNextBatch
void dataLoaderGetNextBatch(dataLoader_t *loader,
tensor_t *batchInput,
tensor_t *batchLabels) {
for (size_t s = 0; s < loader->batchSize; s++) {
// Shuffle wraps around: restart epoch when all samples served
if (loader->currentIdx >= loader->dataset->numberOfEntries) {
loader->currentIdx = 0;
rngShuffleIndices(loader->shuffleIndices,
loader->dataset->numberOfEntries,
&loader->rng);
}
// Get the actual sample index from the shuffle permutation
size_t sampleIdx = loader->shuffleIndices[loader->currentIdx++];
// Get raw data for this sample
dataEntry_t *entry = &loader->dataset->entries[sampleIdx];
// Call user's getSample to convert raw bytes into network input
loader->getSample(entry->dataPTR, entry->numberOfElements,
getRowSlice(batchInput, s));
// Copy label for this sample
copyLabelToRow(batchLabels, sampleIdx, s);
}
}
```

1	`if (loader->currentIdx >= loader->dataset->numberOfEntries)`	Check if we've served all samples in this epoch. If so, reset and reshuffle. This implements epoch-based training: each epoch visits every sample exactly once.
2	`rngShuffleIndices(loader->shuffleIndices, N, &loader->rng);`	Fisher-Yates shuffle of the index array. After shuffling, shuffleIndices is a random permutation of [0..N-1]. Shuffling ensures the network sees samples in different orders each epoch, preventing it from memorising the training order.
3	`size_t sampleIdx = loader->shuffleIndices [loader->currentIdx++];`	Look up which sample to serve next. shuffleIndices[0] might be 47, [1] might be 3, [2] might be 91 — random order. currentIdx++ increments after reading (post-increment): serve current, advance to next.

4	`loader->getSample(entry->dataPTR, entry->numberOfElements, getRowSlice(batchInput, s));`	Call the user-provided getSample callback through a function pointer. This is where application-specific preprocessing happens: normalize audio, extract MFCC features, etc. getRowSlice returns a tensor_t pointing to row s of the batch tensor.

Section 8.4 — The Complete Training Loop (with RigL)

This section shows how every component from the previous chapters combines into a complete training iteration. We also show where the RigL update step would be inserted.

```
// Complete training loop (pseudo-code showing actual ODT API calls)
// === Setup (once) ===
sgd_t sgd = { .learningRate = 0.01f, .weightDecay = 0.0001f };
optimizer_t opt;
initSgdOptimizer(&opt, &sgd);
// RigL config (if using sparse training):
// riglConfig_t riglCfg = { .mask=&maskTensor, .sparsityRatio=0.8f,
// .updateStep=100, .batchCount=0 };
// === Training loop ===
for (int epoch = 0; epoch < numEpochs; epoch++) {
float epochLoss = 0.0f;
for (int batch = 0; batch < numBatches; batch++) {
// ■■ Step 1: Zero gradients ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sgdZeroGrad(&weightParam);
sgdZeroGrad(&biasParam);
// ■■ Step 2: Load next mini-batch ■■■■■■■■■■■■■■■■■■■■■■■■■■■
dataLoaderGetNextBatch(&loader, &batchInput, &batchLabels);
// ■■ Step 3: Forward pass ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
layerForward(&linearLayer, &batchInput, &linear_out);
layerForward(&reluLayer, &linear_out, &relu_out);
layerForward(&softmaxLayer,&relu_out, &softmax_out);
// ■■ Step 4: Compute loss ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
float loss = crossEntropyLossFloat(&softmax_out, &batchLabels, &lossGrad);
epochLoss += loss;
// ■■ Step 5: Backward pass ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Loss gradient dL/dY flows backward through each layer
layerBackward(&softmaxLayer, &lossGrad, &softmax_out, &softmax_grad);
layerBackward(&reluLayer, &softmax_grad,&relu_out, &relu_grad);
layerBackward(&linearLayer, &relu_grad, &batchInput, &input_grad);
// ■■ Step 6: Optimizer step ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
layerUpdate(&linearLayer, &opt, &forwardQ);
// Internally calls: sgdStepFloat(&sgd, &weightParam)
// sgdStepFloat(&sgd, &biasParam)
// ■■ Step 7: RigL update (every T batches) ■■■■■■■■■■■■■■■■■■■
// if (batch % RIGL_T == 0 && batch > 0) {
// riglUpdate(&weightParam, &riglCfg);
// }
}
printf("Epoch %d: avg_loss=%.4f\n", epoch, epochLoss/(float)numBatches);
}
```

1	`sgdZeroGrad(&weightParam);`	Reset gradients to zero BEFORE loading new data. Gradients accumulate during backward. Without zeroing, old gradients from previous batches contaminate new updates.
2	`dataLoaderGetNextBatch(&loader, &batchInput, &batchLabels);`	Load next N samples. Handles shuffling and epoch wrapping automatically.
3	`layerForward(&linearLayer, &batchInput, &linear_out);`	Dispatch to linearForwardFloat. Computes output = input × W^T + b. Stores pointer to batchInput in linearConfig->forwardInput for backward.
4	`float loss = crossEntropyLossFloat(..., &lossGrad);`	Compute loss AND fill lossGrad with the initial gradient. lossGrad[i][j] = (predicted[i][j] - label[i][j]) / batchSize.

5	`layerBackward(&softmaxLayer, &lossGrad, &softmax_out, &softmax_grad);`	Backward through softmax. For cross-entropy + softmax, the gradient is already in lossGrad (computed by crossEntropyLossFloat). softmax backward just passes it through.
6	`layerBackward(&reluLayer, &softmax_grad, &relu_out, &relu_grad);`	Backward through ReLU. For each element: relu_grad[i] = softmax_grad[i] if relu_input[i] > 0, else 0.
7	`layerBackward(&linearLayer, &relu_grad, &batchInput, &input_grad);`	Backward through linear. Computes weight gradients (dL/dW) and input gradients (dL/dX, stored in input_grad for previous layers).
8	`layerUpdate(&linearLayer, &opt, &forwardQ);`	Calls sgdStepFloat for weights and biases. Uses the gradients accumulated during backward to update weights: w = w - lr × (gradient + weight_decay × w).
9	`riglUpdate(&weightParam, &riglCfg);`	The RigL step (commented out). After every T batches, prune the smallest active weights and regrow the highest-gradient inactive ones. Must happen AFTER the optimizer step (so gradients are fresh).

Section 8.5 — Relations With Other Files

File	Direction	Why
LossFunction.c	→ depends on	Tensor.h, Log.h (logf wrapper), math.h
Sgd.c	→ depends on	Tensor.h, DTypes.h, Rounding.h (roundByMode, clamp), TensorConversion.h
DataLoader.c	→ depends on	DataStorage.h, Tensor.h, RNG.h (rngShuffleIndices)
Layer.c	← uses SGD	layerUpdate dispatches to sgdStep via optimizer_t
Linear.c, Conv1d.c	← store forwardInput	Backward pass needs forward input for gradient computation
RigL.c (future)	← calls sgdZeroGrad	riglUpdate must zero gradients after topology change
UserAPI.c	← wraps training loop	trainingEpochDefault encapsulates the loop shown above

Chapter 9

Layer 7 — Support Modules

RNG, Serialization, Bernoulli, and NPY Loader

Section 9.1 — RNG.h/.c: The Random Number Generator

■ WHAT THIS FILE DOES

RNG.h provides a fast, deterministic pseudo-random number generator (PRNG) based on the xorshift32 algorithm. It is used for: (1) dataset shuffling (rngShuffleIndices), (2) Dropout mask generation (via Bernoulli.c), (3) stochastic rounding in FQT (via Rounding.c). The generator is seeded by rngSetSeed and maintains a single global state. Setting the same seed gives the same sequence every time (reproducibility).

```
// RNG.h — the public interface
typedef struct { uint32_t state; } rng32_t;
// Seed and query
void rngSetSeed(uint32_t seed);
uint32_t rngGetSeed(void);
// Generate a uniform float in [0, 1)
float rngNextFloat(void);
// Fisher-Yates shuffle of an index array
void rngShuffleIndices(size_t *indices, size_t n);
```

9.1.1 xorshift32 — How Random Numbers Are Generated

```
// RNG.c — the internal state and xorshift32 step:
static rng32_t rng = { .state = 1 }; // module-level, persistent
static uint32_t rngNext(rng32_t *rng) {
uint32_t x = rng->state;
x ^= x << 13; // step 1: XOR with left-shifted self
x ^= x >> 17; // step 2: XOR with right-shifted result
x ^= x << 5; // step 3: XOR with left-shifted result
rng->state = x;
return x;
}
```

■ C CONCEPT: XOR-Shift PRNG — Why It Works

A PRNG (Pseudo-Random Number Generator) produces a deterministic sequence that LOOKS random but is fully determined by the initial seed. XOR (exclusive-or, ^): bit-by-bit, output 1 if exactly one input is 1. 0 ^ 0 = 0, 0 ^ 1 = 1, 1 ^ 0 = 1, 1 ^ 1 = 0 x ^= x << 13 means: x = x XOR (x shifted left by 13 bits) This mixes bits: each bit becomes a function of multiple original bits. Three shifts (13, 17, 5) are carefully chosen constants that make the generator cycle through all 2^32 - 1 non-zero values before repeating (maximal period). The generator is very fast: 3 XOR + 3 shift = 6 instructions, 1 cycle on ARM. This is significantly faster than rand() from the C standard library, which is important on a microcontroller where every cycle counts.

═■ WORKED EXAMPLE

Trace with seed=1 (state = 0x00000001 = binary 0000...0001): x = 0x00000001 x ^= x << 13: x = 0x00000001 XOR 0x00002000 = 0x00002001 x ^= x >> 17: x = 0x00002001 XOR 0x00000000 = 0x00002001 (17-bit shift of small number = 0) x ^= x << 5: x = 0x00002001 XOR 0x00040020 = 0x00042021 state = 0x00042021 = 270369 decimal Next call: state = 270369, another round of 3 XOR-shifts → new number. Every call produces a different 32-bit integer.

9.1.2 rngBounded — Fair Bounded Range

```
// Get a random uint32_t in [0, bound) with NO modulo bias:
static uint32_t rngBounded(uint32_t bound) {
// Compute largest multiple of bound that fits in uint32_t:
uint32_t limit = UINT32_MAX - (UINT32_MAX % bound);
uint32_t x;
// Rejection sampling: discard values above 'limit'
do {
x = rngNext(&rng);
} while (x >= limit);
return x % bound;
}
```

■ C CONCEPT: Rejection Sampling — Avoiding Modulo Bias

Why not just use 'rngNext() % bound'? Example: bound=3, range=[0,1,2]. UINT32_MAX+1 = 4,294,967,296. 4,294,967,296 / 3 = 1,431,655,765 remainder 1. So values 0 and 1 appear 1 more time than value 2 — NOT uniform! Rejection sampling: discard any generated value above 'limit' (the largest multiple of bound that fits). The remaining values are uniformly distributed when divided by bound. For bound=3: limit = 4,294,967,296 - (4,294,967,296 % 3) = 4,294,967,295. We reject nothing — by luck limit = UINT32_MAX+1 for bound=3. For bound=2: limit = UINT32_MAX+1 - 0 = 4,294,967,296. Even here, values 0..MAX/2-1 map to 0, others to 1 — perfectly balanced.

9.1.3 rngShuffleIndices — Fisher-Yates Shuffle

```
// rngShuffleIndices: Fisher-Yates shuffle of indices [0..n-1]
void rngShuffleIndices(size_t *indices, size_t n) {
// Initialise to identity permutation if needed:
// (caller typically calls setOrderOfDimsForNewTensor to pre-fill)
for (size_t i = n - 1; i >= 1; i--) {
// Pick a random j in [0, i] (inclusive):
size_t j = (size_t)rngBounded((uint32_t)(i + 1));
// Swap indices[i] and indices[j]:
size_t tmp = indices[i];
indices[i] = indices[j];
indices[j] = tmp;
}
}
```

■ C CONCEPT: Fisher-Yates Shuffle — Why This Algorithm?

The Fisher-Yates shuffle produces an UNBIASED permutation: all N! orderings are equally likely. Algorithm (working from end to beginning): for i from N-1 down to 1: pick random j in [0, i] swap elements i and j Example: [A, B, C, D] (N=4): i=3: j=random[0..3]=1 → swap [3] and [1] → [A, D, C, B] i=2: j=random[0..2]=2 → swap [2] and [2] → [A, D, C, B] (no-op) i=1: j=random[0..1]=0 → swap [1] and [0] → [D, A, C, B] Result: [D, A, C, B] — one of 24 equally likely orderings. Why not a naive shuffle (pick random pair and swap N times)? Naive is biased: some permutations are more likely than others.

9.1.4 rngNextFloat — Uniform Float in [0, 1)

```
float rngNextFloat(void) {
// Take top 24 bits of the 32-bit random number:
uint32_t raw = rngNext(&rng) >> 8; // 24-bit value in [0, 2^24 - 1]
return (float)raw / (float)(1 << 24); // divide by 2^24 → [0, 1)
}
```

■ C CONCEPT: 24-Bit Float Resolution

A float32 mantissa has 23 bits of precision (24 bits including the implicit 1). Using more bits would be wasted — they can't be represented exactly in float32. raw >> 8: discard the bottom 8 bits, keep the top 24. Divide by 2^24 = 16,777,216 gives a value in [0, 1). Example: raw = 0x3A7B12C4 = 979763908 >> 8 → 0x003A7B12 = 3833618 / 16777216 = 0.22848... (a float in [0, 1))

9.1.5 rngSetSeed — Reproducibility

```
void rngSetSeed(uint32_t seed) {
// xorshift32 cannot have state=0 (0 XOR anything = 0 forever)
rng.state = (seed == 0) ? UINT32_MAX : seed;
}
```

■ NOTE

Always set the seed before any training run for reproducibility. With the same seed, the same shuffle order, the same dropout masks, and the same stochastic rounding decisions are made. Different seeds → different results even with identical training data and code. Publishing the seed used in your experiments lets others reproduce your results exactly.

Section 9.2 — Serialize.h/.c and Deserialize.h/.c

■ WHAT THIS FILE DOES

Serialization saves a trained model (tensor weights) to a byte stream. Deserialization reads it back. On the STM32, you train the model, then serialize weights to UART (PC receives them). Later, deserialize from flash/UART to restore the model for inference on the MCU. The format is raw bytes — each tensor is written as its metadata followed by its data.

```
// Serialize.h — save model to a FILE stream:
void serializeTensor(tensor_t *tensor, FILE *f);
void serializeParameter(parameter_t *parameter, FILE *f);
void serializeModel(layer_t **model, size_t sizeModel, FILE *f);
// Deserialize.h — restore model from a FILE stream:
void deserializeTensor(tensor_t *tensor, FILE *f);
void deserializeParameter(parameter_t *parameter, FILE *f);
void deserializeModel(layer_t **model, size_t sizeModel, FILE *f);
```

■ C CONCEPT: Serialization — Writing Structs to a Byte Stream

serializeTensor writes in order: 1. tensor->quantization->type (4 bytes: qtype_t) 2. qConfig fields (scale, qMaxBits, etc.) 3. shape: numberOfDimensions, dimensions[], orderOfDimensions[] 4. data: the raw weight bytes deserializeTensor reads in the same order and fills the struct. This only works if both writer and reader use the SAME struct layout — true on one architecture (ARM Cortex-M7 → same ARM PC) but potentially broken across platforms with different endianness or struct packing. The library uses fwrite/fread for simplicity. On the MCU, FILE* maps to UART or flash.

■ WORKED EXAMPLE

Serializing a SYM_INT32 tensor with shape [3,4] and 12 elements: Bytes written: [0-3]: qtype = SYM_INT32 = 2 (uint32_t, 4 bytes) [4-7]: scale = 0.001f (float, 4 bytes = 0x6F126E3A in IEEE754) [8]: qMaxBits = 16 (uint8_t, 1 byte) [9]: roundingMode = 0 (uint8_t, 1 byte) [10-13]: numberOfDimensions = 2 (size_t, 4 bytes) [14-17]: dimensions[0] = 3 (4 bytes) [18-21]: dimensions[1] = 4 (4 bytes) [22-25]: orderOfDimensions[0] = 0 (4 bytes) [26-29]: orderOfDimensions[1] = 1 (4 bytes) [30-77]: 12 × 4 = 48 bytes of int32_t mantissas Total: 78 bytes for this tensor's weights.

Section 9.3 — Bernoulli.h/.c: Sampling for Dropout

■ WHAT THIS FILE DOES

Bernoulli.c provides Bernoulli sampling: for a given probability p, generate a BOOL (0 or 1) where P(1) = p. Used by Dropout to generate the random drop mask: each mask element is 0 (drop, prob p) or 1 (keep, prob 1-p). Relies on RNG.c's rngNextFloat.

```
// Bernoulli.h:
void bernoulliSampleTensor(tensor_t *mask, float p);
// Fills all elements of 'mask' (BOOL tensor):
// mask[i] = 1 with probability p
// mask[i] = 0 with probability 1-p
// Implementation (Bernoulli.c):
void bernoulliSampleTensor(tensor_t *mask, float p) {
size_t n = calcNumberOfElementsByTensor(mask);
for (size_t i = 0; i < n; i++) {
bool val = (rngNextFloat() < p); // true with prob p
tensorBoolSet(mask, i, val);
}
}
```

═■ WORKED EXAMPLE

Generating a dropout mask with p=0.5 for 8 neurons: rngNextFloat() calls: [0.73, 0.12, 0.61, 0.48, 0.92, 0.33, 0.55, 0.09] Compare to p=0.5: [F, T, F, T, F, T, F, T] (T = < 0.5 = keep) Wait — Dropout DROPS with prob p=0.5, KEEPS with prob 0.5. So Dropout calls bernoulliSampleTensor(mask, 1-p=0.5) for the KEEP mask: mask = [0, 1, 0, 1, 0, 1, 0, 1] Activations multiplied by mask: [0, a1, 0, a3, 0, a5, 0, a7] The kept activations are also scaled by 1/(1-p) = 2 (inverted dropout).

Section 9.4 — NPYLoader.h/.c: Loading NumPy Data Files

■ WHAT THIS FILE DOES

NPYLoader reads NumPy .npy files. NumPy is the standard Python array library. Training data prepared on a PC (audio features, spectrograms) is saved as .npy files and loaded onto the MCU via UART or stored in flash. NPYLoader parses the .npy header (array shape and dtype) and fills a tensor_t with the array data.

■ C CONCEPT: The .npy File Format

A .npy file has: Bytes 0-5: magic string 0x93NUMPY Byte 6: major version (1 or 2) Byte 7: minor version (0) Bytes 8-9: header length (uint16_t, little-endian) Bytes 10..(10+header_len-1): Python dict as ASCII text, e.g.: {'descr': '<f4', 'fortran_order': False, 'shape': (1000, 128), } After header: raw array data (C contiguous, little-endian) '<f4' means little-endian float32. '<i4' means little-endian int32. NPYLoader parses this header, extracts the shape and dtype, then reads the raw bytes directly into the tensor's data buffer.

Section 9.5 — Relations With Other Files

File	Direction	Why
RNG.h/.c	→ standalone	No project dependencies — uses only stdint, stddef
Bernoulli.h/.c	→ uses RNG + Tensor	rngNextFloat for sampling, tensorBoolSet to write mask
Dropout.c	← uses Bernoulli	Calls bernoulliSampleTensor to generate the drop mask
Rounding.c	← uses RNG	rngNextFloat for stochastic rounding noise
DataLoader.c	← uses RNG	rngShuffleIndices for epoch-wise shuffle

Serialize.h/.c	← uses Tensor + Layer	Iterates model layers and serializes each tensor
Deserialize.h/.c	← uses Tensor + Layer	Reads bytes and fills tensor_t fields
NPYLoader.h/.c	← uses Tensor	Fills tensor_t.data and tensor_t.shape from .npy bytes
Training loop	← uses all support modules	DataLoader + RNG + Serialize complete the training pipeline

■ RigL GAP IN THIS FILE

RNG.c already provides everything needed for RigL's stochastic elements. RigL itself is deterministic (prune smallest, grow largest gradient) — no extra randomness needed. The Bernoulli sampler is NOT needed for basic RigL (RigL doesn't randomly prune). However, a variant called 'Random pruning + gradient regrow' (simpler than RigL) would use Bernoulli to select which weights to prune randomly.

Chapter 10

Gap Analysis — From ODT to RigL

Every Missing Piece, How It Was Solved, and the Actual C API

Section 10.1 — What RigL Needs (The Complete Picture)

■ C CONCEPT: RigL in One Sentence

RigL (Evci et al., NeurIPS 2020) trains a neural network where a fixed fraction of weights are always zero — but WHICH weights are zero changes periodically: Every T batches: 1. PRUNE: deactivate the S% of active weights with smallest |weight| 2. GROW: reactivate the S% of inactive weights with largest |gradient| Between updates, only active weights participate in forward/backward passes. The gradient of inactive weights is still computed (for the grow step). Inactive weights stay at zero and are never updated by the optimizer.

RigL Component	Needed From ODT	Status Before This Work
Sparsity mask (BOOL tensor)	BOOL quantization + tensorBoolGet/Set	EXISTS in Tensor.c
Masked forward pass	matmul that skips masked weights	MISSING
Full gradient for inactive weights	linearCalcWeightGrads all positions	EXISTS
Mask-aware SGD step	sgdStep skips masked positions	MISSING
riglInitMask()	Initialize mask to target sparsity	MISSING
riglPruneStep()	Sort active by |w|, deactivate bottom S%	MISSING
riglGrowStep()	Sort inactive by |grad|, activate top S%	MISSING
riglUpdate()	Orchestrate prune+grow every T batches	MISSING

Section 10.2 — The Mask: riglConfig_t and BOOL Tensors

```
// riglConfig_t — configuration struct for one sparse layer (RigL.h):
typedef struct {
tensor_t *weights; // the weight tensor being made sparse
tensor_t *grads; // corresponding gradient tensor
tensor_t *mask; // BOOL tensor: mask[i]=1 active, mask[i]=0 pruned
float sparsityRatio; // 0.8 = 80% of weights pruned at all times
size_t updateStep; // batches between prune+grow updates
size_t batchCount; // internal counter
} riglConfig_t;
// Mask allocation for a 128x64 linear layer:
// 128 * 64 = 8192 weights -> 8192 bits -> 1024 bytes
static uint8_t maskBuf[1024];
static tensor_t maskTensor;
// set up maskTensor with BOOL quantization, shape [128,64]
riglConfig_t cfg = {
.weights = &weightTensor,
.grads = &gradTensor,
.mask = &maskTensor,
.sparsityRatio = 0.8f,
.updateStep = 100,
.batchCount = 0,
};
```

1	`tensor_t *mask;`	The mask is a separate tensor_t with BOOL quantization. It stores 1 bit per weight, packed into bytes. tensorBoolGet/Set access individual bits.
2	`float sparsityRatio;`	Fraction of weights that are PRUNED at any time. 0.8 = 80% zero, 20% active. This stays constant throughout training. Only the PATTERN of which weights are pruned changes.
3	`size_t updateStep;`	T in the RigL paper — number of training batches between topology updates. Typical value: 100. Too small = unstable; too large = topology cannot adapt.
4	`size_t batchCount;`	Counter incremented each batch. When batchCount % updateStep == 0, riglUpdate() fires the prune+grow step.

Section 10.3 — Gap #1 Solved: riglInitMask()

The first step is initializing the mask before training begins: exactly (1-sparsityRatio)*N weights start active, randomly selected.

```
void riglInitMask(riglConfig_t *cfg, unsigned int seed) {
tensor_t *mask = cfg->mask;
size_t N = calcNumberOfElementsByTensor(mask);
size_t maskBytes = (N + 7) / 8;
memset(mask->data, 0xFF, maskBytes); // all bits = 1 (all active)
if (N % 8 != 0) { // fix spare bits in last byte
size_t validBits = N % 8;
uint8_t lastMask = (uint8_t)((1u << validBits) - 1);
mask->data[maskBytes - 1] &= lastMask;
}
size_t numToPrune = (size_t)(cfg->sparsityRatio * (float)N);
srand(seed);
size_t pruned = 0;
while (pruned < numToPrune) {
size_t idx = (size_t)((unsigned)rand() % (unsigned)N);
if (tensorBoolGet(mask, idx)) {
tensorBoolSet(mask, idx, false);
pruned++;
}
}
}
```

3	`size_t N = calcNumberOfElementsByTensor(mask);`	Total weight count. For [128,64]: N=8192.
4	`size_t maskBytes = (N + 7) / 8;`	Ceiling division: bytes needed for N bits. 8192 bits = 1024 bytes.
6	`memset(mask->data, 0xFF, maskBytes);`	0xFF = 11111111 binary. Sets every bit to 1 = every weight active.
8-11	`if (N % 8 != 0) { ... &= lastMask; }`	If N is not a multiple of 8, the last byte has extra 1-bits from memset. (1u << validBits)-1 creates a bitmask for only the valid bits. Example N=10: last byte has 2 valid bits, mask=0b00000011.
13	`size_t numToPrune = (size_t)(cfg->sparsityRatio * (float)N);`	sparsityRatio=0.8, N=8192 -> prune 6553 weights, keep 1639 active.
16-22	`while (pruned < numToPrune) { ... rejection sampling ... }`	Pick a random index; if it is still active, prune it. Retry if already pruned. Guarantees exactly numToPrune unique positions pruned.

Section 10.4 — Gap #2 Solved: Masked Forward Pass

During training, pruned weights must not contribute to the output. riglMaskedMatmulFloat32 wraps the standard matmul but skips positions where mask=0. This is where the MCU speedup comes from.

```
void riglMaskedMatmulFloat32(tensor_t *A, tensor_t *B,
tensor_t *out, tensor_t *bias,
tensor_t *mask) {
size_t M = getDimensionsByIndex(A, 0); // batch size
size_t K = getDimensionsByIndex(A, 1); // input features
size_t N = getDimensionsByIndex(B, 0); // output neurons
for (size_t i = 0; i < M; i++) {
for (size_t j = 0; j < N; j++) {
float sum = 0.0f;
for (size_t k = 0; k < K; k++) {
if (mask != NULL) {
size_t flatIdx = j * K + k;
if (!tensorBoolGet(mask, flatIdx)) continue;
}
float aVal = readBytesAsFloat(A->data + (i*K+k)*4);
float bVal = readBytesAsFloat(B->data + (j*K+k)*4);
sum += aVal * bVal;
}
float biasVal = readBytesAsFloat(bias->data + j*4);
writeFloatToByteArray(sum+biasVal, out->data + (i*N+j)*4);
}
}
}
```

3-5	`M, K, N dimensions`	M=batch rows, K=shared dimension (input features), N=output neurons. For input [1,128] and weight [64,128] (transposed): M=1, K=128, N=64.
9	`float sum = 0.0f;`	Accumulator reset to zero before computing each output element out[i][j].
11-13	`if (!tensorBoolGet(mask, flatIdx)) continue;`	flatIdx = j*K+k: the linear index of weight [j][k] in the mask. continue skips this k entirely — no load, no multiply. With 80% sparsity: 80% of inner-loop iterations are just a bit-check and skip.
14-16	`readBytesAsFloat + sum += aVal * bVal`	For active weights only: load input element and weight element, multiply, accumulate.
18-19	`bias + writeFloatToByteArray`	Add the per-output-neuron bias and write result to output tensor.

WORKED EXAMPLE

Speedup example — 128-input, 64-output linear, sparsityRatio=0.8: Dense matmul: 1 x 64 x 128 = 8,192 multiplications Sparse matmul: 1 x 64 x 128 x 0.2 = 1,638 multiplications Saved: 6,554 multiplications (80% fewer) ARM Cortex-M7: 1 FMA = 1 cycle. Real speedup ~3-4x (not 5x) due to mask check overhead.

Section 10.5 — Gap #3 Solved: riglPruneStep()

The prune step finds the weakest active weights (smallest |w|) and deactivates them. Static buffers avoid stack overflow. qsort() provides the sorting.

```
#define RIGL_MAX_WEIGHTS 16384
static size_t s_activeIdx[RIGL_MAX_WEIGHTS]; // in .bss, not stack
static float s_wFloat[RIGL_MAX_WEIGHTS];
static int cmpAbsWeightAsc(const void *a, const void *b) {
size_t ia = *(const size_t *)a; size_t ib = *(const size_t *)b;
float fa = fabsf(s_wFloat[ia]); float fb = fabsf(s_wFloat[ib]);
return (fa > fb) - (fa < fb); // -1, 0, or +1
}
void riglPruneStep(riglConfig_t *cfg) {
size_t N = calcNumberOfElementsByTensor(cfg->weights);
size_t numActive = 0;
for (size_t i = 0; i < N && numActive < RIGL_MAX_WEIGHTS; i++) {
if (tensorBoolGet(cfg->mask, i)) {
s_activeIdx[numActive] = i;
s_wFloat[numActive] = readBytesAsFloat(cfg->weights->data + i*4);
numActive++;
}
}
qsort(s_activeIdx, numActive, sizeof(size_t), cmpAbsWeightAsc);
size_t numToPrune = (size_t)(cfg->sparsityRatio * (float)numActive);
for (size_t i = 0; i < numToPrune; i++) {
size_t idx = s_activeIdx[i];
tensorBoolSet(cfg->mask, idx, false);
writeFloatToByteArray(0.0f, cfg->weights->data + idx*4);
}
}
```

1-2	`static size_t s_activeIdx[]; static float s_wFloat[];`	Static = allocated in .bss (zeroed RAM section), persists between calls. NOT on the stack. A VLA of 8192 size_t = 32 KB would overflow a ~4 KB stack.
5-8	`cmpAbsWeightAsc callback`	qsort calls this with pointers to two array elements (indices). Cast void* to size_t*, dereference to get index, look up weight in s_wFloat. Returns negative if |w[a]| < |w[b]| (a comes first = smaller weights first).
13-18	`Collect active indices and weight values`	Linear scan of all N positions. For each active weight (mask=1), record its index and float value for sorting.
20	`qsort(s_activeIdx, numActive, sizeof(size_t), cmpAbsWeightAsc);`	Standard C library sort. After this, s_activeIdx[0] is the index of the SMALLEST active weight.
21-26	`Deactivate bottom fraction`	The first numToPrune entries are the weakest. Set mask=0 (deactivate) and set weight to 0.0f. These weights will not be updated by the optimizer.

Section 10.6 — Gap #4 Solved: riglGrowStep()

```
static size_t s_inactiveIdx[RIGL_MAX_WEIGHTS];
static float s_gFloat[RIGL_MAX_WEIGHTS];
static int cmpAbsGradDesc(const void *a, const void *b) {
size_t ia = *(const size_t *)a; size_t ib = *(const size_t *)b;
float fa = fabsf(s_gFloat[ia]); float fb = fabsf(s_gFloat[ib]);
return (fb > fa) - (fb < fa); // NOTE: b before a = DESCENDING
}
void riglGrowStep(riglConfig_t *cfg) {
size_t N = calcNumberOfElementsByTensor(cfg->mask);
size_t numActive = 0;
for (size_t i = 0; i < N; i++)
if (tensorBoolGet(cfg->mask, i)) numActive++;
size_t numInactive = 0;
for (size_t i = 0; i < N && numInactive < RIGL_MAX_WEIGHTS; i++) {
if (!tensorBoolGet(cfg->mask, i)) {
s_inactiveIdx[numInactive] = i;
s_gFloat[numInactive] = readBytesAsFloat(cfg->grads->data + i*4);
numInactive++;
}
}
qsort(s_inactiveIdx, numInactive, sizeof(size_t), cmpAbsGradDesc);
size_t targetActive = (size_t)((1.0f - cfg->sparsityRatio) * (float)N);
size_t numToGrow = (targetActive > numActive) ? (targetActive - numActive) : 0;
for (size_t i = 0; i < numToGrow && i < numInactive; i++) {
tensorBoolSet(cfg->mask, s_inactiveIdx[i], true);
// weight stays 0 — grows from gradient updates next T batches
}
}
```

7	`return (fb > fa) - (fb < fa);`	Descending sort: b comes before a when fb > fa. After sort, s_inactiveIdx[0] is the inactive weight with the LARGEST |gradient|.
11-13	`Count numActive`	Count how many weights are currently active. We need this to calculate how many to grow.
15-21	`Collect inactive indices + gradient magnitudes`	For every pruned weight (mask=0), record its index and gradient value.
22	`qsort(..., cmpAbsGradDesc)`	Sort so position [0] = inactive weight with highest gradient magnitude. This weight would reduce the loss the most if re-enabled.
24-25	`targetActive and numToGrow`	targetActive = N * (1-sparsityRatio) = the desired number of active weights. numToGrow = targetActive - current numActive. After prune removed some, grow restores exact sparsity ratio.
26-28	`tensorBoolSet(mask, idx, true)`	Re-enable the weight. Its value is 0.0f. Starting next batch, the optimizer will update it. Over the next T batches it grows from 0 based on gradient information.

Section 10.7 — Gap #5 Solved: riglUpdate() + riglMaskedSgdStepFloat()

```
// riglUpdate() — call once per batch, fires every updateStep batches:
void riglUpdate(riglConfig_t *cfg) {
cfg->batchCount++;
if (cfg->batchCount % cfg->updateStep != 0) return;
riglPruneStep(cfg);
riglGrowStep(cfg);
size_t bytes = calcNumberOfElementsByTensor(cfg->grads) * 4;
memset(cfg->grads->data, 0, bytes); // zero stale gradients
}
// riglMaskedSgdStepFloat() — mask-aware optimizer step:
void riglMaskedSgdStepFloat(riglConfig_t *cfg, float lr) {
tensor_t *w = cfg->weights;
tensor_t *g = cfg->grads;
tensor_t *mask = cfg->mask;
size_t N = calcNumberOfElementsByTensor(w);
for (size_t i = 0; i < N; i++) {
if (!tensorBoolGet(mask, i)) continue; // skip pruned
float wi = readBytesAsFloat(w->data + i*4);
float gi = readBytesAsFloat(g->data + i*4);
writeFloatToByteArray(wi - lr * gi, w->data + i*4);
}
}
```

3	`cfg->batchCount++;`	Incremented every batch. riglUpdate is called unconditionally each batch but returns early if not yet time for a topology update.
4	`if (batchCount % updateStep != 0) return;`	The modulo check: only proceed every updateStep batches. For updateStep=100: triggers at batch 100, 200, 300, ...
5-6	`riglPruneStep(); riglGrowStep();`	Prune first, then grow. Order matters: if we grew first, newly activated weights (value=0) would immediately be the smallest and get pruned in the same step.
7-8	`memset(grads, 0, bytes)`	Zero all gradients after topology change. The gradient computed under the OLD topology is not valid for the NEW topology. Stale gradients would corrupt the NEXT grow step's selection.
17	`if (!tensorBoolGet(mask, i)) continue;`	The single most important line for RigL correctness: pruned weights (mask=0) are never updated by the optimizer. They stay at 0.0f. Only the grow step can reactivate them.

Section 10.8 — Complete RigL Training Loop

```
// One-time setup:
riglConfig_t riglCfg = { &linearWeights, &linearWeightGrads,
&linearMask, 0.8f, 100, 0 };
riglInitMask(&riglCfg, 42); // seed=42, reproducible initial mask
// Training loop:
for (size_t epoch = 0; epoch < NUM_EPOCHS; epoch++) {
for (size_t batch = 0; batch < NUM_BATCHES; batch++) {
// 1. Zero gradients
memset(linearWeightGrads.data, 0, gradBytes);
// 2. Forward pass (sparse matmul skips pruned weights)
riglMaskedMatmulFloat32(&inputBatch, &linearWeightsT,
&outputBatch, &linearBias, &linearMask);
reluForwardFloat(&outputBatch);
// 3. Loss
float loss = crossEntropyLossFloat(&outputBatch, &labelBatch, &lossGrad);
// 4. Backward (gradients computed for ALL weights including pruned!)
reluBackwardFloat(&lossGrad, &outputBatch);
linearCalcWeightGradsFloat(&lossGrad, &inputBatch, &linearWeightGrads);
// 5. Optimizer (only updates active weights)
riglMaskedSgdStepFloat(&riglCfg, LEARNING_RATE);
// 6. RigL update (fires every 100 batches automatically)
riglUpdate(&riglCfg);
}
}
```

■ RigL GAP IN THIS FILE

Key insight: linearCalcWeightGradsFloat computes gradients for ALL N positions, even pruned ones. This is not a bug — it is intentional. The grow step needs the gradient at pruned positions to decide which to reactivate. We deliberately compute more gradients than the optimizer uses, because those extra gradients drive the topology evolution.

■ NOTE

Chapter 21 contains the complete, compilable RigL.h and RigL.c with all functions fully implemented. This chapter explained WHY each piece was needed. Chapter 21 shows the complete HOW.

Chapter 11

How Everything Works Together on the MCU

STM32 Nucleo-F746ZG — Memory, Execution, and the Complete Data Flow

Section 11.1 — The Target Hardware

■ WHAT THIS FILE DOES

The STM32 Nucleo-F746ZG is an ARM Cortex-M7 microcontroller development board. It runs at 216 MHz, has 320 KB of SRAM and 1 MB of flash. There is no operating system (bare-metal), no virtual memory, no malloc (or very limited). All memory must be statically declared. The ODT library is designed specifically for these constraints.

Resource	Amount	What It Means for ODT
SRAM	320 KB	All tensors, activations, gradients, stack — everything live in RAM. No swap space.
Flash	1 MB	Code + read-only data (e.g. dataset in flash). Trained weights saved here.
CPU	ARM Cortex-M7 @ 216 MHz	Integer ops: ~216M/s. FP (with FPU): ~216M single-precision ops/s.
FPU	Double-precision optional	Single-precision floats run in 1 cycle on the Cortex-M7 FPU.
UART	Up to 4.5 Mbps	Used for: loading training data, sending trained weights to PC.
No OS	Bare-metal	No threads, no malloc, no printf (unless routed to UART). All functions are called directly.

Section 11.2 — Memory Layout on the STM32

On a bare-metal embedded system, you manage memory manually. Everything is a statically declared array or struct. Here is how a typical ODT model's memory is laid out in SRAM:

```
// Typical SRAM layout for a 2-layer MLP speaker-ID model on STM32:
// Total SRAM budget: 320 KB
// ■■ LAYER 1 WEIGHTS (Linear 128→64) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
static uint8_t w1Data[128*64*4]; // 32 KB — SYM_INT32 weights
static uint8_t g1Data[128*64*4]; // 32 KB — gradient tensor
static uint8_t b1Data[64*4]; // 256 B — bias
static uint8_t bg1Data[64*4]; // 256 B — bias gradient
// ■■ LAYER 2 WEIGHTS (Linear 64→N_SPEAKERS) ■■■■■■■■■■■■■■■■■■■■■■■■
static uint8_t w2Data[64*N_SPK*4]; // 16 KB (N_SPK=64) — weights
static uint8_t g2Data[64*N_SPK*4]; // 16 KB — gradient
static uint8_t b2Data[N_SPK*4]; // 256 B — bias
static uint8_t bg2Data[N_SPK*4]; // 256 B — bias gradient
// ■■ ACTIVATIONS (max 1 batch at a time) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
static uint8_t inputBuf[BATCH*128*4]; // e.g. 1*128*4 = 512 B for batch=1
static uint8_t act1Buf[BATCH*64*4]; // 256 B — after linear1+relu
static uint8_t act2Buf[BATCH*N_SPK*4];// 256 B — after linear2
static uint8_t outBuf[BATCH*N_SPK*4]; // 256 B — after softmax
// ■■ GRADIENT SIGNALS (backpropagation) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
static uint8_t grad0Buf[BATCH*N_SPK*4]; // initial gradient from loss
static uint8_t grad1Buf[BATCH*64*4]; // gradient after linear2 backward
static uint8_t grad2Buf[BATCH*128*4]; // gradient after relu backward
// ■■ QUANTIZATION CONFIGS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
static symInt32QConfig_t w1QCfg, g1QCfg; // scale, qMaxBits, roundingMode
// ■■ TOTAL: ~100 KB for this model ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Remaining ~220 KB for: stack, dataset buffer, sort temps, code globals
```

■ C CONCEPT: Why static? Why not malloc?

malloc() allocates heap memory at runtime. On bare-metal MCUs: 1. The heap may be very small (or absent) — the linker sets its size. 2. Heap fragmentation: repeated malloc/free can leave no usable large block. 3. malloc can fail — and checking every return value is tedious. 4. Static allocation is DETERMINISTIC: you know exactly how much RAM is used at compile time, printed in the .map file. 'static' keyword: declared at file scope (or with static keyword in a function). These variables live in the .bss (zero-initialized) or .data section — placed in SRAM by the linker before main() runs. They exist for the entire lifetime of the program.

Section 11.3 — Startup Sequence on the MCU

When the STM32 powers on or resets, the following sequence occurs before the training loop begins:

```
// MCU startup sequence (conceptual):
// HARDWARE INIT (generated by CubeMX or written manually):
SystemClock_Config(); // set CPU to 216 MHz
MX_USART3_UART_Init(); // configure UART for data transfer
// ... other peripheral inits ...
// ODT LIBRARY INIT:
// 1. Seed the random number generator (use a fixed seed for reproducibility):
rngSetSeed(42);
// 2. Initialize quantization configs for all tensors:
initSymInt32QConfig(HALF_AWAY, &w1QCfg); // scale=1.0, qMaxBits=16
initSymInt32QConfig(SR_HALF_AWAY, &g1QCfg); // stochastic rounding for grads
initFloat32Quantization(&outQ); // softmax output = float
// 3. Initialize quantization_t wrappers:
initSymInt32Quantization(&w1Q, &w1QCfg);
initFloat32Quantization(&outQObj);
// 4. Initialize shapes:
size_t w1Dims[2] = {128, 64};
size_t w1Order[2]; setOrderOfDimsForNewTensor(2, w1Order);
setShape(&w1Shape, w1Dims, 2, w1Order);
// 5. Link data buffers into tensors:
setTensorValues(&w1Tensor, w1Data, &w1Shape, &w1Q, NULL);
setTensorValues(&g1Tensor, g1Data, &w1Shape, &g1Q, NULL);
setParameterValues(&w1Param, &w1Tensor, &g1Tensor);
// 6. Initialize weights (e.g. random from PC, or He init):
// (NPYLoader loads pre-computed initial weights via UART)
loadWeightsFromUART(&w1Tensor);
// 7. Initialize layer configs:
linearInitConfig(&linear1Cfg, &w1Param, &b1Param, &symInt32Q, &gradQ, &gradQ, &lossQ);
initLayer(&linear1Layer, LINEAR, &linear1LayerConfig);
// 8. Initialize optimizer:
sgdInit(&sgd, 0.001f, 0.9f, 0.0001f); // lr=0.001, momentum=0.9, wd=0.0001
// 9. Initialize DataLoader:
initDataLoader(&loader, myGetSample, myGetDatasetSize, defaultGetBatch, ...);
```

Section 11.4 — The Complete Forward Pass (Step by Step)

During each training iteration, the forward pass computes the network's prediction given the current weights. Here is how data flows through every layer and every function call for a 2-layer FQT MLP:

```
// INPUT: batchInput tensor [1, 128] — a single 128-feature speech frame
// (in SYM_INT32: mantissas in [-32767, 32767], scale ≈ feature normalization factor)
// === FORWARD PASS ===
// LAYER 1: Linear (128 → 64)
layerFunctions[LINEAR].forward(&linear1Layer, &batchInput, &act1);
// Inside linearForward:
// 1. transposeTensor(w1, 0, 1) [128,64] → [64,128] (O(1))
// 2. matmulSymInt32TensorsWithBias(batchInput, w1, act1, b1)
// for each output neuron j in [0,64):
// acc = 0 (int64)
// for each input k in [0,128):
// acc += input[k] × w1[j][k] (int32×int32 → int64)
// realVal = acc × scaleInput × scaleW1 + bias[j] × scaleBias
// act1[j] = round(clamp(realVal / scaleAct1, qMin, qMax))
// (scale act1 is chosen to cover the full output range)
// 3. transposeTensor(w1, 0, 1) [64,128] → [128,64] (O(1) restore)
// LAYER 2: ReLU
layerFunctions[RELU].forward(&reluLayer, &act1, &act2);
// Inside reluForwardSymInt32:
// for each i: act2[i] = (act1[i] > 0) ? act1[i] : 0
// Copy act1's scale to act2 (ReLU doesn't change the scale)
// LAYER 3: Linear (64 → N_SPEAKERS)
layerFunctions[LINEAR].forward(&linear2Layer, &act2, &act3);
// Same matmul as layer 1 but [64 → N_SPEAKERS]
// LAYER 4: Softmax (N_SPEAKERS → probabilities)
layerFunctions[SOFTMAX].forward(&softmaxLayer, &act3, &output);
// Note: softmax requires FLOAT32 — if act3 is SYM_INT32, it gets converted
// Inside softmaxForward (FLOAT32):
// 1. Find max logit for numerical stability
// 2. exp(z - max) for each class
// 3. Divide by sum: output = probability distribution
// OUTPUT: output tensor [1, N_SPEAKERS] — FLOAT32 probabilities summing to 1.0
// e.g. output = [0.02, 0.01, 0.91, 0.03, 0.03] (speaker 2 most likely)
```

Section 11.5 — The Complete Backward Pass (Step by Step)

```
// === BACKWARD PASS (reverse order!) ===
// LOSS GRADIENT: dL/dlogit = softmax_output - one_hot_target
lossFunctions[CROSS_ENTROPY].backward(&output, &target, &grad0);
// grad0 = [0.02, 0.01, -0.09, 0.03, 0.03] (correct class gets -0.09)
// LAYER 4 BACKWARD: Softmax (combined with CE already done above)
// No separate softmax backward needed (CE + softmax combined gradient = above)
// LAYER 3 BACKWARD: Linear2 backward
layerFunctions[LINEAR].backward(&linear2Layer, &act2, &grad0, &grad1);
// Inside linearBackward:
// linearCalcWeightGradsFloat32(grad0, act2, w2.grad):
// dL/dW2 = grad0^T @ act2 [added to w2.grad]
// linearCalcBiasGradsFloat32(b2.grad, grad0):
// dL/db2 = sum(grad0, axis=0) [added to b2.grad]
// linearCalcPropLossFloat32(w2, grad0, grad1):
// grad1 = grad0 @ W2 [propagate loss to layer below]
// LAYER 2 BACKWARD: ReLU backward
layerFunctions[RELU].backward(&reluLayer, &act1, &grad1, &grad2);
// Inside reluBackwardFloat:
// for i: grad2[i] = (act1[i] > 0) ? grad1[i] : 0
// Gates the gradient: block where input was <= 0
// LAYER 1 BACKWARD: Linear1 backward
layerFunctions[LINEAR].backward(&linear1Layer, &batchInput, &grad2, NULL);
// grad propagated to input layer (NULL = no output needed, first layer)
// linearCalcWeightGradsFloat32(grad2, batchInput, w1.grad): dL/dW1
// linearCalcBiasGradsFloat32(b1.grad, grad2): dL/db1
// OPTIMIZER STEP: update all weights
sgdStep(&sgd, &w1Param, &forwardQ); // W1 -= lr * w1.grad
sgdStep(&sgd, &b1Param, &forwardQ); // b1 -= lr * b1.grad
sgdStep(&sgd, &w2Param, &forwardQ); // W2 -= lr * w2.grad
sgdStep(&sgd, &b2Param, &forwardQ); // b2 -= lr * b2.grad
// After all batches in epoch: requantize weights if needed
// (requantSymInt32Tensor updates scales to current value range)
```

Section 11.6 — Memory Flow Diagram

This diagram shows where data lives in SRAM at each stage of one training iteration. Each arrow represents a function call that reads from source buffers and writes to destination buffers.

```
SRAM CONTENTS during one forward + backward pass:
[w1Data] [g1Data] [b1Data] [w2Data] [g2Data] [b2Data] <- WEIGHTS (persistent)
| <- forward reads w1
v
[inputBuf] → [linearForward] → [act1Buf] [act1Buf stores: 64 SYM_INT32]
|
[reluForward] <- gteSymInt32Zero
|
[act2Buf] [act2Buf stores: 64 SYM_INT32]
|
[linear2Forward] <- reads w2
|
[act3Buf] [act3Buf: N_SPEAKERS floats]
|
[softmaxForward]
|
[outBuf] [N_SPEAKERS probabilities]
|
[CE.backward] <- computes loss
|
[grad0] [= output - target]
|
[linear2.backward] -> [g2Data] [b2 grad updated]
|
[grad1] [64-dim gradient]
|
[relu.backward] <- gate by act1
|
[grad2] [64-dim gated gradient]
|
[linear1.backward] -> [g1Data] [b1 grad updated]
|
[sgdStep × 4] -> [w1Data, b1Data, w2Data, b2Data]
Note: grad0, grad1, grad2 buffers can OVERLAP if they are different sizes
and backward is computed sequentially (only one gradient needed at a time).
This is a key memory optimization for embedded systems.
```

Section 11.7 — Performance Considerations on Cortex-M7

■ C CONCEPT: ARM Cortex-M7 Performance Characteristics

The Cortex-M7 has: - Double-issue pipeline: can execute 2 instructions per cycle in ideal cases - Harvard architecture: separate instruction and data caches - 16 KB L1 instruction cache + 16 KB L1 data cache - FPU (Floating-Point Unit): single-precision multiply in 1 cycle - TCM (Tightly Coupled Memory): zero-wait-state memory access Impact on ODT: - float multiply (matmulFloat32): 1 cycle per element (FPU MAC) - int32 multiply: also 1 cycle (MUL instruction) - memcpy, memset: uses DMA or optimised LDMIA instructions - The inner loop of matmul (K multiplies per output): runs in the cache as long as the weight row fits in 16 KB data cache For a [128, 64] matmul: inner row is 128×4 = 512 bytes (fits in 16 KB cache easily). The main bottleneck is memory bandwidth for large tensors that don't fit in cache.

Operation	Cost (approx)	Bottleneck
matmulFloat32 [1,128]×[128,64]	128×64=8192 MACs = ~8192 cycles	FPU throughput
matmulSymInt32 [1,128]×[128,64]	8192 int64 MACs = ~8192 cycles	MUL/MULL throughput
requantSymInt32 (2 passes, 128×64)	2×8192 reads + 8192 writes ≈ 25K cycles	Memory bandwidth
reluForwardSymInt32 (64 elements)	64 comparisons ≈ 64 cycles	Trivial

sgdStep SYM_INT32 (128×64 weights)	2 convertTensor + VLA alloc + loop ≈ 100K cycles	Float conversion
softmaxForward (float, 64 classes)	64 exp() + 64 div ≈ 3000 cycles	exp() latency
Full forward pass (2-layer MLP)	~20K cycles ≈ 0.1 ms at 216 MHz	matmul dominant
Full backward pass	~3× forward ≈ 60K cycles ≈ 0.3 ms	3 matmuls (w1grad,w2grad,prop)
Full training step (batch=1)	~80K cycles ≈ 0.4 ms	Limited by matmul + SGD

═■ WORKED EXAMPLE

Training speed estimate for 1000-sample dataset, 100 epochs: 1000 samples × 100 epochs × 0.4 ms/step = 40,000 ms = 40 seconds With RigL (80% sparse, 5× fewer matmul ops in forward): Forward: 0.02 ms instead of 0.1 ms Backward: still need full gradient → ~0.3 ms unchanged Total: ~0.32 ms/step Training time: 32 seconds (20% faster) The bigger win from RigL is INFERENCE speedup: after training, the sparse model runs 5× faster (no backward needed during inference).

Section 11.8 — The Layer Stack: How All 8 Layers Interact

Every function call in the training loop flows through all 8 architectural layers. Here is the complete chain of dependencies for one linearForward call:

```
// CALL: layerFunctions[LINEAR].forward(&layer, &input, &output)
// Layer 5 (Layers): dispatch
// → linearForward (Linear.c) [Layer 5]
// Layer 5 → Layer 4 (Arithmetic):
// → transposeTensor (Tensor.c) [Layer 3]
// orderOfDimensions swap — no lower layers called
// → matmulSymInt32TensorsWithBias (Matmul.c) [Layer 4]
// Layer 4 → Layer 3 (Tensor):
// → getDimensionsByIndex (Arithmetic.c) → tensor->shape
// → calcNumberOfElementsByTensor (Tensor.c)
// Layer 4 → Layer 1 (DTypes) for each element:
// → readBytesAsInt32 (DTypes.c) [Layer 1] — read A[i][k] and B[j][k]
// → writeInt32ToByteArray (DTypes.c) [Layer 1] — write output[i][j]
// Layer 4 → Layer 4 (Rounding) for quantised output:
// → roundByMode (Rounding.c) → clamp (Rounding.c)
// → roundByMode may call: rngNextFloat (RNG.c) [Layer 7]
// rngNextFloat → rngNext (private) — pure arithmetic
// Layer 4 → Layer 2 (Quantization) to read scales:
// → input->quantization->qConfig cast to symInt32QConfig_t*
// → read scale field (plain struct access, no function call)
// Layer 0 (Common) is active throughout:
// → PRINT_ERROR (Common.h) if any error occurs — wraps printf to UART
// → PRINT_DEBUG (Common.h) if DLEVEL >= DEBUG_LEVEL
// Total call depth: up to 4 levels
// Total functions touched: ~10 across 6 of the 8 layers
// Data touched: w1Data, g1Data buffers (weight) + act1Buf (output)
```

Section 11.9 — Debugging on the MCU

Debugging embedded code is harder than debugging on a PC. The ODT library provides several tools through Common.h:

```
// Common.h debugging tools:
// 1. DLEVEL: set debug verbosity at COMPILE TIME
// DLEVEL=0: silent (production) DLEVEL=3: very verbose
#define DLEVEL 2 // set in build system or CMakeLists.txt
// 2. PRINT_DEBUG: only prints if DLEVEL >= threshold
PRINT_DEBUG(2, "Layer 1 forward: scale=%.6f", scale);
// → uart output: "[TENSOR] Layer 1 forward: scale=0.000123"
// 3. PRINT_ERROR: always prints, then you can set a breakpoint
PRINT_ERROR("matmul: dimension mismatch %lu vs %lu", M, N);
// 4. printTensor: dump all tensor values to UART
printTensor(&act1Tensor);
// → prints all 64 mantissas and the current scale
// 5. printShape: dump shape information
printShape(tensor->shape);
// Common debugging workflow:
// Step 1: set DLEVEL=3, rebuild, flash to MCU
// Step 2: open UART terminal (115200 baud)
// Step 3: watch debug output as training runs
// Step 4: if a PRINT_ERROR fires, check which file/line
// Step 5: use SWD debugger (J-Link/OpenOCD) to set breakpoints
```

■ BEGINNER TIP

PRINT_ERROR uses do { ... } while (false) internally (see Chapter 2 on Common.h). This means you can safely put PRINT_ERROR inside an if/else without curly braces. The SOURCE_FILE macro is set at the top of each .c file so that error messages tell you exactly which module printed the error — very helpful when you see '[ARITHMETIC] Dimensions don't match' — you know to look in Arithmetic.c.

Section 11.10 — The End-to-End Speaker-ID Training Pipeline

Putting it all together: here is the complete pipeline from raw audio to a trained speaker-ID model on the STM32, using all components of the ODT library:

```
=== PHASE 1: DATA PREPARATION (on PC, Python) ===
1. Collect audio clips for N speakers.
2. Extract MFCC or filterbank features → numpy array [num_clips, 128].
3. Save as .npy file: np.save('features.npy', X)
4. Create labels: np.save('labels.npy', Y) (one-hot, shape [num_clips, N])
5. Transfer to STM32 via UART or embed in flash.
=== PHASE 2: INITIALIZATION (in main() on MCU) ===
6. SystemClock_Config() — set 216 MHz
7. MX_USART3_UART_Init() — set up UART
8. rngSetSeed(42) — reproducible training
9. Declare all static buffers, shapes, quantization configs
10. initLayer() for each layer
11. sgdInit() for optimizer
12. initDataLoader() with getSampleFromFlash callback
13. Load initial weights (random or pre-computed) via NPYLoader
=== PHASE 3: TRAINING (loop on MCU) ===
14. for epoch in [0..99]:
15. rngShuffleIndices(indices, datasetSize)
16. for batch in [0..datasetSize/batchSize):
17. sgdZeroGrad(all parameters)
18. load batch from DataLoader
19. linearForward(1) → reluForward → linearForward(2) → softmaxForward
20. crossEntropyForward → print loss
21. crossEntropySoftmaxBackward → linearBackward(2) → reluBackward
→ linearBackward(1)
22. scale gradients by 1/batchSize
23. sgdStep all parameters
24. print epoch loss to UART
=== PHASE 4: MODEL EXPORT (after training) ===
25. serializeModel(layers, numLayers, uartFile)
→ streams model weights to PC via UART
=== PHASE 5: INFERENCE (optional, back on MCU) ===
26. deserializeModel() from flash (pre-stored trained weights)
27. for each new audio clip:
28. extract features → fill inputTensor
29. forward pass (no backward needed)
30. predictedSpeaker = argmax(outputTensor)
```

Section 11.11 — Common Pitfalls and How to Avoid Them

■■ IMPORTANT

PITFALL 1: Forgetting sgdZeroGrad before each batch. Gradients accumulate across batches. The loss goes NaN. ALWAYS call sgdZeroGrad at the start of each batch.

■■ IMPORTANT

PITFALL 2: Mismatched quantization types. Calling linearForwardFloat with a SYM_INT32 weight tensor will silently reinterpret the int32 mantissas as float bit patterns — garbage output. Always check that forwardQ->type matches the actual tensor type.

■■ IMPORTANT

PITFALL 3: Stack overflow from large VLAs. sgdStepSymInt32 declares 'float wFloat[n]' on the stack. For n=8192 elements: 32 KB on the stack. The default ARM stack is often only 4-8 KB. Solution: increase stack size in the linker script, or use static buffers for large tensors.

■■ IMPORTANT

PITFALL 4: Not resetting scale after zeroing a SYM_INT32 tensor. After memset(grad, 0), the old scale is still set. The next gradient addition uses mantissa units relative to the old scale. sgdZeroGrad correctly resets the scale to 1.0 — always use sgdZeroGrad, never raw memset on a SYM_INT32 gradient tensor.

■■ IMPORTANT

PITFALL 5: transposeTensor not called in pairs. linearForwardFloat transposes w, calls matmul, then transposes back. If an error occurs between the two transposes (e.g. dimension mismatch in matmul), the weight tensor stays transposed. The next forward pass uses the wrong layout. Use PRINT_ERROR + return (don't exit) if you need cleanup — or use KeepTogether for transpose-matmul-transpose as an atomic unit.

Chapter 12

Deep Dive — Arithmetic.c and Matmul.c

Every Function Explained Line by Line

Section 12.1 — getDimensionsByIndex: Handling Transposed Tensors

getDimensionsByIndex is subtle. It does NOT just return dimensions[index]. It searches the orderOfDimensions array to find which physical slot holds the requested logical dimension. This is needed because after transposeTensor, the mapping between logical and physical dimensions is scrambled.

```
size_t getDimensionsByIndex(tensor_t *tensor, size_t index) {
size_t numberOfDims = tensor->shape->numberOfDimensions;
for (size_t i = 0; i < numberOfDims; i++) {
if (tensor->shape->orderOfDimensions[i] == index) {
return tensor->shape->dimensions[i];
}
}
PRINT_ERROR("Tensor doesn't have %lu dimensions!", index);
exit(1);
}
```

■ C CONCEPT: Logical vs Physical Dimension Index

A freshly created [3,4] tensor has: dimensions[] = {3, 4} orderOfDimensions = {0, 1} (identity: logical 0 → physical 0) After transposeTensor(t, 0, 1) (makes it a [4,3] transposed view): dimensions[] = {4, 3} (SWAPPED physically) orderOfDimensions = {1, 0} (logical 0 → physical 1) getDimensionsByIndex(t, 0): Loops: orderOfDimensions[0]=1 ≠ 0; orderOfDimensions[1]=0 == 0 → returns dimensions[1] = 3 (logical dim 0 has size 3, which was rows before transpose) This correctly gives the LOGICAL dimension size, accounting for the transpose.

1	`size_t numberOfDims = tensor->shape->numberOfDimensions;`	Cache the number of dimensions to avoid repeated struct field access in the loop.
2-3	`for (size_t i = 0; i < numberOfDims; i++) { if (orderOfDimensions[i] == index)`	Search for the physical slot whose logical label matches 'index'. In the identity case this finds i==index immediately. After transpose, it may find a different i.
4	`return tensor->shape->dimensions[i];`	Return the physical dimension size at that slot. This is the correct SIZE for the requested logical dimension.
5-6	`PRINT_ERROR(...); exit(1);`	If no slot matched, the requested dimension index exceeds the tensor's rank. e.g. asking for dimension 2 of a 2D tensor (valid indices: 0,1).

Section 12.2 — orderDims: Materializing Logical Dimensions

```
void orderDims(tensor_t *tensor, size_t *orderedDims) {
for (size_t i = 0; i < tensor->shape->numberOfDimensions; i++) {
orderedDims[i] = getDimensionsByIndex(tensor, i);
}
}
```

orderDims fills an array with the logical dimension sizes in order [0,1,2,...]. The caller provides a VLA array large enough for numberOfDimensions elements. After this call, orderedDims[0] = logical-size-of-dim-0, etc. This is

used by int32PointWiseArithmetic to compute indices correctly.

═■ WORKED EXAMPLE

For a transposed [3,4] tensor viewed as [4,3]: dimensions[] = {4, 3} orderOfDimensions = {1, 0} orderDims() calls: getDimensionsByIndex(t, 0) → looks for orderOfDims[i]==0 → i=1 → dimensions[1]=3 getDimensionsByIndex(t, 1) → looks for orderOfDims[i]==1 → i=0 → dimensions[0]=4 orderedDims = {3, 4} — the LOGICAL sizes before transpose This is used in calcIndicesByRawIndex so that flat-index-to-indices conversion uses the logical sizes, not the physical ones.

Section 12.3 — doDimensionsMatch: Shape Validation

```
bool doDimensionsMatch(tensor_t *a, tensor_t *b) {
size_t aN = a->shape->numberOfDimensions;
size_t bN = b->shape->numberOfDimensions;
if (aN != bN) {
PRINT_ERROR("Rank mismatch: %lu vs %lu", aN, bN);
exit(1);
}
size_t aOrd[aN], bOrd[bN];
orderDims(a, aOrd);
orderDims(b, bOrd);
for (size_t i = 0; i < aN; i++) {
if (aOrd[i] != bOrd[i]) {
PRINT_DEBUG("Dim mismatch at %lu: %lu vs %lu", i, aOrd[i], bOrd[i]);
return false;
}
}
return true;
}
```

■ C CONCEPT: bool Type in C

C (before C99) had no bool type. C99 introduced _Bool and the <stdbool.h> header which defines 'bool', 'true' (=1), and 'false' (=0). #include <stdbool.h> bool flag = true; // flag holds 1 bool other = false; // other holds 0 if (flag) { ... } // checks if flag != 0 Functions returning bool are used for predicates (yes/no questions). doDimensionsMatch returns true if shapes are compatible for element-wise operations.

Rank check	`if (aN != bN) { PRINT_ERROR(); exit(1); }`	Two tensors with different numbers of dimensions cannot be added/multiplied element-wise. e.g. a 2D matrix and a 3D cube have incompatible shapes regardless of individual sizes.
VLA arrays	`size_t aOrd[aN], bOrd[bN];`	Allocate two temporary arrays on the stack. Size is runtime-determined. Note: aN == bN after the rank check, so aOrd and bOrd are the same size.
orderDims	`orderDims(a, aOrd); orderDims(b, bOrd);`	Materialize LOGICAL dimension sizes for both tensors. This handles transposed tensors correctly.
Dimension comparison	`if (aOrd[i] != bOrd[i]) return false;`	Every logical dimension must match. PRINT_DEBUG prints a message but DOES NOT exit — returns false instead, letting the caller decide whether to abort or handle the mismatch.
return true	`All checks passed: shapes are element-wise compatible.`	The tensors can be used in point-wise arithmetic together.

Section 12.4 — calcElementIndexByIndices: The Transpose-Aware Index

calcElementIndexByIndices converts logical multi-dim indices to a physical flat index, applying the orderOfDimensions permutation. This is the function that makes transposed tensor access correct.

```
size_t calcElementIndexByIndices(size_t numberOfDims, size_t *dims,
size_t *indices, size_t *orderOfDimensions) {
size_t offset = 1;
for (size_t i = 0; i < numberOfDims; i++) {
offset *= dims[i]; // total elements = product of all dims
}
// Apply permutation: map logical indices to physical indices
size_t orderedIndices[numberOfDims];
for (size_t d = 0; d < numberOfDims; d++) {
orderedIndices[orderOfDimensions[d]] = indices[d];
}
// Row-major index calculation using physical indices
size_t outputIndex = 0;
for (size_t i = 0; i < numberOfDims; i++) {
offset /= dims[i];
outputIndex += orderedIndices[i] * offset;
}
return outputIndex;
}
```

═■ WORKED EXAMPLE

Example: access element [i=0, j=1] of a transposed [3,4] matrix (transposed from a physical [4,3] layout): Physical tensor: dims={4,3}, orderOfDimensions={1,0} Logical indices: indices={0, 1} (logical row 0, logical col 1) Phase 1: permute indices d=0: orderedIndices[orderOfDims[0]=1] = indices[0] = 0 → orderedIndices[1]=0 d=1: orderedIndices[orderOfDims[1]=0] = indices[1] = 1 → orderedIndices[0]=1 orderedIndices = {1, 0} Phase 2: row-major index from physical dims {4,3} offset = 4×3 = 12 i=0: offset /= 4 → offset=3; outputIndex += orderedIndices[0]×3 = 1×3 = 3 i=1: offset /= 3 → offset=1; outputIndex += orderedIndices[1]×1 = 0×1 = 0 outputIndex = 3 Physical byte offset = 3 × sizeof(int32_t) = 12 This accesses the element at row 1, col 0 of the ORIGINAL (untransposed) matrix, which is exactly what row 0, col 1 of the TRANSPOSED matrix should be. ✓

Section 12.5 — int32PointWiseArithmetic: Full Line-by-Line

```
// Full implementation from Arithmetic.c:
void int32PointWiseArithmetic(tensor_t *aTensor, tensor_t *bTensor,
int32ElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor) {
if (!doDimensionsMatch(aTensor, bTensor)) {
PRINT_ERROR("Dimensions don't match!"); exit(1);
}
size_t numberOfElements = calcNumberOfElementsByTensor(aTensor);
size_t bytesPerElement = sizeof(int32_t);
size_t numberOfDims = aTensor->shape->numberOfDimensions;
size_t *aDims = aTensor->shape->dimensions;
size_t *bDims = bTensor->shape->dimensions;
size_t aOrderedDims[numberOfDims];
size_t bOrderedDims[numberOfDims];
orderDims(aTensor, aOrderedDims);
orderDims(bTensor, bOrderedDims);
for (size_t i = 0; i < numberOfElements; i++) {
size_t aIndices[numberOfDims];
size_t bIndices[numberOfDims];
calcIndicesByRawIndex(numberOfDims, aOrderedDims, i, aIndices);
size_t aIdx = calcElementIndexByIndices(numberOfDims,
aDims, aIndices, aTensor->shape->orderOfDimensions);
calcIndicesByRawIndex(numberOfDims, bOrderedDims, i, bIndices);
size_t bIdx = calcElementIndexByIndices(numberOfDims,
bDims, bIndices, bTensor->shape->orderOfDimensions);
int32_t aVal = readBytesAsInt32(aTensor->data + aIdx*bytesPerElement);
int32_t bVal = readBytesAsInt32(bTensor->data + bIdx*bytesPerElement);
int32_t result = arithmeticFunc(aVal, bVal);
size_t oIdx = calcElementIndexByIndices(numberOfDims,
outputTensor->shape->dimensions, aIndices,
outputTensor->shape->orderOfDimensions);
writeInt32ToByteArray(result,
outputTensor->data + oIdx*bytesPerElement);
}
}
```

L1-2	`doDimensionsMatch check and exit on failure`	Correctness guard. Both input tensors must have the same shape (after accounting for transposes). Exits immediately on mismatch because continuing with mismatched shapes would write to wrong memory locations.
L3	`numberOfElements = total number of elements in tensor A`	This is the product of all dimensions. We iterate i from 0 to numberOfElements-1, covering every element.
L4	`bytesPerElement = sizeof(int32_t) = 4`	All INT32 elements are 4 bytes. We use this to compute byte offsets.
L5-7	`Cache dimension arrays as local pointers`	Avoids repeatedly dereferencing tensor->shape->dimensions in the hot loop. Modern compilers may do this anyway, but explicit caching makes intent clear.
L8-11	`VLA arrays + orderDims: materialize logical sizes`	aOrderedDims[i] = logical size of dimension i for tensor A (accounting for transpose). Used in calcIndicesByRawIndex to convert flat index → logical indices.
L12	`Main loop: for each element position i in [0, N)`	We process every element of the result tensor.
L13-14	`VLA index arrays — new stack allocation per iteration`	aIndices[d] = logical index along dimension d for element i. These are inside the loop so a new VLA is created each iteration. Compilers typically optimize this to a single stack frame adjustment.
L15	`calcIndicesByRawIndex → convert flat i to logical [row, col, ...]`	For a [3,4] tensor, flat index 6 → logical [1,2]. Uses aOrderedDims (the logical sizes) so the result is in LOGICAL space.

L16	`calcElementIndexByIndices → convert logical indices to PHYSICAL byte offset`	Applies the orderOfDimensions permutation. For a non-transposed tensor this is the same as row-major index. For a transposed tensor, the physical memory offset differs from the logical index.
L17-18	`Same process for tensor B`	B may have a DIFFERENT orderOfDimensions than A. This correctly handles: A is row-major, B is transposed.
L19-20	`readBytesAsInt32 to read values from both tensors`	The byte offset = index × 4 bytes. readBytesAsInt32 copies 4 bytes → int32_t.
L21	`arithmeticFunc(aVal, bVal) — the actual operation`	This is the function pointer call. For addInt32s: returns aVal + bVal. For mulInt32s: returns aVal × bVal. The loop logic is identical for all operations.
L22	`Compute output physical index from the SAME logical indices as A`	The output tensor has the same logical shape as A and B. We reuse aIndices (= logical position of element i) to find where to write.
L23	`writeInt32ToByteArray — write result`	Copies 4 bytes of result into the output tensor's data buffer.

■■ IMPORTANT

Performance note: int32PointWiseArithmetic calls calcIndicesByRawIndex and calcElementIndexByIndices per element. For an N-element tensor: O(N × numDims) work just for index calculations. This is correct but slow compared to a simple flat-index loop (which would ignore orderOfDimensions). If you know your tensors are non-transposed, you can use the inplace variants (int32PointWiseArithmeticInplace) which directly iterate flat indices.

Section 12.6 — matmulIntCore: The Matrix Multiply Engine

matmulIntCore is the shared implementation for all INT32 matrix multiplications. It handles 1D vectors as well as 2D matrices. An optional bias array can be pre-added to the accumulator.

```
static void matmulIntCore(tensor_t *A, tensor_t *B, tensor_t *out,
const int32_t *biasSeed) {
if (A->shape->numberOfDimensions > 2 || B->shape->numberOfDimensions > 2) {
PRINT_ERROR("Matmul only supports up to 2D Tensors"); exit(1);
}
size_t aRows, aColumns;
if (A->shape->numberOfDimensions < 2) {
aRows = 1; aColumns = getDimensionsByIndex(A, 0);
} else {
aRows = getDimensionsByIndex(A, 0);
aColumns = getDimensionsByIndex(A, 1);
}
size_t bRows = getDimensionsByIndex(B, 0);
size_t bColumns = (B->shape->numberOfDimensions < 2)
? 1 : getDimensionsByIndex(B, 1);
if (aColumns != bRows) {
PRINT_ERROR("Rows don't match Columns"); exit(1);
}
size_t resultCounter = 0;
for (size_t row = 0; row < aRows; row++) {
for (size_t col = 0; col < bColumns; col++) {
int32_t result = biasSeed ? biasSeed[col] : 0;
for (size_t k = 0; k < aColumns; k++) {
size_t aIdx[2] = {row, k};
size_t aPhys = calcElementIndexByIndices(2, A->shape->dimensions,
aIdx, A->shape->orderOfDimensions);
int32_t aV = readBytesAsInt32(A->data + aPhys*4);
size_t bIdx[2] = {k, col};
size_t bPhys = calcElementIndexByIndices(2, B->shape->dimensions,
bIdx, B->shape->orderOfDimensions);
int32_t bV = readBytesAsInt32(B->data + bPhys*4);
result += mulInt32s(aV, bV);
}
writeInt32ToByteArray(result, out->data + resultCounter*4);
resultCounter++;
}
}
}
```

1D handling	`if (numberOfDimensions < 2) { aRows=1; aColumns=dim0; }`	A 1D vector [K] is treated as a [1,K] row vector for matrix multiplication. This enables matmul to handle both vector-matrix and matrix-matrix products.
Dimension check	`if (aColumns != bRows) { PRINT_ERROR; exit(1); }`	The inner dimension must match: A is [M,K], B must be [K,N]. If K doesn't match, matrix multiplication is undefined.
biasSeed	`result = biasSeed ? biasSeed[col] : 0;`	Start the accumulator with the bias value for column 'col'. This elegantly fuses the bias addition into the matmul loop, avoiding a separate addition pass over the output.
Inner loop	`for k: aPhys = calcElementIndex(A, [row,k]);`	Look up A[row][k] using the transpose-aware index function. For linearForward: A has been transposed, so A[row][k] accesses the COLUMN of the weight matrix before the transpose.

result += mullnt32s(aV, bV)	Accumulate in int32_t.	For small K (e.g. 64), the maximum accumulation is 64 × 32767^2 ≈ 68B — which OVERFLOWS int32_t (max 2.1B). For safe INT32 accumulation, qMaxBits must be small enough that K × qMax^2 < INT32_MAX. The SYM_INT32 variant uses int64_t to avoid this overflow.
resultCounter++	Output is written in row-major order.	output[row*bColumns + col] = result. resultCounter is a flat index that increments for each output element.

═■ WORKED EXAMPLE

Trace matmulIntCore for A=[2,3] and B=[3,2] (no bias): A = [[1,2,3],[4,5,6]] B = [[7,8],[9,10],[11,12]] Expected C = A×B = [[1×7+2×9+3×11, 1×8+2×10+3×12], [4×7+5×9+6×11, ...]] = [[58, 64], [139, 154]] Loop trace: row=0, col=0: result=0; k=0: 1×7=7; k=1: +2×9=25; k=2: +3×11=58. Write 58. row=0, col=1: result=0; k=0: 1×8=8; k=1: +2×10=28; k=2: +3×12=64. Write 64. row=1, col=0: result=0; k=0: 4×7=28; k=1: +5×9=73; k=2: +6×11=139. Write 139. row=1, col=1: result=0; k=0: 4×8=32; k=1: +5×10=82; k=2: +6×12=154. Write 154. Output flat: [58, 64, 139, 154] ✓

Section 12.7 — matmulSymInt32TensorsWithBias: The FQT Core

```
void matmulSymInt32TensorsWithBias(tensor_t *A, tensor_t *B,
tensor_t *out, tensor_t *bias) {
symInt32QConfig_t *aCfg = A->quantization->qConfig;
symInt32QConfig_t *bCfg = B->quantization->qConfig;
symInt32QConfig_t *outCfg = out->quantization->qConfig;
symInt32QConfig_t *biasCfg = bias->quantization->qConfig;
float scaleA = aCfg->scale;
float scaleB = bCfg->scale;
float scaleOut = outCfg->scale;
float scaleBias = biasCfg->scale;
int32_t outQMax = (1 << (outCfg->qMaxBits - 1)) - 1; // e.g. 32767
int32_t outQMin = -outQMax;
size_t M = getDimensionsByIndex(A, 0);
size_t K = getDimensionsByIndex(A, 1);
size_t N = getDimensionsByIndex(B, 1);
size_t resultCounter = 0;
for (size_t row = 0; row < M; row++) {
for (size_t col = 0; col < N; col++) {
int64_t acc = 0; // 64-bit to prevent overflow
for (size_t k = 0; k < K; k++) {
// A[row][k] and B[k][col] — respecting orderOfDimensions
int32_t aV = /* readBytesAsInt32 at [row][k] */;
int32_t bV = /* readBytesAsInt32 at [k][col] */;
acc += (int64_t)aV * (int64_t)bV;
}
// Convert from mantissa-product to real value:
float realResult = (float)acc * scaleA * scaleB;
// Add bias (bias is already in real-value units × scaleBias):
float biasReal = readBytesAsInt32(bias->data + col*4) * scaleBias;
realResult += biasReal;
// Quantise to output SYM_INT32:
int32_t q = roundByMode(
clamp(realResult / scaleOut, outQMin, outQMax),
outCfg->roundingMode);
writeInt32ToByteArray(q, out->data + resultCounter*4);
resultCounter++;
}
}
}
```

Read scales	`scaleA, scaleB, scaleOut, scaleBias from qConfig`	Each tensor has its own scale. real_A[i][k] = A_mantissa[i][k] × scaleA. The product real_A × real_B = mantissa_A × mantissa_B × scaleA × scaleB.
outQMax	`(1 << (qMaxBits-1)) - 1`	qMaxBits=16 → outQMax = 2^15 - 1 = 32767. The << operator is a bit shift: 1 << 15 = 32768. Subtract 1 = 32767. This is the maximum representable mantissa.
int64_t acc = 0	`64-bit accumulator for the inner product.`	Each product aV × bV fits in int32_t, but K products may overflow. Casting to int64_t before multiply prevents the overflow.
(int64_t)aV × (int64_t)bV	`Cast to 64-bit BEFORE the multiply, not after.`	Without the cast: int32 × int32 = int32 (potential overflow). With the cast: int64 × int64 = int64 (no overflow for K up to ~2M).

float realResult = acc × scaleA × scaleB	`Convert mantissa-product accumulation to real value.`	acc holds sum(aM[k] × bM[k]). real_result = sum(aM[k] × scaleA × bM[k] × scaleB) = acc × scaleA × scaleB.
biasReal = biasM × scaleBias	`Bias is also in SYM_INT32. Convert to real value and add.`	The bias is NOT converted to float VLA here — we read one bias element at a time.
clamp(..., outQMin, outQMax)	`Clip the real value to the output's representable range.`	If realResult / scaleOut = 40000 but outQMax = 32767: clamp to 32767. This saturation is INTENTIONAL — values outside the range are clipped.
roundByMode(..., roundingMode)	`Round the float to nearest int32_t using the configured rounding mode.`	HALF_AWAY: standard round(). SR_HALF_AWAY: add noise before rounding (stochastic rounding for gradient estimation).

Section 12.8 — TRACK_INSTRUCTIONS: Profiling Support

Matmul.c has a conditional compile feature for profiling: if TRACK_INSTRUCTIONS is defined, each matmul call increments a counter. This lets you measure how many multiply-accumulate operations your model uses.

```
// At the top of Matmul.c:
#ifdef TRACK_INSTRUCTIONS
#define MATMUL_FUNC_INT matmulIntTensorsWithInstructionCounter
#else
#define MATMUL_FUNC_INT matmulIntTensors
#endif
// The counter:
size_t matmulInstructionCounter = 0;
// Accessed via:
size_t getMatmulInstructionCounter() { return matmulInstructionCounter; }
// Add.h also has addInstructionCounter:
size_t getAddInstructionCounter() { return addInstructionCounter; }
```

■ C CONCEPT: #ifdef — Conditional Compilation

#ifdef SYMBOL compiles the following block only if SYMBOL is defined. #endif ends the conditional block. #ifndef compiles only if SYMBOL is NOT defined. In CMakeLists.txt or Makefile: add_definitions(-DTRACK_INSTRUCTIONS) → defines TRACK_INSTRUCTIONS (Without this flag, the profiling code is completely absent from the binary) Why conditional? Profiling adds overhead (counter increment per matmul call). In production (deployment), you want maximum speed. #ifdef lets you enable profiling for benchmarking and remove it for deployment with a single build flag, without changing source code.

Chapter 13

C Language Deep Dive for Embedded Systems

Pointers, Memory Layout, Structs, and All C Patterns Used in ODT

Section 13.1 — Pointers: The Most Important C Concept

■ C CONCEPT: What is a Pointer?

A pointer is a variable that holds a MEMORY ADDRESS (a location in RAM). Instead of storing a value, it stores the address WHERE a value is stored. int x = 42; // x holds the VALUE 42, lives at some address (say 0x2000) int *p = &x; // p holds the ADDRESS 0x2000 (& = 'address of') int y = *p; // *p = dereference: READ the value at address 0x2000 = 42 *p = 100; // write 100 to address 0x2000 → x is now 100 Types of pointers in ODT: uint8_t *data → points to a byte array (tensor storage) size_t *dimensions → points to an array of sizes tensor_t *tensor → points to a tensor_t struct void *qConfig → generic pointer to any config struct forwardFn_t forward → pointer to a function

```
// Pointer examples from the ODT library:
// 1. tensor_t has a pointer to its data:
tensor_t t;
uint8_t buf[48]; // 12 float32 elements
t.data = buf; // t.data now holds the address of buf[0]
t.data[0] = 0x3F; // write byte to buf[0] (same as buf[0] = 0x3F)
// 2. Pointer arithmetic — move a pointer forward by N bytes:
uint8_t *elem3 = t.data + 3 * sizeof(float); // points to element 3
// This is equivalent to &t.data[3 * sizeof(float)]
// 3. Arrow operator → : dereference + field access in one step:
tensor_t *pt = &t;
size_t n = pt->shape->numberOfDimensions; // same as (*pt).shape->numberOfDimensions
// 4. NULL pointer: a pointer that doesn't point to anything valid:
tensor_t *nothing = NULL;
// Accessing nothing->data would crash (segfault / HardFault on STM32)
if (nothing != NULL) { // ALWAYS check before using
size_t x = nothing->shape->numberOfDimensions;
}
// 5. const pointer: can't modify what it points to:
const uint8_t *readOnly = buf; // can read readOnly[0] but NOT write it
// readOnly[0] = 5; ← compiler error
```

■ C CONCEPT: Why Pointers are Essential for ODT

On a microcontroller with 320 KB of RAM, copying data wastes memory and time. ODT passes tensor_t* everywhere instead of tensor_t (by value) because: 1. sizeof(tensor_t) = ~16 bytes (4 pointers). Passing by pointer: 4 bytes on 32-bit ARM. 2. Modification: functions that update tensors (forward pass writes output) MUST receive a pointer to modify the original, not a copy. 3. Large buffers: tensor->data points to potentially megabytes of weight data. Copying by value would copy that too — catastrophic. Rule: if the function READS a tensor, pass const tensor_t*. If it MODIFIES a tensor, pass tensor_t*.

Section 13.2 — Structs: Grouping Related Data

```
// Struct basics:
typedef struct point {
int x;
int y;
} point_t;
// Creating and using:
point_t p; // p is on the stack, uninitialized
p.x = 3; // set field x
p.y = 4;
int dist = p.x * p.x + p.y * p.y; // = 25
// Struct on the heap (ODT avoids this, uses static instead):
point_t *p2 = malloc(sizeof(point_t));
p2->x = 5; // arrow: dereference then access
free(p2);
// Struct layout in memory (no padding for same-sized fields):
// [x: 4 bytes][y: 4 bytes] = 8 bytes total
// sizeof(point_t) = 8
// Struct with pointer fields (the ODT pattern):
typedef struct shape {
size_t numberOfDimensions; // 4 or 8 bytes
size_t *dimensions; // 4 bytes (pointer)
size_t *orderOfDimensions; // 4 bytes (pointer)
} shape_t;
// sizeof(shape_t) ≈ 12 bytes on 32-bit ARM
// The actual dimension data is ELSEWHERE, pointed to by dimensions*
```

■ C CONCEPT: Struct vs Union vs Enum

struct: ALL fields exist simultaneously. Total size = sum of field sizes (+ alignment). struct { int a; float b; } → 8 bytes, both a and b are valid. union: ONLY ONE field valid at a time. Total size = LARGEST field size. union { linearConfig_t *linear; reluConfig_t *relu; } → 4 bytes (one pointer). Reading the wrong field is UNDEFINED BEHAVIOR. enum: named integer constants. enum {A, B, C} → A=0, B=1, C=2. sizeof(enum) = sizeof(int) = 4 bytes on ARM. ODT uses all three: - tensor_t, shape_t, linearConfig_t: structs (all fields always present) - layerConfig_t: union (holds one of 11 config pointers) - qtype_t, layerType_t, lossFuncType_t: enums (small integer sets)

Section 13.3 — Arrays, Pointer Arithmetic, and Multi-Dimensional Access

```
// Arrays in C are CONTIGUOUS blocks of memory:
int arr[5] = {10, 20, 30, 40, 50};
// [0] [1] [2] [3] [4]
// Memory: 10 20 30 40 50 (each int = 4 bytes)
// Address of arr[0]: 0x1000
// Address of arr[1]: 0x1004 (0x1000 + 4)
// Address of arr[2]: 0x1008 etc.
// Pointer arithmetic: arr+i points to arr[i]
int *p = arr; // p points to arr[0]
p++; // p now points to arr[1] (advances by sizeof(int) = 4 bytes)
// *(p+2) = arr[3] (two ints forward from arr[1])
// ODT tensor access: element i at byte offset i*sizeof(int32_t):
int32_t val = readBytesAsInt32(tensor->data + i * sizeof(int32_t));
// tensor->data is uint8_t*
// Adding i*4 gives the byte address of the i-th int32_t element
// 2D array as flat 1D (row-major):
// Matrix [3][4] stored as: [row0: 4 ints][row1: 4 ints][row2: 4 ints]
// Element [r][c] at flat index r*4 + c
// In bytes: (r*4 + c) * sizeof(int32_t)
// The ODT equivalent using calcElementIndexByIndices:
size_t indices[2] = {r, c};
size_t flatIdx = calcElementIndexByIndices(2, dims, indices, order);
int32_t v = readBytesAsInt32(tensor->data + flatIdx * 4);
```

═■ WORKED EXAMPLE

Memory layout of a [3×4] INT32 tensor with data [0,1,...,11]: Address: 0x2000 0x2004 0x2008 0x200C 0x2010 ... Value: 0 1 2 3 4 ... Meaning: [0][0] [0][1] [0][2] [0][3] [1][0] ... tensor->data = 0x2000 (pointer to start) Element [1][2] = flat index 1×4 + 2 = 6 Byte offset = 6 × 4 = 24 bytes Address = 0x2000 + 24 = 0x2018 Value at 0x2018 = 6 ✓

Section 13.4 — Function Pointers: C's Dispatch Mechanism

```
// A function pointer stores the ADDRESS of a function.
// Step 1: define the function signature type:
typedef int32_t (*int32BinaryOp_t)(int32_t a, int32_t b);
// This says: 'int32BinaryOp_t is a pointer to any function
// that takes two int32_t and returns int32_t'
// Step 2: write concrete functions matching this signature:
int32_t addInt32s(int32_t a, int32_t b) { return a + b; }
int32_t mulInt32s(int32_t a, int32_t b) { return a * b; }
int32_t maxInt32s(int32_t a, int32_t b) { return (a > b) ? a : b; }
// Step 3: store or pass a function pointer:
int32BinaryOp_t op = addInt32s; // op now holds the address of addInt32s
int32_t result = op(3, 4); // calls addInt32s(3,4) = 7
op = mulInt32s;
result = op(3, 4); // calls mulInt32s(3,4) = 12
// Step 4: pass as argument (like int32PointWiseArithmetic):
void applyToAll(int32_t *arr, size_t n, int32_t x, int32BinaryOp_t op) {
for (size_t i = 0; i < n; i++)
arr[i] = op(arr[i], x);
}
// Usage:
int32_t data[4] = {1, 2, 3, 4};
applyToAll(data, 4, 10, addInt32s); // adds 10 to each: {11,12,13,14}
applyToAll(data, 4, 2, mulInt32s); // multiplies by 2: {22,24,26,28}
```

■ C CONCEPT: Function Pointer Typedef Syntax — The Tricky Part

typedef void (*forwardFn_t)(layer_t*, tensor_t*, tensor_t*); Read this inside-out: *forwardFn_t → it's a pointer (*forwardFn_t) → to a function (layer_t*, tensor_t*, tensor_t*) → taking these parameters void → returning void So forwardFn_t is a type alias for 'pointer to function(layer_t*, tensor_t*, tensor_t*) returning void'. An alternative syntax (C23/GCC extension) using __typeof__ or function alias macros exists but is non-standard. The typedef form above is portable C99.

Section 13.5 — Storage Classes: static, extern, and Global Variables

```
// 1. LOCAL (automatic) — lives on the stack, dies when function returns:
void foo() {
int x = 42; // created on stack entry, freed on return
}
// 2. STATIC (file-scope) — lives in BSS/data section for entire program:
static uint8_t weightData[32768]; // declared outside functions
// OR static inside a function:
void bar() {
static int count = 0; // persists between calls!
count++;
}
// 3. EXTERN — 'this variable is defined in another .c file':
// In Layer.h:
extern layerFunctions_t layerFunctions[];
// This declares that layerFunctions[] is DEFINED in Layer.c
// Any .c file that includes Layer.h can use layerFunctions[] without redefining it
// In Layer.c:
layerFunctions_t layerFunctions[] = { // ← the actual DEFINITION
[LINEAR] = { linearForward, linearBackward, linearCalcOutputShape },
// ...
};
// 4. STATIC + function scope = module-level state (hidden from other files):
// In RNG.c:
static rng32_t rng = { .state = 1 }; // only RNG.c can access this
// Other files cannot accidentally modify rng — it's encapsulated in RNG.c
```

■ C CONCEPT: Why static at File Scope?

In C, a function or variable declared static at file scope is PRIVATE to that .c file. Other files cannot see or use it (the linker doesn't export it). This is C's primary encapsulation mechanism: static rng32_t rng → only RNG.c's functions can modify the RNG state static uint32_t rngNext(...) → only RNG.c can call rngNext; public API is rngNextFloat Without static, rng would be visible to ALL .c files (global scope). Any file could accidentally reset the seed to 0 and corrupt training.

Section 13.6 — Memory Sections on ARM Cortex-M7

	Location	Contents	Lifetime
	Flash (1MB)	Compiled code, string literals, const arrays	Forever (read-only)
	SRAM (copied from flash at startup)	Initialized global/static variables (non-zero initial value)	Forever
	SRAM (zero-initialized at startup)	Uninitialized global/static variables (zeroed by startup)	Forever
	SRAM (grows downward)	Function call frames, local variables, VLAs	Duration of function
	SRAM (between .bss and stack)	malloc'd memory	Until free()

```
// Example: where does each variable live?
static uint8_t weightData[32768]; // → .bss (large, initialized to 0)
static float lr = 0.001f; // → .data (non-zero initial value)
const char *msg = "hello"; // msg ptr → .data; "hello" string → .text
void train() {
int epoch = 0; // → stack
float tmpBuf[1024]; // → stack (VLA: 4KB! careful with stack size)
float *grad = malloc(N*4); // → heap (should use static instead!)
// After train() returns:
// epoch and tmpBuf are gone (stack frame freed)
// grad is LEAKED unless free(grad) is called
free(grad);
}
// Memory map at runtime (typical Cortex-M7):
// 0x00000000 - 0x001FFFFF: Flash (1MB)
// .text + .rodata (.bss/.data initializers)
// 0x20000000 - 0x2004FFFF: SRAM (320KB)
// .data → .bss → heap → (gap) → stack (grows down)
```

Section 13.7 — Preprocessor Directives in Detail

```
// The preprocessor runs BEFORE compilation. It does text substitution.
// #define: simple text replacement
#define MAX_LAYERS 16
layer_t *layers[MAX_LAYERS]; // becomes: layer_t *layers[16];
// #define with arguments (function-like macro):
#define ABS(x) ((x) < 0 ? -(x) : (x))
int a = ABS(-5); // becomes: int a = ((-5) < 0 ? -(-5) : (-5)) = 5
// Note the extra parentheses around x: ABS(a+b) → ((a+b) < 0 ? -(a+b) : (a+b))
// Without them: ABS(a+b) → (a+b < 0 ? -a+b : a+b) — WRONG due to precedence!
// #include: paste file contents
#include <stdint.h> // system header (angle brackets: search system paths)
#include "Tensor.h" // project header (quotes: search current dir first)
// #ifdef / #ifndef / #endif: conditional compilation
#ifndef ODT_TENSOR_H // if ODT_TENSOR_H is NOT defined:
#define ODT_TENSOR_H // define it (marking this file as included)
// ... header content ...
#endif // end of conditional
// #pragma: compiler-specific directives (non-portable):
#pragma once // alternative to header guards (GCC/Clang extension)
// __VA_ARGS__: variadic macros
#define PRINT_DEBUG(level, fmt, ...) \
do { if (DLEVEL >= level) printf(fmt, ##__VA_ARGS__); } while(0)
// ## before __VA_ARGS__: removes the comma if no args follow fmt
PRINT_DEBUG(2, "Loss: %.4f", loss); // → printf("Loss: %.4f", loss)
PRINT_DEBUG(1, "starting"); // → printf("starting") (no comma issue)
```

Section 13.8 — Type Casting in C

```
// Explicit cast: (type)expression
// 1. Widening cast — safe, no information loss:
int32_t a = 32767;
int64_t b = (int64_t)a; // 32-bit → 64-bit, always safe
// 2. Narrowing cast — may lose information:
int64_t big = 100000000000LL;
int32_t small = (int32_t)big; // TRUNCATION — only lower 32 bits kept
// Always clamp before narrowing!
// 3. Float ↔ integer cast:
float f = 3.14f;
int32_t i = (int32_t)f; // rounds TOWARD ZERO: i = 3
// Use roundf() to round to nearest: (int32_t)roundf(f) = 3
// 4. Pointer cast — the most dangerous!
uint8_t *bytes = ...
float *floats = (float *)bytes; // ONLY safe if bytes is properly aligned!
// On ARM, unaligned float access may trigger HardFault.
// Safe alternative: memcpy into a float variable.
// 5. void* cast — used for qConfig:
void *ptr = &symInt32QCfg;
symInt32QConfig_t *cfg = (symInt32QConfig_t *)ptr; // explicit cast
// In C (not C++), void* can be implicitly assigned to any pointer.
// But explicit cast makes intent clear.
// 6. The safe cast pattern from ODT:
// WRONG:
float *wArr = (float *)w->data; // only safe for FLOAT32 tensors!
// RIGHT:
if (w->quantization->type == FLOAT32) {
float *wArr = (float *)w->data; // now guaranteed safe
}
```

Section 13.9 — Embedded C Patterns Specific to ODT

13.9.1 Designated Initializers

```
// Designated initializer: initialize specific fields by name
static rng32_t rng = { .state = 1 };
// Equivalent to: rng.state = 1; (all other fields = 0)
// Array designated initializer (used in dispatch table):
layerFunctions_t layerFunctions[] = {
[LINEAR] = { linearForward, linearBackward, linearCalcOutputShape },
[RELU] = { reluForward, ... },
[SOFTMAX] = { softmaxForward, ... },
};
// [LINEAR] = 0, [RELU] = 1, etc. (from the enum values)
// Slots not initialized (e.g. indices between 0 and LINEAR) are zero-filled
// This syntax requires C99 or later.
```

13.9.2 Compound Literals

```
// Compound literal: create a temporary struct/array inline:
size_t aIdx[2] = {row, k}; // stack array
// Or inline (C99):
calcElementIndexByIndices(2, dims, (size_t[]){row, k}, order);
// (size_t[]){row, k} creates a temporary 2-element array
// Used in matmulIntCore:
size_t aIndices[] = {rowIndex, i}; // local array, no separate declaration
```

13.9.3 Flexible Array Members and Zero-Length Arrays

```
// The ODT dispatch tables use a flexible-size approach:
layerFunctions_t layerFunctions[]; // no size specified = extern (defined elsewhere)
// In the definition file, size is determined by the initializer:
layerFunctions_t layerFunctions[] = {
[LINEAR] = ..., [RELU] = ..., ..., [LAYERNORM] = ...
};
// The compiler counts the initializers and sets the array size automatically.
// sizeof(layerFunctions) / sizeof(layerFunctions[0]) = number of layer types
```

13.9.4 exit(1) vs Return Error Codes

```
// ODT uses exit(1) for fatal errors:
if (aColumns != bRows) {
PRINT_ERROR("Dimension mismatch");
exit(1);
}
// Alternative: return error code (not used in ODT for simplicity):
int matmul(...) {
if (aColumns != bRows) return -1; // error
// ... compute ...
return 0; // success
}
// Caller must check: if (matmul(...) != 0) { /* handle error */ }
// On bare-metal MCU, exit(1) typically calls _exit() which loops forever
// (or resets the MCU, depending on the startup code).
// This is acceptable: a dimension mismatch is a programming error,
// not a runtime condition to recover from.
// For production code, a reset might be more appropriate:
// NVIC_SystemReset(); // ARM reset via CMSIS
```

Section 13.10 — Common C Mistakes in Embedded Systems

■■ IMPORTANT
MISTAKE 1: Integer overflow in accumulation. int32_t acc += a * b; // a*b may overflow before being added to acc FIX: acc += (int64_t)a * (int64_t)b; — cast BEFORE multiply

■■ IMPORTANT
MISTAKE 2: Signed/unsigned comparison. for (int i = n-1; i >= 0; i--) // works fine for (size_t i = n-1; i >= 0; i--) // INFINITE LOOP! size_t is unsigned, wraps to HUGE after 0 FIX: use 'int' for loop counters that must go negative.

■■ IMPORTANT
MISTAKE 3: Dangling pointer. tensor_t *t = &localTensor; // t points to a local variable return t; // localTensor is freed on return! t now points to garbage FIX: use static/global arrays or heap allocation for tensors that must outlive functions.

■■ IMPORTANT
MISTAKE 4: Missing volatile for hardware registers. while (!(UART->SR & 0x40)) {} // busy-wait for UART ready Without 'volatile', the compiler may cache UART->SR and loop forever. FIX: hardware registers are declared volatile in STM32 HAL headers.

■■ IMPORTANT
MISTAKE 5: Forgetting NULL check before pointer dereference. tensor->sparsity->config->mask // if sparsity==NULL: HardFault! FIX: always check: if (tensor->sparsity != NULL && tensor->sparsity->config != NULL)

Chapter 14

Complete Function Reference Index

Every Public Function in the ODT Library

Section 14.1 — DTypes Module

Function	Signature	Purpose
readBytesAsInt32	int32_t f(const uint8_t *bytes)	Copy 4 bytes into int32_t via memcpy. Used to read int32 from tensor byte buffer.
readBytesAsFloat	float f(const uint8_t *bytes)	Copy 4 bytes into float via memcpy. Used to read float32 from tensor byte buffer.
writeInt32ToByteArray	void f(int32_t value, uint8_t *bytes)	Copy int32_t into 4 bytes via memcpy. Used to write int32 results into tensor.
writeFloatToByteArray	void f(float value, uint8_t *bytes)	Copy float into 4 bytes via memcpy. Used to write float results into tensor.
writeBitsToByteArray	void f(uint8_t *ba, size_t start, size_t end, uint8_t data)	Write sub-byte bits into a byte array. Used for BOOL tensors and sparse masks.
readBitsFromByteArray	uint8_t f(const uint8_t *ba, size_t start, size_t end)	Read sub-byte bits from a byte array. Companion to writeBitsToByteArray.

Section 14.2 — Quantization Module

Function	Signature	Purpose
initInt32Quantization	void f(quantization_t *q)	Set type=INT32, qConfig=NULL.
initFloat32Quantization	void f(quantization_t *q)	Set type=FLOAT32, qConfig=NULL.
initSymInt32Quantization	void f(quantization_t *q, symInt32QConfig_t *cfg)	Set type=SYM_INT32, qConfig=cfg.
initBoolQuantization	void f(quantization_t *q, boolQConfig_t *cfg)	Set type=BOOL, qConfig=cfg.
initInt32QConfig	void f(int32QConfig_t *cfg)	Initialize empty INT32 config (no-op, placeholder).
initFloat32QConfig	void f(float32QConfig_t *cfg)	Initialize empty FLOAT32 config.
initSymInt32QConfig	void f(roundingMode_t rm, symInt32QConfig_t *cfg)	Set scale=1.0, qMaxBits=16, roundingMode=rm.
calcNumberOfBytesForData	size_t f(quantization_t *q, size_t N)	Return bytes needed for N elements of type q->type.
calcBitsPerElement	size_t f(quantization_t *q)	Return bits per element for the given quantization type.

Section 14.3 — Tensor Module

Function	Signature	Purpose
setTensorValues	void f(tensor_t*, uint8_t*, shape_t*, quantization_t*, sparsity_t*)	Link all tensor fields. No allocation — stores pointers.
setParameterValues	void f(parameter_t*, tensor_t* param, tensor_t* grad)	Link weight and gradient into parameter_t.
setShape	void f(shape_t*, size_t* dims, size_t N, size_t* order)	Link shape fields. No allocation.

setOrderOfDimsForNewTensor	void f(size_t N, size_t* order)	Fill order with identity {0,1,...,N-1}.
transposeTensor	void f(tensor_t*, size_t dim0, size_t dim1)	Swap two axes in orderOfDimensions (O(1)).
copyTensor	void f(tensor_t* dest, tensor_t* src)	memcpy data bytes from src to dest.
calcNumberOfElementsByShape	size_t f(shape_t*)	Product of all dimensions.
calcNumberOfElementsByTensor	size_t f(tensor_t*)	Calls calcNumberOfElementsByShape on tensor->shape.
calcBytesPerElement	size_t f(quantization_t*)	Bytes per element based on quantization type.
calcBytesPerTensor	size_t f(tensor_t*)	Total bytes = elements × bytesPerElement.
tensorBoolGet	bool f(tensor_t*, size_t flatIndex)	Read 1 bit from BOOL tensor at flatIndex.
tensorBoolSet	void f(tensor_t*, size_t flatIndex, bool val)	Write 1 bit to BOOL tensor at flatIndex.
getParamFromParameter	tensor_t* f(parameter_t*)	Return p->param (weight tensor).
getGradFromParameter	tensor_t* f(parameter_t*)	Return p->grad (gradient tensor).
printTensor	void f(tensor_t*)	Print all values to UART (debug use).
printShape	void f(shape_t*)	Print shape info to UART.

Section 14.4 — TensorConversion Module

Function	Signature	Purpose
convertTensor	void f(tensor_t* in, tensor_t* out)	Convert in to out's quantization type. Dispatches via conversionMatrix[fromType][toType].
requantSymInt32Tensor	void f(tensor_t* in, tensor_t* out)	Two-pass SYM_INT32 → SYM_INT32 with auto-computed scale. In-place safe.
requantSymInt32TensorToScale	void f(tensor_t* in, tensor_t* out)	SYM_INT32 → SYM_INT32 using pre-set target scale. Caller must set out->qConfig->scale first.
quantTypeToString	char* f(qtype_t t)	Return type name as string (for debug).
conversionMatrix[6][6]	conversionFunction_t table	6×6 table of type conversion functions. Index by [fromType][toType].

Section 14.5 — Arithmetic Module

Function	Signature	Purpose
getDimensionsByIndex	size_t f(tensor_t*, size_t index)	Logical dimension size at axis 'index', accounting for transpose.
orderDims	void f(tensor_t*, size_t* out)	Fill out[] with logical dimension sizes [0..N-1].
doDimensionsMatch	bool f(tensor_t* a, tensor_t* b)	True if all logical dimensions match (accounts for transpose).
calcTensorIndexByIndices	size_t f(size_t N, size_t* dims, size_t* indices)	Row-major flat index from multi-dim indices (no transpose).
calcIndicesByRawIndex	void f(size_t N, size_t* dims, size_t raw, size_t* out)	Reverse: flat raw index → multi-dim logical indices.
calcElementIndexByIndices	size_t f(size_t N, size_t* dims, size_t* idx, size_t* order)	Physical flat index from logical indices, applying orderOfDimensions permutation.
int32PointWiseArithmetic	void f(tensor_t* a, tensor_t* b, fn, tensor_t* out)	Apply binary fn to each pair of elements. Transpose-aware.
floatPointWiseArithmetic	void f(tensor_t* a, tensor_t* b, fn, tensor_t* out)	Float version of point-wise arithmetic.

int32PointWiseArithmeticInplace	void f(tensor_t* a, tensor_t* b, fn)	Inplace: a[i] = fn(a[i], b[i]). Faster (no output tensor).
int32ElementWithTensorArithmetic	void f(tensor_t* a, int32_t x, fn, tensor_t* out)	Apply fn to each tensor element and scalar x.

Section 14.6 — Add / Sub / Mul / Div Modules

Function	Signature	Purpose
addInt32s / addFloat32s	int32_t/float f(a, b)	Primitive element operations: return a+b.
addInt32Tensors	void f(tensor_t* a, tensor_t* b, tensor_t* out)	Element-wise INT32 add.
addFloat32TensorsInplace	void f(tensor_t* a, tensor_t* b)	a[i] += b[i] for all i, FLOAT32.
addInt32TensorToSymInt32TensorInplace	void f(tensor_t* sym, tensor_t* int32)	sym_mantissa[i] += int32[i] for all i (mixed-type accumulation).
addFloat32TensorToSymInt32TensorInplace	void f(tensor_t* sym, tensor_t* f32)	sym[i] += f32[i] / scale (converts float grad to SYM_INT32 units).
addSymInt32Tensors / addSymInt32TensorsInplace	void f(tensor_t* a, tensor_t* b, ...)	SYM_INT32 addition with scale matching.
subInt32Tensors / subFloat32Tensors / subSymInt32Tensors	void f(a, b, out)	Element-wise subtraction variants.
mulInt32s / mulFloat32s	int32_t/float f(a, b)	Primitive element multiplication.
mulInt32Tensors / mulFloat32Tensors / mulSymInt32Tensors	void f(a, b, out)	Element-wise tensor multiplication.
divInt32Tensors / divFloat32Tensors	void f(a, b, out)	Element-wise tensor division.
getAddInstructionCounter	size_t f()	Return count of add operations (for profiling).

Section 14.7 — Matmul, MinMax, Sum, Log, Rounding, Comparison

Function	Signature	Purpose
matmulInt32Tensors	void f(a, b, out)	INT32 matrix multiply without bias.
matmulFloat32Tensors	void f(a, b, out)	FLOAT32 matrix multiply without bias.
matmulSymInt32Tensors	void f(a, b, out)	SYM_INT32 matrix multiply without bias.
matmulFloat32TensorsWithBias	void f(a, b, out, bias)	FLOAT32 Y = X @ W + b.
matmulSymInt32TensorsWithBias	void f(a, b, out, bias)	SYM_INT32 Y = X @ W + b with scale handling.
getMatmulInstructionCounter	size_t f()	Return MACs counter.
findAbsMaxFloat	float f(uint8_t* data, size_t N)	Max |x[i]| over N FLOAT32 elements.
findMaxFloat / findMinFloat	float f(data, N)	Max/min over N FLOAT32 elements.
findMaxInt32 / findMinInt32	int32_t f(data, N)	Max/min over N INT32 elements.
sumint32 / sumFloat	int32_t/float f(values, N)	Sum of N values.
logFloat	float f(float x)	Natural log. Used in cross-entropy loss.
roundByMode	int32_t f(float x, roundingMode_t)	Round float to int32 with HALF_AWAY or SR_HALF_AWAY.
clamp	float f(float x, float min, float max)	Clip x to [min, max].

gteFloatValue	void f(in, float thresh, float belowVal, out)	Output = input where input>=thresh, else belowVal.
gteInt32Value	void f(in, int32_t thresh, int32_t belowVal, out)	INT32 threshold gate.
gteSymInt32Zero	void f(in, out)	SYM_INT32 ReLU: zero mantissas below 0, copy scale.

Section 14.8 — Layer Module

Function	Signature	Purpose
initLayer	void f(layer_t*, layerType_t, layerConfig_t*)	Set layer->type and layer->config.
linearInitConfig	void f(linearConfig_t*, weights, bias, forwardQ, ...)	Fill all linearConfig_t fields.
linearForward / linearForwardFloat / linearForwardSymInt32	void f(layer, in, out)	Forward pass: transpose W, matmul, transpose W back.
linearBackward / backwardFloat / backwardSymInt32	void f(layer, fwdIn, loss, propLoss)	Backward pass: compute weight grads, bias grads, propagated loss.
linearCalcWeightGradsFloat32 / SymInt32	void f(loss, fwdIn, weightGrads)	dL/dW = loss^T @ forwardInput. Accumulates (+=) into weightGrads.
linearCalcBiasGradsFloat32 / SymInt32	void f(biasGrads, loss)	dL/db = sum(loss, axis=batch). Accumulates.
linearCalcPropLossFloat32 / SymInt32	void f(weights, loss, propLoss)	dL/dX = loss @ W (propagated gradient for layer below).
linearCalcOutputShape	void f(layer, inShape, outShape)	outShape[0]=inShape[0]=batch; outShape[1]=weights.dims[0]=out_features.
reluInitConfig / reluInit / reluForward / reluBackward	...	ReLU config + dispatch forward/backward.
reluCalcOutputShape	void f(layer, in, out)	Output shape = input shape (ReLU is identity shape).
softmaxInitConfig / softmaxForward / softmaxBackward	...	Softmax layer forward (exp normalization) and backward.
conv1dForward / conv1dBackward / conv1dCalcOutputShape	...	1D convolution forward and backward.
dropoutForward / dropoutBackward	...	Dropout: random mask during training, identity during eval.
flattenForward / flattenBackward / flattenCalcOutputShape	...	Flatten: reshape metadata without copying data.
initKernel / initKernelExplicit	void f(kernel_t*, size, padding, stride, dilation)	Set up convolution kernel parameters.

Section 14.9 — Training Infrastructure

Function	Signature	Purpose
defaultLossConfig	lossConfig_t f(lossFuncType_t)	Returns {funcType, REDUCTION_MEAN, NULL}. Quick setup.
lossFunctions[MSE].forward	float f(out, label, reduction)	MSE loss: mean((out-label)^2).
lossFunctions[CE].forward	float f(out, label, reduction)	Cross-entropy: -sum(label * log(out)).
lossFunctions[CE].backward	void f(out, label, result)	CE+softmax grad: result = out - label.

lossFunctions[X].computeMeanScale	float f(totalSamples, out)	Scale factor for REDUCTION_MEAN gradient.
sgdInit	void f(sgd_t*, float lr, float mom, float wd)	Initialize SGD with learning rate, momentum, weight decay.
sgdStep	void f(sgd_t*, parameter_t*, quantization_t*)	Update: w -= lr * grad. Dispatches float or SYM_INT32 path.
sgdStepM	void f(sgd_t*, parameter_t*, quantization_t*, state)	SGD+Momentum: v = m*v + g; w -= lr*v.
sgdZeroGrad	void f(parameter_t*)	memset grad data to 0. Reset SYM_INT32 scale to 1.0.
initDataLoader	void f(dataLoader_t*, getSample, getSize, getBatch, batchSize, ...)	Set up DataLoader with callbacks and config.
optimizerFunctions[SGD].step / .zeroGrad	stepFn_t / zeroFn_t	Dispatch table for optimizer variants.

Section 14.10 — Support Modules

Function	Signature	Purpose
rngSetSeed	void f(uint32_t seed)	Set xorshift32 state. seed=0 → UINT32_MAX (state=0 is illegal).
rngGetSeed	uint32_t f(void)	Return current RNG state.
rngNextFloat	float f(void)	Uniform float in [0,1). Uses top 24 bits of xorshift32.
rngShuffleIndices	void f(size_t* indices, size_t n)	Fisher-Yates shuffle of indices[0..n-1].
bernoulliSampleTensor	void f(tensor_t* mask, float p)	Fill BOOL mask: each bit = 1 with prob p, 0 with prob 1-p.
serializeTensor	void f(tensor_t*, FILE*)	Write tensor (quantization + shape + data) to byte stream.
serializeParameter	void f(parameter_t*, FILE*)	Serialize both param and grad tensors.
serializeModel	void f(layer_t** model, size_t N, FILE*)	Serialize all N layers' parameters.
deserializeTensor	void f(tensor_t*, FILE*)	Read tensor from byte stream into pre-allocated tensor_t.
deserializeModel	void f(layer_t** model, size_t N, FILE*)	Restore all N layers from byte stream.

Section 14.11 — Quick Lookup: Data Type Sizes

	Size (ARM 32-bit)	Value Range	Used For
	1 byte	0 to 255	Tensor data buffer, byte arrays
	4 bytes	-2,147,483,648 to +2,147,483,647	INT32 and SYM_INT32 mantissas, indices
	8 bytes	-9.2e18 to +9.2e18	Matmul accumulator (avoids overflow)
	4 bytes	0 to 4,294,967,295	RNG state, bitmasks
	4 bytes	~1.2e-38 to ~3.4e38 (7 sig. digits)	FLOAT32 tensors, scales, learning rate
	4 bytes (32-bit)	0 to 4,294,967,295	Sizes, indices, counts
	1 byte (implementation)	0 or 1	Boolean flags, BOOL tensor elements (in C)
	4 bytes (32-bit ARM)	0x00000000 to 0xFFFFFFFF	tensor_t*, function pointers, all references

Section 14.12 — Quick Lookup: All Struct Fields

tensor_t — The central data structure:

Field	Type	Purpose
data	uint8_t*	Raw bytes. Size = calcBytesPerTensor(tensor).

shape	shape_t*	Dimensions and axis permutation.
quantization	quantization_t*	Storage format (type + config).
sparsity	sparsity_t*	Sparse pattern (currently a stub, NULL for dense).

shape_t — Dimension descriptor:

Field	Type	Purpose
numberOfDimensions	size_t	Tensor rank (number of axes).
dimensions	size_t*	Physical size of each dimension slot.
orderOfDimensions	size_t*	Permutation mapping logical → physical axes.

quantization_t — Quantization descriptor:

Field	Type	Purpose
type	qtype_t	INT32, FLOAT32, SYM_INT32, SYM, ASYM, or BOOL.
qConfig	void*	Pointer to type-specific config struct. NULL for INT32/FLOAT32.

symInt32QConfig_t — SYM_INT32 parameters:

Field	Type	Purpose
scale	float	real = mantissa × scale. Updated by requantSymInt32Tensor.
qMaxBits	uint8_t	Mantissa range: [-2^(qMaxBits-1)+1, +2^(qMaxBits-1)-1].
roundingMode	uint8_t	HALF_AWAY or SR_HALF_AWAY.

Chapter 15

Layer Implementations Deep Dive

Every forward and backward pass, line by line

15.1 What Are Neural Network Layers?

A neural network is nothing more than a chain of mathematical operations called layers. Each layer takes a tensor in, does some math, and produces a tensor out. The magic is that when you stack many of these layers and tune their internal parameters (weights and biases) using gradient descent, the network learns to map inputs to outputs.

In the ODT library, every layer follows the same interface contract defined in Layer.h. This means you can add any new layer type without touching the training loop — you just register three function pointers: forward, backward, and calcOutputShape.

■ C CONCEPT: Forward Pass vs Backward Pass

The forward pass computes the layer's output from its input — this is inference. The backward pass computes gradients — how much does changing each parameter affect the loss? The backward pass flows in reverse through the network, which is why it is called backpropagation.

15.2 Layer.h: The Interface Contract

Every layer is described by a <code>layer_t</code> struct. Let's look at each piece:

Layer.h — layer_t struct

```
typedef struct {
layerType_t type; // Which kind of layer is this?
layerConfig_t config; // Pointer to the layer's settings
} layer_t;
```

The <code>layerType_t</code> is an enum — an integer constant with a name. ODT defines: LINEAR, RELU, SOFTMAX, CONV1D, DROPOUT, FLATTEN, BATCHNORM, EMBEDDING, ATTENTION, POOLING, LOSS.

The <code>layerConfig_t</code> is a union — a single block of memory that can hold a pointer to any of the 11 config structs. A union is like a parking space: only one car (value) at a time, but any car that fits can park there.

layerConfig_t union — one memory slot for all layer config types

```
typedef union {
linearConfig_t *linear;
reluConfig_t *relu;
softmaxConfig_t *softmax;
conv1dConfig_t *conv1d;
dropoutConfig_t *dropout;
flattenConfig_t *flatten;
batchNormConfig_t *batchNorm;
embeddingConfig_t *embedding;
attentionConfig_t *attention;
poolingConfig_t *pooling;
lossConfig_t *loss;
} layerConfig_t;
```

■ C CONCEPT: Why Use a Union Here?

All pointers on a 32-bit CPU are 4 bytes. A union that holds any pointer therefore costs exactly 4 bytes — the same as a single pointer. Without the union, you'd need 11 pointer fields (44 bytes) even though only one is ever used at a time. The union is pure memory efficiency.

15.2.1 The Dispatch Table — layerFunctions[]

The dispatch table connects a layer type (an integer) to its three function pointers. It is an array where the index IS the layer type.

Layer.h — dispatch table entry and full table

```
typedef void (*forwardFn_t)(layer_t*, tensor_t*, tensor_t*);
typedef void (*backwardFn_t)(layer_t*, tensor_t*, tensor_t*,
tensor_t*, tensor_t*);
typedef void (*calcOutputShapeFn_t)(layer_t*, shape_t*, shape_t*);
typedef struct {
forwardFn_t forward;
backwardFn_t backward;
calcOutputShapeFn_t calcOutputShape;
} layerFunctions_t;
extern layerFunctions_t layerFunctions[]; // declared in Layer.h
// In Layer.c — definition using designated initializers:
layerFunctions_t layerFunctions[] = {
[LAYER_LINEAR] = { linearForward, linearBackward, linearCalcOutputShape },
[LAYER_RELU] = { reluForward, reluBackward, reluCalcOutputShape },
[LAYER_SOFTMAX] = { softmaxForward, softmaxBackward, softmaxCalcOutputShape },
[LAYER_CONV1D] = { conv1dForward, conv1dBackward, conv1dCalcOutputShape },
[LAYER_DROPOUT] = { dropoutForward, dropoutBackward, dropoutCalcOutputShape },
[LAYER_FLATTEN] = { flattenForward, flattenBackward, flattenCalcOutputShape },
};
```

When the training loop calls <code>layerFunctions[layer->type].forward(layer, input, output)</code>, C looks up the function pointer at index <code>layer->type</code> and calls it. This is called a vtable pattern — it is how object-oriented languages implement virtual methods, but here we do it explicitly with C arrays.

═■ WORKED EXAMPLE

Calling a layer generically: layer_t myLayer; // could be linear, relu, anything // The same call works for ANY layer type: layerFunctions[myLayer.type].forward(&myLayer, &inputTensor, &outputTensor); // C resolves the correct function at runtime based on myLayer.type

15.3 Linear Layer — Full Deep Dive

The Linear (fully connected / dense) layer is the fundamental building block of neural networks. It computes: output = input × W^T + b, where W is the weight matrix and b is the bias vector.

Mathematically: if input has shape [batchSize, inputFeatures] and W has shape [outputFeatures, inputFeatures], then output has shape [batchSize, outputFeatures]. Each output neuron is a dot product of the input with one row of W.

15.3.1 linearConfig_t — All 7 Fields

Linear.h — linearConfig_t struct

```
typedef struct {
parameter_t weights; // {tensor_t param; tensor_t grad;}
parameter_t bias; // bias parameter + its gradient
bool hasBias; // true = add bias; false = skip bias
bool freezeWeights; // true = don't update weights during backward
bool freezeBias; // true = don't update bias during backward
bool isTraining; // true during training, false during inference
tensor_t *inputCache; // stores input for use in backward pass
} linearConfig_t;
```

weights		A parameter_t holds both the value tensor (.param) and its gradient tensor (.grad). For a linear layer mapping 64→32, weights.param has shape [32,64].
bias		Shape [outputFeatures]. Added to each row of the output.
hasBias		If false, the bias addition is skipped entirely — useful when a BatchNorm layer follows (it would cancel the bias anyway).
freezeWeights/freezeBias		Skip gradient accumulation for these parameters. Used in transfer learning where you want to train only part of the network.
inputCache		Pointer to the forward pass input tensor. The backward pass needs this to compute the weight gradient: dW = dOutput^T × input.

15.3.2 linearForwardFloat — Complete Walkthrough

Linear.c — linearForwardFloat

```
void linearForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
linearConfig_t *cfg = layer->config.linear; // Step 1
cfg->inputCache = input; // Step 2
// Step 3: transpose input from [batch, in] → [in, batch]
transposeTensor(input);
// Step 4: output = weights × input^T + bias
// weights shape: [out, in]
// input (transposed) shape: [in, batch]
// result shape: [out, batch]
matmulFloat32TensorsWithBias(
&cfg->weights.param, // A: [out, in]
input, // B: [in, batch] (transposed)
cfg->hasBias ? &cfg->bias.param : NULL,
output // C: [out, batch]
);
// Step 5: un-transpose input (restore caller's tensor)
transposeTensor(input);
// Step 6: transpose output from [out, batch] → [batch, out]
transposeTensor(output);
}
```

Step 1		Cast the generic config union to linearConfig_t*. This is safe because we know this function is only called when layer->type == LAYER_LINEAR.
Step 2		Store the input pointer in inputCache. The backward pass will read this later to compute weight gradients. Note: only the pointer is stored, not a copy. The caller must keep the input tensor alive until backward completes.
Step 3		transposeTensor is O(1) — it just swaps orderOfDimensions[0] and orderOfDimensions[1]. Now what was [batch, in] logically looks like [in, batch] when accessed via tensor index functions.
Step 4		matmulFloat32TensorsWithBias computes A×B+bias. The shapes are [out,in] × [in,batch] = [out,batch]. If hasBias is false, NULL is passed and the matmul skips the bias addition.
Step 5		Restore the input tensor to its original orientation. The caller expects the input to be unchanged after the forward pass.
Step 6		The matmul result is [out,batch]. Transpose to [batch,out] so that the next layer receives data in the conventional [batch, features] format.

═■ WORKED EXAMPLE

Shape trace through linearForwardFloat: Input: [4, 64] (batch=4, input_features=64) Weights: [32, 64] (output_features=32, input_features=64) Bias: [32] (one bias per output neuron) After transposeTensor(input): logically [64, 4] After matmul([32,64] × [64,4]): result [32, 4] After transposeTensor(output): output is [4, 32] ✓

15.3.3 linearBackward — Computing All Three Gradients

The backward pass receives <code>dOutput</code> — the gradient of the loss with respect to this layer's output. It must compute:

1. dWeights = dOutput^T × input (shape: [out, in])
2. dBias = sum over batch of dOutput (shape: [out])
3. dInput = dOutput × weights (shape: [batch, in]) — propagates loss upstream

Linear.c — linearBackward (simplified)

```
void linearBackward(layer_t *layer, tensor_t *input,
tensor_t *dOutput, tensor_t *dInput) {
linearConfig_t *cfg = layer->config.linear;
if (!cfg->freezeWeights) {
// dWeights = dOutput^T x inputCache
// dOutput shape: [batch, out] → transpose → [out, batch]
// inputCache shape: [batch, in]
// [out, batch] x [batch, in] = [out, in] ← same as weights shape
transposeTensor(dOutput);
matmulFloat32TensorsWithBias(dOutput, cfg->inputCache,
NULL, &cfg->weights.grad);
transposeTensor(dOutput); // restore
}
if (cfg->hasBias && !cfg->freezeBias) {
// dBias[j] = sum over batch of dOutput[batch, j]
for (size_t j = 0; j < biasSize; j++) {
float sum = 0.0f;
for (size_t b = 0; b < batchSize; b++)
sum += dOutput[b * outFeatures + j];
biasGradPtr[j] += sum;
}
}
if (dInput != NULL) {
// dInput = dOutput x weights (no transpose on weights)
// [batch, out] x [out, in] = [batch, in]
matmulFloat32TensorsWithBias(dOutput, &cfg->weights.param,
NULL, dInput);
}
}
```

■ C CONCEPT: Why += for gradient accumulation?

Gradients use <code>+=</code> because a single parameter (like a weight) can affect multiple outputs. Each path through the network contributes a partial gradient, and the total gradient is the sum of all contributions. This is the chain rule applied across a batch.

15.3.4 linearCalcOutputShape

Linear.c — linearCalcOutputShape

```
void linearCalcOutputShape(layer_t *layer, shape_t *inputShape,
shape_t *outputShape) {
linearConfig_t *cfg = layer->config.linear;
size_t outFeatures = cfg->weights.param.shape.dimensions[0];
// Output has same batch dimensions, but last dim changes
*outputShape = *inputShape; // copy all dims
outputShape->dimensions[inputShape->numberOfDimensions - 1] = outFeatures;
}
```

This function is used before allocating output tensors. By knowing input shape and layer config, the training infrastructure can pre-allocate the correct output buffer.

15.4 ReLU Layer — Complete Analysis

ReLU stands for Rectified Linear Unit. It is the most widely used activation function in neural networks. The formula is simple: ReLU(x) = max(0, x). Negative values become 0; positive values pass through unchanged.

Why is this useful? Without non-linear activation functions like ReLU, stacking multiple linear layers is mathematically equivalent to a single linear layer — the network cannot learn complex patterns. ReLU introduces non-linearity cheaply.

15.4.1 reluForwardFloat — Line by Line

Relu.c — reluForwardFloat

```
void reluForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
size_t n = calcNumberOfElementsByShape(&input->shape);
float *inPtr = (float*) input->data.dataPTR;
float *outPtr = (float*) output->data.dataPTR;
for (size_t i = 0; i < n; i++) {
outPtr[i] = gteFloatValue(inPtr[i], 0.0f) ? inPtr[i] : 0.0f;
}
}
```

Line 2		<code>calcNumberOfElementsByShape</code> multiplies all dimensions together. For shape [4, 32], n = 128. We loop over every element.
Line 3-4		Cast the void* data pointer to float*. This lets us use array indexing with [i] instead of byte-level pointer arithmetic.
Line 5-7		For each element: if it is ≥ 0, copy it. Otherwise write 0. The <code>gteFloatValue(a, b)</code> helper returns true if a ≥ b.

Note that ReLU has no trainable parameters — <code>reluConfig_t</code> exists only to hold a pointer to the input tensor cached for the backward pass.

15.4.2 reluForwardSymInt32 — Quantized ReLU

Relu.c — reluForwardSymInt32

```
void reluForwardSymInt32(layer_t *layer, tensor_t *input, tensor_t *output) {
size_t n = calcNumberOfElementsByShape(&input->shape);
int32_t *inPtr = (int32_t*) input->data.dataPTR;
int32_t *outPtr = (int32_t*) output->data.dataPTR;
for (size_t i = 0; i < n; i++) {
outPtr[i] = gteSymInt32Zero(inPtr[i]) ? inPtr[i] : 0;
}
// Copy scale from input to output (ReLU doesn't change scale)
symInt32QConfig_t *inQ = input->quantization.qConfig;
symInt32QConfig_t *outQ = output->quantization.qConfig;
outQ->scale = inQ->scale;
outQ->qMaxBits = inQ->qMaxBits;
}
```

gteSymInt32Zero		For SYM_INT32, zero is represented as the integer 0 (because the quantization is symmetric). So this is just checking if the integer mantissa ≥ 0, which is equivalent to checking if the real value ≥ 0.
Scale copy		ReLU does not change the magnitude of positive values, so the output has the same quantization scale as the input. This is important — if we forgot to copy the scale, the next layer would decode the tensor wrong.

■ C CONCEPT: Symmetric Quantization and Zero

In SYM_INT32: real_value = mantissa × scale. Zero mantissa → zero real value. Checking mantissa ≥ 0 is exactly the same as checking real_value ≥ 0, because scale is always positive. This is why symmetric quantization is convenient for ReLU.

15.4.3 reluBackward

Relu.c — reluBackward

```
void reluBackward(layer_t *layer, tensor_t *dOutput, tensor_t *dInput) {
size_t n = calcNumberOfElementsByShape(&dOutput->shape);
float *dOutPtr = (float*) dOutput->data.dataPTR;
float *dInPtr = (float*) dInput->data.dataPTR;
float *cachedIn = (float*) layer->config.relu->inputCache->data.dataPTR;
for (size_t i = 0; i < n; i++) {
// If the forward input was > 0, gradient passes through
// If the forward input was <= 0, gradient is blocked (= 0)
dInPtr[i] = (cachedIn[i] > 0.0f) ? dOutPtr[i] : 0.0f;
}
}
```

The ReLU backward pass implements the derivative of max(0,x): it equals 1 when x>0 and 0 otherwise. By multiplying the upstream gradient by this derivative, gradients only flow back through neurons that were 'on' (positive) during the forward pass. Neurons that were 'off' (negative) get zero gradient — they are 'dead' for this sample.

■■ IMPORTANT

The Dying ReLU Problem: If a neuron always receives negative input, its gradient is always 0, so it never gets updated. This is called a 'dead ReLU'. Solutions include: careful weight initialization, lower learning rates, or using LeakyReLU which allows a small negative slope instead of zero.

15.5 Softmax Layer — Probability Output

Softmax converts a vector of arbitrary real numbers (called logits) into a probability distribution — all values are in [0,1] and they sum to 1. It is used as the last layer in classification networks.

The formula: softmax(x)[i] = exp(x[i]) / sum(exp(x[j]) for all j)

■ C CONCEPT: Numerical Stability Trick

Computing exp(x) directly can overflow for large x values (e.g., exp(1000) = inf). The trick: subtract the maximum value first. Since softmax(x) = softmax(x - max(x)), this shifts all values to ≤ 0 before exponentiating, preventing overflow.

Softmax.c — softmaxForward (simplified)

```
void softmaxForward(layer_t *layer, tensor_t *input, tensor_t *output) {
size_t batch = input->shape.dimensions[0];
size_t n = input->shape.dimensions[1]; // number of classes
float *in = (float*) input->data.dataPTR;
float *out = (float*) output->data.dataPTR;
for (size_t b = 0; b < batch; b++) {
float *row_in = in + b * n;
float *row_out = out + b * n;
// Step 1: find max for numerical stability
float maxVal = row_in[0];
for (size_t i = 1; i < n; i++)
if (row_in[i] > maxVal) maxVal = row_in[i];
// Step 2: compute exp(x - max) and sum
float sum = 0.0f;
for (size_t i = 0; i < n; i++) {
row_out[i] = expf(row_in[i] - maxVal);
sum += row_out[i];
}
// Step 3: divide by sum to normalize
for (size_t i = 0; i < n; i++)
row_out[i] /= sum;
}
}
```

b * n		Pointer arithmetic to jump to the start of batch row b. If each row has n elements, batch b starts at offset b×n from the beginning of the array.
expf		C standard library function for float exponentiation. Use expf (not exp) for float — exp works on double, which is slower and unnecessary on an MCU.
Dividing by sum		This ensures the outputs sum to exactly 1.0, making them valid probabilities.

15.5.1 Why Softmax and Cross-Entropy Are Usually Combined

If you compute softmax backward separately and cross-entropy backward separately, you get a numerically unstable and complicated gradient. But if you combine them, the gradient simplifies beautifully: d(CE after softmax)/d(logits) = pred - target.

This is why ODT implements <code>crossEntropySoftmaxBackward</code> as a single function that assumes softmax was applied just before cross-entropy. The combined backward is both simpler and more numerically stable.

15.6 Conv1D Layer — 1D Convolution for Sequences

1D convolution processes sequential data — like audio waveforms, time series, or sequences of feature vectors. It slides a small kernel (filter) across the input and computes dot products at each position.

For speaker identification on LibriSpeech audio, Conv1D extracts local temporal patterns — things like the shape of phonemes or short acoustic events that characterize a speaker's voice.

15.6.1 conv1dConfig_t — Every Field Explained

Conv1d.h — conv1dConfig_t

```
typedef struct {
parameter_t weights; // shape: [outChannels, inChannels, kernelSize]
parameter_t bias; // shape: [outChannels]
bool hasBias;
size_t inChannels; // input feature dimension
size_t outChannels; // output feature dimension
kernel_t kernel; // size, padding, stride, dilation
bool isTraining;
tensor_t *inputCache;
} conv1dConfig_t;
typedef struct {
size_t size; // kernel size (e.g. 3, 5, 7)
paddingType_t paddingType; // VALID, SAME, or EXPLICIT
size_t stride; // step between kernel positions
size_t dilation; // spacing between kernel elements
size_t padding; // explicit padding amount (if EXPLICIT)
} kernel_t;
```

kernelSize		How many input time steps each output value looks at. A kernel of size 3 means each output is computed from 3 consecutive inputs.
stride		How many steps to advance the kernel each time. stride=1 gives maximum overlap (most detailed). stride=2 halves the output length.
dilation		Gaps between kernel elements. dilation=2 means the kernel samples at positions [0, 2, 4] instead of [0, 1, 2]. This gives a larger receptive field without more parameters.
paddingType=SAME		Adds zeros at the edges so the output length equals the input length (assuming stride=1).
paddingType=VALID		No padding. Output length = floor((L-K)/stride)+1 where L is input length and K is kernel size.

15.6.2 Conv1D Mathematical Formula

For each output channel c_out, output position t, and batch b:

Conv1D formula

```
output[b, c_out, t] = bias[c_out]
+ sum over c_in, k of:
weights[c_out, c_in, k] * input[b, c_in, t*stride + k*dilation]
```

WORKED EXAMPLE

Conv1D with kernel=3, stride=1, no padding: input: [1, 1, 5] (batch=1, channels=1, length=5): [1, 2, 3, 4, 5] kernel: [1, 1, 3] (1 out_channel, 1 in_channel, size=3): [0.5, 1.0, 0.5] output[t=0] = 1*0.5 + 2*1.0 + 3*0.5 = 0.5+2.0+1.5 = 4.0 output[t=1] = 2*0.5 + 3*1.0 + 4*0.5 = 1.0+3.0+2.0 = 6.0 output[t=2] = 3*0.5 + 4*1.0 + 5*0.5 = 1.5+4.0+2.5 = 8.0 Output shape: [1, 1, 3] (length reduced from 5 to 3 by VALID padding)

15.7 Dropout Layer — Regularization

Dropout randomly sets a fraction p of the neurons to zero during training. This prevents co-adaptation: neurons cannot rely on specific other neurons, so they must learn more robust features. During inference, dropout is disabled.

ODT uses inverted dropout: during training, surviving neurons are scaled by 1/(1-p) so that the expected value of the output is the same as without dropout. This means no adjustment is needed at inference time.

15.7.1 dropoutConfig_t Fields

Dropout.h — dropoutConfig_t

```
typedef struct {
float p; // dropout probability (fraction to zero out)
bool training; // true = apply dropout; false = pass-through
tensor_t *mask; // boolean mask (1=keep, 0=drop), saved for backward
quantization_t *forwardQ; // quantization for mask tensor
quantization_t *backwardQ;
bool ownsQuantizations; // true = we allocated forwardQ/backwardQ
} dropoutConfig_t;
```

15.7.2 dropoutForward — Step by Step

Dropout.c — dropoutForward

```
void dropoutForward(layer_t *layer, tensor_t *input, tensor_t *output) {
dropoutConfig_t *cfg = layer->config.dropout;
size_t n = calcNumberOfElementsByShape(&input->shape);
float *in = (float*) input->data.dataPTR;
float *out = (float*) output->data.dataPTR;
if (!cfg->training) {
// Inference: copy input to output unchanged
memcpy(out, in, n * sizeof(float));
return;
}
// Training: generate random binary mask
bernoulliSampleTensor(cfg->mask, 1.0f - cfg->p);
// 1-p = probability of keeping a neuron
float scale = 1.0f / (1.0f - cfg->p); // inverted dropout scale
for (size_t i = 0; i < n; i++) {
bool keep = tensorBoolGet(cfg->mask, i);
out[i] = keep ? in[i] * scale : 0.0f;
}
}
```

bernoulliSampleTensor		Fills cfg->mask with random bits: each bit is 1 with probability (1-p) and 0 with probability p. Uses the RNG module.
scale = 1/(1-p)		For p=0.5: scale=2.0. Surviving neurons are doubled, so their expected contribution equals the full (no-dropout) case.
tensorBoolGet		Reads a single bit from the packed boolean mask tensor. The mask uses 1 bit per element, so a 512-element mask costs only 64 bytes.

out[i] = 0.0f		Dropped neurons output exactly zero. During backward, zeros also block gradient flow through dropped neurons.

15.7.3 dropoutBackward

Dropout.c — dropoutBackward

```
void dropoutBackward(layer_t *layer, tensor_t *dOutput, tensor_t *dInput) {
dropoutConfig_t *cfg = layer->config.dropout;
size_t n = calcNumberOfElementsByShape(&dOutput->shape);
float *dOut = (float*) dOutput->data.dataPTR;
float *dIn = (float*) dInput->data.dataPTR;
float scale = 1.0f / (1.0f - cfg->p);
for (size_t i = 0; i < n; i++) {
bool kept = tensorBoolGet(cfg->mask, i);
dIn[i] = kept ? dOut[i] * scale : 0.0f;
}
}
```

The backward pass uses the SAME mask as the forward pass (saved in cfg->mask). This ensures that gradients only flow through neurons that were active during the forward pass. The scale factor is applied again for consistency.

15.8 Flatten Layer — Zero-Copy Reshape

Flatten converts a multi-dimensional tensor into a 1D vector (per batch item). For example, after a Conv1D layer with output [batch, channels, length], Flatten produces [batch, channels × length]. This is needed before a Linear layer which expects 2D input.

The key insight in ODT's implementation: no data is copied. The underlying memory is identical — we just change the shape metadata.

Flatten.c — flattenForward

```
void flattenForward(layer_t *layer, tensor_t *input, tensor_t *output) {
// Share the data buffer — no copy needed
output->data.dataPTR = input->data.dataPTR;
output->data.numberOfElements = input->data.numberOfElements;
// Compute flat size: product of all dims except batch
size_t flatSize = 1;
for (size_t d = 1; d < input->shape.numberOfDimensions; d++)
flatSize *= input->shape.dimensions[d];
// Set output shape: [batchSize, flatSize]
size_t batchSize = input->shape.dimensions[0];
output->shape.numberOfDimensions = 2;
output->shape.dimensions[0] = batchSize;
output->shape.dimensions[1] = flatSize;
output->shape.orderOfDimensions[0] = 0;
output->shape.orderOfDimensions[1] = 1;
}
```

output->data.dataPTR = input->data.dataPTR		Both tensors point to the SAME memory. A write through output->data changes what input->data sees too. This is safe here because flatten's output is only read, not written.
flatSize		Multiply dimensions [1], [2], ... together. For Conv1D output [4, 16, 8] (batch=4, channels=16, length=8): flatSize = 16×8 = 128.
orderOfDimensions = {0, 1}		Reset to natural order after flattening. The 2D output is not transposed.

═■ WORKED EXAMPLE

Flatten shape transformation: Input shape: [4, 16, 8] (batch=4, 16 channels, length=8) Output shape: [4, 128] (batch=4, flat=16×8=128) Data in memory: unchanged — both tensors read the same bytes Memory saved: 0 bytes copied, 0 extra allocations

■ NOTE

Flatten and Memory Safety: Because flatten shares memory, the caller must not free the input tensor's data buffer while the output tensor is still in use. The training loop manages this by keeping all layer activations alive until the backward pass completes.

15.9 Layer Summary Table

uct	Has Params?	Forward Cost	Key Feature
g_t	Yes (W, b)	O(in × out × batch)	Fully connected
t	No	O(n)	Element-wise,
nfig_t	No	O(classes × batch)	Normalizes to p
fig_t	Yes (W, b)	O(out×in×K×L×batch)	Local pattern e
nfig_t	No	O(n) + RNG	Regularization,
g_t	No	O(1)	Zero-copy resh

■ **RELATIONS WITH OTHER FILES**

Chapter 15 connects to: Chapter 6 (Arithmetic) — matmulFloat32TensorsWithBias, gteFloatValue called by layers · Chapter 5 (Tensor) — transposeTensor, calcNumberOfElementsByShape used everywhere · Chapter 8 (Training) — layerFunctions dispatch table called by training loop · Chapter 9 (RNG) — bernoulliSampleTensor used by dropout · Chapter 11 (MCU) — All layers execute on-device, data flows through SRAM buffers

Chapter 16

Mathematical Foundations

The math behind every operation in the ODT library

16.1 Linear Algebra Essentials

Neural networks are, at their core, linear algebra. Every forward pass is a sequence of matrix multiplications and element-wise operations. Understanding the shapes and operations on matrices is essential to understanding the code.

16.1.1 Vectors and Matrices

A vector is a list of numbers: v = [v1, v2, ..., vn]. A matrix is a 2D grid of numbers with rows and columns. An m × n matrix has m rows and n columns.

Matrix notation example

```
W = [[1, 2, 3], // row 0
[4, 5, 6]] // row 1
// W is a 2x3 matrix: 2 rows, 3 columns
// W[i][j] means row i, column j
// W[1][2] = 6
```

In C, a 2D matrix is typically stored as a flat 1D array in row-major order: all elements of row 0 come first, then row 1, etc. Element W[i][j] is at index i × numCols + j.

═■ WORKED EXAMPLE

Row-major memory layout: Matrix W (2 rows, 3 cols) stored in memory: Index: 0 1 2 3 4 5 Value: 1 2 3 4 5 6 ↑row0↑ ↑row1↑ W[1][2] = W_flat[1*3 + 2] = W_flat[5] = 6 ✓

16.1.2 Matrix Multiplication

If A has shape [m, k] and B has shape [k, n], then C = A×B has shape [m, n]. Each element C[i][j] = sum over t of A[i][t] × B[t][j]. The inner dimension k must match — this is the 'contracted' dimension.

Matrix multiply formula

```
// C[i][j] = sum_{t=0}^{k-1} A[i][t] * B[t][j]
//
// Example: A is [2,3], B is [3,2], C is [2,2]
A = [[1,2,3], [4,5,6]] // shape [2,3]
B = [[7,8], [9,10], [11,12]] // shape [3,2]
C[0][0] = 1*7 + 2*9 + 3*11 = 7+18+33 = 58
C[0][1] = 1*8 + 2*10 + 3*12 = 8+20+36 = 64
C[1][0] = 4*7 + 5*9 + 6*11 = 28+45+66 = 139
C[1][1] = 4*8 + 5*10 + 6*12 = 32+50+72 = 154
```

In ODT, <code>matmulFloat32TensorsWithBias</code> computes exactly this, with an optional bias added to each row of the result.

16.1.3 Matrix Transpose

The transpose of A (written A^T) swaps rows and columns: A^T[i][j] = A[j][i]. If A has shape [m, n], then A^T has shape [n, m].

In ODT, <code>transposeTensor</code> is O(1) — it does not move any data. It swaps the orderOfDimensions array, so that when the tensor is accessed logically, dimensions are looked up in swapped order.

═■ WORKED EXAMPLE

Why transpose is needed in Linear forward pass: We want: output = input × W^T (so each row of W is a set of weights for one output neuron) input shape: [batch=4, in=64] W shape: [out=32, in=64] W^T shape: [in=64, out=32] input × W^T = [4,64] × [64,32] = [4,32] ← correct output shape But ODT's matmul expects row × column = scalar, so instead we transpose input and compute W × input^T = [32,64] × [64,4] = [32,4], then transpose result to [4,32]

16.2 Gradient Descent and Backpropagation

The goal of training is to find weight values that minimize the loss function. We do this by computing the gradient (direction of steepest increase) and moving the weights in the opposite direction.

16.2.1 What Is a Gradient?

For a function f(x), the derivative f'(x) tells us: 'if I increase x by a tiny amount ε, f increases by approximately f'(x) × ε'. The gradient is the multidimensional generalization: ∇f is a vector where each component is the partial derivative with respect to one variable.

Gradient intuition

```
// f(w) = w^2 → f'(w) = 2w
// At w=3: f'(3) = 6
// Meaning: if w increases by 0.01, f increases by ~0.06
// To DECREASE f, move w in direction -f'(w) = -6
// Update: w_new = w - lr * f'(w) = 3 - 0.1*6 = 2.4
// Now f(2.4) = 5.76 < f(3) = 9 ✓ loss went down
```

16.2.2 Chain Rule — The Engine of Backpropagation

Neural networks are composed functions: loss = L(softmax(linear(relu(linear(x))))). To find how much the loss changes when we change a weight deep in the network, we use the chain rule: if z = f(g(x)), then dz/dx = (dz/dg) × (dg/dx).

In a network with layers A → B → C → loss:

Chain rule in backprop

```
// Forward: x → A(x) → B(A(x)) → C(B(A(x))) → loss
// Backward (chain rule):
// dL/dA = dL/dC × dC/dB × dB/dA
//
// In code, each layer's backward receives dOutput = dL/d(this layer's output)
// and must compute dInput = dL/d(this layer's input) = dOutput × d(output)/d(input)
// This dInput becomes dOutput for the previous layer.
```

This is exactly what ODT implements: the training loop calls backward on each layer from last to first, passing the gradient signal backwards through the network.

16.2.3 Stochastic Gradient Descent (SGD)

Pure gradient descent computes gradients over the entire dataset. Stochastic Gradient Descent (SGD) computes gradients over small random batches (mini-batches). This is noisier but much faster per update.

SGD update rule

```
// For each parameter w:
// w_new = w - learning_rate * gradient(w)
//
// With momentum (helps escape local minima):
// velocity = momentum * velocity - learning_rate * gradient
// w_new = w + velocity
//
// With weight decay (L2 regularization):
// w_new = w - learning_rate * (gradient + weight_decay * w)
```

ODT's <code>sgdStepFloat</code> implements the basic version: w -= lr × grad. The <code>sgd_t</code> struct stores momentumFactor and weightDecay fields for optional use in extended implementations.

16.3 Loss Functions — Measuring Error

A loss function measures how wrong the network's predictions are. It takes the network's output and the true target, and returns a single scalar number. Lower loss = better predictions.

16.3.1 Mean Squared Error (MSE)

MSE = (1/n) × sum((predicted - target)^2). Used for regression tasks (predicting a continuous value). The squared term penalizes large errors more heavily.

MSE example

```
// predictions: [2.3, 0.5, 1.8]
// targets: [2.0, 0.0, 2.0]
// errors: [0.3, 0.5, -0.2]
// squared: [0.09, 0.25, 0.04]
// MSE = (0.09 + 0.25 + 0.04) / 3 = 0.127
//
// MSE gradient w.r.t. predictions: 2/n * (pred - target)
// = 2/3 * [0.3, 0.5, -0.2] = [0.2, 0.333, -0.133]
```

16.3.2 Cross-Entropy Loss

Cross-entropy is used for classification. If the network outputs probabilities (after softmax), cross-entropy is: CE = -sum(target × log(prediction)).

For one-hot targets (only one class is correct), this simplifies to -log(probability of the correct class). Perfect prediction → loss = 0. Wrong prediction → loss = infinity.

Cross-entropy example — 3-class classification

```
// targets (one-hot): [0, 1, 0] ← class 1 is correct
// predictions (softmax output): [0.1, 0.7, 0.2]
// CE = -(0*log(0.1) + 1*log(0.7) + 0*log(0.2))
// = -log(0.7)
// = 0.357 (low loss — prediction was fairly good)
//
// If predictions were [0.4, 0.2, 0.4] (wrong):
// CE = -log(0.2) = 1.609 (high loss)
```

The backward gradient through softmax+cross-entropy combined is simply: dL/d(logits) = predictions - targets. This is what <code>crossEntropySoftmaxBackward</code> computes.

✏■ WORKED EXAMPLE

Cross-entropy backward example: predictions: [0.1, 0.7, 0.2] targets: [0, 1, 0] gradient: [0.1-0, 0.7-1, 0.2-0] = [0.1, -0.3, 0.2] Interpretation: the network should increase logit for class 1 (negative gradient means: decreasing this value decreases loss) and decrease logits for classes 0 and 2.

16.4 Fixed-Point Quantization — The Mathematics

Floating-point numbers (float32) are convenient but expensive on MCUs. Quantization maps floating-point values to integers, enabling faster arithmetic with less memory. FQT (Fixed-point Quantization Training) in ODT trains with quantized weights from the start.

16.4.1 SYM_INT32 Encoding

In ODT's SYM_INT32 format: real_value = mantissa × scale, where mantissa is an int32_t and scale is a float. The range of representable values is approximately [-qMax × scale, +qMax × scale] where qMax = 2^(qMaxBits-1) - 1.

SYM_INT32 quantization formulas

```
// To quantize a float to SYM_INT32:
mantissa = clamp(round(real_value / scale), -qMax, +qMax)
// To dequantize back to float:
real_value = mantissa * scale
// Example: scale=0.01, qMaxBits=8, qMax=127
// Quantize 0.735: mantissa = round(0.735/0.01) = round(73.5) = 74
// Dequantize: 74 * 0.01 = 0.74 (small rounding error)
```

16.4.2 Choosing Scale — Two-Pass Requantization

To minimize quantization error, we want the scale to be as small as possible while still representing the maximum value in the tensor. The optimal scale is: scale = absMax / qMax.

ODT's <code>requantSymInt32Tensor</code> does this in two passes:

Two-pass requantization algorithm

```
// Pass A: find absolute maximum value in the tensor
float absMax = 0.0f;
for each element e in tensor:
float val = e.mantissa * e.scale; // dequantize
if (fabsf(val) > absMax) absMax = fabsf(val);
// Compute new scale to use the full integer range
float newScale = absMax / qMax;
// Pass B: re-encode every element with the new scale
for each element e in tensor:
float realVal = e.mantissa * e.scale; // old representation
e.mantissa = clamp(round(realVal / newScale), -qMax, qMax);
e.scale = newScale; // update scale (same for whole tensor)
```

■ NOTE

Scale is Per-Tensor, Not Per-Element: In ODT's SYM_INT32, a single scale value is shared across all elements of a tensor. This is called per-tensor quantization and is the simplest scheme. Per-channel quantization (one scale per output channel) can give better accuracy but is more complex to implement.

16.4.3 Integer Arithmetic for Matmul

When multiplying two SYM_INT32 tensors, the result has a combined scale: scale_C = scale_A × scale_B.

SYM_INT32 matmul math

```
// A[i] = mantissa_A[i] * scale_A
// B[j] = mantissa_B[j] * scale_B
// C[i][j] = sum_k(A[i][k] * B[k][j])
// = sum_k(mantissa_A[i][k] * scale_A * mantissa_B[k][j] * scale_B)
// = (sum_k(mantissa_A[i][k] * mantissa_B[k][j])) * scale_A * scale_B
// = int64_accumulator * (scale_A * scale_B)
// Why int64_t? Because int32_t * int32_t can overflow int32_t!
// int32 max = ~2.1 billion
// (2.1e9)^2 = 4.4e18, which overflows int32 but fits in int64
```

This is why <code>matmulSymInt32TensorsWithBias</code> uses <code>int64_t acc = 0</code> as the accumulator before converting back to int32_t with the new combined scale.

16.4.4 Quantization Noise and Stochastic Rounding

Every quantization step introduces a small error (the quantization noise). Regular rounding (round-half-away) is deterministic: the same values always round the same way. Over many training steps, this can cause a systematic bias.

Stochastic rounding (SR_HALF_AWAY in ODT) adds random noise before rounding: mantissa = round(real/scale + U(-0.5, 0.5)). This makes rounding errors random rather than systematic, which helps training converge better with low-bit quantization.

16.5 Sparse Training Mathematics (RigL)

Sparse neural networks have many weights set to exactly zero. A network that is 90% sparse (90% zeros) uses 10× less memory and can compute 10× faster if the zeros are exploited.

16.5.1 Magnitude Pruning

The simplest pruning criterion: remove the weights with the smallest absolute value. The intuition: small weights contribute little to the output and can be set to zero without much loss in accuracy.

Magnitude pruning math

```
// Given weights W and target sparsity ratio s (e.g., 0.9 for 90% sparse):
// 1. Sort |W[i]| in ascending order
// 2. Set to zero the smallest floor(n*s) weights
// 3. Keep the remaining ceil(n*(1-s)) weights
// Example: W = [0.5, -0.1, 0.8, -0.03, 0.2], s=0.4 (prune 40%)
// |W| sorted: [0.03, 0.1, 0.2, 0.5, 0.8]
// floor(5*0.4) = 2 → prune 2 smallest: 0.03 and 0.1
// Result mask: W[-0.1]=0, W[-0.03]=0, others remain
```

16.5.2 Gradient-Based Regrowth (RigL)

Magnitude pruning only prunes. RigL (Rigging the Lottery by Evci et al., 2020) also grows new connections where the gradient signal is largest. The criterion for growing: weights with the largest |gradient|.

The gradient of a pruned (zero) weight is meaningful — it tells us 'if this weight were non-zero, how much would it help?' Computing this gradient requires a special backward pass that also propagates through masked weights.

RigL update algorithm

```
// At step T (e.g., every 100 batches):
//
// 1. PRUNE: find k weights with smallest |magnitude|
// mask[i] = 0 for these k weights
// W[i] = 0 (explicitly zero)
//
// 2. GROW: find k positions with largest |gradient|
// (among currently masked/pruned positions)
// mask[i] = 1 (activate these weights)
// W[i] = 0 (start at zero, will grow during training)
//
// 3. Continue normal training with the new mask
// Active weights (mask=1): updated by SGD
// Pruned weights (mask=0): skipped by SGD
```

■ RigL GAP IN THIS FILE

RigL Gap in ODT: ODT has the foundation (sparsity_t struct stub, BOOL tensor type, RNG for random init) but lacks the riglUpdate() function and the mask-aware matmul and SGD. See Chapter 10 for the complete implementation checklist.

16.5.3 Why Sparse Training Is Hard on Embedded Systems

On a CPU or GPU, sparse operations use special hardware instructions or libraries. On the STM32 MCU, there is no hardware sparse support. The options are:

1. Dense storage with mask: Keep W in full [out, in] memory but multiply by the mask during forward/backward. No memory savings, but computation savings if you check the mask before each multiply (skip if mask=0).

2. Compressed Sparse Row (CSR): Store only non-zero values and their indices. Memory-efficient but adds index lookup overhead and more complex C code.

ODT currently targets option 1 (dense + mask) as the simpler starting point. The mask tensor uses 1 bit per weight via tensorBoolSet/Get, costing only n/8 bytes for an n-element weight matrix.

16.6 Why This Math Applies to Speaker Identification

The LibriSpeech speaker identification task requires the network to map audio features to speaker identities. Let's trace the math through one inference:

Full forward pass — mathematical view

```
// Input: x = audio feature vector, shape [1, 64] (MFCC features)
// Layer 1 (Linear 64→128):
// h1 = x × W1^T + b1
// shape: [1,64] × [64,128] + [128] = [1,128]
// Layer 2 (ReLU):
// h2 = max(0, h1) element-wise
// shape: [1,128]
// Layer 3 (Linear 128→num_speakers):
// logits = h2 × W2^T + b2
// shape: [1,128] × [128,num_speakers] + [num_speakers] = [1,num_speakers]
// Layer 4 (Softmax):
// probs = softmax(logits)
// probs[k] = exp(logits[k]) / sum(exp(logits[:]))
// shape: [1,num_speakers], all values in (0,1), sum=1
// Prediction: speaker = argmax(probs)
```

Cross-entropy loss during training: CE = -log(probs[true_speaker]). The network learns to assign high probability to the correct speaker for each audio segment.

After training is complete on the MCU, inference is just the forward pass without any gradient computation — fast and deterministic.

16.7 Numerical Concepts Reference

Concept	Value / Formula	Where Used in ODT
int32_t max	2,147,483,647 ≈ 2.1×10^9	SYM_INT32 mantissa range
int64_t max	9.2×10^18	matmulSymInt32 accumulator
float32 precision	~7 decimal digits	SGD, loss computation
float32 range	±3.4×10^38	Intermediate activations
exp overflow	exp(89) ≈ float max	Softmax uses max subtraction trick
qMax (8-bit SYM)	2^7 - 1 = 127	SYM_INT32 with qMaxBits=8
qMax (16-bit SYM)	2^15 - 1 = 32767	SYM_INT32 with qMaxBits=16
UINT32_MAX	4,294,967,295 = 2^32 - 1	xorshift32 state, rngBounded limit
2^24 float normalization	16,777,216	rngNextFloat: x>>8 then /2^24 → [0,1)

■ RELATIONS WITH OTHER FILES

Chapter 16 connects to: Chapter 4 (Quantization) — SYM_INT32 encoding and scale computation · Chapter 6 (Arithmetic) — matmul accumulator in int64_t, roundByMode · Chapter 8 (Training) — SGD update rule, loss functions forward and backward · Chapter 10 (RigL gap) — Pruning and regrowth math that needs to be implemented · Chapter 11 (MCU) — Why integer math is preferred on Cortex-M7

Chapter 17

End-to-End Worked Example

Building, training, and running a 2-layer MLP — step by step with real numbers

17.1 The Task: 3-Class Speaker Identification

To make the example concrete, we will build a tiny network that classifies audio feature vectors into one of 3 speaker identities (A, B, C). This is a simplified version of the full LibriSpeech task.

Network architecture

```
Input: 4 features (simplified MFCC coefficients)
Linear 1: 4 → 8 neurons, with bias
ReLU 1: element-wise activation
Linear 2: 8 → 3 neurons (3 speaker classes), with bias
Softmax: normalize to probabilities
Loss: CrossEntropy
```

Parameter count: Linear1 weights = 8×4 = 32, bias = 8. Linear2 weights = 3×8 = 24, bias = 3. Total: 67 parameters. This is tiny — the full model for LibriSpeech would have thousands.

17.2 Step 1 — Memory Layout on the MCU

Before writing any code, plan where each tensor lives in SRAM. The STM32F746ZG has 320 KB RAM. For our tiny example:

	Type	Bytes	Purpose
	float32	128	Linear1 weights
	float32	128	Linear1 weight
	float32	32	Linear1 bias
	float32	32	Linear1 bias gr
	float32	96	Linear2 weights
	float32	96	Linear2 weight
	float32	12	Linear2 bias
	float32	12	Linear2 bias gr
	float32	16	Input feature ve
	float32	32	After Linear1
	float32	32	After ReLU1
	float32	12	After Linear2 (l
	float32	12	After Softmax
	float32	32	Gradient at h2
	float32	32	Gradient at h1
	float32	12	One-hot target
		716 B	Well within 320

17.3 Step 2 — Initialization Code

All tensors and layer configs must be initialized before training begins. Here is the complete initialization sequence:

main.c — initialization

```
#include <string.h>
#include 'Tensor.h'
#include 'Layer.h'
#include 'Linear.h'
#include 'Relu.h'
#include 'LossFunction.h'
#include 'Sgd.h'
#include 'Rng.h'
// ■■ Static buffers (in .bss — zeroed at startup) ■■■■■■■■■■■■■■■■■■■
static float w1Data[8*4], g1Data[8*4];
static float b1Data[8], gb1Data[8];
static float w2Data[3*8], g2Data[3*8];
static float b2Data[3], gb2Data[3];
static float inputBuf[4], h1Buf[8], h2Buf[8];
static float outBuf[3], probBuf[3];
static float dh2Buf[8], dh1Buf[8];
static float targetBuf[3];
// ■■ Tensor objects (metadata only, point into buffers) ■■■■■■■■■■■■■
static tensor_t w1, g1, b1, gb1, w2, g2, b2, gb2;
static tensor_t input, h1, h2, output, probs, dh2, dh1, target;
// ■■ Parameter wrappers ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
static parameter_t p1, p2, pb1, pb2;
// ■■ Layer configs ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
static linearConfig_t lin1cfg, lin2cfg;
static reluConfig_t relu1cfg;
static layer_t lin1, lin2, relu1;
// ■■ RNG for weight init ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
static rng32_t rng;
void initNetwork(void) {
rngSetSeed(&rng, 42);
// Shape setup for all tensors
setShape(&w1.shape, 2, (size_t[]){8, 4});
setShape(&g1.shape, 2, (size_t[]){8, 4});
setShape(&b1.shape, 1, (size_t[]){8});
setShape(&gb1.shape, 1, (size_t[]){8});
setShape(&w2.shape, 2, (size_t[]){3, 8});
// ... (similar for all other tensors)
// Assign data pointers
w1.data.dataPTR = w1Data; w1.data.numberOfElements = 32;
g1.data.dataPTR = g1Data; g1.data.numberOfElements = 32;
// ... (similar for all other tensors)
// Xavier weight initialization: w ~ Uniform(-sqrt(6/(fan_in+fan_out)))
float lim1 = 0.5f; // sqrt(6/(4+8)) ≈ 0.707, simplified to 0.5
for (int i = 0; i < 32; i++)
w1Data[i] = (rngNextFloat(&rng) - 0.5f) * 2.0f * lim1;
// Wire up parameter structs
p1.param = w1; p1.grad = g1;
pb1.param = b1; pb1.grad = gb1;
// Configure layers
lin1cfg.weights = p1;
lin1cfg.bias = pb1;
lin1cfg.hasBias = true;
lin1cfg.freezeWeights = false;
lin1cfg.freezeBias = false;
lin1cfg.isTraining = true;
lin1cfg.inputCache = NULL; // filled during forward pass
lin1.type = LAYER_LINEAR;
lin1.config.linear = &lin1cfg;
// Similar for relu1, lin2...
}
```

static		All arrays and structs are static — they live in .bss/.data sections in flash/SRAM, not on the stack. This avoids stack overflow for large buffers.
rngSetSeed(&rng, 42)		Initialize the xorshift32 RNG with seed 42 for reproducibility.
Xavier init		A well-known initialization scheme that scales initial weights to prevent activations from vanishing or exploding. Scale = sqrt(6/(fan_in+fan_out)).

17.4 Step 3 — One Training Step with Real Numbers

Let's trace a complete training step with specific numbers. We use a single sample (batch_size=1) for clarity.

Initial state (after random init, simplified):

Initial weights and input

```
Input (MFCC features): x = [0.5, -0.3, 0.8, 0.1]
W1 (8x4) - showing first 3 rows for brevity:
[0.2, 0.5, -0.3, 0.1] // row 0 → neuron h1[0]
[-0.4, 0.1, 0.6, -0.2] // row 1 → neuron h1[1]
[0.3, -0.2, 0.1, 0.4] // row 2 → neuron h1[2]
... (5 more rows)
b1 (8): [0.1, -0.1, 0.05, 0.0, 0.1, -0.05, 0.02, 0.0]
Target (speaker A = class 0, one-hot): [1, 0, 0]
```

17.4.1 Forward Pass — Layer by Layer

Linear1 forward: h1 = W1 × x + b1

```
h1[0] = 0.2*0.5 + 0.5*(-0.3) + (-0.3)*0.8 + 0.1*0.1 + 0.1
= 0.10 - 0.15 - 0.24 + 0.01 + 0.10
= -0.18
h1[1] = (-0.4)*0.5 + 0.1*(-0.3) + 0.6*0.8 + (-0.2)*0.1 + (-0.1)
= -0.20 - 0.03 + 0.48 - 0.02 - 0.10
= 0.13
h1[2] = 0.3*0.5 + (-0.2)*(-0.3) + 0.1*0.8 + 0.4*0.1 + 0.05
= 0.15 + 0.06 + 0.08 + 0.04 + 0.05
= 0.38
// (complete h1 for all 8 neurons in practice)
h1 = [-0.18, 0.13, 0.38, 0.05, -0.22, 0.31, -0.07, 0.44]
```

ReLU1 forward: h2 = max(0, h1)

```
h2[0] = max(0, -0.18) = 0.00 ← neuron is OFF (dead for this sample)
h2[1] = max(0, 0.13) = 0.13
h2[2] = max(0, 0.38) = 0.38
h2[3] = max(0, 0.05) = 0.05
h2[4] = max(0, -0.22) = 0.00 ← neuron is OFF
h2[5] = max(0, 0.31) = 0.31
h2[6] = max(0, -0.07) = 0.00 ← neuron is OFF
h2[7] = max(0, 0.44) = 0.44
h2 = [0.00, 0.13, 0.38, 0.05, 0.00, 0.31, 0.00, 0.44]
```

Linear2 forward: logits = W2 × h2 + b2

```
// W2 shape [3,8], h2 shape [8] → logits shape [3]
// Assume W2 and b2 initialized similarly to W1
logits[0] = W2[0,:] · h2 + b2[0] = 0.25 // speaker A score
logits[1] = W2[1,:] · h2 + b2[1] = 0.40 // speaker B score
logits[2] = W2[2,:] · h2 + b2[2] = 0.10 // speaker C score
logits = [0.25, 0.40, 0.10]
```

Softmax forward: probs = softmax(logits)

```
maxVal = 0.40 (for numerical stability)
shifted = [0.25-0.40, 0.40-0.40, 0.10-0.40]
= [-0.15, 0.00, -0.30]
exp_vals = [exp(-0.15), exp(0.00), exp(-0.30)]
= [0.861, 1.000, 0.741]
sum = 0.861 + 1.000 + 0.741 = 2.602
probs = [0.861/2.602, 1.000/2.602, 0.741/2.602]
= [0.331, 0.384, 0.285]
// Network assigns 33.1% to speaker A, 38.4% to B, 28.5% to C
// True class is A (0) — not great prediction, but we just started training
```

CrossEntropy loss

```
target = [1, 0, 0] (speaker A)
CE = -(1*log(0.331) + 0*log(0.384) + 0*log(0.285))
= -log(0.331)
= 1.107
// High loss — the network is not confident about the correct speaker
```

17.4.2 Backward Pass with Real Numbers

CrossEntropy+Softmax backward: d_logits = probs - target

```
d_logits = [0.331 - 1, 0.384 - 0, 0.285 - 0]
= [-0.669, 0.384, 0.285]
// Meaning: logits[0] should increase (negative gradient → decrease loss)
// logits[1] and [2] should decrease
```

Linear2 backward: dW2, db2, dh2

```
// dW2[i][j] = d_logits[i] * h2[j]
dW2[0][:] = -0.669 * [0.00, 0.13, 0.38, 0.05, 0.00, 0.31, 0.00, 0.44]
= [0.00, -0.087, -0.254, -0.033, 0.00, -0.207, 0.00, -0.294]
dW2[1][:] = 0.384 * h2 = [0.00, 0.050, 0.146, 0.019, 0.00, 0.119, 0.00, 0.169]
dW2[2][:] = 0.285 * h2 = [0.00, 0.037, 0.108, 0.014, 0.00, 0.088, 0.00, 0.125]
db2 = d_logits = [-0.669, 0.384, 0.285]
// dh2 = W2^T × d_logits (propagate loss upstream)
dh2[j] = sum_i(W2[i][j] * d_logits[i]) for j=0..7
// This gives the gradient at each h2 neuron
```

ReLU1 backward: dh1 = dh2 * (h1 > 0)

```
// Only neurons that were ON in forward get gradient
// h1 = [-0.18, 0.13, 0.38, 0.05, -0.22, 0.31, -0.07, 0.44]
// mask = [0, 1, 1, 1, 0, 1, 0, 1]
dh1[0] = dh2[0] * 0 = 0.0 ← h1[0] was negative, gradient blocked
dh1[1] = dh2[1] * 1 = dh2[1] ← passes through
dh1[2] = dh2[2] * 1 = dh2[2]
// etc.
```

Linear1 backward: dW1, db1

```
// dW1[i][j] = dh1[i] * x[j] (input was x = [0.5, -0.3, 0.8, 0.1])
// db1[i] = dh1[i]
// (dInput computed but not used — x is the network input, no upstream layer)
```

17.4.3 SGD Weight Update

SGD update (learning_rate = 0.01)

```
// For each parameter w with gradient dw:
w_new = w - 0.01 * dw
// Example — W1 row 0 first element:
// W1[0][0]_old = 0.20
// dW1[0][0] = dh1[0] * x[0] = 0.0 * 0.5 = 0.0 (neuron 0 was OFF!)
// W1[0][0]_new = 0.20 - 0.01 * 0.0 = 0.20 (no change)
// W1[1][0] (neuron 1 was ON):
// W1[1][0]_old = -0.40
// dW1[1][0] = dh1[1] * x[0] = dh2[1] * 0.5
// W1[1][0]_new = -0.40 - 0.01 * (dh2[1] * 0.5)
// After one step, loss will be slightly lower.
// After many steps (hundreds or thousands), network converges.
```

17.5 Step 4 — The Training Loop in C

Now let's write the actual C training loop that ties everything together:

main.c — training loop

```
void trainEpoch(dataLoader_t *loader, sgd_t *sgd,
layer_t *lin1, layer_t *relu1,
layer_t *lin2, lossFunc_t *lossFn,
tensor_t *input, tensor_t *h1, tensor_t *h2,
tensor_t *output, tensor_t *dh2, tensor_t *dh1,
tensor_t *target) {
size_t numBatches = loader->getDatasetSize(loader) / loader->batchSize;
for (size_t batch = 0; batch < numBatches; batch++) {
// 1. Zero all gradients
sgdZeroGrad(sgd, lin1->config.linear);
sgdZeroGrad(sgd, lin2->config.linear);
// 2. Load one batch
loader->getBatch(loader, batch, input, target);
// 3. Forward pass
layerFunctions[LAYER_LINEAR].forward(lin1, input, h1);
layerFunctions[LAYER_RELU].forward(relu1, h1, h2);
layerFunctions[LAYER_LINEAR].forward(lin2, h2, output);
// 4. Compute loss (includes softmax inside)
float loss = lossFn->forward(lossFn, output, target);
// 5. Backward pass (reverse order)
lossFn->backward(lossFn, output, target, dh2);
layerFunctions[LAYER_LINEAR].backward(lin2, h2, dh2, dh1);
layerFunctions[LAYER_RELU].backward(relu1, dh1, dh1); // in-place
layerFunctions[LAYER_LINEAR].backward(lin1, input, dh1, NULL);
// 6. Update weights
sgdStep(sgd, lin1->config.linear);
sgdStep(sgd, lin2->config.linear);
if (batch % 10 == 0)
printf('Batch %zu loss: %.4f\n', batch, loss);
}
}
```

sgdZeroGrad		CRITICAL: must be called at the start of every batch. Otherwise gradients from previous batches accumulate, corrupting the update.
getBatch		Fills the input and target tensors with data from the dataset. In the DataLoader, this calls getSample in a loop to fill the batch.
NULL for dInput in lin1		The first layer has no upstream layer to send gradients to, so we pass NULL. linearBackward checks for NULL before computing dInput.
printf on MCU		On the STM32, printf can be redirected to UART for debugging. This is usually done by overriding the _write() system call to use HAL_UART_Transmit.

17.6 Step 5 — Serialization and Inference

After training, the weights need to be saved to flash so they survive power cycles, or transmitted to a host PC for analysis.

Saving trained weights with serializeTensor

```
// Write trained weights to UART (connected to PC)
// PC-side Python script reads and saves as .npy
void saveModel(layer_t *lin1, layer_t *lin2) {
serializeTensor(&lin1->config.linear->weights.param);
serializeTensor(&lin1->config.linear->bias.param);
serializeTensor(&lin2->config.linear->weights.param);
serializeTensor(&lin2->config.linear->bias.param);
}
// For inference (no gradients needed):
void runInference(tensor_t *x, tensor_t *probs) {
lin1cfg.isTraining = false; // disable inputCache saving
layerFunctions[LAYER_LINEAR].forward(&lin1, x, &h1);
layerFunctions[LAYER_RELU].forward(&relu1, &h1, &h2);
layerFunctions[LAYER_LINEAR].forward(&lin2, &h2, &outBuf_t);
softmaxForward(NULL, &outBuf_t, probs);
// probs now holds probabilities for each speaker class
}
```

17.7 Common Mistakes and How to Avoid Them

Mistake	Symptom	Fix
Forgot sgdZeroGrad	Loss oscillates, gradients grow	Call sgdZeroGrad before each batch
Wrong layer order in backward	Loss doesn't decrease	Backward must be exact reverse of forward
transposeTensor not in pairs	Wrong matmul shapes, garbage output	Every transposeTensor must be undone
Scale not copied in ReLU	Next layer decodes wrong values	Copy scale from input to output quantization
inputCache is NULL in backward	Segfault / wrong gradients	Check isTraining=true and forward called first
VLA too large	Stack overflow, MCU crashes	Use static buffers for large tensors
float/int32 mix without casting	Compiler warning, wrong results	Explicit cast: (float)(int32_val)
log(0) in cross-entropy	NaN loss	Guard: max(pred, EPSILON) before log

■ RELATIONS WITH OTHER FILES

Chapter 17 connects to: Chapters 7-8 (Layers + Training) — All layer and SGD functions used in the loop · Chapter 6 (Arithmetic) — Matmul and add operations underneath each layer · Chapter 9 (Support) — RNG for weight init, serialization for saving weights · Chapter 11 (MCU) — Static buffer strategy, UART printf, STM32 startup · Chapter 16 (Math) — Xavier init formula, cross-entropy gradient derivation

Chapter 18

Embedded Systems Deep Dive

STM32, ARM Cortex-M7, bare-metal C, and how the MCU executes your code

18.1 What Is an Embedded System?

An embedded system is a computer designed to perform a specific task, built into a larger device. Unlike a laptop or desktop, it typically has no operating system (or a very minimal one), limited resources, and must run reliably for years without human intervention.

The STM32 Nucleo-F746ZG is a development board containing an STM32F746ZG microcontroller — a single chip with CPU, RAM, flash, and peripherals all integrated together. Your C code runs directly on the metal.

Component	Specification	Relevance to ODT
CPU	ARM Cortex-M7 @ 216 MHz	Executes all C code
RAM (SRAM)	320 KB total	All tensors, stack, globals
Flash	1 MB	Code + constant data
FPU	Double-precision FPU	Float operations without software emulation
DSP	DSP instructions	SIMD-style operations (not used in ODT)
UART	Multiple UARTs	Serial communication, printf debugging
Timers	Multiple 16/32-bit	Training loop timing
DMA	DMA controller	Memory-to-memory transfers
I-Cache/D-Cache	16 KB instruction, 16 KB data	Speeds up repeated code/data access
Bus width	32-bit AHB/APB	All memory accesses are 32-bit aligned

18.2 ARM Cortex-M7 Architecture

The ARM Cortex-M7 is a 32-bit RISC (Reduced Instruction Set Computer) processor. Understanding its architecture helps explain why ODT makes certain design choices.

18.2.1 Registers

The CPU has 16 general-purpose 32-bit registers (R0–R15) plus status registers. All computation happens in registers — data must be loaded from memory into a register before it can be used, then stored back.

ARM Cortex-M7 key registers

```
R0-R3: Function arguments and return values
R4-R11: Callee-saved registers (function must preserve these)
R12: Intra-procedure-call scratch register
R13 (SP): Stack Pointer — points to top of current stack frame
R14 (LR): Link Register — return address for function calls
R15 (PC): Program Counter — address of next instruction to execute
xPSR: Status register — zero flag, negative flag, carry flag, etc.
FPU registers (for float operations):
S0-S31: 32 single-precision float registers (32-bit each)
D0-D15: 16 double-precision float registers (64-bit, alias S pairs)
FPSCR: FPU status/control register
```

When the compiler generates code for <code>float a = b * c</code>, it emits VLDR instructions to load b and c into S0 and S1, then VMUL.F32 to multiply them and store the result in another S register, then VSTR to save back to RAM.

18.2.2 The Pipeline

The Cortex-M7 has a 6-stage pipeline: Fetch → Decode → Issue → Execute → Write-back → Retire. While one instruction is executing, the next is being decoded, and the one after that is being fetched. This overlap is called pipelining.

Pipeline hazards occur when an instruction depends on the result of the previous one. The CPU must stall (wait) for the first result. This is why tight loops with data dependencies (like accumulating a sum) can be slower than expected.

═■ WORKED EXAMPLE

Pipeline in matrix multiply inner loop: acc += A[i][k] * B[k][j] Cycle 1: Load A[i][k] into S0 Cycle 2: Load B[k][j] into S1, (waiting for S0 to be ready) Cycle 3: VMUL.F32 S2, S0, S1 (multiply) Cycle 4: VADD.F32 S3, S3, S2 (accumulate — must wait for VMUL result) Cycle 5: Increment k, check loop condition Total: ~5 cycles per inner loop iteration at 216 MHz → ~43M inner iterations/second

18.2.3 Memory Architecture

The Cortex-M7 uses a Harvard architecture variant with separate instruction and data buses. This means it can fetch the next instruction while loading/storing data for the current instruction simultaneously.

STM32F746ZG memory map (simplified)

```
0x00000000 - 0x000FFFFF: Flash (1 MB) — code + const data
0x20000000 - 0x2004FFFF: DTCM SRAM (320 KB) — fastest RAM
0x40000000 - 0x5FFFFFFF: Peripheral registers (UART, timers, GPIO, etc.)
0xE0000000 - 0xFFFFFFFF: System/debug registers
Stack grows DOWN from top of SRAM (0x2004FFFF → 0x20040000)
.bss + .data grow UP from bottom of SRAM (0x20000000 → ...)
```

18.3 Memory Sections in Detail

Your compiled C program is divided into named memory sections. Understanding these is critical for embedded programming.

18.3.1 The .text Section (Flash)

All compiled C functions and constant data go into the .text section, stored in flash memory. Flash is read-only at runtime (you can't write to it during normal execution). It survives power cycles.

What lives in .text

```
// All function code:
void matmulFloat32TensorsWithBias(...) { ... } // compiled to ARM opcodes in flash
void linearForwardFloat(...) { ... }
// String literals:
const char *name = 'ODT'; // 'ODT\0' stored in flash
// Constant arrays:
const float lut[256] = { 0.1, 0.2, ... }; // lookup table in flash
```

18.3.2 The .data Section (RAM — Initialized Globals)

Global and static variables that have initial values go into .data. The initial values are stored in flash and copied to RAM at startup.

.data example

```
float learningRate = 0.01f; // in .data — initial value 0.01
int epochCount = 0; // in .bss — zero initial value
// At startup, the MCU's reset handler copies the flash image
// of .data into RAM. This is why your globals have their initial values.
```

18.3.3 The .bss Section (RAM — Zero-Initialized)

Variables declared without an explicit initial value default to zero. These go in .bss (Block Started by Symbol). At startup, the reset handler calls memset(0) over the entire .bss region. This is cheap in flash because we only store the size, not 320KB of zeros.

.bss example — ODT static buffers

```
// All of these go in .bss — zero at startup, no flash storage needed
static float w1Data[8*4]; // 128 bytes in .bss
static float g1Data[8*4]; // 128 bytes in .bss
static tensor_t w1; // tensor metadata in .bss
static linearConfig_t lin1cfg; // layer config in .bss
// Cost in flash: ~4 bytes to record the size
// Cost in RAM: 32*4 = 128 bytes at runtime
```

18.3.4 The Stack

Every function call creates a stack frame — a region of memory for local variables and saved registers. The stack grows downward from the top of SRAM. Stack frames are automatically created on function entry and destroyed on return.

Stack frame example

```
void myFunction(int x) {
float localArray[64]; // 256 bytes on stack (VLA style if runtime-sized)
int count = 0; // 4 bytes on stack
// ...
} // stack frame is automatically freed here
// Total stack depth example for ODT training loop:
main() → 64 bytes
trainEpoch() → 32 bytes
layerFunctions[].forward() → 32 bytes
matmulFloat32TensorsWithBias() → 128 bytes
matmulIntCore() → 48 bytes
// Total stack depth: ~304 bytes
// STM32 has 320KB RAM — stack and static buffers must not overlap
```

■■ IMPORTANT

Stack Overflow on MCUs: On a desktop OS, stack overflow triggers a segfault. On the STM32, stack overflow silently corrupts other data (it overwrites .bss or .data). This causes bizarre behavior that is very hard to debug. Always calculate maximum stack depth for embedded code.

18.4 Startup Sequence — Before main()

Most developers assume C programs start at main(). On embedded systems, a lot happens before main() is called.

STM32 startup sequence

```
// 1. CPU resets: PC = address stored in vector table at 0x00000004
// (this is the Reset_Handler address)
// 2. Reset_Handler (in startup_stm32f746zg.s, written in assembly):
Reset_Handler:
// Set stack pointer to top of SRAM
LDR R0, =_estack
MSR MSP, R0
// Copy .data section from flash to RAM
LDR R0, =_sdata // destination in RAM
LDR R1, =_edata // end of .data in RAM
LDR R2, =_sidata // source in flash
CopyLoop:
LDR R3, [R2], #4
STR R3, [R0], #4
CMP R0, R1
BLT CopyLoop
// Zero out .bss section
LDR R0, =_sbss
LDR R1, =_ebss
MOV R2, #0
ZeroLoop:
STR R2, [R0], #4
CMP R0, R1
BLT ZeroLoop
// 3. Call SystemInit() — configure PLL, flash wait states
// 4. Call main()
BL main
```

This is why your static float w1Data[] array is automatically zeroed — the startup code clears .bss before main() runs.

18.5 Clock Configuration — Running at 216 MHz

The STM32F746ZG has an internal 16 MHz RC oscillator as the default clock. To reach 216 MHz, it uses a Phase-Locked Loop (PLL) which multiplies the input frequency.

SystemClock_Config (STM32CubeMX generated)

```
void SystemClock_Config(void) {
RCC_OscInitTypeDef osc = {0};
RCC_ClkInitTypeDef clk = {0};
// Enable HSE (external crystal, usually 8 or 25 MHz)
osc.OscillatorType = RCC_OSCILLATORTYPE_HSE;
osc.HSEState = RCC_HSE_BYPASS;
osc.PLL.PLLState = RCC_PLL_ON;
osc.PLL.PLLSource = RCC_PLLSOURCE_HSE;
// PLL multiplier: 8 MHz * (N/M) / P = 216 MHz
osc.PLL.PLLM = 4; // divide by 4 → 2 MHz
osc.PLL.PLLN = 216; // multiply by 216 → 432 MHz
osc.PLL.PLLP = 2; // divide by 2 → 216 MHz
osc.PLL.PLLQ = 9; // USB clock = 432/9 = 48 MHz
HAL_RCC_OscConfig(&osc);
// Set flash wait states (needed at high clock speeds)
__HAL_FLASH_SET_LATENCY(FLASH_LATENCY_7);
clk.ClockType = RCC_CLOCKTYPE_SYSCLK | ...
clk.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
clk.AHBCLKDivider = RCC_SYSCLK_DIV1; // AHB = 216 MHz
clk.APB1CLKDivider = RCC_HCLK_DIV4; // APB1 = 54 MHz
clk.APB2CLKDivider = RCC_HCLK_DIV2; // APB2 = 108 MHz
HAL_RCC_ClockConfig(&clk, FLASH_LATENCY_7);
}
```

HAL		Hardware Abstraction Layer — STM32's C driver library. HAL functions have names like HAL_RCC_OscConfig, HAL_UART_Transmit. They wrap the raw register writes into readable function calls.

18.6 UART — Debugging and Data Transfer

UART (Universal Asynchronous Receiver/Transmitter) is a serial communication protocol. On the Nucleo board, one UART is connected to the ST-Link debugger chip, which bridges to USB. This creates a virtual COM port on the PC.

This is how ODT's <code>serializeTensor</code> sends weight data to the PC, and how <code>printf</code> outputs debugging information.

UART setup and use

```
static UART_HandleTypeDef huart3; // UART3 connected to ST-Link on Nucleo
void MX_USART3_UART_Init(void) {
huart3.Instance = USART3;
huart3.Init.BaudRate = 115200; // 115200 bits per second
huart3.Init.WordLength = UART_WORDLENGTH_8B;
huart3.Init.StopBits = UART_STOPBITS_1;
huart3.Init.Parity = UART_PARITY_NONE;
huart3.Init.Mode = UART_MODE_TX_RX;
HAL_UART_Init(&huart3);
}
// Send data:
uint8_t data[] = {0x01, 0x02, 0xFF};
HAL_UART_Transmit(&huart3, data, 3, HAL_MAX_DELAY);
// 115200 baud → ~11520 bytes/second
// Sending 128-byte weight matrix: ~11 ms
// Redirect printf to UART:
int _write(int file, char *ptr, int len) {
HAL_UART_Transmit(&huart3, (uint8_t*)ptr, len, HAL_MAX_DELAY);
return len;
}
// Now printf works over UART!
```

115200 baud		Baud rate = bits per second. With 8 data bits + 1 stop bit = 9 bits per byte. At 115200 baud: 115200/9 ≈ 12800 bytes/second.
HAL_MAX_DELAY		Wait until the transmission is complete. On an MCU without an OS, blocking is usually fine for debugging.

18.7 Interrupts and the NVIC

An interrupt is a hardware signal that pauses the CPU's current task and calls a special function (the Interrupt Service Routine, or ISR). Common interrupt sources: UART receive, timer overflow, GPIO edge.

ODT's training loop is designed to run in the main loop (not in an ISR). But the data loader could use a UART receive interrupt to gather audio samples while computation runs.

Timer interrupt example — measure training loop time

```
volatile uint32_t timerTicks = 0; // volatile! Modified by ISR
// ISR — called every 1 ms by Timer 6
void TIM6_DAC_IRQHandler(void) {
if (__HAL_TIM_GET_FLAG(&htim6, TIM_FLAG_UPDATE)) {
__HAL_TIM_CLEAR_FLAG(&htim6, TIM_FLAG_UPDATE);
timerTicks++;
}
}
// In main loop:
uint32_t start = timerTicks;
layerFunctions[LAYER_LINEAR].forward(&lin1, &input, &h1);
uint32_t elapsed = timerTicks - start;
printf('Linear forward: %u ms\n', elapsed);
```

■ C CONCEPT: The volatile Keyword

When a variable can be modified by an interrupt (or any external event), it must be declared volatile. Without volatile, the compiler may cache the variable in a register and never re-read it from RAM, missing updates made by the ISR. Always use volatile for ISR-shared variables.

18.8 The FPU — Why Float Is Fast on Cortex-M7

Older Cortex-M0 and M0+ processors have no hardware FPU — float operations are emulated in software, taking ~100 cycles each. The Cortex-M7 has a double-precision FPU that handles float operations in 1-3 cycles.

FPU enabling (in system_stm32f7xx.c)

```
// Enable FPU — must be done before any float operations
#define FPU_CPACR (*((volatile uint32_t*)0xE000ED88))
void FPU_Enable(void) {
FPU_CPACR |= (0xF << 20); // CP10 and CP11 = full access
__DSB(); // Data Synchronization Barrier
__ISB(); // Instruction Synchronization Barrier
}
// With FPU enabled:
// float a = b * c; → VMUL.F32 S0, S1, S2 (1-3 cycles)
// Without FPU (software emulation):
// float a = b * c; → BL __aeabi_fmul (~80-100 cycles)
```

The FPU is why ODT can run full float32 neural networks on the STM32. Without it, even a small matmul would be prohibitively slow.

18.9 Instruction and Data Cache

The Cortex-M7 has separate 16 KB instruction cache (I-cache) and 16 KB data cache (D-cache). Flash has 7-cycle wait states at 216 MHz, so without the I-cache, instruction fetches would take 7× longer.

The D-cache is especially important for ODT: the matmul inner loop repeatedly reads the same rows of the weight matrix. With D-cache, these reads come from fast cache (1-4 cycles) rather than slow SRAM (3-4 cycles, but still faster than flash at 216 MHz).

Enabling caches (in main.c)

```
SCB_EnableICache(); // Enable instruction cache
SCB_EnableDCache(); // Enable data cache
// These are usually the first two lines in main()
// Typical speedup: 2-4x for instruction-heavy code
```

■■ IMPORTANT

Cache Coherency: If you use DMA (Direct Memory Access) to transfer data without going through the CPU, the D-cache can become stale. You must explicitly invalidate or clean the cache after DMA transfers. ODT uses memcpy (CPU-driven), so cache coherency is not an issue in the current implementation.

18.10 Linking and the Linker Script

The linker script (STM32F746ZGTx_FLASH.ld) tells the linker where to place each section in the memory map. This is what ensures your code goes to flash and your buffers go to SRAM.

Linker script excerpt (STM32F746ZGTx_FLASH.ld)

```
MEMORY {
RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 320K
FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 1024K
}
SECTIONS {
/* Code and read-only data go to flash */
.text : {
KEEP(*(.isr_vector)) /* Vector table must be first */
*(.text) /* All .text sections */
*(.rodata) /* Constant data */
} > FLASH
/* Initialized globals: stored in flash, copied to RAM at startup */
.data : {
_sdata = .;
*(.data)
_edata = .;
} > RAM AT > FLASH
/* Zero-initialized globals: only in RAM */
.bss : {
_sbss = .;
*(.bss)
*(COMMON)
_ebss = .;
} > RAM
/* Stack at top of RAM */
_estack = ORIGIN(RAM) + LENGTH(RAM);
}
```

AT > FLASH		The .data section lives in RAM at runtime (> RAM) but is stored in flash during programming (AT > FLASH). The startup code copies it.
KEEP(*(.isr_vector))		The vector table (interrupt handler addresses) must be at address 0x08000000 (start of flash). KEEP prevents the linker from removing it even if it looks unused.
_sdata, _edata, _sbss, _ebss, _estack		These symbols are used by the Reset_Handler in startup assembly to know where to copy .data and zero .bss.

18.11 Summary: Why Embedded C Is Different

Feature	Desktop C	Embedded C (STM32)
Memory	Virtual memory, GBs available	Physical only, 320 KB fixed
OS	Linux/Windows manages resources	Bare-metal, you manage everything
malloc/free	Generally safe	Avoid — fragmentation causes crashes
printf	Writes to terminal	Must redirect to UART
Stack overflow	Segfault + OS recovery	Silent corruption, very hard to debug
Interrupts	Rarely need to think about	Must use volatile, avoid race conditions
Float operations	Always hardware-accelerated	Need to enable FPU explicitly
Startup	OS handles initialization	Reset_Handler copies .data, zeros .bss
Debugging	gdb, IDE, print statements	SWD/JTAG probe, UART printf, blinking LED
Power	Not a concern	May need low-power modes for battery life

■ RELATIONS WITH OTHER FILES

Chapter 18 connects to: Chapter 11 (MCU chapter) — Static buffers, startup sequence, SRAM layout · Chapter 13 (C language) — volatile, static, memory sections in C · Chapter 17 (Worked example) — printf via UART, static buffers for tensors · Chapter 9 (Support/RNG) — rng32_t using DTCM SRAM for fast access · Chapter 6 (Arithmetic) — FPU acceleration for matmul float operations

Chapter 19

Debugging, Testing, and Extending ODT

How to find bugs, validate correctness, and add new layers or features

19.1 Debugging Embedded C — General Strategy

Debugging embedded code is harder than debugging desktop code because you cannot easily inspect memory or stop the program without special hardware. The main tools are: UART printing, LED blinking, hardware debugger (SWD/JTAG), and the ODT-specific PRINT_DEBUG/PRINT_ERROR macros.

19.1.1 PRINT_DEBUG and PRINT_ERROR

ODT's <code>Common.h</code> defines conditional print macros. When <code>#define DEBUG</code> is set in the build, they expand to printf. In release builds (no DEBUG), they expand to nothing — zero overhead.

Common.h — debug macros

```
#ifdef DEBUG
#define PRINT_DEBUG(fmt, ...) printf('[DEBUG] ' fmt, ##__VA_ARGS__)
#define PRINT_ERROR(fmt, ...) printf('[ERROR] ' fmt, ##__VA_ARGS__)
#else
#define PRINT_DEBUG(fmt, ...) do { } while(0)
#define PRINT_ERROR(fmt, ...) do { } while(0)
#endif
// Usage in code:
PRINT_DEBUG('matmul: A[%zu,%zu] B[%zu,%zu]\n',
A->shape.dimensions[0], A->shape.dimensions[1],
B->shape.dimensions[0], B->shape.dimensions[1]);
```

#ifdef DEBUG		The compiler checks if DEBUG was defined. If using GCC: 'gcc -DDEBUG file.c' defines it. In STM32CubeIDE: add DEBUG to Project Properties → C/C++ Build → Settings → Preprocessor → Defined symbols.
##__VA_ARGS__		The ## before __VA_ARGS__ suppresses the trailing comma if no variadic arguments are passed — allows PRINT_DEBUG('message') without ','.

19.1.2 Adding Debug Prints to Trace a Bug

When a layer produces wrong output, add debug prints to trace the data flow:

Debug tracing example

```
// Add to linearForwardFloat to trace the matmul inputs:
void linearForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {
linearConfig_t *cfg = layer->config.linear;
PRINT_DEBUG('Linear forward: input[%zu,%zu] weights[%zu,%zu]\n',
input->shape.dimensions[0], input->shape.dimensions[1],
cfg->weights.param.shape.dimensions[0],
cfg->weights.param.shape.dimensions[1]);
float *inPtr = (float*) input->data.dataPTR;
PRINT_DEBUG(' input[0] = %.4f, input[1] = %.4f\n',
inPtr[0], inPtr[1]);
// ... rest of function
float *outPtr = (float*) output->data.dataPTR;
PRINT_DEBUG(' output[0] = %.4f\n', outPtr[0]);
}
```

Common causes of wrong output: wrong shape in setShape (e.g., rows and cols swapped), data pointer pointing to wrong buffer, scale not initialized (stays 0 → all values 0), or transposeTensor called odd number of times.

19.2 The TRACK_INSTRUCTIONS Macro

ODT includes a special debug mode for counting arithmetic operations. When <code>#define TRACK_INSTRUCTIONS</code> is active, the code counts multiplications and additions per forward pass.

Arithmetic.c — TRACK_INSTRUCTIONS usage

```
#ifdef TRACK_INSTRUCTIONS
static size_t g_numMul = 0;
static size_t g_numAdd = 0;
#endif
// In matmul inner loop:
for (size_t k = 0; k < K; k++) {
acc += (int64_t)A_val * B_val;
#ifdef TRACK_INSTRUCTIONS
g_numMul++;
g_numAdd++;
#endif
}
// After forward pass:
#ifdef TRACK_INSTRUCTIONS
PRINT_DEBUG('Operations: %zu mul, %zu add\n', g_numMul, g_numAdd);
#endif
```

This is used to estimate how many sparse operations RigL would save (if mask zeros are skipped, that many multiplications are avoided).

19.3 Hardware Debugger — SWD and GDB

The Nucleo board includes an ST-Link debugger. It connects to the STM32's SWD (Serial Wire Debug) pins and lets you use GDB to inspect the MCU's state while it is running.

GDB commands for embedded debugging

```
# Connect to target (in STM32CubeIDE or arm-none-eabi-gdb):
(gdb) target remote localhost:3333 # openocd bridge
(gdb) monitor reset halt
# Set breakpoints:
(gdb) break linearForwardFloat
(gdb) break Matmul.c:45
# Inspect variables:
(gdb) print input->shape.dimensions[0]
(gdb) print *(float*)input->data.dataPTR @ 8 # print 8 floats
# Step through code:
(gdb) next # step over (don't enter functions)
(gdb) step # step into (enter function calls)
(gdb) continue # run until next breakpoint
# Inspect memory:
(gdb) x/32f w1Data # print 32 floats starting at w1Data
(gdb) x/8xb &cfg # print 8 bytes in hex at cfg address
```

19.4 Testing ODT Code — Unit Tests on Desktop

A key advantage of ODT's design: the C code is standard C99 with no MCU-specific headers in the core library. This means you can compile and test it on a desktop Linux/Mac/Windows machine using gcc.

19.4.1 Writing a Unit Test for matmul

test_matmul.c — unit test

```
#include <stdio.h>
#include <assert.h>
#include <math.h>
#include 'Tensor.h'
#include 'Matmul.h'
void test_matmulFloat_2x3_times_3x2(void) {
// A = [[1,2,3],[4,5,6]] shape [2,3]
// B = [[7,8],[9,10],[11,12]] shape [3,2]
// Expected C = A*B = [[58,64],[139,154]] shape [2,2]
float aData[] = {1,2,3, 4,5,6};
float bData[] = {7,8, 9,10, 11,12};
float cData[4] = {0};
tensor_t A, B, C;
setShape(&A.shape, 2, (size_t[]){2,3});
setShape(&B.shape, 2, (size_t[]){3,2});
setShape(&C.shape, 2, (size_t[]){2,2});
A.data.dataPTR = aData; A.data.numberOfElements = 6;
B.data.dataPTR = bData; B.data.numberOfElements = 6;
C.data.dataPTR = cData; C.data.numberOfElements = 4;
matmulFloat32TensorsWithBias(&A, &B, NULL, &C);
float *result = (float*) C.data.dataPTR;
assert(fabsf(result[0] - 58.0f) < 0.001f);
assert(fabsf(result[1] - 64.0f) < 0.001f);
assert(fabsf(result[2] - 139.0f) < 0.001f);
assert(fabsf(result[3] - 154.0f) < 0.001f);
printf('test_matmulFloat_2x3_times_3x2: PASS\n');
}
int main(void) {
test_matmulFloat_2x3_times_3x2();
return 0;
}
```

Compiling and running on desktop

```
# Compile:
gcc -o test_matmul test_matmul.c Matmul.c Tensor.c Arithmetic.c DTypes.c -lm
# Run:
./test_matmul
# Output: test_matmulFloat_2x3_times_3x2: PASS
```

Testing on desktop first is much faster than flashing to the MCU for every change. Only deploy to the MCU when the logic is proven correct on desktop.

19.4.2 Comparing MCU Output to Python

A powerful validation technique: run the same computation in Python (with NumPy) and compare results with the MCU output.

Python reference computation (run on PC)

```
import numpy as np
# Same weights as MCU (loaded from serialized output)
W1 = np.array([[0.2, 0.5, -0.3, 0.1],
[-0.4, 0.1, 0.6, -0.2]], dtype=np.float32)
b1 = np.array([0.1, -0.1], dtype=np.float32)
x = np.array([[0.5, -0.3, 0.8, 0.1]], dtype=np.float32)
# Forward pass in NumPy
h1_py = x @ W1.T + b1
h2_py = np.maximum(0, h1_py) # ReLU
print('h1:', h1_py)
print('h2:', h2_py)
# Compare with MCU UART output
# If they match (within float32 precision), implementation is correct
```

x @ W1.T		NumPy matrix multiply with transpose. Equivalent to ODT's transpose-matmul-untranspose sequence. Should give identical float32 results.

19.5 Adding a New Layer to ODT

ODT's dispatch table design makes it easy to add new layer types. Here is the complete procedure for adding a new layer (e.g., Tanh activation).

Step 1 — Add to layerType_t enum (Layer.h)

```
typedef enum {
LAYER_LINEAR = 0,
LAYER_RELU,
LAYER_SOFTMAX,
LAYER_CONV1D,
LAYER_DROPOUT,
LAYER_FLATTEN,
LAYER_TANH, // ← ADD HERE
LAYER_COUNT
} layerType_t;
```

Step 2 — Add config struct and union field (Layer.h)

```
typedef struct {
tensor_t *inputCache; // for backward pass
} tanhConfig_t;
// In layerConfig_t union, add:
tanhConfig_t *tanh;
```

Step 3 — Create Tanh.c and Tanh.h

```
// Tanh.h
void tanhForward(layer_t *layer, tensor_t *input, tensor_t *output);
void tanhBackward(layer_t *layer, tensor_t *dOutput, tensor_t *dInput);
void tanhCalcOutputShape(layer_t *layer, shape_t *in, shape_t *out);
// Tanh.c
#include <math.h>
void tanhForward(layer_t *layer, tensor_t *input, tensor_t *output) {
tanhConfig_t *cfg = layer->config.tanh;
cfg->inputCache = input;
size_t n = calcNumberOfElementsByShape(&input->shape);
float *in = (float*) input->data.dataPTR;
float *out = (float*) output->data.dataPTR;
for (size_t i = 0; i < n; i++)
out[i] = tanhf(in[i]); // tanhf = float version of tanh
}
void tanhBackward(layer_t *layer, tensor_t *dOutput, tensor_t *dInput) {
tanhConfig_t *cfg = layer->config.tanh;
size_t n = calcNumberOfElementsByShape(&dOutput->shape);
float *dOut = (float*) dOutput->data.dataPTR;
float *dIn = (float*) dInput->data.dataPTR;
float *cachedIn = (float*) cfg->inputCache->data.dataPTR;
for (size_t i = 0; i < n; i++) {
float t = tanhf(cachedIn[i]);
dIn[i] = dOut[i] * (1.0f - t * t); // d/dx tanh(x) = 1 - tanh(x)^2
}
}
void tanhCalcOutputShape(layer_t *l, shape_t *in, shape_t *out) {
*out = *in; // tanh doesn't change shape
}
```

Step 4 — Register in layerFunctions[] dispatch table (Layer.c)

```
layerFunctions_t layerFunctions[] = {
[LAYER_LINEAR] = { linearForward, linearBackward, linearCalcOutputShape },
[LAYER_RELU] = { reluForward, reluBackward, reluCalcOutputShape },
[LAYER_SOFTMAX] = { softmaxForward, softmaxBackward, softmaxCalcOutputShape },
[LAYER_TANH] = { tanhForward, tanhBackward, tanhCalcOutputShape }, // NEW
// ...
};
```

After these 4 steps, you can use LAYER_TANH anywhere in the training loop exactly like LAYER_RELU — the dispatch table handles the routing.

19.6 Adding a New Optimizer

ODT currently only implements SGD. Adding Adam (Adaptive Moment Estimation) is a common next step. Adam requires storing two additional state vectors per parameter (first and second moment estimates).

Adam optimizer struct

```
typedef struct {
float lr; // learning rate (default 0.001)
float beta1; // first moment decay (default 0.9)
float beta2; // second moment decay (default 0.999)
float epsilon; // numerical stability (default 1e-8)
size_t t; // step counter (for bias correction)
tensor_t m; // first moment (mean of gradients)
tensor_t v; // second moment (mean of squared gradients)
} adam_t;
// Adam update step:
void adamStep(adam_t *adam, parameter_t *param) {
adam->t++;
float *w = (float*) param->param.data.dataPTR;
float *g = (float*) param->grad.data.dataPTR;
float *m = (float*) adam->m.data.dataPTR;
float *v = (float*) adam->v.data.dataPTR;
size_t n = param->param.data.numberOfElements;
// Bias correction factors
float bc1 = 1.0f - powf(adam->beta1, adam->t);
float bc2 = 1.0f - powf(adam->beta2, adam->t);
for (size_t i = 0; i < n; i++) {
// Update biased first moment
m[i] = adam->beta1 * m[i] + (1.0f - adam->beta1) * g[i];
// Update biased second moment
v[i] = adam->beta2 * v[i] + (1.0f - adam->beta2) * g[i] * g[i];
// Bias-corrected moments
float m_hat = m[i] / bc1;
float v_hat = v[i] / bc2;
// Update weight
w[i] -= adam->lr * m_hat / (sqrtf(v_hat) + adam->epsilon);
}
}
```

■ NOTE

Adam Memory Cost: Adam requires 2 extra float tensors per parameter tensor (m and v). For a 64×128 weight matrix: 2 × 64×128 × 4 bytes = 65,536 extra bytes. This is significant on the STM32's 320 KB RAM. SGD needs no extra state.

19.7 Adding a New Loss Function

The lossFunctions[] dispatch table works the same way as layerFunctions[]. Here's how to add Mean Absolute Error (MAE) loss:

MAE loss implementation

```
// MAE.c
float maeForward(lossFunc_t *loss, tensor_t *pred, tensor_t *target) {
size_t n = pred->data.numberOfElements;
float *p = (float*) pred->data.dataPTR;
float *t = (float*) target->data.dataPTR;
float sum = 0.0f;
for (size_t i = 0; i < n; i++)
sum += fabsf(p[i] - t[i]);
return sum / n;
}
void maeBackward(lossFunc_t *loss, tensor_t *pred, tensor_t *target,
tensor_t *dPred) {
size_t n = pred->data.numberOfElements;
float *p = (float*) pred->data.dataPTR;
float *t = (float*) target->data.dataPTR;
float *dp = (float*) dPred->data.dataPTR;
for (size_t i = 0; i < n; i++) {
// d/dp |p-t| = sign(p-t)
dp[i] = (p[i] > t[i]) ? 1.0f/n : -1.0f/n;
}
}
```

19.8 Performance Profiling on the STM32

To measure exactly how long each part of the training loop takes, use the DWT (Data Watchpoint and Trace) cycle counter built into Cortex-M7.

Cycle counter profiling

```
// Enable DWT cycle counter (one-time setup in main.c):
CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
DWT->CYCCNT = 0;
DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
// Measure a function:
#define CYCLES_START() (DWT->CYCCNT)
#define CYCLES_END(start) (DWT->CYCCNT - (start))
uint32_t t0 = CYCLES_START();
matmulFloat32TensorsWithBias(&W, &x, NULL, &output);
uint32_t cycles = CYCLES_END(t0);
// At 216 MHz: 1 cycle = 4.63 ns
float time_us = (float)cycles / 216.0f; // microseconds
printf('matmul: %u cycles (%.1f us)\n', cycles, time_us);
```

═■ WORKED EXAMPLE

Measured performance on STM32F746ZG (estimated): Operation | Cycles | Time (216 MHz) Linear forward [64→32], f32 | ~18,000 | ~83 µs ReLU forward [32], f32 | ~200 | ~1 µs Softmax forward [10 classes] | ~300 | ~1.4 µs Cross-entropy backward | ~80 | ~0.37 µs SGD step [64×32 weights] | ~2,200 | ~10 µs Full training step (1 sample)| ~42,000 | ~194 µs Throughput: ~5,000 training samples/second

19.9 Checklist: Before Flashing to MCU

Check	How to Verify
All shapes correct	Add PRINT_DEBUG for shape in each layer init
Scale initialized	Check qConfig->scale != 0.0f before first use
sgdZeroGrad called	Add assert(gradBuffer[0] == 0) after sgdZeroGrad
transposeTensor pairs	Count calls: must be even for each tensor
No VLA larger than ~4KB	Calculate: sizeof(type) * maxDim1 * maxDim2
Stack depth estimated	Add up all local variables in the call chain
Loss decreasing	Print loss every 10 batches — should trend down
Gradients not NaN/inf	Print grad[0] every batch; NaN means overflow
Weights updated	Print W[0] before/after sgdStep — should differ

■ RELATIONS WITH OTHER FILES

Chapter 19 connects to: Chapter 2 (Common.h) — PRINT_DEBUG, PRINT_ERROR, TRACK_INSTRUCTIONS macros · Chapter 7 (Layers) — Adding a new layer follows the dispatch table pattern · Chapter 8 (Training) — Adding new optimizers and loss functions · Chapter 13 (C language) — assert, volatile, test compilation on desktop · Chapter 18 (Embedded) — DWT cycle counter, SWD debugger, UART printf

Chapter 20

SGD, DataLoader, and the Training Infrastructure

How the optimizer, data pipeline, and training loop work together

Section 20.1 — SGD Optimizer: Inside sgdStep

The Stochastic Gradient Descent optimizer is the simplest but most fundamental optimizer in deep learning. ODT implements it in Sgd.h and Sgd.c. Understanding every line of sgdStep reveals what 'training' really means at the lowest level.

Sgd.h — sgd_t struct

```
// sgd_t: the optimizer's persistent state
typedef struct {
float learningRate; // α — how large each update step is
float momentumFactor; // β■ — fraction of velocity to carry forward (0 = no momentum)
float weightDecay; // λ — L2 regularization strength (0 = no decay)
} sgd_t;
```

Three fields, three different training behaviours. The learning rate (α) controls the size of each gradient step. Momentum (β■) smooths updates by combining the current gradient with a running average of past gradients — like a ball rolling downhill, it builds speed. Weight decay (λ) adds a penalty proportional to the weight magnitude, preventing any single weight from becoming too large (L2 regularisation).

Sgd.c — sgdStepFloat line-by-line

```
// Sgd.c — sgdStepFloat (annotated)
void sgdStepFloat(sgd_t *sgd, parameter_t *param) {
size_t n = param->param.data.numberOfElements;
float *w = (float*) param->param.data.dataPTR; // weight array
float *g = (float*) param->grad.data.dataPTR; // gradient array
for (size_t i = 0; i < n; i++) {
// Core SGD update: w = w - lr * grad
w[i] -= sgd->learningRate * g[i];
// Optional weight decay: w = w - lr * lambda * w
if (sgd->weightDecay != 0.0f)
w[i] -= sgd->learningRate * sgd->weightDecay * w[i];
}
}
```

n	`param.data.numberOfElements`	Total number of elements in this parameter tensor. For a [32,64] weight matrix: n=2048.
w	`(float*) param.param.data.dataPTR`	Cast the void* buffer pointer to float* so we can index it as an array. This is the actual weight values that get updated.
g	`(float*) param.grad.data.dataPTR`	Cast the gradient buffer. These were computed and accumulated during backward pass.
w[i] -= lr * g[i]	`w[i] -= sgd->learningRate * g[i];`	THE CORE OF TRAINING. Subtract (learning rate × gradient) from each weight. If the gradient is positive (increasing weight → increasing loss), we subtract to decrease the weight, which should decrease the loss.
weightDecay	`w[i] -= lr * weightDecay * w[i];`	If non-zero, also subtract lr × λ × w[i]. This is equivalent to adding a term λ×||w||² to the loss function, which penalises large weights and encourages simpler (smaller-weight) models — a form of regularisation.

Notice what sgdStepFloat does NOT do: it does not zero the gradients. That is the job of sgdZeroGrad, which must be called at the start of every batch.

Sgd.c — sgdZeroGrad: why scale must be reset

```
// Sgd.c — sgdZeroGrad
void sgdZeroGrad(sgd_t *sgd, linearConfig_t *cfg) {
// Zero the weight gradient buffer
size_t wBytes = cfg->weights.grad.data.numberOfElements * sizeof(float);
memset(cfg->weights.grad.data.dataPTR, 0, wBytes);
// Reset scale to 1.0 (important for SYM_INT32 — memset alone sets scale to 0)
if (cfg->weights.grad.quantization.type == SYM_INT32) {
symInt32QConfig_t *qcfg = cfg->weights.grad.quantization.qConfig;
qcfg->scale = 1.0f;
}
// Zero the bias gradient buffer if present
if (cfg->hasBias) {
size_t bBytes = cfg->bias.grad.data.numberOfElements * sizeof(float);
memset(cfg->bias.grad.data.dataPTR, 0, bBytes);
if (cfg->bias.grad.quantization.type == SYM_INT32) {
symInt32QConfig_t *qcfg = cfg->bias.grad.quantization.qConfig;
qcfg->scale = 1.0f;
}
}
}
```

■■ IMPORTANT

The scale=1.0 reset after memset: memset(ptr, 0, n) sets all bytes to zero. For a float, 0x00000000 = 0.0f — correct. But for SYM_INT32, the scale field is also a float stored in the qConfig struct. If the qConfig struct is inside the zeroed region, scale becomes 0.0f. Then the first dequantize call divides by scale = 0.0f → infinity → NaN → disaster. Always reset scale to 1.0f after zeroing quantized tensors.

Section 20.2 — sgdStepSymInt32: Training with Quantized Weights

When weights are stored as SYM_INT32 (fixed-point quantised), the SGD step must convert to float, update, and re-quantise. This is the core of FQT (Fixed-point Quantisation Training) — training happens in float, but weights are stored and used for forward/backward passes in integer format.

Sgd.c — sgdStepSymInt32: dequantise → update → requantise

```
// Sgd.c — sgdStepSymInt32 (simplified)
void sgdStepSymInt32(sgd_t *sgd, parameter_t *param) {
size_t n = param->param.data.numberOfElements;
int32_t *w = (int32_t*) param->param.data.dataPTR;
float *g = (float*) param->grad.data.dataPTR; // grad always float
symInt32QConfig_t *qcfg = param->param.quantization.qConfig;
float scale = qcfg->scale;
int32_t qMax = (1 << (qcfg->qMaxBits - 1)) - 1;
// VLA: temporary float buffer on the stack
float wFloat[n]; // e.g. 2048 floats = 8 KB if n=2048
// Step 1: dequantise all weights to float
for (size_t i = 0; i < n; i++)
wFloat[i] = (float)w[i] * scale;
// Step 2: SGD update in float
for (size_t i = 0; i < n; i++)
wFloat[i] -= sgd->learningRate * g[i];
// Step 3: re-quantise back to int32
// Find new abs-max for optimal scale
float absMax = findAbsMaxFloat(wFloat, n);
float newScale = (absMax > 0) ? absMax / (float)qMax : scale;
qcfg->scale = newScale;
for (size_t i = 0; i < n; i++) {
int32_t q = roundByMode(wFloat[i] / newScale, qcfg->roundingMode);
w[i] = clamp(q, -qMax, qMax);
}
}
```

■ C CONCEPT: VLA — Variable Length Array

float wFloat[n]; declares a VLA. The size n is only known at runtime. VLAs are allocated on the STACK — the compiler adjusts the stack pointer by n*4 bytes. For n=2048: 8KB of stack. On the STM32 with 320KB RAM, the stack might only be 8-16KB total. This is why large tensors MUST use static buffers instead of VLAs. For small layers (n ≤ 256), VLAs are safe.

═■ WORKED EXAMPLE

SYM_INT32 SGD step trace (scale=0.01, qMaxBits=8, qMax=127, lr=0.001) w[0] = 73 (real value = 73 × 0.01 = 0.73) g[0] = 2.5 (gradient) Step 1 (dequantise): wFloat[0] = 73 × 0.01 = 0.730 Step 2 (update): wFloat[0] = 0.730 - 0.001 × 2.5 = 0.7275 Step 3 (requantise): absMax found = 0.9 → newScale = 0.9/127 = 0.00709 q = round(0.7275 / 0.00709) = round(102.6) = 103 w[0] = clamp(103, -127, 127) = 103 Real value stored: 103 × 0.00709 = 0.730 (very close to 0.7275 — tiny quantisation error)

Section 20.3 — DataLoader: Every Field Explained

The DataLoader is the bridge between the raw dataset and the training loop. It handles batching, shuffling, and calling user-provided data retrieval functions. ODT's DataLoader uses function pointers so that any data source (SD card, UART, hardcoded array, etc.) can be plugged in without modifying the training loop.

DataLoader.h — complete dataLoader_t struct

```
// DataLoader.h — dataLoader_t: the complete struct
typedef struct {
// ■■ User-provided callbacks ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
getSampleFn_t getSample; // Get one sample by index
getDatasetSizeFn_t getDatasetSize; // Total number of samples
getBatchFn_t getBatch; // Fill a batch tensor
// ■■ Batch configuration ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
size_t batchSize; // Number of samples per batch
bool dropLast; // Drop incomplete last batch?
// ■■ Transform callbacks ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
transformFn_t transform; // Applied to each input sample
transformFn_t targetTransform; // Applied to each target sample
// ■■ Shuffling state ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
bool shuffle; // Shuffle sample order each epoch?
size_t shuffleSeed; // Initial RNG seed for shuffling
size_t *indices; // Permuted sample indices (caller-allocated)
} dataLoader_t;
```

getSample	`void (*)(dataLoader_t*, size_t idx, tensor_t* out, tensor_t* target)`	Fetches a single sample at position idx and writes it into out and target. The user implements this — it might read from a hardcoded array, an SD card, or a ring buffer filled by a UART interrupt.
getDatasetSize	`size_t (*)(dataLoader_t*)`	Returns the total count of samples. Used to compute the number of batches per epoch: numBatches = getDatasetSize() / batchSize.
getBatch	`void (*)(dataLoader_t*, size_t batchIdx, tensor_t* out, tensor_t* target)`	Higher-level: fills a full batch. The default implementation calls getSample batchSize times, stacking results into the batch tensor.
batchSize	`size_t`	How many samples per gradient update. Larger batches = stabler gradients but more memory and slower per-update. On the STM32 with limited RAM, small batches (1-8) are typical.
dropLast	`bool`	If the dataset size is not divisible by batchSize, the last batch is incomplete. dropLast=true discards it; dropLast=false uses it (smaller gradients for last batch).
transform	`void (*)(tensor_t*)`	Optional: applied to each input after getSample. E.g., normalise audio features to zero mean. If NULL, no transform is applied.
shuffle	`bool`	If true, the indices array is shuffled using rngShuffleIndices at the start of each epoch. This randomises the order in which samples are seen, preventing the network from memorising the presentation order.
indices	`size_t*`	Array of size getDatasetSize(), holding sample indices. When shuffle=true, Fisher-Yates permutes this array. getSample then accesses dataset[indices[i]] instead of dataset[i]. Caller must allocate and pass this buffer.

Complete DataLoader setup example

```
// Typical DataLoader initialisation for a small hardcoded dataset
// User-defined dataset
#define NUM_SAMPLES 120
static float audioFeatures[NUM_SAMPLES][64]; // 120 samples, 64 MFCC features
static int speakerLabels[NUM_SAMPLES]; // 0, 1, or 2 for 3 speakers
static size_t shuffleIndices[NUM_SAMPLES];
// Callback: get one sample
void myGetSample(dataLoader_t *dl, size_t idx, tensor_t *x, tensor_t *y) {
float *xPtr = (float*) x->data.dataPTR;
float *yPtr = (float*) y->data.dataPTR;
// Copy features
memcpy(xPtr, audioFeatures[idx], 64 * sizeof(float));
// One-hot encode label
memset(yPtr, 0, 3 * sizeof(float));
yPtr[speakerLabels[idx]] = 1.0f;
}
// Callback: dataset size
size_t myGetSize(dataLoader_t *dl) { return NUM_SAMPLES; }
// Initialise DataLoader
dataLoader_t loader;
initDataLoader(&loader, myGetSample, myGetSize, defaultGetBatch,
4, // batchSize = 4
true, // dropLast = true
NULL, NULL, // no transforms
true, // shuffle = true
42, // shuffleSeed
shuffleIndices);
```

Section 20.4 — Training Loop Analysis: Every Line Explained

The training loop ties everything together. Here is the complete loop with annotations explaining WHY each step is required and what happens if you skip it.

Complete annotated training loop

```
void trainOneEpoch(dataLoader_t *loader, sgd_t *sgd,
layer_t *layers[], size_t numLayers,
lossFunc_t *loss,
tensor_t *activations[], tensor_t *gradients[]) {
size_t N = loader->getDatasetSize(loader);
size_t numBatches = N / loader->batchSize;
if (!loader->dropLast && (N % loader->batchSize) != 0)
numBatches++;
// Shuffle indices for this epoch
if (loader->shuffle) {
rng32_t rng;
rngSetSeed(&rng, loader->shuffleSeed);
rngShuffleIndices(&rng, loader->indices, N);
loader->shuffleSeed = rng.state; // advance seed for next epoch
}
float epochLoss = 0.0f;
for (size_t b = 0; b < numBatches; b++) {
// ■■ Step 1: Zero gradients ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
for (size_t l = 0; l < numLayers; l++)
if (layers[l]->type == LAYER_LINEAR)
sgdZeroGrad(sgd, layers[l]->config.linear);
// ■■ Step 2: Load batch ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
loader->getBatch(loader, b, activations[0], gradients[numLayers]);
// ■■ Step 3: Forward pass ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
for (size_t l = 0; l < numLayers; l++)
layerFunctions[layers[l]->type].forward(
layers[l], activations[l], activations[l+1]);
// ■■ Step 4: Compute loss ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
float batchLoss = loss->forward(loss,
activations[numLayers], gradients[numLayers]);
epochLoss += batchLoss;
// ■■ Step 5: Backward pass ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
loss->backward(loss,
activations[numLayers], gradients[numLayers],
gradients[numLayers-1]);
for (int l = (int)numLayers - 1; l >= 0; l--)
layerFunctions[layers[l]->type].backward(
layers[l], activations[l],
gradients[l], (l>0) ? gradients[l-1] : NULL);
// ■■ Step 6: Update weights ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
for (size_t l = 0; l < numLayers; l++)
if (layers[l]->type == LAYER_LINEAR)
sgdStep(sgd, layers[l]->config.linear);
}
PRINT_DEBUG('Epoch loss: %.4f\n', epochLoss / numBatches);
}
```

Step 1 — sgdZeroGrad		Gradient buffers accumulate via +=. Without zeroing, gradients from all previous batches add up. After 1000 batches, the effective gradient would be 1000× too large, causing wildly incorrect weight updates. MUST be the very first step.

Step 2 — getBatch		Loads batchSize samples from the dataset into the input activation buffer. The target tensor is also filled (one-hot encoded labels). With shuffle=true, sample order is randomised each epoch.
Step 3 — Forward pass		Sequentially calls each layer's forward function: Linear→ReLU→Linear→Softmax (example). Each activation is stored in activations[l+1]. The final activation is the network output.
Step 4 — Compute loss		Compares the network output to the target. Returns a scalar loss value. For cross-entropy: loss = -log(probability assigned to true class). Lower is better. Also computes the initial gradient (dL/d_output).
Step 5 — Backward pass		Propagates gradients in REVERSE layer order. Each layer receives dOutput (gradient flowing in) and computes: dInput (to pass upstream) and dWeights/dBias (accumulated into gradient buffers). The chain rule in action.
Step 6 — Update weights		For each LINEAR layer: w -= lr * gradient. This is the actual learning step. Note: only LINEAR layers have learnable parameters. ReLU, Softmax, Flatten have none.

■ NOTE

Why the Loop Iterates Batches, Not Individual Samples A batch of 4 samples produces 4 sets of gradients. The backward pass accumulates them all (via +=) into the gradient buffers. The SGD step then updates weights based on the AVERAGE gradient over the batch. This is more numerically stable than updating after every single sample (1-sample batches = pure stochastic gradient descent, very noisy).

Section 20.5 — Loss Functions: Forward and Backward in Detail

ODT's loss function dispatch table works exactly like the layer dispatch table. A lossFunctions_t struct holds forward, backward, and computeMeanScale function pointers.

LossFunction.h — dispatch table

```
// LossFunction.h
typedef float (*lossFwdFn_t)(lossFunc_t*, tensor_t* pred, tensor_t* target);
typedef void (*lossBwdFn_t)(lossFunc_t*, tensor_t* pred, tensor_t* target,
tensor_t* dPred);
typedef struct {
lossFwdFn_t forward;
lossBwdFn_t backward;
// Optional: scale gradient by batch mean (mean reduction)
computeMeanScaleFn_t computeMeanScale;
} lossFunctions_t;
extern lossFunctions_t lossFunctions[]; // indexed by lossFuncType_t
// Registration:
lossFunctions_t lossFunctions[] = {
[LOSS_MSE] = { mseForward, mseBackward, ... },
[LOSS_CROSS_ENTROPY] = { crossEntropyFwdFloat, crossEntropySMBackFloat, ... },
};
```

crossEntropyForwardFloat — annotated

```
// LossFunction.c — crossEntropyForwardFloat
float crossEntropyForwardFloat(lossFunc_t *loss,
tensor_t *pred, tensor_t *target) {
size_t batch = pred->shape.dimensions[0];
size_t classes = pred->shape.dimensions[1];
float *p = (float*) pred->data.dataPTR;
float *t = (float*) target->data.dataPTR;
float totalLoss = 0.0f;
for (size_t b = 0; b < batch; b++) {
for (size_t c = 0; c < classes; c++) {
float predVal = p[b * classes + c];
float targVal = t[b * classes + c];
if (targVal > 0.0f) {
// Guard against log(0) = -infinity
float safeP = (predVal < 1e-7f) ? 1e-7f : predVal;
totalLoss -= targVal * logf(safeP);
}
}
}
return totalLoss / (float)batch; // mean over batch
}
```

targVal > 0.0f check		For one-hot targets, only one class is non-zero. This check skips the log computation for all zero-target classes (saves n-1 logf calls per sample).
predVal < 1e-7f guard		log(0) = -infinity. If the network is very confident and wrong (predicts 0 for the true class), we clamp to 1e-7 to prevent NaN loss. In practice, after softmax all outputs are > 0, but numerical precision can push them to ~0.

/ (float) batch		Divide by batch size to get mean loss. This makes the loss comparable across different batch sizes (a batch of 4 and a batch of 8 give similar loss magnitudes).

crossEntropySoftmaxBackwardFloat — the 1-line gradient

```
// LossFunction.c — crossEntropySoftmaxBackwardFloat
// This implements the combined softmax + cross-entropy backward pass
// Math result: dL/d(logit_i) = softmax_output_i - target_i
void crossEntropySoftmaxBackwardFloat(
lossFunc_t *loss, tensor_t *pred, tensor_t *target, tensor_t *dPred) {
size_t batch = pred->shape.dimensions[0];
size_t classes = pred->shape.dimensions[1];
float *p = (float*) pred->data.dataPTR; // softmax outputs
float *t = (float*) target->data.dataPTR; // one-hot targets
float *dp = (float*) dPred->data.dataPTR; // output gradient buffer
for (size_t i = 0; i < batch * classes; i++)
dp[i] = (p[i] - t[i]) / (float)batch;
// That's it. One line. The combined derivative is beautiful.
}
```

■ C CONCEPT: Why softmax+CE backward is so simple

Separately, softmax backward is complex (involves the Jacobian matrix). Cross-entropy backward is -target/pred. Combined, almost everything cancels: d(CE■Softmax)/d(logit_i) = softmax_i - target_i. This is why ODT (and all major frameworks) fuse these two operations. The result: if the network predicted 0.7 for the true class (target=1.0), the gradient is 0.7 - 1.0 = -0.3 → the logit should increase.

■ RELATIONS WITH OTHER FILES

Chapter 20 uses: Sgd.h/c (sgdStep, sgdZeroGrad) · DataLoader.h/c (initDataLoader, getBatch) · LossFunction.h/c (crossEntropyForward, crossEntropySoftmaxBackward) · Layer.h (layerFunctions dispatch) · Rng.h/c (rngShuffleIndices for shuffle) · Tensor.h (tensor_t for all data) · Chapter 17 worked example shows these functions called together.

Chapter 21

Full RigL C Implementation

Complete compilable code for rigl.h and rigl.c — every line explained

Section 21.1 — What We Are Building

■ WHAT THIS FILE DOES

This chapter presents a COMPLETE, COMPILABLE C implementation of RigL (Rigging the Lottery, Evci et al. NeurIPS 2020) for the ODT library. Every function is presented in full with line-by-line explanations. This is not pseudocode — this is the actual code you would add to the repository. Files to create: RigL.h (header), RigL.c (implementation). Files to modify: Tensor.h (sparsityConfig fields), Sgd.c (mask-aware step), Linear.c (masked forward), Matmul.c (masked multiply).

Function	File	Purpose
riglConfig_t (struct)	RigL.h	Configuration: mask tensor, sparsity ratio, update interval
riglInitMask()	RigL.c	Randomly initialize mask: set sparsityRatio fraction to pruned
riglComputeCountToPrune()	RigL.c	Helper: how many weights to prune/grow this step

riglPruneStep()	RigL.c	Deactivate weights with smallest |w|
riglGrowStep()	RigL.c	Reactivate pruned weights with largest |gradient|
riglUpdate()	RigL.c	Combined: check if it's time to update, then prune+grow
riglMaskedMatmulFloat32()	Matmul.c (modified)	Matmul that skips pruned weights
riglMaskedSgdStepFloat()	Sgd.c (modified)	SGD update that skips pruned weights
riglGetSparsityRatio()	RigL.c	Compute actual current sparsity ratio
riglPrintStats()	RigL.c	Debug: print how many weights active/pruned

Section 21.2 — RigL.h: The Complete Header File

```
// ================================================================
// RigL.h — src/training/include/RigL.h
//
// Implements Evci et al. 2020: 'Rigging the Lottery'
// Sparse training: periodically prune smallest weights, grow by gradient.
// ================================================================
#ifndef RIGL_H
#define RIGL_H
#include <stddef.h>
#include <stdint.h>
#include <stdbool.h>
#include "Tensor.h"
#include "Quantization.h"
#include "Sgd.h"
// ■■ RigL configuration for one sparse layer ■■■■■■■■■■■■■■■■■■■■■■
typedef struct riglConfig {
tensor_t *mask; // BOOL tensor, shape = weight shape
float sparsityRatio; // fraction pruned, e.g. 0.8 = 80%
size_t updateStep; // batches between topology updates
size_t batchCount; // internal counter — do not set manually
} riglConfig_t;
// ■■ Public API ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Initialise the mask: randomly set sparsityRatio fraction to 0 (pruned)
void riglInitMask(riglConfig_t *cfg, unsigned int seed);
// One prune+grow cycle (call this every updateStep batches)
void riglUpdate(parameter_t *parameter, riglConfig_t *cfg);
// The prune step alone (deactivate smallest |w| fraction)
void riglPruneStep(parameter_t *parameter, riglConfig_t *cfg);
// The grow step alone (reactivate highest |g| pruned weights)
void riglGrowStep(parameter_t *parameter, riglConfig_t *cfg);
// Utility: compute actual current sparsity (active/total)
float riglGetSparsityRatio(riglConfig_t *cfg, size_t totalWeights);
// Debug: print mask statistics over UART
void riglPrintStats(riglConfig_t *cfg, size_t totalWeights);
#endif // RIGL_H
```

1	`#ifndef RIGL_H / #define RIGL_H / #endif`	Include guard: prevents the header from being processed more than once if multiple .c files include RigL.h. The preprocessor checks: if RIGL_H is not yet defined, define it and process the file. On a second inclusion: RIGL_H is already defined, so the entire file is skipped.
2	`#include <stdbool.h>`	Provides the 'bool', 'true', and 'false' keywords for C (prior to C99). In C, there is no built-in boolean type — stdbool.h defines bool as _Bool, true as 1, false as 0. tensorBoolGet returns bool, so we need this.
3	`tensor_t *mask;`	Pointer to a BOOL tensor with the same shape as the weight tensor. mask[i] = 1 (true) → weight i is ACTIVE (participates in computation). mask[i] = 0 (false) → weight i is PRUNED (forced to zero, skip in matmul).
4	`float sparsityRatio;`	Target fraction of pruned weights. sparsityRatio=0.8 → 80% of weights pruned, 20% active. The SAME sparsity ratio is maintained throughout training. Only the pattern changes (which 20% are active).
5	`size_t updateStep;`	How many batches between topology updates. Typical: 100. Every 100 batches, call riglUpdate() to prune+grow. Too frequent: topology oscillates before weights converge. Too rare: suboptimal topology persists too long.

6	`size_t batchCount;`	Internal counter incremented each batch. Do not set this manually. riglUpdate() uses batchCount % updateStep to know when to run the prune+grow cycle.

Section 21.3 — riglInitMask(): Initialize the Sparsity Pattern

Before training, the mask must be initialized so that exactly sparsityRatio of the weights are pruned and the rest are active. We use a simple random approach: set all bits to 1 (all active), then randomly deactivate sparsityRatio × N of them.

```
// RigL.c — riglInitMask
#include "RigL.h"
#include "DTypes.h" // tensorBoolGet, tensorBoolSet
#include "Tensor.h" // calcNumberOfElementsByTensor
#include <string.h> // memset
#include <stdlib.h> // srand, rand
void riglInitMask(riglConfig_t *cfg, unsigned int seed) {
tensor_t *mask = cfg->mask;
size_t N = calcNumberOfElementsByTensor(mask);
// Step 1: Activate ALL weights (set every mask bit to 1)
size_t maskBytes = (N + 7) / 8; // ceil(N/8)
memset(mask->data, 0xFF, maskBytes);
// Handle non-multiple-of-8: clear the padding bits in the last byte
if (N % 8 != 0) {
size_t validBits = N % 8;
uint8_t lastMask = (uint8_t)((1u << validBits) - 1);
mask->data[maskBytes - 1] &= lastMask;
}
// Step 2: Randomly prune sparsityRatio fraction
size_t numToPrune = (size_t)(cfg->sparsityRatio * (float)N);
srand(seed);
size_t pruned = 0;
while (pruned < numToPrune) {
// Pick a random weight index
size_t idx = (size_t)((unsigned)rand() % (unsigned)N);
// If it is still active, prune it
if (tensorBoolGet(mask, idx)) {
tensorBoolSet(mask, idx, false);
pruned++;
}
}
}
```

1	`size_t N = calcNumberOfElementsByTensor(mask);`	Total weight count. For a linear layer [128, 64]: N = 128 × 64 = 8192.
2	`size_t maskBytes = (N + 7) / 8;`	A BOOL tensor stores 1 bit per weight. N bits need ceil(N/8) bytes. (N + 7) / 8 is integer division that rounds UP. For N=8192: (8192+7)/8 = 8199/8 = 1024.875 → integer division = 1024 bytes.
3	`memset(mask->data, 0xFF, maskBytes);`	0xFF = 0b11111111 = all 8 bits set to 1. memset fills every byte with 0xFF, setting all mask bits to 1 (all weights active). This is the starting point before we randomly prune.
4	`if (N % 8 != 0) { ... }`	If N is not a multiple of 8, the last byte has some unused bits. memset sets them all to 1, but they represent indices beyond N-1 (non-existent weights). We clear those padding bits to avoid tensorBoolGet returning true for invalid indices.
5	`uint8_t lastMask = (uint8_t)((1u << validBits) - 1);`	Create a bitmask with only the valid bits set. Example: N=10, so last byte has 2 valid bits (indices 8 and 9). validBits=2, 1u<<2=4, 4-1=3=0b00000011. ANDing with this keeps only bits 0 and 1, clearing bits 2-7.

6	`size_t numToPrune = (size_t)(cfg->sparsityRatio * (float)N);`	How many weights to prune. For sparsityRatio=0.8, N=8192: numToPrune = (size_t)(0.8 × 8192.0) = (size_t)(6553.6) = 6553. The cast truncates (not rounds), so actual sparsity may be slightly less than 80%.
7	`size_t idx = (size_t)((unsigned)rand() % (unsigned)N);`	Generate a random index in [0, N-1]. rand() returns int in [0, RAND_MAX]. % N gives the remainder, mapping to [0, N-1]. Casting to unsigned before % avoids signed-modulo issues on some platforms.
8	`if (tensorBoolGet(mask, idx)) { tensorBoolSet(mask, idx, false); pruned++; }`	Only prune if not already pruned. Without this check, the same index could be picked twice, and pruned++ would count an already-pruned weight. The while loop retries until it finds an unpruned weight.

■■ IMPORTANT

The random-retry approach is simple but slow if sparsityRatio is very high (>95%). At 95% sparsity, only 5% of random picks find an unset bit — many retries. A more efficient approach: build a shuffled index array (Fisher-Yates), then prune the first numToPrune entries. rngShuffleIndices from RNG.h does this.

Section 21.4 — riglPruneStep(): Remove Smallest Weights

The prune step deactivates the sparsityRatio fraction of currently active weights that have the smallest absolute value. Small-magnitude weights contribute little to the output, so removing them has minimal impact on accuracy.

■ C CONCEPT: Sorting by Absolute Value

To prune the 'bottom k%' of active weights by |w|, we need to sort them. C's standard library provides qsort() — the quicksort algorithm. qsort(array, count, element_size, comparison_function) The comparison function receives two const void* pointers to elements and must return: negative if a < b, 0 if a == b, positive if a > b. For our case: we sort an array of INDICES (not weights) by the |weight| at each index. This lets us sort cheaply and then use the sorted indices to both prune and grow.

```
// RigL.c — sort helper for prune step
// Global pointer used by the qsort comparison function
// (qsort callback cannot receive extra arguments)
static float *g_sortWeights = NULL;
// Compare two indices by |weight| ascending (smallest first)
static int cmpAbsWeightAsc(const void *a, const void *b) {
size_t ia = *(const size_t*)a;
size_t ib = *(const size_t*)b;
float fa = g_sortWeights[ia];
float fb = g_sortWeights[ib];
if (fa < 0.0f) fa = -fa; // |fa|
if (fb < 0.0f) fb = -fb; // |fb|
if (fa < fb) return -1;
if (fa > fb) return 1;
return 0;
}
// Compare two indices by |gradient| descending (largest first)
static float *g_sortGrads = NULL;
static int cmpAbsGradDesc(const void *a, const void *b) {
size_t ia = *(const size_t*)a;
size_t ib = *(const size_t*)b;
float fa = g_sortGrads[ia];
float fb = g_sortGrads[ib];
if (fa < 0.0f) fa = -fa;
if (fb < 0.0f) fb = -fb;
if (fa > fb) return -1; // reversed: descending
if (fa < fb) return 1;
return 0;
}
```

1	`static float *g_sortWeights = NULL;`	A global (module-level) pointer to the weight array. 'static' limits its scope to RigL.c — it is not visible to other files. The qsort comparison function cannot receive extra arguments (it's a fixed signature), so we pass the weight array via this global pointer. This is NOT thread-safe, but the MCU runs single-threaded.
2	`size_t ia = *(const size_t*)a;`	Cast the void* pointer to size_t* and dereference to get the actual index. qsort passes &array[i] (address of element), not array[i] (value). We need to dereference (*) to get the index value.
3	`if (fa < 0.0f) fa = -fa;`	Manual absolute value. Equivalent to fabsf(fa). We avoid including math.h for this single operation.
4	`if (fa < fb) return -1;`	qsort convention: return negative if a should come BEFORE b. For ascending sort: smaller absolute value comes first (return -1 when fa < fb).

5	`if (fa > fb) return -1; (in cmpAbsGradDesc)`	For DESCENDING sort: LARGER absolute gradient comes first. Reversed: return -1 (a before b) when fa > fb (a is larger).

```
// RigL.c — riglPruneStep
// Static buffers: avoid stack overflow on MCU
// Max supported weight count for one layer
#define RIGL_MAX_WEIGHTS 16384
static size_t s_activeIdx[RIGL_MAX_WEIGHTS];
static float s_wFloat[RIGL_MAX_WEIGHTS];
void riglPruneStep(parameter_t *parameter, riglConfig_t *cfg) {
tensor_t *w = getParamFromParameter(parameter);
tensor_t *mask = cfg->mask;
size_t N = calcNumberOfElementsByTensor(w);
if (N > RIGL_MAX_WEIGHTS) {
PRINT_ERROR("riglPruneStep: layer too large\n");
return;
}
// === Collect active indices and their float weights ===
size_t numActive = 0;
for (size_t i = 0; i < N; i++) {
s_wFloat[i] = readBytesAsFloat(w->data + i*4);
if (tensorBoolGet(mask, i)) {
s_activeIdx[numActive++] = i;
}
}
// === Sort active indices by |w| ascending ===
g_sortWeights = s_wFloat;
qsort(s_activeIdx, numActive, sizeof(size_t), cmpAbsWeightAsc);
// === Prune the bottom sparsityRatio fraction ===
size_t numToPrune = (size_t)(cfg->sparsityRatio * (float)numActive);
for (size_t i = 0; i < numToPrune; i++) {
size_t idx = s_activeIdx[i];
tensorBoolSet(mask, idx, false);
writeFloatToByteArray(0.0f, w->data + idx*4);
}
}
```

1	`static size_t s_activeIdx[RIGL_MAX_WEIGHTS]; static float s_wFloat[RIGL_MAX_WEIGHTS];`	Static arrays at file scope (NOT on the stack). On MCU, the stack is ~4-8 KB. A 16384-element size_t array would be 64 KB on the stack — fatal overflow. Static arrays live in .bss or .data section (RAM, but not the stack). They persist between calls; this is safe because riglPruneStep is called one layer at a time.
2	`#define RIGL_MAX_WEIGHTS 16384`	Compile-time constant limiting layer size. For STM32 with 320 KB RAM, a 16384-weight layer uses 64 KB for weights alone (float32). If you need larger layers, increase this constant and allocate more static buffer.
3	`if (N > RIGL_MAX_WEIGHTS) { PRINT_ERROR(...); return; }`	Guard against buffer overflow. If the layer is larger than our static buffer, print an error and return without crashing. A silent buffer overflow would corrupt memory unpredictably.
4	`for (size_t i = 0; i < N; i++) { s_wFloat[i] = readBytesAsFloat(...); ...}`	Build s_wFloat: a float copy of all weights (for the sort comparison function). Simultaneously collect s_activeIdx: indices of all active (mask=1) weights.
5	`g_sortWeights = s_wFloat; qsort(s_activeIdx, numActive, sizeof(size_t), cmpAbsWeightAsc);`	Set the global pointer, then sort. qsort rearranges s_activeIdx so that s_activeIdx[0] holds the index of the smallest-|w| active weight, s_activeIdx[1] holds the second smallest, etc.

6	`size_t numToPrune = (size_t)(cfg->sparsityRatio * (float)numActive);`	How many active weights to prune this step. Note: we prune relative to CURRENTLY ACTIVE count (numActive), not total N. This maintains the target sparsity ratio precisely.
7	`tensorBoolSet(mask, idx, false);`	Clear the mask bit: this weight is now pruned.
8	`writeFloatToByteArray(0.0f, w->data + idx*4);`	Zero the weight value. Pruned weights must be 0 so the forward pass can skip them (or if the forward pass doesn't check the mask, multiplying by 0 contributes nothing to the output).

Section 21.5 — riglGrowStep(): Reactivate High-Gradient Weights

The grow step reactivates the same number of weights that were just pruned. It selects from currently-pruned weights, choosing those with the largest gradient magnitude. A large gradient at a pruned weight means: 'if this weight existed, it would receive a large update — it would be useful.'

```
// RigL.c — riglGrowStep
static size_t s_inactiveIdx[RIGL_MAX_WEIGHTS];
static float s_gFloat[RIGL_MAX_WEIGHTS];
void riglGrowStep(parameter_t *parameter, riglConfig_t *cfg) {
tensor_t *g = getGradFromParameter(parameter);
tensor_t *w = getParamFromParameter(parameter);
tensor_t *mask = cfg->mask;
size_t N = calcNumberOfElementsByTensor(g);
// === Collect inactive indices and their gradient magnitudes ===
size_t numInactive = 0;
size_t numActive = 0;
for (size_t i = 0; i < N; i++) {
s_gFloat[i] = readBytesAsFloat(g->data + i*4);
if (!tensorBoolGet(mask, i)) {
s_inactiveIdx[numInactive++] = i;
} else {
numActive++;
}
}
// === Sort inactive indices by |g| descending ===
g_sortGrads = s_gFloat;
qsort(s_inactiveIdx, numInactive, sizeof(size_t), cmpAbsGradDesc);
// === Regrow the same number that was pruned ===
// numToPrune was (sparsityRatio * numActive_before_prune)
// After prune: numActive_now = numActive (already reduced by prune step)
// We want to restore the count: grow back what was pruned
size_t targetActive = (size_t)((1.0f - cfg->sparsityRatio) * (float)N);
size_t numToGrow = (numActive < targetActive) ?
(targetActive - numActive) : 0;
for (size_t i = 0; i < numToGrow && i < numInactive; i++) {
size_t idx = s_inactiveIdx[i];
tensorBoolSet(mask, idx, true);
// Weight starts at 0 — new gradients will grow it
// DO NOT copy the old gradient into the weight
// (the gradient tells us DIRECTION, not the VALUE the weight should have)
}
}
```

1	`tensor_t *g = getGradFromParameter(parameter);`	We need the GRADIENT tensor, not the weight tensor. The grow step selects which pruned weights have the highest gradient magnitude. These gradients were computed during the backward pass — even for pruned weights (because linearCalcWeightGrads computes dL/dW for ALL positions, not just active ones).
2	`if (!tensorBoolGet(mask, i)) { s_inactiveIdx[numInactive++] = i; }`	Collect INACTIVE (pruned) indices. !tensorBoolGet returns true when the bit is 0 (pruned). We store these indices in s_inactiveIdx for sorting.
3	`size_t targetActive = (size_t)((1.0f - cfg->sparsityRatio) * (float)N);`	How many weights should be active after the grow step. For sparsityRatio=0.8, N=8192: targetActive = 0.2 × 8192 = 1638. We want exactly 20% of weights active after prune+grow.

4	`size_t numToGrow = (numActive < targetActive) ? (targetActive - numActive) : 0;`	How many to grow: the deficit between current active count and target. If prune reduced active from 1638 to 0 (extreme), we grow back 1638. If prune reduced by 500 (e.g., 1638→1138), we grow back 500. Ternary operator: condition ? value_if_true : value_if_false.
5	`tensorBoolSet(mask, idx, true);`	Set the mask bit to 1 (active). This weight will now participate in the forward pass, receive gradient updates in the backward pass, and be updated by SGD.
6	`// Weight starts at 0 — new gradients will grow it`	The weight was pruned (set to 0) in the prune step. We leave it at 0 after reactivation. The very next forward+backward will compute a gradient for it, and SGD will update it to a non-zero value. This is correct: RigL paper initializes regrown weights at 0.

Section 21.6 — riglUpdate(): The Main Entry Point

```
// RigL.c — riglUpdate
// Call this every batch. It internally decides when to run prune+grow.
void riglUpdate(parameter_t *parameter, riglConfig_t *cfg) {
// Increment batch counter
cfg->batchCount++;
// Only run prune+grow every updateStep batches
if (cfg->batchCount % cfg->updateStep != 0) {
return; // not time yet
}
PRINT_DEBUG("RigL update at batch %zu\n", cfg->batchCount);
// Run prune step first, then grow step
riglPruneStep(parameter, cfg);
riglGrowStep(parameter, cfg);
// Zero gradients: the stale gradients that triggered grow
// should not be applied as weight updates
sgdZeroGrad(parameter);
}
```

1	`cfg->batchCount++;`	Increment the counter every time riglUpdate is called (every batch). This counter persists across batches because cfg is a pointer to a struct that lives in the caller's scope.
2	`if (cfg->batchCount % cfg->updateStep != 0) return;`	% is the modulo operator: remainder after division. batchCount % 100 == 0 when batchCount is 100, 200, 300, etc. So the prune+grow runs every 100 batches (or whatever updateStep is set to). On all other batches, return immediately.
3	`riglPruneStep(parameter, cfg); riglGrowStep(parameter, cfg);`	Always prune BEFORE growing. Pruning sets some mask bits to 0 and zeros their weights. Growing then sets different bits to 1 and leaves their weights at 0. The net effect: the active set shifts toward high-gradient weights.
4	`sgdZeroGrad(parameter);`	CRITICAL: zero gradients after the topology change. The gradients that the grow step used to select weights are now stale: they were computed for the OLD topology. If not zeroed, the next SGD step would apply these old gradients, potentially undoing the grow step (by updating the freshly-regrown weights with a gradient computed when they were inactive and had value 0).

Section 21.7 — Utility Functions: Statistics and Debug

```
// RigL.c — utility functions
// Compute fraction of weights currently active (1-sparsity)
float riglGetSparsityRatio(riglConfig_t *cfg, size_t totalWeights) {
size_t activeCount = 0;
for (size_t i = 0; i < totalWeights; i++) {
if (tensorBoolGet(cfg->mask, i)) activeCount++;
}
// Return the PRUNED fraction (0.8 = 80% pruned = 20% active)
return 1.0f - (float)activeCount / (float)totalWeights;
}
// Print statistics for debugging
void riglPrintStats(riglConfig_t *cfg, size_t totalWeights) {
float actualSparsity = riglGetSparsityRatio(cfg, totalWeights);
size_t activeCount = (size_t)((1.0f - actualSparsity) * totalWeights);
PRINT_DEBUG(
"RigL stats: batch=%zu, active=%zu/%zu, sparsity=%.1f%%\n",
cfg->batchCount,
activeCount, totalWeights,
actualSparsity * 100.0f
);
}
```

■ NOTE

riglGetSparsityRatio is O(N): it scans every bit in the mask. For a 16384-weight layer, this is 16384 bit reads — cheap (each is one bit operation). Call this once per epoch for monitoring; do not call inside the inner training loop.

Section 21.8 — riglMaskedSgdStepFloat(): Mask-Aware Weight Update

This function modifies the sgdStepFloat from Sgd.c to respect the sparsity mask. Active weights (mask=1) are updated normally. Pruned weights (mask=0) are skipped — their value remains 0.

```
// Sgd.c — riglMaskedSgdStepFloat
// (Add this function to Sgd.c; call from sgdStep when mask is non-NULL)
void riglMaskedSgdStepFloat(sgd_t *sgd, parameter_t *param,
riglConfig_t *cfg) {
tensor_t *w = getParamFromParameter(param);
tensor_t *g = getGradFromParameter(param);
tensor_t *mask = cfg->mask;
size_t n = calcNumberOfElementsByTensor(w);
float lr = sgd->learningRate;
float wd = sgd->weightDecay;
for (size_t i = 0; i < n; i++) {
// Skip pruned weights — do not update them
if (!tensorBoolGet(mask, i)) {
continue;
}
float wi = readBytesAsFloat(w->data + i*4);
float gi = readBytesAsFloat(g->data + i*4);
float grad_with_wd = gi + wd * wi;
float wNew = wi - lr * grad_with_wd;
writeFloatToByteArray(wNew, w->data + i*4);
}
}
```

1	`if (!tensorBoolGet(mask, i)) { continue; }`	The key RigL modification to SGD. 'continue' skips the rest of the current loop iteration and jumps to the next i. For pruned weights (mask=0): the weight stays at 0, gradient is ignored. For active weights (mask=1): normal gradient descent update applies.
2	`float gi = readBytesAsFloat(g->data + i*4);`	The gradient for this weight IS computed (by linearCalcWeightGrads) even if the weight was pruned. But we use the gradient for the GROW STEP decision (in riglGrowStep), not for weight updates. The 'continue' above prevents this gradient from actually being applied to the (zero) weight value.

Section 21.9 — Integration: Putting It All Together

Here is a complete setup and training loop showing exactly how to integrate RigL into an existing ODT training script for a speaker ID model.

```
// ===== Complete RigL integration example =====
// 1. Static buffers for a single linear layer [128, 64]
#define IN_FEATURES 64
#define OUT_FEATURES 128
#define N_WEIGHTS (IN_FEATURES * OUT_FEATURES) // 8192
// Mask: ceil(8192 / 8) = 1024 bytes
static uint8_t maskData[(N_WEIGHTS + 7) / 8];
static tensor_t maskTensor;
// Mask quantization (BOOL type)
static quantization_t maskQ;
// 2. Setup (called once before training)
void setupRigL(void) {
// 2a. Init mask quantization
initBoolQuantization(&maskQ);
// 2b. Mask shape (same as weight shape)
static size_t maskDims[2] = {OUT_FEATURES, IN_FEATURES};
static size_t maskOrder[2] = {0, 1};
static shape_t maskShape;
setShape(&maskShape, maskDims, 2, maskOrder);
// 2c. Build mask tensor
setTensorValues(&maskTensor, maskData, &maskShape, &maskQ, NULL);
// 2d. RigL config
static riglConfig_t riglCfg = {
.mask = &maskTensor,
.sparsityRatio = 0.8f,
.updateStep = 100,
.batchCount = 0,
};
// 2e. Initialize mask randomly (80% pruned)
riglInitMask(&riglCfg, 42); // seed=42 for reproducibility
}
// 3. Training loop
void trainWithRigL(int numEpochs) {
for (int epoch = 0; epoch < numEpochs; epoch++) {
for (int batch = 0; batch < numBatches; batch++) {
// Standard forward/backward/update
sgdZeroGrad(&weightParam);
dataLoaderGetNextBatch(&loader, &batchInput, &batchLabels);
layerForward(&linearLayer, &batchInput, &linear_out);
// ... rest of forward pass ...
float loss = crossEntropyLossFloat(...);
layerBackward(&linearLayer, ...);
// Use masked SGD (skip pruned weights)
riglMaskedSgdStepFloat(&sgd, &weightParam, &riglCfg);
// RigL topology update (every 100 batches)
riglUpdate(&weightParam, &riglCfg);
}
// Optional: print sparsity stats each epoch
riglPrintStats(&riglCfg, N_WEIGHTS);
}
}
```

1	`static uint8_t maskData[(N_WEIGHTS + 7) / 8];`	Static array for the mask bits. On MCU, this must be static (not stack). 8192 weights → 1024 bytes for the mask. Negligible memory overhead.
2	`initBoolQuantization(&maskQ);`	Set up the BOOL quantization type for the mask tensor. BOOL tensors pack 8 bits per byte, enabling tensorBoolGet/Set operations.

3	`.sparsityRatio = 0.8f, .updateStep = 100, .batchCount = 0`	Configuration: 80% sparsity, update topology every 100 batches, counter starts at 0. The compound literal initializer sets all fields by name — order doesn't matter.
4	`riglInitMask(&riglCfg, 42);`	Initialize mask with 80% bits set to 0 (pruned), 20% to 1 (active). seed=42 for reproducibility — same seed gives same initial pattern every run.
5	`riglMaskedSgdStepFloat(&sgd, &weightParam, &riglCfg);`	Replace the normal sgdStepFloat with the mask-aware version. Only active weights get gradient updates.
6	`riglUpdate(&weightParam, &riglCfg);`	Called every batch. Internally: only runs prune+grow when batchCount % 100 == 0. Increments batchCount every call.

═■ WORKED EXAMPLE

Expected training behaviour with RigL (sparsity=80%, updateStep=100): Batches 0-99: Train with initial random sparse topology. Loss decreases. Batch 100: riglUpdate: prune 20% smallest active, grow 20% best-gradient inactive. Topology shifts. Loss may temporarily increase (new weights start at 0). Batches 101-199: New topology trains. The regrown weights adapt quickly. Batch 200: Another prune+grow. Topology converges toward better patterns. ... After 500+ updates: topology stabilises. 20% of weights carry nearly all signal. Final model: 80% of weights are 0. Inference skips those. ~5× speedup on MCU.

Section 21.10 — Memory Budget on STM32 Nucleo-F746ZG

Component	Size (linear [128,64])	Notes
Weight tensor (dense)	8192 × 4 = 32 KB	float32 or SYM_INT32
Gradient tensor	8192 × 4 = 32 KB	Same shape as weight
RigL mask tensor	(8192+7)/8 = 1024 B ≈ 1 KB	BOOL, 1 bit per weight
Sort buffer (s_activeIdx)	16384 × 4 = 64 KB (static)	Shared across all layers, reused
Sort buffer (s_wFloat)	16384 × 4 = 64 KB (static)	Reused per call
riglConfig_t struct	≈ 16 bytes	Mask pointer + float + 2× size_t
Total overhead vs dense	+1 KB mask + shared sort bufs	Sort bufs reuse existing space
Total RAM usage	~193 KB for this layer alone	Leaves 127 KB for other layers

■■ IMPORTANT

The static sort buffers (s_activeIdx, s_wFloat, s_inactiveIdx, s_gFloat) are the largest RigL overhead: 4 × RIGL_MAX_WEIGHTS × 4 bytes = 256 KB for RIGL_MAX_WEIGHTS=16384. This exceeds the STM32's 320 KB RAM if used naively. Optimization: reduce RIGL_MAX_WEIGHTS to match the actual largest layer size, OR reuse the activation buffer (which is not in use during the RigL update phase).

■ NOTE

The gradient buffer is already allocated for training (it is the parameter_t.grad tensor). So the gradient sort buffer (s_gFloat) can be replaced by reading directly from g->data in the comparison function, eliminating the 64 KB copy. The g_sortGrads global would then point to (float*)g->data directly.

Chapter 22

C Language Deep Dive

Every concept used in ODT explained: cast, pointer, struct, memory, and more

Section 22.1 — Why Learn C for Embedded ML?

All ODT source code is written in C. Unlike Python, C gives you direct control over every byte in memory. On a microcontroller with 320 KB of RAM and no operating system, this control is not optional — it is required. This chapter explains every C concept that appears in the ODT library, from the most basic (variables, types) to the advanced (function pointers, bit operations).

Section 22.2 — Integer Types and Their Sizes

■ C CONCEPT: Why Exact-Width Integer Types?

In standard C, 'int' is NOT guaranteed to be 32 bits. On a 16-bit platform, int is 16 bits. On a 64-bit platform, it might be 64 bits. This makes code non-portable and causes subtle bugs. The solution: use exact-width types from <stdint.h>: int8_t — exactly 8 bits signed (-128 to +127) uint8_t — exactly 8 bits unsigned (0 to 255) int16_t — exactly 16 bits signed (-32768 to +32767) uint16_t — exactly 16 bits unsigned (0 to 65535) int32_t — exactly 32 bits signed (-2,147,483,648 to +2,147,483,647) uint32_t — exactly 32 bits unsigned (0 to 4,294,967,295) int64_t — exactly 64 bits signed (large range) size_t — unsigned, large enough for any array index (32-bit on STM32)

lue	Max Value	Use in ODT
	255	Raw bytes (dat
	+127	Small signed va
	65,535	qBits, qMaxBits
	+2.1B	SYM_INT32 ma
	4.3B	Unsigned 32-bi
egative	very positive	Matmul accumu
38	~+3.4e38	FLOAT32 weig
	4.3B	Array indices, s

═■ WORKED EXAMPLE

Choosing the right type: uint8_t qBits = 8; // fine: 8 fits in [0, 255] uint8_t qMaxBits = 16; // fine: 16 fits in [0, 255] // qMaxBits=16 means 'use 16-bit quantization range', not 'this field is 16 bits' int32_t mantissa = 32767; // fine: 32767 fits in int32_t int64_t acc = 0; // needed: sum of int32×int32 products can overflow int32 size_t n = calcNumberOfElements(tensor); // correct: n is always positive

Section 22.3 — Type Casting: What It Is and When to Use It

■ C CONCEPT: What is a Cast?
A cast tells the compiler: 'treat this value as if it has this other type'. Syntax: (target_type) expression Examples: int a = 65; char c = (char)a; // cast int to char: reinterpret 65 as 'A' float f = 3.7f; int i = (int)f; // cast float to int: truncates decimal → i = 3 void *ptr = buffer; float *fp = (float*)ptr; // cast void* to float*: pointer reinterpretation

There are four situations where casts appear in ODT:

22.3.1 Void Pointer Casts (the most common in ODT)

```
// tensor->data is void* or uint8_t* — raw bytes, no type
// To read elements, you must cast to the correct pointer type:
// Reading float values:
float *fptr = (float*)tensor->data;
float val = fptr[5]; // element 5 as float
// OR using readBytesAsFloat (which does the cast internally):
float val2 = readBytesAsFloat(tensor->data + 5*4);
// Reading int32_t values:
int32_t *iptr = (int32_t*)tensor->data;
int32_t ival = iptr[5];
// Accessing quantization config (void* qConfig):
symInt32QConfig_t *cfg = (symInt32QConfig_t*)tensor->quantization->qConfig;
float scale = cfg->scale;
// DO NOT cast incorrectly:
// Treating a float tensor as int32:
// int32_t wrong = ((int32_t*)floatTensor->data)[0]; // reads float bits as int!
```

■■ IMPORTANT
A cast does NOT convert the data — it reinterprets the SAME bits. float x = 3.14f stores as 0x4048F5C3 in memory (IEEE 754). Casting that memory to int32_t gives -1010580029 — the same 4 bytes, wrong meaning. Only cast to a type whose memory layout matches the actual stored data.

22.3.2 Numeric Casts (widen or narrow)

```
// Widening: smaller type → larger type (safe, automatic)
int32_t a = 32767;
int64_t b = (int64_t)a; // explicit is better — shows intent
// Without cast: int32_t * int32_t computed in 32-bit (may overflow)
// With cast: multiplication happens in 64-bit (safe)
int64_t product = (int64_t)a * (int64_t)a; // 32767^2 = 1,073,676,289 (fits!)
// Narrowing: larger type → smaller type (potentially lossy)
float f = 3.9999f;
int32_t n = (int32_t)f; // truncates decimal: n = 3 (NOT rounded!)
// Use roundf() to round before truncating:
int32_t rounded = (int32_t)roundf(f); // rounded = 4
// Integer to float
size_t count = 1000;
float ratio = (float)pruned / (float)count; // 0.0f–1.0f
// Without casts: integer division 100/1000 = 0 (truncated!)
// With casts: float division 100.0/1000.0 = 0.1f (correct!)
```

1	`int64_t product = (int64_t)a * (int64_t)a;`	Cast BOTH operands before multiplication. If only one is cast: (int64_t)a * b — C promotes the other operand too (integer promotion rules). Casting both is explicit and always correct.

2	`int32_t n = (int32_t)f;`	Truncation: the decimal is dropped, NOT rounded. 3.9 → 3, -3.9 → -3 (always toward zero). If you want rounding, use roundf(f) first, then cast.
3	`float ratio = (float)pruned / (float)count;`	When dividing two integers and expecting a float result, cast at least one operand to float. Integer division drops the remainder: pruned=1, count=3 → 1/3=0 (int), but (float)1/(float)3=0.333f (correct).

Section 22.4 — Pointers and Pointer Arithmetic

■ C CONCEPT: What is a Pointer?

A pointer is a variable that stores the MEMORY ADDRESS of another variable. int x = 42; // x is an integer, lives at some address in RAM int *p = &x; // p stores the ADDRESS of x (&x = 'address of x') int val = *p; // *p = 'value at the address p holds' = 42 Memory diagram: Address: 0x20001000 → value 42 ← x Address: 0x20001004 → value 0x20001000 ← p (stores address of x) Dereferencing (*p): follow the pointer to read/write the value it points to. Address-of (&x): get the memory address where x is stored. In ODT, pointers are everywhere: void *dataPTR; → points to the raw tensor data buffer tensor_t *input; → points to the input tensor struct parameter_t *param; → points to the weight+gradient pair

22.4.1 Pointer Arithmetic

```
// Pointer arithmetic: adding to a pointer moves by sizeof(type) bytes
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr; // p points to arr[0] (address 0x1000, say)
p + 1; // address 0x1004 (1 int = 4 bytes forward)
p + 2; // address 0x1008 (2 ints = 8 bytes forward)
*(p + 2); // value at 0x1008 = 30
p[2]; // same as *(p+2) — array notation is just shorthand
// In ODT: how readBytesAsFloat is called
// tensor->data is uint8_t* (1 byte per step)
float val = readBytesAsFloat(tensor->data + i * sizeof(float));
// tensor->data + i*4 = address of element i
// (uint8_t* + N) moves N bytes forward
// If tensor->data were float*, + i would move i*4 bytes — same result
// Why uint8_t* and not float*?
// Because the tensor might hold int32, float, or BOOL data.
// uint8_t* is the universal 'raw bytes' pointer — works for any type.
// The caller decides how to interpret the bytes via a cast.
```

■ C CONCEPT: sizeof() — The Size Operator

sizeof(type) returns the number of bytes a type occupies. sizeof(expression) returns the number of bytes the result would occupy. sizeof(uint8_t) = 1 sizeof(int32_t) = 4 sizeof(float) = 4 sizeof(int64_t) = 8 sizeof(tensor_t) = sum of all fields (may include padding) sizeof is evaluated at COMPILE TIME — no runtime cost. Always use sizeof(type) rather than hardcoding 4 — it adapts if you change the type.

22.4.2 Passing by Pointer vs Passing by Value

```
// Pass by value: function gets a COPY — changes don't affect original
void addOne_value(int x) { x += 1; } // modifies LOCAL copy
int a = 5;
addOne_value(a); // a is still 5
// Pass by pointer: function gets ADDRESS — can modify original
void addOne_pointer(int *x) { *x += 1; } // modifies the REAL variable
int b = 5;
addOne_pointer(&b); // b is now 6
// In ODT: ALL tensor functions take tensor_t* (pointer)
// Reason: tensor_t is a large struct. Copying it on every call would waste
// 20-40 bytes per call. Using a pointer is O(1) regardless of tensor size.
void matmulFloat32TensorsWithBias(
tensor_t *A, // pointer: no copy of A's struct
tensor_t *B, // pointer: no copy of B's struct
tensor_t *out, // pointer: writes result directly into caller's tensor
tensor_t *bias
);
```

═■ WORKED EXAMPLE

Why ODT uses pointers everywhere: tensor_t has fields: data (4B pointer), shape (4B pointer), quantization (4B pointer), sparsity (4B pointer) Total struct size: ~16 bytes If matmulFloat32TensorsWithBias took tensor_t by VALUE: Each call copies 4 tensor_t structs = 64 bytes pushed onto the stack. Plus, any modifications inside the function would NOT be visible to the caller. Using pointers (tensor_t*): Each call passes 4 pointers = 16 bytes (4 × 4 bytes). Modifications (writing to out->data) are visible to the caller immediately. The actual tensor data (potentially MB) is never copied — just the address.

Section 22.5 — Structs, Typedef, and Memory Layout

■ C CONCEPT: struct in C

A struct groups related variables into a single named type. struct point { float x; float y; }; struct point p; // declare a variable of type 'struct point' p.x = 3.0f; // access field x via '.' operator p.y = 4.0f; struct point *pp = &p; // pointer to p pp->x = 3.0f; // access field x via pointer using '->' operator The '->' operator is shorthand for (*pp).x — dereference then access field.

```
// typedef: create an alias so you don't need 'struct' every time
// Without typedef:
struct tensor { uint8_t *data; ... };
struct tensor t; // must write 'struct tensor' every time
// With typedef (ODT approach):
typedef struct tensor {
uint8_t *data;
shape_t *shape;
quantization_t *quantization;
sparsity_t *sparsity;
} tensor_t;
// Now: tensor_t t; (cleaner)
// Accessing fields:
tensor_t *t = &myTensor;
t->data // the raw data pointer
t->shape // the shape pointer
t->quantization // the quantization pointer
// Accessing nested structs:
t->shape->dimensions[0] // first dimension of the shape
// t->shape is a shape_t* pointer, so we use -> again
// ->dimensions[0] is the first element of the dimensions array
```

■ C CONCEPT: Struct Memory Layout and Padding

Structs in C are laid out in memory in field order, but the compiler may add 'padding' bytes between fields to ensure alignment. typedef struct { uint8_t a; // 1 byte // 3 bytes padding (to align b to 4-byte boundary) int32_t b; // 4 bytes uint8_t c; // 1 byte // 3 bytes padding (to align next to 4-byte boundary) } example_t; // sizeof = 12 bytes (not 1+4+1=6!) Rule of thumb: members should be ordered largest-first to minimize padding. In ODT, all struct fields are pointers (4 bytes each), so no padding occurs.

Section 22.6 — Enums: Named Integer Constants

```
// enum: a set of named integer constants
// From Quantization.h:
typedef enum {
INT32 = 0, // explicit values (optional)
FLOAT32 = 1,
SYM_INT32= 2,
SYM = 3,
ASYM = 4,
BOOL = 5
} quantizationType_t;
// Usage:
quantizationType_t t = FLOAT32; // t = 1
if (t == SYM_INT32) {
// handle SYM_INT32 case
}
switch (t) {
case FLOAT32: ...; break;
case SYM_INT32:...; break;
default: ...; break;
}
// WITHOUT enum, you'd write:
// if (t == 1) { ... } — not readable!
// WITH enum: if (t == FLOAT32) { ... } — self-documenting
```

■ NOTE

Enum values are ints. You can compare them with ==, use them in switch statements, and use them as array indices (as in the layer dispatch table: forwardFuncs[RELU]). Without the explicit = 0, 1, 2 assignments, enum values are automatically 0, 1, 2, ...

Section 22.7 — Memory Sections: Stack, Heap, .data, .bss

■ C CONCEPT: Where Variables Live in Memory

C variables can live in different memory regions: .text (ROM/Flash): → Function code, string literals, const global arrays. → Read-only. On STM32: 1 MB Flash. .data (RAM): → Global/static variables WITH initial values. → Example: static float lr = 0.01f; (initialized to 0.01) → Copied from Flash to RAM at startup. .bss (RAM): → Global/static variables WITHOUT initial values (initialized to 0 by startup). → Example: static size_t counter; (automatically = 0) → No copy needed from Flash; just zeroed in RAM. Stack (RAM, grows downward): → Local variables inside functions. Automatically allocated on function entry, freed on return. Fast but limited (4-8 KB on STM32). → VLAs (Variable-Length Arrays) also go here. Heap (RAM): → Dynamically allocated memory via malloc(). ODT avoids this. → Problem: malloc on MCU causes fragmentation; free() order matters. → Better: use static arrays (predictable, no fragmentation).

```
// Examples of memory section placement:
// .text (Flash) — function code
void myFunc(void) { ... }
// .data (RAM, initialized) — global with initial value
float learningRate = 0.01f;
// .bss (RAM, zero-initialized) — global without initial value
size_t batchCount; // automatically 0 at startup
// Stack — local variable inside function
void trainStep(void) {
float sum = 0.0f; // on stack, freed when trainStep returns
size_t i; // on stack
}
// Static local — like .bss but scoped to function
void riglUpdate(void) {
static size_t count = 0; // .data or .bss, persists across calls
count++;
}
// Static array (RigL buffer) — .bss, avoids stack overflow
static size_t s_activeIdx[16384]; // 64 KB in .bss, safe
// size_t s_activeIdx[16384]; // This would be on stack — 64 KB CRASH!
```

1	`float learningRate = 0.01f;`	Global with initialiser → .data section. At startup, the MCU's startup code copies this from Flash to RAM. ODT avoids many globals because they consume precious RAM permanently.
2	`size_t batchCount;`	.bss: global with no initialiser → automatically 0. ODT's riglConfig_t has batchCount as a struct field, set to 0 in the initialiser.
3	`float sum = 0.0f;`	Local variable → stack. Allocated when the function is called, freed when it returns. If you recurse deeply enough, stack overflow.
4	`static size_t count = 0;`	Static local: one allocation, shared across ALL calls to this function. count persists between calls (unlike a normal local which resets each call). First call: count=0. After count++: count=1. Second call: count=1 (not 0).
5	`static size_t s_activeIdx[16384];`	Static array: 64 KB in .bss, NOT on the stack. ODT's RigL buffers MUST be static for this reason. A non-static 64 KB local array would immediately cause a stack overflow on STM32.

Section 22.8 — Bit Operations (used in DTypes.c and BOOL tensors)

■ C CONCEPT: Bitwise Operators in C

C provides direct bit manipulation operators: & — bitwise AND: 1010 & 1100 = 1000 | — bitwise OR: 1010 | 1100 = 1110 ^ — bitwise XOR: 1010 ^ 1100 = 0110 ~ — bitwise NOT: ~1010 = 0101 (inverts all bits) << — left shift: 0001 << 3 = 1000 (multiply by 2^3 = 8) >> — right shift: 1000 >> 2 = 0010 (divide by 2^2 = 4) These are used in tensorBoolGet/Set to pack 8 booleans per byte.

```
// tensorBoolGet: read bit 'flatIndex' from a BOOL tensor
bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex) {
size_t byteIdx = flatIndex / 8; // which byte?
size_t bitIdx = flatIndex % 8; // which bit in that byte?
return (tensor->data[byteIdx] >> bitIdx) & 1u;
}
// tensorBoolSet: write bit 'flatIndex' to a BOOL tensor
void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value) {
size_t byteIdx = flatIndex / 8;
size_t bitIdx = flatIndex % 8;
if (value)
tensor->data[byteIdx] |= (1u << bitIdx); // set bit
else
tensor->data[byteIdx] &= ~(1u << bitIdx); // clear bit
}
```

1	`size_t byteIdx = flatIndex / 8;`	Integer division: which byte contains this bit? Element 0–7 are in byte 0, 8–15 in byte 1, etc. flatIndex=13: byteIdx = 13/8 = 1.
2	`size_t bitIdx = flatIndex % 8;`	Modulo: which bit within that byte? flatIndex=13: bitIdx = 13%8 = 5. Bit 5 of byte 1.
3	`(tensor->data[byteIdx] >> bitIdx) & 1u`	Right-shift the byte by bitIdx positions, then AND with 1. Example: byte=0b10100000, shift right by 5 → 0b00000101. AND with 1: 0b00000001 → result is 1 (true). The bit was set.
4	`(1u << bitIdx)`	Create a mask with ONLY bit bitIdx set. 1u=0b00000001, << 5 → 0b00100000. This is the 'bitmask' for bit 5.
5	`data[byteIdx] |= (1u << bitIdx);`	Set the bit: OR with the bitmask. 0b10000000 | 0b00100000 = 0b10100000. Bit 5 is now set.
6	`data[byteIdx] &= ~(1u << bitIdx);`	Clear the bit: AND with the INVERTED bitmask. ~0b00100000 = 0b11011111. 0b10100000 & 0b11011111 = 0b10000000. Bit 5 is cleared. Other bits unchanged.

Section 22.9 — Macros and the C Preprocessor

■ C CONCEPT: The C Preprocessor

Before compiling C code, the compiler runs the PREPROCESSOR. The preprocessor handles lines starting with #: #include <file.h> — paste the contents of file.h here #define NAME value — replace all occurrences of NAME with value #ifdef NAME — compile the following block only if NAME is defined #ifndef NAME — compile the following block only if NAME is NOT defined #endif — end an #ifdef or #ifndef block The preprocessor runs BEFORE the compiler and produces modified C source. Macros are text substitution — they are not functions and have no types.

```
// From Common.h — debug macros using ifdef
#ifdef DEBUG
#define PRINT_DEBUG(fmt, ...) printf("[DEBUG] " fmt, ##__VA_ARGS__)
#define PRINT_ERROR(fmt, ...) printf("[ERROR] " fmt, ##__VA_ARGS__)
#else
#define PRINT_DEBUG(fmt, ...) do { } while(0)
#define PRINT_ERROR(fmt, ...) do { } while(0)
#endif
// Usage:
PRINT_DEBUG("weight[%zu] = %.4f\n", i, val);
// In DEBUG build, expands to: printf("[DEBUG] weight[%zu] = %.4f\n", i, val);
// In release build, expands to: do { } while(0); (nothing — zero overhead)
// Function-like macro with argument:
#define MAX(a, b) ((a) > (b) ? (a) : (b))
int result = MAX(x, y);
// Expands to: int result = ((x) > (y) ? (x) : (y));
// Parentheses around args prevent operator-precedence bugs:
// MAX(1+2, 3) without parens: 1+2 > 3 ? 1+2 : 3 → ambiguous precedence
// With parens: ((1+2)) > ((3)) ? ((1+2)) : ((3)) → correct
// Constant macro:
#define RIGL_MAX_WEIGHTS 16384
// All occurrences of RIGL_MAX_WEIGHTS in the code are replaced with 16384
// Change this once → all array sizes and checks update automatically
```

1	`#ifdef DEBUG ... #else ... #endif`	Conditional compilation: the PRINT_DEBUG macro includes real printf only when DEBUG is defined (e.g., via gcc -DDEBUG). In release builds, PRINT_DEBUG expands to an empty do-while — the compiler optimises it away, leaving zero overhead.
2	`##__VA_ARGS__`	Variadic macro argument. '...' accepts any number of arguments. __VA_ARGS__ pastes them into the expansion. ## before it suppresses the trailing comma when no variadic args are given.
3	`do { } while(0)`	The 'do nothing' idiom for empty macros. Using just an empty ; would cause issues in if/else without braces. do { } while(0) is a safe, complete statement.
4	`#define MAX(a,b) ((a) > (b) ? (a) : (b))`	A function-like macro. Note: 'a' is evaluated TWICE. MAX(x++, y++) would increment x and y each twice — dangerous. For simple values (no side effects), macros are fine.

Section 22.10 — const, static, and extern Qualifiers

	Applies to	Meaning	Example in ODT
	variables, pointers	Value cannot be changed after initialization. Compiler may place in Flash (ROM) to save RAM.	const float PI = 3.14159f;
	local variables	Variable persists across function calls (in .bss or .data). Not on the stack.	static size_t s_activeIdx[16384]; in RigL.c
	global variables, functions	Limits scope to the current file. Not visible from other .c files.	static int32_t addInt32s(int32_t a, int32_t b)
	variables, functions	Declares that the variable/function is DEFINED elsewhere. Tells linker to resolve the symbol.	extern float learningRate; in a header
	variables	Tells compiler: this variable can change outside normal code flow (hardware register, interrupt). Do not cache in register.	volatile uint32_t *DWT_CYCCNT;

```
// const examples:
const float SCALE = 0.001f; // cannot assign SCALE = 0.002f
void foo(const tensor_t *t) { // cannot modify t->data inside foo
// t->data = NULL; // COMPILE ERROR
}
// static (local) — persists between calls:
void countCalls(void) {
static int n = 0; // initialized once, persists
n++;
PRINT_DEBUG("Called %d times\n", n);
}
// static (global) — private to file:
// In Sgd.c:
static void sgdStepFloat(sgd_t *sgd, parameter_t *p) { ... }
// sgdStepFloat cannot be called from Linear.c — it is private to Sgd.c
// extern — declare, not define:
// In a .h file:
extern float g_learningRate; // declared: 'it exists somewhere'
// In a .c file:
float g_learningRate = 0.01f; // defined: 'it actually lives here'
```

Section 22.11 — Integer Overflow and Promotion Rules

■ C CONCEPT: Integer Overflow in C

Signed integer overflow is UNDEFINED BEHAVIOUR in C — the result is unpredictable. Unsigned integer overflow wraps around (modular arithmetic) — well defined. int32_t a = 2147483647; // INT32_MAX int32_t b = a + 1; // UNDEFINED BEHAVIOUR (signed overflow) uint32_t c = 4294967295; // UINT32_MAX uint32_t d = c + 1; // 0 (wraps around — defined!) In ODT matmul: using int64_t for accumulation prevents overflow: Max product: 32767 × 32767 = 1,073,676,289 < INT32_MAX (2,147,483,647) ONE product fits. But 100 such products: 107,367,628,900 > INT32_MAX → OVERFLOW int64_t max: 9,223,372,036,854,775,807 → thousands of products fit safely

```
// Integer promotion rules:
// When two operands have different types, the 'narrower' is promoted
// Case 1: int32_t * int32_t stays int32_t
int32_t a = 32767, b = 32767;
int32_t c = a * b; // 1,073,676,289 — fits! (but if K products are summed → risk)
// Case 2: cast before multiply to use 64-bit
int64_t d = (int64_t)a * b; // b is promoted to int64_t → 64-bit multiply
// Case 3: int / int = int (truncated!)
int x = 5, y = 2;
int result = x / y; // 2 (not 2.5 — decimal truncated)
float ratio = (float)x / y; // 2.5f (x promoted to float first)
// Case 4: uint8_t + uint8_t: both promoted to int first
uint8_t a8 = 200, b8 = 100;
uint8_t sum = a8 + b8; // WARNING: 300 doesn't fit in uint8_t!
// 300 % 256 = 44 (wraps around — silent bug!)
uint16_t safeSum = (uint16_t)a8 + b8; // correct: 300
```

■■ IMPORTANT

uint8_t arithmetic silently wraps around! 200 + 100 = 44 (not 300) when stored in uint8_t. In ODT, this is why we use int32_t for mantissas (not int8_t) and int64_t for accumulation (not int32_t). Always think about the maximum value an expression can produce before choosing the result type.

Section 22.12 — memset and memcpy: Raw Memory Operations

```
// memset: fill N bytes with a given byte value
// Signature: void *memset(void *ptr, int value, size_t count)
uint8_t buf[10];
memset(buf, 0, 10); // set all 10 bytes to 0x00
memset(buf, 0xFF, 10); // set all 10 bytes to 0xFF
// In sgdZeroGrad:
memset(grad->data, 0, bytes); // zero all gradient mantissas
// In riglInitMask:
memset(mask->data, 0xFF, maskBytes); // set all bits to 1 (all active)
// WARNING: memset uses BYTE values (0–255)
// memset(arr, 1, 4*n); does NOT set int32 elements to 1!
// It sets each BYTE to 1: each int32 = 0x01010101 = 16,843,009
// To zero int32 array: memset(arr, 0, 4*n) — works because 0x00000000 = 0
// memcpy: copy N bytes from source to destination
// Signature: void *memcpy(void *dst, const void *src, size_t count)
float src_val = 3.14f;
float dst_val;
memcpy(&dst_val, &src_val, sizeof(float)); // copy 4 bytes
// dst_val is now 3.14f
// In readBytesAsFloat (DTypes.c):
float readBytesAsFloat(const uint8_t *bytes) {
float value;
memcpy(&value, bytes, sizeof(float)); // copy 4 bytes from buffer into float
return value;
}
// WHY memcpy and not (float)*bytes?
// Direct cast via pointer (float*)bytes is undefined behaviour when bytes
// is not float-aligned. memcpy is always safe regardless of alignment.
```

1	`memset(buf, 0, 10);`	Fill 10 bytes with 0x00. The 'value' argument (0) is an int but only the lowest byte (0x00) is written. Safe for zeroing int32 arrays because 0x00000000 = 0 in all integer representations.
2	`memset(mask->data, 0xFF, maskBytes);`	0xFF = 0b11111111: sets all 8 bits of each byte to 1. For a BOOL tensor: this marks all elements as active (bit=1 means active).
3	`memcpy(&value, bytes, sizeof(float));`	Copy 4 bytes from bytes[] into the float variable. memcpy avoids the undefined-behaviour of direct pointer casting. The sizeof(float) = 4 bytes copied exactly reconstruct the original float.

Section 22.13 — Quick Reference: C Concepts in ODT

Concept	Symbol/Syntax	Where in ODT
Pointer declaration	type *name;	tensor_t *input everywhere
Dereference	*ptr	*cfg = access value at pointer
Address-of	&var	&cfg to pass struct by pointer
Arrow operator	ptr->field	layer->config.linear, tensor->data
Cast (pointer)	(type*)ptr	(float*)tensor->data, (symInt32QConfig_t*)qConfig
Cast (numeric)	(type)value	(int64_t)aVal * bVal, (float)count
sizeof	sizeof(type)	sizeof(float)=4, sizeof(int32_t)=4
Enum	typedef enum { A,B } t;	quantizationType_t, layerType_t
Struct	typedef struct { } t;	tensor_t, shape_t, riglConfig_t
Union	typedef union { } t;	layerConfig_t (one pointer for any layer)
Function pointer	typedef ret (*name)(args);	layerForwardFunc_t, int32ArithFunc_t
Static local	static type name;	RigL sort buffers, batch counter
memset	memset(ptr, val, n)	sgdZeroGrad, riglInitMask
memcpy	memcpy(dst, src, n)	readBytesAsFloat, writeFloatToByteArray
Bitwise ops	& | ^ ~ << >>	tensorBoolGet/Set in DTypes.c
Modulo	a % b	riglUpdate batch counter, byteIdx for BOOL
Ternary	cond ? a : b	riglBackward: dX = (x>0) ? dY : 0
VLA	type arr[n];	Index arrays in Arithmetic.c (nDim small)
#define	#define NAME value	RIGL_MAX_WEIGHTS, DEBUG macros
#ifdef	#ifdef NAME ... #endif	PRINT_DEBUG only in debug builds

Chapter 23

Complete Glossary

Every Technical Term Used in This Book, Explained in Plain Language

This glossary defines every term, abbreviation, and concept introduced in the book. Entries are grouped by topic so related terms appear together. References to ODT source files are given where relevant.

A — C Language Fundamentals

Term	Definition
.bss	The Block Started by Symbol section. A region of RAM for global and static variables that are zero-initialized at program start. Costs no flash (linker fills it at boot). ODT uses .bss for static sort buffers in RigL.c.
.data	A RAM section for initialized global/static variables (e.g., float x = 3.14f). The initial values are stored in flash and copied to RAM at boot.
.text	The code section — where compiled machine instructions live. Stored in flash. Read-only at runtime.
address-of operator (&)	& before a variable returns its memory address as a pointer. int x = 5; int *p = &x; — p now holds the address where x lives.
bit field	A struct field declared with a width in bits: uint8_t flag : 1; Packs multiple flags into one byte. Used in mask representations.

cast (type cast)	(target_type)expression — instructs the compiler to treat a value as a different type. (float)x converts integer x to float. (symI nt32QConfig_t*)qConfi g converts void* to the actual config type. Widening cast (int to long) is always safe. Narrowing cast may lose data.
const	Tells the compiler this value must not change. const float PI = 3.14159f; Placed in flash (.rodata) by the compiler. const tensor_t *t means 't points to a tensor that cannot be modified.'
dereference (*)	*pointer reads or writes the value at the address stored in pointer. int *p = &x; int y = *p; — y gets the value of x.
enum	An enumeration — a named set of integer constants. typedef enum { FLOAT32=0, INT32=1, SYM_INT32=2, BOOL=3 } quantizationType_t; Makes code readable (FLOAT32 instead of 0).
extern	Declares a symbol defined in ANOTHER translation unit. Tells the compiler 'this exists somewhere, the linker will find it.' Used to share global variables or function declarations across .c files.
fused-multiply-accumulate (FMA)	A hardware instruction that computes a*b + c in ONE step, with only one rounding. ARM Cortex-M7 has an FPU with single-precision FMA. matmul inner loop: acc += aVal * bVal — compiled to VFMA on Cortex-M7.

function pointer	A variable holding the address of a function. typedef void (*forwardFn_t)(linearConfig_t*, tensor_t*, tensor_t*); Allows dispatching different layer types through the same interface. Used in Layer.c's forwardFuncs[] dispatch table.
guard (include guard)	#ifndef FILENAME_H / #define FILENAME_H / ... / #endif. Prevents a header file from being included more than once per compilation unit.
heap	Dynamic memory region managed by malloc/free. Exists on hosted systems (PC). On bare-metal STM32 with ODT: malloc is NOT used. All memory is static or stack.
macro	A preprocessor text substitution. #define MAX(a,b) ((a)>(b)?(a):(b)). Expanded before compilation. No type safety. PRINT_DEBUG in ODT is a macro that can be compiled away with #ifdef.
memcpy	void memcpy(dst, src, n_bytes) — copies exactly n_bytes from src to dst. Used in readBytesAsFloat and writeFloatToByteArray for safe type-punning. Faster than element-wise copy for large buffers.
memset	void memset(dst, byte_value, n_bytes) — fills n_bytes at dst with byte_value. memset(buf, 0, N*4) zeros N float32 values. memset(mask, 0xFF, N/8) sets all mask bits to 1 (all active).

NULL	A null pointer constant (typically 0). Represents 'no valid pointer.' Dereferencing NULL causes a hard fault on STM32. Always check != NULL before use.
pointer arithmetic	Adding an integer to a pointer moves it by integer * sizeof(element) bytes. float *p = buf; *(p+3) reads the 4th float. In ODT: data + i*4 moves i*4 bytes into the byte buffer.
signed vs unsigned	signed int holds -2^31 to 2^31-1. unsigned int holds 0 to 2^32-1. Overflow of signed int is undefined behavior in C. Overflow of unsigned int wraps modulo 2^32 (defined).
size_t	An unsigned integer type large enough to hold any array index. On 32-bit STM32: size_t = uint32_t = 4 bytes. Used for all array sizes, loop indices, and byte counts in ODT.
stack	Automatic memory for local variables and function call frames. Grows downward in memory. On STM32: typically 4-8 KB. Stack overflow (e.g., large VLAs) causes hard fault with no warning.
static	Two meanings: (1) At file scope: symbol is private to this .c file (not linked elsewhere). (2) At function scope: variable persists between calls, lives in .bss/.data. ODT's s_activeIdx in RigL.c is static to avoid stack overflow.

struct	A composite type grouping named fields: struct Point { int x; int y; }. Fields are laid out sequentially in memory (with possible alignment padding). tensor_t, layer_t, riglConfig_t, etc. are all structs.
typedef	Creates a type alias. typedef unsigned char uint8_t; Makes types more readable and portable.
union	Like a struct but ALL fields share the SAME memory location. Size = largest member. Used in layerConfig_t to hold either linearConfig_t* or reluConfig_t* without extra memory per layer type.
uint8_t / uint16_t / uint32_t / uint64_t	Fixed-width unsigned integers. uint8_t = 1 byte (0-255), uint16_t = 2 bytes (0-65535), uint32_t = 4 bytes (0 - 4.29 billion), uint64_t = 8 bytes. Defined in stdint.h. Essential for embedded code where int sizes vary by platform.
void*	A generic (typeless) pointer. Can point to anything. Must be cast to the actual type before dereferencing. Used in qConfig fields and qsort callbacks to accept any type.
VLA (Variable Length Array)	int arr[n] inside a function with non-constant n. Allocated on the stack at runtime. DANGEROUS on embedded: no size check, stack overflow crashes silently.
XOR (^)	Exclusive OR bitwise operation. a ^ b: output 1 if exactly one input is 1. 0 ^ 0=0, 0 ^ 1=1, 1 ^ 0=1, 1 ^ 1=0. Used in xorshift32 RNG and bit-manipulation operations.

B — Machine Learning and Neural Networks

Term	Definition
activation function	A non-linear function applied after a linear layer. Without it, stacked linear layers collapse to one linear transformation. ODT includes ReLU, Softmax, Dropout.
backpropagation (backprop)	The algorithm to compute dL/dW for all weights in a neural network. Applies the chain rule layer by layer from the loss back to the input. ODT: linearCalcWeightGradsFloat + linearCalcInputGradsFloat implement this.
batch	A subset of training samples processed together. Batch size = 1 (single sample per step). Larger batches give more accurate gradient estimates but use more memory.
bias	An additive term in a linear layer: Y = X @ W + b. The bias allows the layer to shift its output independently of the input. ODT stores bias as a separate tensor in linearConfig_t.
chain rule	d(f(g(x)))/dx = f'(g(x)) * g'(x). Backpropagation is the chain rule applied recursively through layers. Each layer's backward pass multiplies the incoming gradient by the local Jacobian.
cross-entropy loss	-sum(y_true * log(y_pred)). Measures distance between predicted probability distribution and true one-hot label. Gradient w.r.t. softmax output: y_pred - y_true.

dense (fully-connected) layer	A layer where every input connects to every output neuron. Computation: Y = X @ W^T + b. Weight matrix shape: [out_features, in_features]. Called 'linear layer' or 'FC layer' in code.
dropout	Randomly sets a fraction of activations to zero during training. Probability p = drop rate (e.g., 0.5 = drop 50%). Reduces overfitting by preventing co-adaptation of neurons. Disabled during inference (dropoutForwardFloat is a no-op in inference mode).
epoch	One complete pass through the entire training dataset. Training typically runs for 10-100 epochs.
forward pass	Computing the network output from input to loss. Each layer applies its transformation in sequence. The result is a loss value and predicted output.
gradient	The partial derivative of the loss with respect to a parameter. Tells us: if we increase weight w by a tiny epsilon, how much does the loss change? SGD moves weights in the negative gradient direction to reduce loss.
gradient descent	Optimization algorithm: w = w - lr * dL/dw. Repeatedly moves each weight in the direction that reduces the loss. ODT implements vanilla SGD (no momentum, no Adam).
learning rate (lr)	Step size for gradient descent. Too large: training diverges. Too small: training is very slow. Typical values: 0.01, 0.001. Requires tuning per problem.

logits	The raw unnormalized scores output by the last linear layer, before softmax. Logits can be any real number.
loss	A scalar measure of how wrong the model's predictions are. We minimize the loss during training. ODT uses cross-entropy loss for speaker ID classification.
one-hot encoding	A label vector where only one element is 1.0 and the rest are 0.0. Speaker 3 of 10: [0, 0, 1, 0, 0, 0, 0, 0, 0, 0].
overfitting	When a model memorizes training data but fails to generalize to new data. Signs: low training loss, high validation loss. Mitigated by dropout, weight decay, or collecting more data.
ReLU (Rectified Linear Unit)	f(x) = max(0, x). The most common activation function. Simple, fast to compute, avoids vanishing gradients. ODT: reluForwardFloat, reluBackwardFloat.
softmax	Converts a vector of logits to a probability distribution. softmax(x)_i = exp(x_i) / sum(exp(x_j)). Output sums to 1.0. ODT subtracts max(x) before exp for numerical stability.
speaker identification	The task of determining WHICH speaker produced a given audio clip. A classification task: N speakers = N output classes. The thesis uses LibriSpeech data: 2484 speakers.
weight	A trainable parameter in a neural network. Stored as a float32 or SYM_INT32 (quantized) value in ODT. Updated by the optimizer each training step.

weight gradient (dL/dW)	How much the loss changes per unit change in a weight. Computed by backpropagation. ODT stores gradients in parameter_t->grad tensor.

C — Quantization and Fixed-Point Training (FQT)

Term	Definition
FQT (Fixed-point Quantization Training)	Training directly with quantized weights (integers with a scale factor) instead of full float32. Deutel-style: maintain SYM_INT32 weights, compute forward/backward in quantized form. Benefit: smaller model, faster inference on MCU without float hardware.
INT32	A 32-bit signed integer. Range -2^31 to 2^31-1. Used as the underlying type for SYM_INT32 mantissas in ODT.
int64_t accumulator	In SYM_INT32 matmul, int32 * int32 can produce up to 2^62. To avoid overflow during accumulation, the running sum is stored in int64_t. The final result is scaled back to int32 at the end.
mantissa	The integer part of a SYM_INT32 value. real_value = mantissa * scale. Example: mantissa=-1503, scale=0.001 -> real_value=-1.503. Stored in tensor data as int32_t bytes.
qBits	The quantization bit width for storage/deployment. qBits=8 means weights are stored as int8 values (range -127 to +127). Determines how many distinct values can be represented.

qMax	The maximum representable quantized value: 2^(qMaxBits-1) - 1. qMaxBits=16 -> qMax=32767. Weights are clamped to [-qMax, +qMax] during training.
qMaxBits	The bit width used during training for the RANGE check. Default: 16 bits (qMax=32767). Why 16: 32767 * 32767 = 1.07 billion < int32_t max (2.1 billion). Products of 16-bit mantissas fit safely in a 32-bit accumulator without overflow.
quantization	The process of approximating real-valued numbers with a discrete set of values. float32 has 2^32 possible values. int8 has only 256. The trade-off: less memory and faster arithmetic vs slightly lower accuracy.
requantization	Rescaling a quantized tensor after arithmetic operations. After matmul, the accumulated sum may have a different scale. Requantization adjusts scale and clamps to valid range. ODT: quantizeSymInt32() performs this.
rounding mode	How to round a real value to the nearest integer. Options: ROUND_HALF_EVEN (banker's rounding), ROUND_HALF_AWAY (round 0.5 up), SR_HALF_AWAY (stochastic rounding). Stochastic rounding is essential for FQT convergence.
scale	Multiplier that converts a quantized integer to a real value. real = mantissa * scale. Scale is learned/set per tensor. Smaller scale = finer resolution.

stochastic rounding (SR_HALF_AWAY)	Add uniform random noise in [0, 0.5) before rounding. Expected value of rounded result = true real value. Preserves small gradient updates that would otherwise round to zero. Critical for FQT: without SR, small gradients have no effect on quantized weights.
symmetric quantization (SYM_INT32)	Quantization where the zero point is always 0. Range is symmetric around 0: [-qMax, +qMax]. Simpler than asymmetric (no zero-point offset needed in matmul).
type-punning	Reading the same bytes as a different type. E.g., reading 4 bytes of a float as a uint32_t to examine its bits. In C, only safe via memcpy (direct casts via pointer violate strict aliasing). ODT uses readBytesAsFloat/writeFloatToByteArray for safe type-punning.

D — RigL and Sparse Training

Term	Definition
active weight	A weight where mask=1. Participates in forward pass computation. Updated by the optimizer. Currently non-zero (or allowed to be non-zero).
dynamic sparsity	Sparse training where the PATTERN of zero weights changes during training. RigL is dynamic sparse. Opposite: static sparsity (mask fixed after init).
Evci et al. 2020	The paper 'Rigging the Lottery: Making All Tickets Winners' by Utku Evci et al., NeurIPS 2020. Introduced the RigL algorithm. Available at: https://arxiv.org/abs/1911.11134

grow step	The second half of a RigL topology update. Selects inactive weights with the highest gradient magnitude and reactivates them. Newly grown weights start with value 0.0 and grow from future gradient steps.
lottery ticket hypothesis	The observation (Frankle & Carlin, 2019) that dense networks contain sparse subnetworks ('winning tickets') that can train to full accuracy. RigL's title 'Rigging the Lottery' refers to finding these tickets during training.
mask	A BOOL tensor (one bit per weight) indicating which weights are active (1) or pruned (0). riglConfig_t->mask in ODT.
masked matmul	Matrix multiplication that skips positions where mask=0. riglMaskedMatmulFloat32() in RigL.c. Reduces FLOPS proportionally to sparsity ratio.
prune step	The first half of a RigL topology update. Deactivates the active weights with the smallest absolute value. Sets their mask=0 and weight value=0.
pruned weight	A weight where mask=0. Excluded from forward pass. Not updated by optimizer. Set to 0.0. Its gradient is still computed (needed for grow step).
rejection sampling	A technique to sample from a distribution by generating candidates and discarding those outside the target domain. Used in riglInitMask to ensure unique pruned positions.

RigL (Rigging the Lottery)	Sparse training algorithm: maintain fixed sparsity ratio, periodically update which weights are zero by pruning small weights and growing high-gradient weights.
sparsity pattern	Which specific weight positions are zero at a given point in training. In RigL, the pattern evolves every T batches while sparsity ratio stays constant.
sparsity ratio	Fraction of weights that are zero (pruned). sparsityRatio=0.8 means 80% of weights are zero. Also called 'density' (1-sparsityRatio) or 'fraction of non-zeros.'
static sparsity	Sparse model where the zero positions are fixed (set once and never changed). Simpler than RigL but typically lower accuracy for a given sparsity ratio.
T (update interval)	Number of training batches between consecutive RigL topology updates. riglConfig_t->updateStep. Typical value: 100. RigL in the paper uses a cosine decay schedule for T.
topology	The structure of the sparse network — which weights are active. RigL updates the topology every T batches by prune+grow.

E — Embedded Systems and STM32

Term	Definition

ARM Cortex-M7	A 32-bit RISC processor core used in the STM32F746. Features: branch predictor, instruction cache, data cache, double-precision FPU (optional), SIMD DSP instructions. Maximum clock: 216 MHz on the F746.
bare-metal	Running code directly on hardware with no operating system. No processes, no virtual memory, no filesystem (without additional hardware). The programmer manages all resources directly.
CMSIS (Cortex Microcontroller Software Interface Standard)	A vendor-independent hardware abstraction layer for ARM Cortex-M processors. Provides standardized access to CPU registers, system timer (SysTick), and NVIC.
flash memory	Non-volatile (persistent) storage on the STM32. Holds the program code (.text), read-only constants (.rodata), and initialized variable values (.data init copy). STM32F746: 1 MB flash.
FPU (Floating Point Unit)	Hardware that executes floating-point operations (add, mul, div) without software emulation. Cortex-M7 has a full double-precision FPU. Single-precision FMA executes in 1 clock cycle.
HAL (Hardware Abstraction Layer)	STM32CubeHAL is ST's library of functions for configuring peripherals (UART, GPIO, I2C, SPI, etc.) without writing register-level code. ODT uses HAL_UART for transmitting trained weights to the PC.

hard fault	An unrecoverable processor exception on ARM. Causes: NULL pointer dereference, stack overflow, unaligned memory access, divide by zero (in integer code). On bare-metal with no handler: the MCU freezes or resets.
LibriSpeech	A corpus of 1000 hours of English audiobook speech from LibriVox. The thesis uses a subset: 2484 speakers. Audio features (MFCCs or spectrograms) are extracted on PC, stored as .npy files.
MFCC (Mel-Frequency Cepstral Coefficients)	Audio features commonly used for speech tasks. Computed by: FFT -> mel filterbank -> log -> DCT. Produces a compact representation of the vocal tract shape. A 128-dimensional MFCC vector is a typical input feature for speaker ID.
Nucleo-F746ZG	An STM32 development board with: STM32F746ZG MCU, USB, UART, LEDs, buttons, Arduino headers. The target hardware for the thesis experiments.
NVIC (Nested Vectored Interrupt Controller)	ARM hardware that manages interrupts. In ODT bare-metal usage: interrupts are mostly disabled or unused.
SRAM (Static Random Access Memory)	Fast volatile (loses data on power-off) memory for runtime data storage. STM32F746: 320 KB SRAM. Holds stack, heap (if used), and all ODT tensor buffers.
STM32CubeIDE	ST Microelectronics' integrated development environment for STM32. Based on Eclipse + GCC. Used to compile, flash, and debug ODT on the board.

UART (Universal Asynchronous Receiver-Transmitter)	A serial communication protocol. Used in ODT to stream training data from PC to STM32 and to send trained model weights back to the PC for evaluation.

F — ODT Library: Key Structs and Functions

Term	Definition
calcNumberOfElementsByTensor()	Returns the total number of elements in a tensor (product of all dimensions). Shape [3,4,5] -> 60 elements.
calcNumberOfBytesForData()	Returns byte count for N elements of a given quantization type. FLOAT32: N*4. INT32: N*4. SYM_INT32: N*4. BOOL: ceil(N/8).
getDimensionsByIndex()	Returns the size of a LOGICAL dimension index, accounting for any transposes (orderOfDimensions).
layer_t	A struct representing one neural network layer. Fields: layerType, config (union of layer-specific configs), forwardInput (pointer to last input for backprop).
layerConfig_t	A union holding a pointer to the specific config for each layer type: linearConfig_t* OR reluConfig_t* OR softmaxConfig_t* OR dropoutConfig_t*.
linearConfig_t	Config for a linear (fully-connected) layer. Fields: weights (parameter_t), bias (parameter_t).
parameter_t	Bundles a weight tensor and its gradient tensor together. Fields: param (tensor_t for weights), grad (tensor_t for gradients).
quantization_t	Describes how tensor values are encoded. Fields: type (quantizationType_t enum), qConfig (void* to specific config).

readBytesAsFloat()	Safely reads 4 bytes from a uint8_t* buffer as a float via memcpy. Avoids strict aliasing undefined behavior.
setTensorValues()	Initializes all fields of a tensor_t: data, shape, quantization, sparsity.
shape_t	Describes tensor dimensions. Fields: numberOfDimensions, dimensions[], orderOfDimensions[].
symInt32QConfig_t	Configuration for SYM_INT32 quantization. Fields: scale (float), qMaxBits (uint8_t), roundingMode.
tensor_t	The central data structure: a multi-dimensional array. Fields: data (uint8_t* raw bytes), shape (shape_t*), quantization (quantization_t*), sparsity (sparsity_t* for RigL mask).
tensorBoolGet()	Reads one bit from a BOOL tensor at a given linear index.
tensorBoolSet()	Writes one bit in a BOOL tensor at a given linear index.
transposeTensor()	Swaps two dimensions in orderOfDimensions. Does NOT move data — just changes the index mapping. O(1) operation.
writeFloatToByteArray()	Safely writes a float as 4 bytes into a uint8_t* buffer via memcpy.

G — Mathematics and Algorithms

Term	Definition
Fisher-Yates shuffle	An O(N) algorithm for uniformly random permutations. For i from N-1 down to 1: swap element i with random element j in [0,i]. All N! orderings equally likely. Used in rngShuffleIndices().

IEEE 754	The standard for floating-point arithmetic. float32: 1 sign bit, 8 exponent bits, 23 mantissa bits. Defines special values: +/-inf, NaN. All modern CPUs and the STM32 FPU implement this standard.
matmul (matrix multiplication)	Output[i][j] = sum_k(A[i][k] * B[k][j]). O(M*N*K) multiplications. The dominant operation in neural network forward and backward passes.
modulo operator (%)	a % b = remainder when a is divided by b. 5 % 3 = 2. Used for wrap-around counters (batchCount % updateStep).
numerical stability	Techniques to avoid floating-point errors from large/small intermediate values. Softmax: subtract max before exp to avoid overflow. SYM_INT32 matmul: int64 accumulator to avoid int32 overflow.
qsort()	Standard C library sort. O(N log N) average. Signature: void qsort(void *base, size_t n, size_t size, int(*cmp)(void*,void*)). cmp callback returns negative/0/positive for a<b/a=b/a>b.
uniform distribution	All values in a range [a,b] are equally likely. rngNextFloat() returns values uniformly distributed in [0,1).
xorshift32	A fast PRNG based on XOR and bit-shift operations. Period 2^32-1 (all non-zero 32-bit values). 3 operations: x ^= x<<13; x ^= x>>17; x ^= x<<5.

■ NOTE

This glossary covers every major term in the book. If a term still seems unclear after reading its definition, check the chapter indicated in brackets, or search the ODT source code for the struct or function name — the code itself is always the ground truth.